Pamantasan ng Lungsod ng Maynila
University of the City of Manila
Intramuros, Manila, Philippines

CONSO COMPILER

A Compiler Proposal presented to the
Computer Science Department
College of Information System and Technology Management

In Partial Fulfillment of the Requirements for the Degree
Bachelor of Science in Computer Science (BSCS)

Submitted by:
CONSOLE

Atencia, Dan Erdz B.
Dela Cruz, John Bryan P.
Kalaw, Miguel Kerdie B.
Lumeda, Lauvigne G.
Toribio, Louisse Andrea Mae M.

Submitted to:
PROF. LEISYL M. MAHUSAY
PROF. JONATHAN C. MORANO

May 2025

PAMANTASAN NG LUNGSOD NG MAYNILA
University of the City of Manila
Intramuros, Manila

College of Information System and Technology Management
Computer Science Department

APPROVAL SHEET

The proposed compiler entitled **"CONSO"** presented and submitted by **Console** is hereby approved and accepted as partial fulfillment in CSC 0322 - Compiler Design for the degree of Bachelor of Science in Computer Science, has been examined, and is recommended for acceptance and approval for **FINAL DEFENSE**.

Leisyl M. Mahusay, MEM/SM
Adviser

Jonathan C. Morano
Adviser

Accepted and approved in partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science.

Raymund M. Dioses, MAEd
Chairperson, CS Department

Khatalyn E. Mata, DIT
Dean, CISTM

ACKNOWLEDGEMENT

The successful culmination of the Conso compiler project, culminating in the creation of our programming language Conso and its compiler CNS, owes its completion to the invaluable assistance and steadfast encouragement of several key individuals and institutions. We wish to express our profound appreciation to all who contributed to this endeavor.

Our deepest gratitude is extended to our esteemed faculty advisors, Prof. Leisyl M. Mahusay and Prof. Jonathan C. Morano. Their exceptional guidance, critical insights, and unwavering dedication were absolutely foundational to navigating the complexities of compiler development. Their profound expertise profoundly shaped our technical approach and fostered a rigorous academic discipline essential for this undertaking.

We are equally indebted to the Computer Science Department of the College of Information Systems and Technology Management at the Pamantasan ng Lungsod ng Maynila. A special note of thanks goes to Dr. Khatalyn E. Mata, our respected dean, and Mr. Raymund M. Dioses, our department chair, for championing an environment that encourages innovative projects and provides the necessary academic framework for such intricate research and development.

This project is a testament to the collaborative spirit of the Conso team. The collective intelligence, tireless effort, and mutual support of every member were indispensable in overcoming the numerous challenges encountered. The synergy within the team was a driving force, transforming initial concepts into a tangible, functional system.

We also extend our most sincere thanks to our families and loved ones. Their unwavering patience, continuous encouragement, and deep understanding provided the vital personal support that sustained us throughout this demanding period. The knowledge and practical experience gained from developing Conso have been immense, significantly enhancing our capacity to approach and resolve complex, real-world computational problems.

Finally, we humbly acknowledge the divine grace and enduring strength provided by Almighty God, whose guidance illuminated our path and allowed us to bring this ambitious project to a successful conclusion

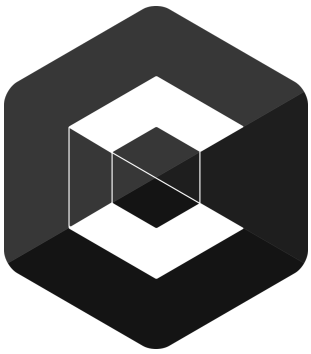
TABLE OF CONTENTS

I. Introduction

This document outlines the design and specifications of Conso, a novel programming language that aims to combine the simplicity of Python with the performance of C. It features a unique vowel-less syntax and supports multiple programming paradigms. The document details key aspects of Conso, including its reserved words and symbols, specific rules for variable and constant declarations, and the implementation of various data types such as integers, doubles, booleans, characters, and strings. Additionally, it provides guidelines on how to declare and access arrays within the Conso language environment. This comprehensive overview serves as a foundational reference for developers working with Conso. The programming language C, developed in the early 1970s, is a powerful and efficient low-level procedural language. Its strength lies in providing programmers with significant control over system resources, making it a preferred choice for developing operating systems, system software, embedded systems, game engines, and applications where performance is critical. C follows a procedural programming paradigm, organizing code into a sequence of procedures or functions. A key feature of C is its ability to directly access memory through pointers, contributing to its efficiency and close interaction with hardware. Notably, C code is often portable, capable of running on different platforms with relatively few changes. Furthermore, C has served as a foundational language, influencing the design and development of many other popular programming languages like C++, Java, and Python.

Python, in contrast to C, is a high-level, interpreted, and general-purpose programming language renowned for its readability and ease of use. Created in the late 1980s, Python has become widely adopted across diverse fields, including web development, data science, machine learning, scripting, and automation. As an interpreted language, Python executes code line by line, which simplifies the development and debugging processes. Its high-level abstraction allows developers to express complex operations with a concise and human-readable syntax. Python's dynamic typing means that variable types are checked during the program's execution, providing a degree of flexibility. The language boasts a large standard library, offering numerous built-in modules and functions, and an even more extensive ecosystem of third-party libraries and frameworks. Python also supports multiple programming paradigms, including procedural, object-oriented and functional styles, making it a versatile language for various programming needs.

II. System Overview



Conso is an innovative programming language that fuses the simplicity and elegance of Python with the performance and versatility of C. Its vowel-less syntax, support for multiple programming paradigms, and robust features make it a powerful tool for developers of all skill levels. Whether you are writing a quick script or developing a complex application, **Conso** provides the flexibility, efficiency, and clarity needed to bring your ideas to life.

One of the core strengths of **Conso** is its support of multiple programming paradigms, such as procedural and functional programming. This allows developers to write code in the style that best fits their problem-solving approach, making **Conso** a versatile tool for a wide range of applications, from small scripts to complex systems programming. The language's strict case sensitivity and explicit syntax rules also help reduce errors and ensure that code is well-structured and easy to maintain.

III. List of Reserved Words

RESERVED WORDS	C COUNTERPART	DESCRIPTION
INPUT AND OUTPUT		
npt	**scanf**	Represents input operations.
prnt	**printf**	Represents output operations.
DATA TYPES		
nt	**int**	Represents an integer data type.
dbl	**double**	Represents a decimal number.
strng	**char**	Represents a sequence of characters or text.
bln	**boolean**	Represents a boolean data type (true or false).
chr	**char**	Represents a one-character value.
CONDITIONAL STATEMENTS		
f	**if**	Executes a block of code if a condition is true.
ls	**else**	Executes a block of code if the previous condition(s) are false.
lsf	**elif**	Combines else and if for additional conditional checks.
swtch	**switch**	Marking the beginning of a switch block.
LOOP STATEMENTS		
fr	**for**	Creates a loop that iterates over a sequence.
whl	**while**	Creates a loop that continues as long as a condition is true.
d	**do**	Creates a loop that executes the block of code once and checks if

		the condition is true, the loop continues.
OTHERS		
mn	**main**	Serves as the main function of the program.
cs	**case**	Indicating a specific condition within the switch block.
dflt	**default**	The fallback when no cases match.
brk	**break**	Exiting the switch block after a case is executed.
cnst	**const**	Used to declare constants that hold values that should remain unchanged throughout the program.
tr	**true**	Represents the boolean value true.
fls	**false**	Represents the boolean value false.
fnctn	**function**	Defines a function in the language.
rtrn	**return**	Exits a function and optionally returns a value.
nll	**none**	Represents the absence of a value or a null value.
cntn	**continue**	Allows the program to skip the current iteration of a loop and continue.
strct	**struct**	Used to define a structured data type.
dfstrct	**struct**	Used to instantiate a structured data type.
vd	**void**	Return type of a function that returns normally but does not

		provide a result value to its caller.
end	**return**	Exits the main (mn) function.

IV. List of Reserved Symbols

RESERVED SYMBOLS	DESCRIPTION
ARITHMETIC OPERATORS	
+	Arithmetic symbol for addition.
-	Arithmetic symbol for subtraction.
*	Arithmetic symbol for multiplication.
/	Arithmetic symbol for division.
%	Arithmetic symbol for modulus.
ASSIGNMENT OPERATORS	
=	Used to assign the value on the right side to the variable on the left side
+=	Adds the value on the right side to the current value of the variable on the left and assigns the result back to the variable
-=	Subtracts the value on the right side from the current value of the variable on the left and assigns the result back to the variable.
*=	Multiplies the current value of the variable on the left by the value on the right and assigns the result back to the variable.
/=	Divides the current value of the variable on the left by the value on the right and assigns the result back to the variable.
%=	Computes the remainder of dividing the current value of the variable on the left by the value on

	the right and assigns the result back to the variable.
LOGICAL OPERATORS	
&&	Returns true only if both the operands are true or if both the conditions are satisfied. If not, it returns false.
||	Returns true if either operand is true. It also returns true if both the operands are true. If neither operand is true, it returns false.
!	Returns true if the operand is false and vice versa. Used to reverse the logical state of its single operand.
INCREMENT/DECREMENT OPERATORS	
++	Arithmetic symbol for increment by 1
--	Arithmetic symbol for decrement by 1
COMPARISON/RELATIONAL OPERATORS	
==	Returns true if both operands are equal
!=	Returns true if both operands are not equal
>	Returns true if the left operand is greater than the right operand.
<	Returns true if the left operand is less than the right operand.
>=	Returns true if the left operand is greater than or equal to the right operand.
<=	Returns true if the left operand is less than or equal to the right operand.
OTHERS	
;	Used for indicating the end of the statement.

:	Used for switch cases.
.	Used for identifying a double data type.
,	Used for concatenating strings. Used for declaring multiple variables.
{	Used for enclosing blocks of executable statements in a function.
}	Used for enclosing blocks of executable statements in a function.
“	Used for identifying a string data type.
”	Used for identifying a string data type.
(	Used for enclosing conditional statements, arguments, and parameters of a function.
)	Used for enclosing conditional statements, arguments, and parameters of a function.
#	Used for single-line comment.
[	Used for enclosing elements of a list/array.
]	Used for enclosing elements of a list/array.
‘	Used for identifying char data types.
’	Used for identifying char data types.
~	Used for identifying negative numbers.
\n	Used for creating a new line.
\t	Used to create a tab space
\”	Used to create a quotation

V. Specific Rules

VARIABLES

Variables are used to store values or information to be referenced and manipulated in a computer program. The identifiers are linked to these storage places.

Rules for Declaring Variables:

1. Variables are declared by stating the data type first, followed by an identifier. The data types are **nt** for integers, **dbl** for floating points, **strng** for series of characters, **bln** for boolean, and **chr** for single characters.
2. Variables can be declared both global and local. For **global variable declarations,** it should be done before the **mn()** function.
3. In Conso, variables can be declared without being assigned an initial value. Unlike traditional behavior where uninitialized variables have undefined values, Conso automatically assigns default values based on the variable type: nt default to **0**, dbl to **0.00**, bln to **fls**, chr to a null character **'\0'**, and strng to a null value **NULL**. This ensures predictable behavior when using uninitialized variables.
4. Multiple declarations of variables with the same data type are allowed and should be separated using a comma (**,**). **Ex**: nt a, b, c;
5. It is not allowed to declare more than one variable with a different data type on the same line.

SYNTAX

<data_type> <identifier>;
<data_type> <identifier>, <identifier>, <identifier> …;
<data_type> <identifier> = <value>;
<data_type> <identifier> = <value>, <identifier> = <value> …;

INTEGERS

An integer is a whole number that can be either positive, negative, or zero, without any fractional or decimal component. The keyword **nt** is used to declare integer variables in Conso.

Rules in Declaring Integers:

1. Positive integers are **unsigned** by default and are written without any prefix.
2. Negative integers are declared using the tilde (~) symbol as a prefix.
3. You can declare multiple integer variables on the same line, separated by commas (,).
4. Each variable's value can be initialized during the declaration but is not required.
5. Positive and negative integers can be mixed in a single declaration.
6. All declarations must end with a semicolon (;).

SYNTAX

nt <identifier>;
nt <identifier>, <identifier>, <identifier>...;
nt <identifier> = <value>;

nt <identifier> = <value>, <identifier> = <value> …;

EXAMPLE

nt x = 10;	Allowed
nt num, num1, num2;	Allowed
nt a = 5, b = ~17, c = 0, d = 22 + 1;	Allowed
nt pos = 45, neg = ~15, zero = 0;	Allowed
nt numx = 'c';	Not Allowed
chr digit = 1;	Not Allowed

DOUBLES

A **double** (dbl) in Conso is a data type that represents real numbers with decimal points, capable of handling precise calculations. It is used for scenarios where fractional values or high precision are required. Use the keyword **dbl** to declare a variable of double type.

Rules for Declaring Doubles:

1. Doubles can be declared without an initial value or assigned a default value of 0.00
2. Doubles must always include a decimal point. Even when declared with fewer digits after the decimal, Conso automatically formats the value to 2 decimal places by default.
3. Conso allows up to 8 digits to the left of the decimal point. A maximum of 6 digits is allowed to the right of the decimal point.
4. Negative doubles are represented with a ~ prefix, similar to negative integers in Conso.
5. You can declare multiple double variables in a single statement, separated by commas.
6. If no value is assigned, doubles default to **0.00** to prevent undefined behavior.

SYNTAX

dbl <identifier>;
dbl <identifier>, <identifier>, <identifier>...;
db <identifier> = <value>;
dbl <identifier> = <value>, <identifier> = <value> …;

EXAMPLE

dbl a = 12345.678901;	Allowed
dbl b = ~12345678.123456;	Allowed
dbl c = 2.2;	Allowed
dbl d = 0.000123;	Allowed
dbl e;	Allowed
dbl f, g = ~9.00, h = 10.25;	Allowed
dbl h = ~1234.0;	Allowed
dbl i = 12345678.9;	Allowed
dbl k = 1.2345678;	Not Allowed

dbl l = 0.0000001;	Not Allowed
dbl m = 999999999.99;	Not Allowed

BOOLEANS

In Conso, the bln data type represents logical values with two possible states: tr (true) and fls (false). Use the keyword **bln** followed by the variable name.

Rules in Declaring Booleans:

1. Use the keyword **bln** followed by the variable name. Optionally, assign an initial value of tr or fls.
2. If a **bln** variable is declared without an initial value, it defaults to **fls**.
3. Multiple bln variables can be declared on the same line, separated by commas.
4. A **bln** variable cannot store values other than tr or fls. Assigning anything else will result in an error.
5. When a **bln** value is displayed, tr will be **1** and fls will be **0.**

SYNTAX

bln <identifier>;
bln <identifier>, <identifier>, <identifier>...;
bln <identifier> = <value>;
bln <identifier> = <value>, <identifier> = <value> …;

EXAMPLE

bln a = tr;	Allowed
bln b = fls;	Allowed
bln c;	Allowed
bln d, e = tr, f = fls;	Allowed
bln g = 1;	Not Allowed
bln h = "true";	Not Allowed
bln i = ~tr;	Not Allowed

CHARACTERS

The chr data type in Conso represents a single character enclosed in single quotes ('). It is used to store a single ASCII character, which can include letters, digits, punctuation marks, or special symbols.

Rules in Declaring Characters:

1. Use the keyword **chr** followed by the variable name. Optionally, assign an initial value enclosed in single quotes.
2. If a chr variable is declared without an initial value, it defaults to a null terminator (**'\\0'**).
3. Multiple chr variables can be declared on the same line, separated by commas.
4. A **chr** variable can only store a single character. Assigning a string or multiple characters is invalid.
5. Any valid **ASCII** character can be assigned to a **chr** variable, including letters, digits, symbols, or whitespace.

SYNTAX

chr <identifier>;
chr <identifier>, <identifier>, <identifier>...;
chr <identifier> = <value>;
ch <identifier> = <value>, <identifier> = <value> …;

EXAMPLE

chr a = 'A';	Allowed
chr b = '1';	Allowed
chr c;	Allowed
chr d = ' ';	Allowed
chr e = '\n';	Allowed
chr f = "";	Not Allowed
chr g = 'XY';	Not Allowed
chr h = 65;	Not Allowed
chr i = "A";	Not Allowed

STRINGS

The **strng** data type in Conso represents a sequence of characters stored as a single entity, not as an array of characters. It is used to store textual data. Unlike traditional programming languages where strings may be arrays of characters, in Conso, strng is treated as an **immutable** string literal.

Rules in Declaring Strings:

1. Use the keyword **strng** followed by the variable name. Strings must be enclosed in double quotes (").
2. If a strng variable is declared without an initial value, it defaults to a **NULL**.
3. Multiple strng variables can be declared on the same line, separated by commas.
4. You can do **strng concatenations** using the operator (`) during declaration.
5. There is no explicit length limit for strng values, but the length of the string determines its truthiness and behavior in operations.
6. Special characters like **newline (\n), tab (\t)**, and **quotes** can be used within strings with escape sequences.

7. Strings must always be enclosed in **double quotes**. Using single quotes or omitting quotes is invalid.

SYNTAX

strng <identifier>;
strng <identifier>, <identifier>, <identifier>...;
strng <identifier> = <value>;
strng <identifier> = <value>, <identifier> = <value> …;

EXAMPLE

strng greeting = "Hello";	Allowed
strng message = "";	Allowed
strng word;	Allowed
strng quote = "Conso rocks!" ` “Yeah!;	Allowed
strng invalid = Hello;	Not Allowed
strng alsoInvalid = 'Hi';	Not Allowed
strng another = 123;	Not Allowed

ARRAYS

An array is a fixed-size collection of similar data items stored in contiguous memory locations. In Conso, arrays are used to store objects of the same data type.

Rules for Declaring an Array:

1. When declaring an array, its size should be strictly initialized. The initialization with elements is optional, and these elements must be enclosed with curly braces (**{}**).
2. Similar to variable declarations, if arrays were initialized without value(s) outside of the main function (mn), Conso will automatically assign default values depending on the array’s data type.
   i. **Ex**. nt num[3]; num = { 0, 0, 0 };
   ii. **Ex**. nt digit[4] = { 1 }; digit[4] = { 1, 0, 0, 0 };
3. However, if an array is declared without a value inside the main function (mn), it will result in an unidentified behavior.
4. Arrays can be declared both **globally and locally**.
5. Index starts at **0** in an array.
6. The **elements** inside an array must be literals of **nt, dbl, strng, bln, or chr.**
7. Placing a **strct** member inside an array is not allowed.
8. An array can be **one or two dimensions** in size.
9. Boolean (bln) arrays should only hold **tr** (true) or **fls** (false) values
10. Variable-sized arrays may not be initialized except with an empty value.

Rules for One-Dimensional Array:

1. The one-dimensional array must be declared with a variable name that belongs to the same data type, followed by **square brackets []**.

2. A one-dimensional array only needs one pair of square brackets.
3. Accessing elements in a one-dimensional array should be done using the index operator ([]).
4. When accessing an index of an array, you can also use a negative index operator as long as it is within the range of the elements.

SYNTAX

<data type> <identifier>[size] = {<elements>};
<data type> <identifier>[<size>];

ACCESSING ELEMENT

<identifier>[index];

EXAMPLE

***[Declaration & Initialization*]:**

nt numbers[3];	Allowed
strng names[2] = {“dan”, “erdz”};	Allowed
nt ages[3] = {19, 12, 30};	Allowed
chr letters[3] = {'A', 'B', 'C'};	Allowed
nt values[3] = {10, 20, 30};	Allowed
bln responses[4] = {fls, tr, tr, fls, ‘c’};	Not Allowed
nt ages[] = {25, 30, 40};	Not Allowed
dbl prices[] = {itemPrice, 50.75};	Not Allowed
nt values[] = 10, 20, 30; *#not enclosed in curly braces*	Not Allowed
nt data[] = {"string1", "string2"}; *#wrong data type*	Not Allowed
nt numbers[a+b]; *#initialization shouldn't hold expressions*	Not Allowed

***[Accessing*]:**

names[1];	Allowed
ages[0];	Allowed
letters[d-1]; *(where **d** is a value)*	Allowed
values[~1];	Allowed
prices[3+1];	Allowed
data[n]; *(where **n** is a nt value)*	Allowed
person[d - e]; *(where both **d** & **e** are nt literals)*	Allowed
group; *(where group is an array)*	Not Allowed

Rules for Two-Dimensional Array:

1. When declaring a two-dimensional array, it should start with a data type followed by a variable name.
2. After stating the data type and variable name, it should be followed by first square brackets indicating the size of the first dimension or row and followed by another square bracket indicating the size of the second dimension or column.

3. A two-dimensional array contains sets of curly braces inside of an array {{},{}}. To separate the array, a comma is used (,).
4. To access the element of the array, the identifier is written followed by two square bracket symbols [index][index]. The index must be an integer that starts at 0.
5. The size of a two-dimensional array will depend on the sizes declared inside the two opening and closing square brackets, and it is strictly required when initializing.

SYNTAX

<data type> <identifier>[<size1>][<size2>] = {{<elements>}, {<elements>}};
<data type> <identifier>[<size1>][<size2>];

EXAMPLE

***[Declaration & Initialization*]:**

nt names[2][4];	Allowed
nt dates[3][4] = {{1,2,3,4}, {5,6,7,8}, {9,10,11,12}};	Allowed
dbl values[2][2] = {{1.1, 2.2}, {3.3, 4.4}};	Allowed
strng strArray[2][2] = {{"Hello", "World"}, {"Hi", "There"}};	Allowed
dbl prices[][] = {{1.1, 2.2}, {3.3, 4.4}};	Not Allowed
nt data[2][2] = {{1, 2}, {3}};	Not Allowed
strng mixedArray[2][2] = {{"Hello", "World"}, {1, 2}};	Not Allowed
nt grid[][] = {{{1, 2}, {3, 4}}};	Not Allowed
nt numbers[a+b][y-x]; *#initialization shouldn't hold expressions*	Not Allowed

***[Accessing*]:**

names[1][3];	Allowed
dates[i][j];	Allowed
nums[i -1][j-1]; *(where i & j are both nt literals)*	Allowed
Values[d][e]; *(where d & e are both nt literals)*	Allowed
strArray[2+1][3-2];	Allowed
prices[~2][~1];	Allowed
data[d+1][1];	Allowed
grid; *(where grid is an array)*	Not Allowed
cups[3][];	Not Allowed

CONSTANTS

Constants are read-only variables whose values cannot be modified once they are declared in the program.

Rules in Declaring Constants:

1. For declaring constants, it must start with the **cnst** keyword followed by a **data type** (**nt, dbl, strng, bl, chr),** then the identifier, and then the equal sign, and after that, the value is assigned. An array constant declaration is allowed as long as it has a data type.
2. Constants can be declared **global** and **local**. For global declarations, it should be stated before the **mn()** function. However, you **cannot declare a cnst** inside functions except for the mn() function.

3. All constant declarations must be initialized upon declaration.

SYNTAX

cnst <data type> <identifier> = <value>;
cnst <data type>[array size] = {<elements>};

EXAMPLE

cnst nt cost = 500;	Allowed
cnst chr a = 'A';	Allowed
cnst dbl price = 12.50;	Allowed
cnst string newline = "\n"	Allowed
cnst nt cost = 300, labor = 40;	Allowed
cnst nt x = 10, y = 20;	Allowed
cnst dbl formula[2] = {3.14, 1.61];	Allowed
cnst nt z;	Not Allowed

IDENTIFIERS

Identifiers are names assigned to various program elements such as variables, arrays, and functions. It can be considered a pointer to a location for storing data.

Rules in Naming Identifiers:

1. Identifiers are case-sensitive. Therefore, **a** is not equal to **A**.
2. Identifiers can only start with a letter followed by either an alphanumeric or the special character ***underscore*** "_". Identifiers can have one character up to 16 characters.
3. Special characters like @#$%^&*()-+={}|~\/<>,.;:'?! are not allowed except for the underscore symbol **_**.

SYNTAX

<alpha><alpha/digitzero/Ø/_>^{15}

EXAMPLE

iden	Allowed
Iden123	Allowed
iden_	Allowed
3x	Not Allowed
Iden$%	Not Allowed
my iden	Not Allowed

STRUCTS

Structs, or structures, are a way to group several related variables into one location.

Rules in Declaring Structs:

1. Structs are declared using **strct** keyword, followed by an **identifier** and followed by enclosing Curly brackets (**{ }**)
2. Inside the struct declaration, you cannot assign a value to a variable.
3. Structs can be declared **globally** only.
4. Members of the **strct** variables are any of the data types (**nt, dbl, strng, bl, chr**).
5. When declaring multiple members within a single structure, each member must be on a newline and ending it with a semicolon (**;**).

Rules when creating an instance of Structs:

1. **dfstrct** keyword is used to create an instance. You can also declare multiple instances in a single line.
2. You can instantiate **strct** locally and globally.
3. You cannot initialize a value when instantiating.

Rules when accessing a member of Structs:

1. Must use **dot notation** (.) to access members.
2. Values must match the member's **data type**.
3. You **cannot use or access** an uninitialized struct member.

SYNTAX

```
strct <identifier> {
    <data type> <identifier>;
    <data type> <identifier>;
    …
};
```

EXAMPLE

***[Declaration & Instantiation]*:**

```
strct myStruct {                                Allowed
    nt age;
    dbl grade;
};

dfstrct myStruct struct1, struct2;              Allowed

strct myStruct{                                 Not  Allowed
    nt age;
    dbl grade = 1.25;
```

```
}
```

[Accessing]

```
mn(){                                   Allowed
    dfstrct myStruct s1;
    s1.age = 10;                        # Assigning value to a member
    s1.grade = 1.5;                     # Assigning value to another member
}

mn() {                                  Allowed
  dfstrct myStruct s1;
   s1.age = 30;
   nt doubleAge = s1.age * 2;           # Using structure member in an operation
}

mn() {                                  Allowed
   dfstrct myStruct s1;
   s1.age = 30;
   nt doubleAge = s1.age * 2;           # Using structure member in an operation
}

mn() {                                  Not Allowed
   dfstrct myStruct s1;
   prnt "Age: ", s1.age;                        # Accessing uninitialized member
}
```

FUNCTION

A function is a self-contained, reusable block of code that performs a specific task or set of tasks. Functions are a basic concept in many programming languages and serve several purposes, such as organization, reusability, and abstraction.

Rules in a Function Declaration:

1. **Declaration**:
    a. Start with the reserved word **fnctn** followed by a data type (void or a return type like nt, dbl, bln, chr, strng, or strct) and an identifier.
    b. Parameters are optional but must be enclosed within parentheses (). Use valid types like primitives, arrays, or structs.
    c. Functions can only be declared and created globally, before the mn() function.
2. **Parameters and Arguments:**
    a. **Valid parameter types**: nt, dbl, string, bln, chr, arrays (nt[], etc.), and strct.
    b. The order and type of parameters in the declaration must match those passed during the function call.
    c. Parameter and argument names must be between 1 and 16 characters long.
    d. Only letters (a-z, A-Z), digits (0-9), and underscores (_) are allowed. Names cannot start with a number or an underscore.
    e. Both parameter and argument names are case-sensitive

3. **Body**:
    a. Enclose executable statements within curly brackets ({}).
    b. End the function with **rtrn**. If **vd**, you cannot use the keyword **rtrn**; otherwise, the return type must match the declared type.
    c. Function body requires at least one statement. “Declarations only” are not allowed inside the function body, as it will be illogical and will throw a syntax error.

SYNTAX

```
fnctn vd <identifier>(<parameters>) {
   <statements>;
   rtrn;
         }

         fnctn <data_type> <identifier>(<parameters>) {
            <statements>;
            rtrn <value>;
         }
```

EXAMPLE

[Declaration and Initialization]

```
fnctn vd greet() {
   prnt("Welcome to Conso!");
}
```
Allowed

```
strct Student {
   nt id;
   dbl grade;
}
```
Allowed

```
fnctn dbl calculateGrade(strct Student) {
   rtrn s.grade * 1.1;
}
```
Not Allowed
#strct as parameters

```
fnctn nt[] getNumbers() {
   nt[] nums = [10, 20, 30];
   rtrn nums;
}
```
Not Allowed
#array as return type

```
void greet() {
   prnt "Hello, World!";
   rtrn;
{
```
Not Allowed
#missing fnctn keyword

```
fnctn nt add(nt a, nt b){
   a + b;  # Calculates but doesn't return the result
}
```
Not Allowed
#missing return statement

```
mn() {
   fnctn nt nestedFn() {                                    Not Allowed
      rtrn 10;                                              #declaring fnctn inside a function
(main)
   }
{
```

FUNCTION CALL

Function Call Rules:

1. The function **must be defined first** before calling the function.
2. When calling a function, use the identifier following with parenthesis **()**.
3. If the function has parameters, the call must include the same number, order, and data types of arguments as specified in the function definition.
4. Nested function calls are allowed inside a function call as long as all the call rules are followed.
5. Passing a strct member as an argument in a function call is allowed.

SYNTAX

<identifier>(<arguments>);

EXAMPLE

[Function Call]

```
hello();                                                    Allowed
name("Dan");    #passing a strng parameter                  Allowed
age(19);        #passing an nt parameter                    Allowed

fnctn addNum(nt a, nt b) {                                  Not Allowed
   rtrn a + b;
}
mn() {
   addNum(5);     #not the same number of arguments

}

fnctn void displayStudent(nt id, str name) {                Not Allowed

   prnt id, name;

   rtrn;

}

mn() {

   displayStudent("John", 101);     # Reversed argument order
```

{

CONDITIONALS

Statements that specify an action to be performed only in situations in which certain conditions have been met are known as conditional statements. The reserved words **f, ls, lsf, swtch** are used for conditional statements.

Rules in Conditional Statement (f)

1. The conditional statement 'if' starts with the corresponding reserved word **(f)** followed by the opening bracket symbol "**(“** inside is the expression and closing it with the closing bracket symbol").”
2. The body of the conditional statement should be enclosed with opening and closing brackets "{}”.
3. If there are multiple conditions inside the condition statement, this should be enclosed with () to avoid ambiguities when evaluating values.
4. Forms of expression:
    a. **Relational**: x > y, x <= y
    b. **Equality**: x == y, x != y
    c. **Logical**: x>0 && y>0, x<=5 || y >= 20
    d. **Negation**: !condition

SYNTAX

```
f (<condition>) {
<statement>
{

f (<condition> <logical operators><condition>....) {
<statement>
}
```

EXAMPLE

```
f (3>2) {                                                    Allowed
  prnt(“3 is greater than 2”);
}

f ((x > 5) && (y <= 10)) {                                   Allowed
      prnt("x is greater than 5 and y is less than or equal to 10.");
   }

f (!((x == 0) && (y == 0))) {                                Allowed
      prnt("Neither x nor y are both zero.");
   }

f ((grade == 'A') || (grade == 'B')) {                       Allowed
```

```
    prnt("Good job!");
  }
```

```
f 3>2{      #condition not enclosed with parenthesis                    Not Allowed
  prnt (“3 is greater than 2”);
{
```

Rules in Conditional Statement (f-ls)

1. Only in the instance that **there is an f statement should there be an ls statement**. If the condition is not met, the statements within its body will be carried out.
2. The body of **ls** statement should also be enclosed with the opening and closing brackets ”{}”. This ensures consistency and prevents ambiguity in conditional structures.
3. **ls** statements must be written either after an **f** or **lsf** statements.
4. Forms of expression:
    a. **Relational**: x > y, x <= y
    b. **Equality**: x == y, x != y
    c. **Logical**: x>0 && y>0, x<=5 || y >= 20
    d. **Negation**: !condition

SYNTAX

```
f (<condition>) {
    <statement>
}
ls {
    <statement>
}
```

EXAMPLE

```
nt a = 20;                                                    Allowed
f (a<18) {
  prnt (“Good Morning”);
}
ls {
  prnt (“Good Night”);
}
```

```
f ((x > 5) && (y > 5)) {                                      Allowed
    prnt("Both x and y are greater than 5.");
}
ls {
    prnt("Either x or y is not greater than 5.");
}
```

```
f (!isAuthenticated) {                                        Allowed
      prnt("Access denied.");
```

```
  }
  ls {
    prnt("Welcome, user.");
  }
```

```
ls { prnt("Invalid condition."); } #no prior f condition
```
Not Allowed

Rules in Conditional Statement (f-lsf-ls)

1. Similar to the **f** statement, the conditional statement starts with the reserved word **lsf** followed by conditions or a set of conditions in-between logical operators. These condition(s) are enclosed in a single block of opening and closing parenthesis **()**.
2. The body of **lsf** statement should also be enclosed with the opening and closing brackets ”{}”. Else if statement only exists if there is an **f** statement and should be written after the **f** statement.
3. The conditional statement **lsf** may or may not have an **ls** statement.
4. Forms of expression:
    a. **Relational**: x > y, x <= y
    b. **Equality**: x == y, x != y
    c. **Logical**: x>0 && y>0, x<=5 || y >= 20
    d. **Negation**: !condition

SYNTAX

```
f (<condition>) {
  <statement>
}
lsf (<condition) {
  <statement>
}
ls {
  <statement>
}
```

EXAMPLE

```
nt a = 10;
f (a == 5) {
  prnt("a is 5");
}
lsf (a == 10) {
  prnt("a is 10");
}
lsf (a == 15) {
  prnt("a is 15");
}
ls {
  prnt("a is not present");
```
Allowed

```
}

nt age = 19;                                                    Allowed
f (age < 18) {
    prnt("You are a minor.");
}
lsf ((age >= 18) && (age <= 60)) {
    prnt("You are an adult.");
}
ls {
    prnt("You are a senior citizen.");
}

f (x > 5) {                                                     Not Allowed
   prnt("x is greater than 5");      # lsf should be used after an f statement, not inside the f block itself.
   lsf (x > 10) {
      prnt("x is greater than 10");
   }
}
```

Rules in Nested Conditional Statement (f, f-ls, f-lsf-ls)

1. In such cases where there are different conditions, Conso supports nested f statements wherein another f, f-ls, or f-lsf-ls statement is written in the body of the primary f, lsf, and ls statements.
2. **Outer block first**: The outermost f, lsf, or ls statement is evaluated first.
3. **Inner blocks next**: If the outer condition is true, it will then evaluate and execute the inner conditional statements one by one.
4. Forms of expression:
    a. **Relational**: x > y, x <= y
    b. **Equality**: x == y, x != y
    c. **Logical**: x>0 && y>0, x<=5 || y >= 20
    d. **Negation**: !condition

SYNTAX

```
f (<condition>) {
  <statement>
  f (<nested_condition>) {
     <nested_statement>
  }
  lsf (<nested_condition>) {
     <nested_statement>
  }
  ls {
     <statement>
  }
}
```

EXAMPLE

Allowed

```
nt a = 12, b = 20;
f (a > 10) {
    f  (b < 5) {
        prnt(“a is greater than 10 and b is less than 5”);
    }
    lsf (b == 5) {
        prnt(“a is greater than 10 and b is equal to 5”);
    }
    ls {
        prnt(“a is greater than 10 and b is greater than 5”);
    }
}
lsf (a == 10) {
    prnt(“a is equal to 10”);
}
ls {
    prnt(“a is less than 10”);
}
```

Allowed

```
nt temp = 28;
nt humidity = 65;
        f (temp > 30) {
            f (humidity > 70) {
                prnt("It's hot and humid");
            }
            ls {
                prnt("It's hot but not humid");
            }
        }
        lsf ((temp >= 20) && (temp <= 30)) {
            f (humidity > 60) {
                prnt("It's warm and humid");
            }
            ls {
                prnt("It's warm but comfortable");
            }
        }
        ls {
            prnt("It's cold outside");
        }
```

Rules in Switch Statement (swtch)

1. For switch statements, the reserved word (**swtch**) must be used first, followed by an identifier that is enclosed in a parenthesis **()**.

2. The identifier will be compared to the case value, which can be found in the body of the switch statement. The case statement should be written as follows: **cs <value>:** and followed by statements, the reserved word (**brk)** and one semicolon "**;**".
3. Switch can have multiple cases, where every case must have a corresponding **brk**.
4. The cs expression must be a valid literal or identifier that can be compared with the switch identifier. Valid data types for **cs** include:
    a. nt (integer)
    b. chr (character)
5. Default using the reserved word **dflt** may or may not be used in a switch. It should have a **brk**. If there is a default, it must be written at the bottom of the body, after all the cases.
6. The body of the switch statement must be enclosed with the opening and closing brackets "**{}**".

SYNTAX

```
swtch(identifier) {
        cs literal:
        <statement>
        brk;
        ;
        dflt:
        <statement>
        brk;
        ;
    }
```

EXAMPLE

```
nt num = 1;                                          Allowed
swtch(num) {
   cs 1:
      prnt("One");
      brk;
   cs 2:
      prnt("Two");
      brk;
   dflt:
      prnt("Other");
      brk;
}

strct Student {                                      Allowed
   nt id;
   str name;
};
```

```
swtch (n) {                                                    Not Allowed
   cs 1:
      prnt("One");
   cs 2:
      prnt("Two");
   brk;                                                        #missing 'brk' or 'dflt' after case statements
}
```

ITERATIVE

Iterative or loop statements allow the execution of a block of code multiple times, given that the condition is met. There are three iterative statements that Conso supports, which are fr, whl, and d-whl statements.

Rules in Iterative Statement (fr)

1. To declare a fr loop statement, the reserved word **fr** must be written first, followed by the opening round bracket **(**, the ***initialization expression***, *the* ***test condition***, and the ***increment or decrement statement*** for the control variable, with each part separated by a semicolon **;**, and the closing round bracket **)**.
    a. **Initialization value**: The initialization must be of type **nt** (integer).
    b. The ***test condition*** and the ***increment or decrement statement*** should also be between nt values.
    c. You cannot declare an nt (integer) type in the ***initialization expression***. It should be declared first before the iterative statement, and then you can initialize its value.
    d. The body of the frloop must be enclosed with the opening and closing brackets "{}".

SYNTAX

```
fr(<variable_declaration>; <condition>; <decrement_or_increment variable>) {
<statements>
}

<datatype> <identifier> = <initial_value>;
fr(<identifier_initialization>; <condition>; <increment/decrement>) {
   <statements>;
}
```

EXAMPLE

```
fr(a = 0;  a < 5; a++) {                                       Allowed
   prnt("\n");
}

nt i = 0;
```

```
fr(i;  i < 5; i++) {                                    Allowed
   prnt(“\n”);
}

nt i;
fr(i = 0;  i < 5; i+=i) {                               Allowed
   prnt(“\n”);
}

fr(a = 0 a < 5 a++) {      #no colons                   Not Allowed
   prnt("\n");
}

fr(; a < 5; a++) {                 #no variable counter  Not Allowed
   prnt("\n");
}

fr(a = 0; a < 5;) {  #no increment/decrement operation  Not Allowed
   prnt("\n");
}

fr(a = 0; a < 5; a = a +) {  #not valid increment       Not Allowed
   prnt("\n");
}

fr(a = 0; a < 5; a++) {      #dbl is not allowed        Not Allowed
   prnt("\n");
}
```

Rules in Iterative Statement (whl):

1. The reserved word **whl** must be used for the ‘while’ loop statement followed by the condition, which is enclosed in a bracket **().**
2. The body of the **whl** statement must be enclosed in the opening and closing brackets “{}”.

SYNTAX

```
whl(<condition>) {
  <statements>
}
```

EXAMPLE

```
nt c = 6;
whl(c>0) {                                              Allowed
```

```
   prnt(c);
   c- - ;
}
whl c && 'o' {                                    Not Allowed
   prnt(c);
   c++ ;
}
```

Rules in Iterative Statement (d-whl)

1. The do-while statement must start with the reserved word (**d**), followed by its body, the reserved word (**whl**), and the condition, which is enclosed in a round bracket symbol "**()**".
2. The body of the **d-whl** statement is enclosed in the opening and closing brackets "**{}**".

SYNTAX

```
d{
    <statements>
} whl (<condition>);
```

EXAMPLE

```
nt b = 5;
d{                                                Allowed
   prnt(b);
   b- -;
} whl(b>0);

d{                                                Not Allowed
   prnt(b);
   b- -;
}    # no whl statement


d{                                                Not Allowed
   prnt(b);
   b- -;
} whl b < 0;   # condition not inside a () block
```

Rules in Nested Iterative Statement

1. Conso supports nested looping statements wherein another **fr**, **whl**, and **d-whl** loop statement can be written inside the body of a loop statement.

SYNTAX

```
fr(<variable_declaration>; <condition>; <decrement_or_increment variable>)  {
```

```
        fr(<variable_declaration>; <condition>; <decrement_or_increment variable>)  {
          <statements>
        }

        whl (<condition>) {
           <statements>
        }

        d {
           <statements>
        } whl(<expression list>);
}
```

EXAMPLE

```
fr(a = 0; a < 5; a++) {
   prnt(“\n”);
    b = 1;
      whl(b<a) {
         prnt(“*”);
         b++;
      }
}
```

Allowed

```
nt a = 5;
whl(a>0) {
   prnt(“\n");
   nt b = 1;
      whl(b<=a) {
         prnt(“*”);
         b++;
      }
  a- - ;
}
```

Allowed

CONDITIONAL EXPRESSION

Conditional expressions are methods for deciding inside of a single line of code. To choose between the two values, it uses the value of the bool expression.

Rules for Conditional Expression:

1. Conditional expression can be used once **f**, **lsf**, **fr**, **whl**, and **d-whl** are stated.
2. When reading the values within the expression, it should always **start from left to right,** minding the presence of the **parenthesis symbols,** just like the closing and opening parenthesis.
3. If there are multiple conditions inside the conditional expression, this should be enclosed with () to avoid ambiguities when evaluating values.

4. A conditional expression will throw a bool value.

SYNTAX

<variable or value> <comparison operator> <variable or value>

EXAMPLE

a > 10	Allowed
b < 4	Allowed
c >= d	Allowed
e <= f	Allowed
c + 3 <= 10	Allowed
o != 0	Allowed
(x > 0) && (y < 10)	Allowed
a <= 100 || b >= 50	Not Allowed
(x > 10 || y < 20) && z == 5	Not Allowed
x > 5 && y < 10 || z == 0	Not Allowed
x > && y < 10 *#Logical operator '&&' has no valid operands.*	Not allowed
name > "Bob" *#Strings cannot be compared using relational operators.*	Not allowed

f (x = 5) { *# '=' is an assignment operator, not a comparison.*
prnt("Invalid condition.");
}
	Not allowed

ASSIGNMENT EXPRESSION

The assignment expression provides a succinct method for giving a variable in an expression a value. When a value needs to be used and assigned to a variable within the same expression, this is quite helpful.

Rules for Assignment Expression:

1. The **assignment expression** must include:
    a. An identifier on the left-hand side (**LHS**),
    b. An assignment operator, and
    c. A value or expression on the right-hand side (**RHS**), followed by a semicolon (;).
2. The assignment operator symbol is =. Compound assignment operators such as +=, -=, *=, /=, and %= are also valid.
3. The expression on the **RHS** is evaluated only once, and the result is stored in the identifier on the LHS.
4. Assignment within parentheses is allowed only when it is part of a larger expression. For example: y = (x = 5) + 2.
5. **Compound operators** inside parentheses are not allowed unless they are part of a larger assignment. For instance, y = (x += 5) is valid, but (x += 5) alone is not.
6. Increment (++) and decrement (--) operators are valid but require spacing when used as part of an expression (e.g., x + ++x).

7. Identifiers must follow naming rules (e.g., must not include invalid characters or start with numbers).

SYNTAX

identifier <assignment operator> <value> ;

EXAMPLE	
a = 1;	Allowed
b += 40;	Allowed
b -= 20;	Allowed
f *= 15;	Allowed
d /= 20;	Allowed
e %= 10;	Allowed
y = (x = 5) + 2;	Allowed
result = (10 + 5) * 2;	Allowed
y = (5 += 2);	Not Allowed

INPUT and OUTPUT

The input statement is used to get user input and store it in a variable.

Rules for Input (npt):

1. **Purpose:** The npt reserved word is used to accept user input. It pauses program execution and waits for the user to provide input.
2. **Syntax**: Input must be assigned to a declared variable, like this **identifier = npt("prompt text");**
    a. ***identifier***: The variable that will store the input value. It must already be declared before using npt.
    b. **"*prompt text*"**: A required string literal enclosed in double quotes to guide the user.
3. **Behavior:**
    a. Input values are automatically typecast to the variable's data type. If typecasting fails, an error will be thrown.
    b. Only one variable can be specified per npt call.
4. **Data Type Rules:**
    a. **Strings (strng):** Accept any input, including plain text or input enclosed in double quotes.
    b. **Characters (chr):** Accept a single character. Entering more than one character results in an error.
    c. **Integers (nt) and Doubles (dbl):** Must be valid numerical input. Non-numeric input will throw an error.
    d. **Booleans (bln):** Accept only tr or fls. Other input causes an error.
5. **Default Values:** If npt is not used, variables retain their default values as per Conso rules:
    a. **nt: 0**

b. **dbl: 0.00**
c. **bln: fls (0)**
d. **chr: '\0'**
e. **strng: NULL**

6. **Prompting:**
   a. The npt function requires a string literal inside its parentheses to serve as the prompt for the user. This replaces the need for a separate prnt statement.

SYNTAX

<identifier> = npt("prompt text");

EXAMPLE

# Accepting a string strng name; name = npt("Enter your name: ");	Allowed
# Accepting an integer nt age; age = npt("Enter your age: ");	Allowed
# Accepting a double dbl salary; salary = npt("Enter your salary: ");	Allowed
# Accepting a boolean bln isStudent; isStudent = npt("Are you a student? (tr/fls): ");	Allowed
# Accepting a character chr grade; grade = npt("Enter your grade: ");	Allowed
npt("Enter your name: "); *# Error: `npt` must be assigned to a variable.*	Not Allowed
nt age, year; age, year = npt("Enter age and year: "); *# Error: Only one variable can be used.*	Not Allowed
strng name; name = npt(); *# Error: Prompt is required inside `npt`.*	Not Allowed
npt(identifier); *# Error: Direct input into identifier is not allowed.*	Not Allowed

Rules for Output (prnt):

1. The print statement has the reserved word (**prnt**).

2. The print statement should start with the reserved word (**prnt**), followed by the strng, or identifier enclosed in the open and close parenthesis **()**.
3. **prnt** strictly receives only the string literal, identifier, and set of expressions.
4. The **prnt** statement prints only the values of the identifiers. Calling of functions, methods, and classes is not allowed.
5. You may use the **prnt** statement to print individual elements of an array directly, e.g., prnt(num[1]);.
6. The symbol (,) in **the prnt** statement is strictly for concatenation.
7. When printing a bln value using prnt, it will only represent either **1** or **0**. 1 represents **tr** and 0 represents f**ls**.
8. When printing a dbl value, it automatically rounds it to **two** decimal places.

SYNTAX

prnt(<strnglit> <add str>);
prnt(<strnglit>, <identifier>);
prnt(<strnglit> | <identifier>, <strnglit> | <identifier>...);
prnt(expression);

EXAMPLE

```
nt a = 10;
prnt("The variable A is equal to " , a);          Allowed

nt c[] = {1, 2};
prnt(c); # prints {1, 2}                          Allowed

nt o = {1, 2};
prnt(n[0]); # prints 1                            Allowed

strng greet = "Conso";
prnt("Hello, " , greet); # prints Hello, Conso    Allowed

dbl pi = 3.14159;
prnt("The value of pi is ", pi);                  Allowed

bln isActive = tr;
prnt ("The value is ", isActive);                 Allowed

prnt((7 + 3) && (fls == 'c');                     Allowed

chr letter = 'A';
prnt("The first letter is " , letter);            Allowed
prnt("The letter is " letter);                    Not Allowed
prnt "Hello, Conso";                              Not Allowed
prnt(Hello,World);                                Not Allowed
prnt(someFunction());                             Not Allowed
```

EXPRESSION

In Conso, an expression is a strictly evaluated combination of **values, variables, operators, and functions**, where each component must adhere to predefined type constraints. Expressions can be:

- **Literals** (e.g., 5, 3.14, 'A', true)
- **Variable references** (e.g., x, y[2], status)
- **Function calls** (e.g., ComputeSum(4, 7))
- Operator-based evaluations, provided they match type-specific rules.

Rules in Creating Expressions:

1. Expressions can include **arithmetic**, **logical**, **comparison**, and **assignment** operators.
2. **Parentheses** are used to group expressions and control the order of evaluation.
3. Each expression can assign a **value** to an identifier, or the result can be used immediately.
4. Expressions can **mix different types** (e.g., combining arithmetic with a comparison operator).
5. When initializing an **nt** and **dbl** data types using expression, **only arithmetic operations are allowed**.

SYNTAX
<expression1>, <expression2>, ..., <expressionN>

EXAMPLE	
a = b + 5	Allowed
flag = x > 10	Allowed
a = b + 5, , c = a * 2	Not Allowed
x > 10 && y < 5, z == 3 prnt(x);	Not Allowed

ARITHMETIC EXPRESSION

An arithmetic expression is made up of operators, numeric values, and occasional parentheses. This kind also yields a numerical result. The operators employed in arithmetic operators like addition, subtraction, and so on are used in these expressions.

Rules for Arithmetic Expressions in Conso

1. **Operator Precedence & Evaluation Order**
    - Arithmetic expressions follow **strict precedence rules**, similar to PEMDAS (Parentheses, Exponents, Multiplication/Division, Addition/Subtraction).
    - Expressions are evaluated **left to right**, unless overridden by parentheses.
2. **Valid Arithmetic Structure**
    - An arithmetic expression **must** start with a **value or variable**, followed by an **operator**, and then another **value or variable**.
    - Parentheses **( )** can be used to group expressions and modify evaluation order.
3. **Allowed Operations**

- Integer (int) to Double (dbl) type arithmetic operations are allowed.
- Compound assignment operators (+=, -=, *=, /=, %=) and increment/decrement operators (++, --) are valid but must follow type restrictions.

4. **Unsupported Types**
   - Characters (chr), booleans (bln), and strings (strng) cannot be used in arithmetic operations.

SYNTAX

<variable or value> <arithmetic operator> <variable or value>
<variable> <compound operator> <value>
<identifier><<arithmetic operator><identifier>

EXAMPLE

a + b	Allowed
10 * 5	Allowed
(x - 4) * 3 + y	Allowed
c += 5	Allowed
++x + y	Allowed
x-- - 3	Allowed
x + ++x + 3	Allowed
10+5 *#No space between operands and operator*	Not Allowed
x++y *#No whitespace between ++ and y*	Not Allowed
x + -- y *#Whitespace between decrement operator -- and variable y*	Not Allowed
++ x *#Whitespace between increment operator ++ and variable x*	Not Allowed
c += *#Missing value after compound operator)*	Not Allowed

RELATIONAL EXPRESSION

A relational expression is an expression that compares two values or variables using a relational operator to determine their relationship. It evaluates to either **tr** or **fls** based on whether the specified condition is satisfied.

Rules for Relational Expressions:

1. **Strings (str)**: You can only use relational operations on string into another string. Comparing to other data types is not allowed. When comparing strings to strings using relational operators, only (==) and (**!**=) operators are allowed. This means you are comparing if the other string is perfectly equal to the other string.
2. **Characters (chr)**: You can only use relational operations on character into another character. Comparing to other data types is not allowed. When comparing characters to other characters, only (==) and (**!**=) operators are allowed. This means you are comparing if the other character is perfectly equal to the other character.
3. **Booleans (bln)**: You can only use relational operations on booleans into another boolean. Comparing to other data types is not allowed. When comparing booleans to other booleans, only

(==) and (**!**=) operators are allowed. This means you are comparing if the other boolean is perfectly equal to the other boolean.

4. **Integers (nt) and Doubles (dbl)**: integers to integers and doubles to doubles can be directly compared using relational operators. When a relational operation involves both an integer and a double, it will throw an error.
5. **Conso's relational operators include:**
    a. == : Equal to
    b. **!**= : Not equal to
    c. > : Greater than
    d. < : Less than
    e. >= : Greater than or equal to
    f. <= : Less than or equal to

SYNTAX

<variable or value> <relational_operator> <variable or value>|
<identifer><relational_operator><variable or value>

EXAMPLE	
5 == 5	Allowed
3.14 != 2.71	Allowed
'a' != 'b'	Allowed
"hello" == "hello"	Allowed
tr == fls	Allowed
5 == "hello"	Not Allowed
3.14 > "test"	Not Allowed
a' == tr	Not Allowed
tr == "string"	Not Allowed
5 > 'a'	Not Allowed

LOGICAL EXPRESSION

A logical expression is a statement that may or may not be true. They are essential for setting up parameters, making choices, and managing the program's flow. They are essential to building responsive and dynamic code.

Rules for Logical Expressions:

1. **Logical Operators**
    a. The main logical operators are:
        i. && (AND)
        ii. || (OR)
        iii. ! (NOT)

2. **Valid Logical Expressions**
    a. A logical expression **must** contain **at least one logical operator** to be considered valid.

b. Expressions are evaluated **left to right**, following operator precedence, unless altered by parentheses.

3. **Boolean-Only Logical Operations**
   a. **Logical operations are only allowed on boolean values.**
   b. The operands in logical operations must be one of the following:
      i. A **boolean literal** (tr, fls)
      ii. A **boolean variable** (bln identifier)
      iii. A **boolean evaluation of a relational expression** (e.g., (3 > 2), (x == y))
   c. Other data types (nt, dbl, chr, strng) cannot be used in logical expressions.

SYNTAX

<conditional expression> <logical operator> <conditional expression>

EXAMPLE	
((a > 0) || (b > 0))	Allowed
((a>10) && (b> 10))	Allowed
((c == conso) && (s > 0))	Allowed
((a == 0) && (b == 0))	Allowed
((c >= 10) || (d <= 20))	Allowed
((a == b) || (c != d))	Allowed
(!(z <= 0))	Allowed
(a === b)	Not Allowed
(x > 5 y < 10)	Not Allowed
(x > 5 && ! 10)	Not Allowed

ARITHMETIC, RELATIONAL, AND LOGICAL EXPRESSIONS TABLE

ARITHMETIC OPERATIONS (+, -, *, /, %)	**INTEGER (nt)**	nt to nt ✔	**RELATIONAL OPERATIONS (==, !=, <, >, <=, >=)**	**INTEGER (nt)**	nt to nt ✔
		nt to chr ❌			nt to chr ❌
		nt to dbl ❌			nt to dbl ❌
		nt to bln ❌			nt to bln ❌
		nt to strng ❌			nt to strng ❌
	DOUBLE (dbl)	dbl to nt ❌		**DOUBLE (dbl)**	dbl to nt ❌
		dbl to chr ❌			dbl to chr ❌
		dbl to dbl ✔			dbl to dbl ✔
		dbl to bln ❌			dbl to bln ❌
		dbl to strng ❌			dbl to strng ❌
	CHARACTER (chr)	chr to nt ❌		**CHARACTER (chr)** *only (==, !=)*	chr to nt ❌
		chr to chr ❌			chr to chr ✔
		chr to dbl ❌			chr to dbl ❌
		chr to bln ❌			chr to bln ❌
		chr to strng❌			chr to strng ❌
	BOOLEAN (bln)	bln to nt ❌		**BOOLEAN (bln)** *only (==, !=)*	bln to nt ❌
		bln to chr ❌			bln to chr ❌
		bln to dbl ❌			bln to dbl ❌
		bln to bln ❌			bln to bln ✔
		bln to strng❌			bln to strng ❌
	STRING (str)	strng to nt ❌		**STRING (str)** *only (==, !=)*	str to nt ❌
		strng to chr ❌			str to chr ❌
		strng to dbl ❌			str to dbl ❌
		strng to bln ❌			str to bln ❌
		strng to strng❌			str to strng ✔

LOGICAL OPERATIONS (&&, ||, !)	**INTEGER (nt)**	nt to nt ❌
		nt to chr ❌
		nt to dbl ❌
		nt to bln ❌
		nt to strng ❌
	DOUBLE (dbl)	dbl to nt ❌
		dbl to chr ❌
		dbl to dbl ❌
		dbl to bln ❌
		dbl to strng ❌
	CHARACTER (chr)	chr to nt ❌
		chr to chr ❌
		chr to dbl ❌
		chr to bln ❌
		chr to strng ❌
	BOOLEAN (bln)	bln to nt ❌
		bln to chr ❌
		bln to dbl ❌
		bln to bln ✔
		bln to strng ❌
	STRING (str)	str to nt ❌
		str to chr ❌
		str to dbl ❌
		str to bln ❌
		str to strng ❌

VI. Regular Expression

RESERVED WORDS	REGULAR EXPRESSION	TOKEN
INPUT AND OUTPUT		
npt	(n)(p)(t)	npt
prnt	(p)(r)(n)(t)	prnt
DATA TYPES		
nt	(n)(t)	nt
dbl	(d)(b)(l)	dbl
strng	(s)(t)(r)(n)(g)	strng
bln	(b)(l)(n)	bln
chr	(c)(h)(r)	chr
CONDITIONAL STATEMENTS		
f	(f)	f
ls	(l)(s)	ls
lsf	(l)(s)(f)	lsf
swtch	(s)(w)(t)(c)(h)	swtch
LOOP STATEMENTS		
fr	(f)(r)	fr
whl	(w)(h)(l)	whl
d	(d)	d
OTHERS		
mn	(m)(n)	mn
cs	(c)(s)	cs
dflt	(d)(f)(l)(t)	dflt
brk	(b)(r)(k)	br

cnst	**(c)(n)(s)(t)**	**cnst**
tr	**(t)(r)**	**tr**
fls	**(f)(l)(s)**	**fls**
fnctn	**(f)(n)(c)(t)(n)**	**fnctn**
rtrn	**(r)(t)(r)(n)**	**rtrn**
nll	**(n)(l)(l)**	**nll**
cntn	**(c)(n)(t)(n)**	**cntn**
strct	**(s)(t)(r)(c)(t)**	**strct**
vd	**(v)(d)**	**vd**

LITERAL	REGULAR EXPRESSION	TOKEN
Identifier	**(alpha)(alpha/digitzero/Ø/_)^{15}**	**id**
Integer	**(~/Ø)(digitzero)(digitzero/Ø)^{7}**	**ntlit/~ntlit**
Double	**(~/Ø)(digitzero/Ø)^{15}(digitzero)(.)(digitzero)(digitzero/Ø)^{7}**	**dbllit/~dbllit**
String	**(“)(ascii)*(“)**	**strnglit**
Char	**(‘)(ascii)(‘)**	**chrlit**
Boolean	**(true/false)**	**blnlit**
Comments	**(#)(ascii)(newline)**	**comment**

VII. Regular Definition

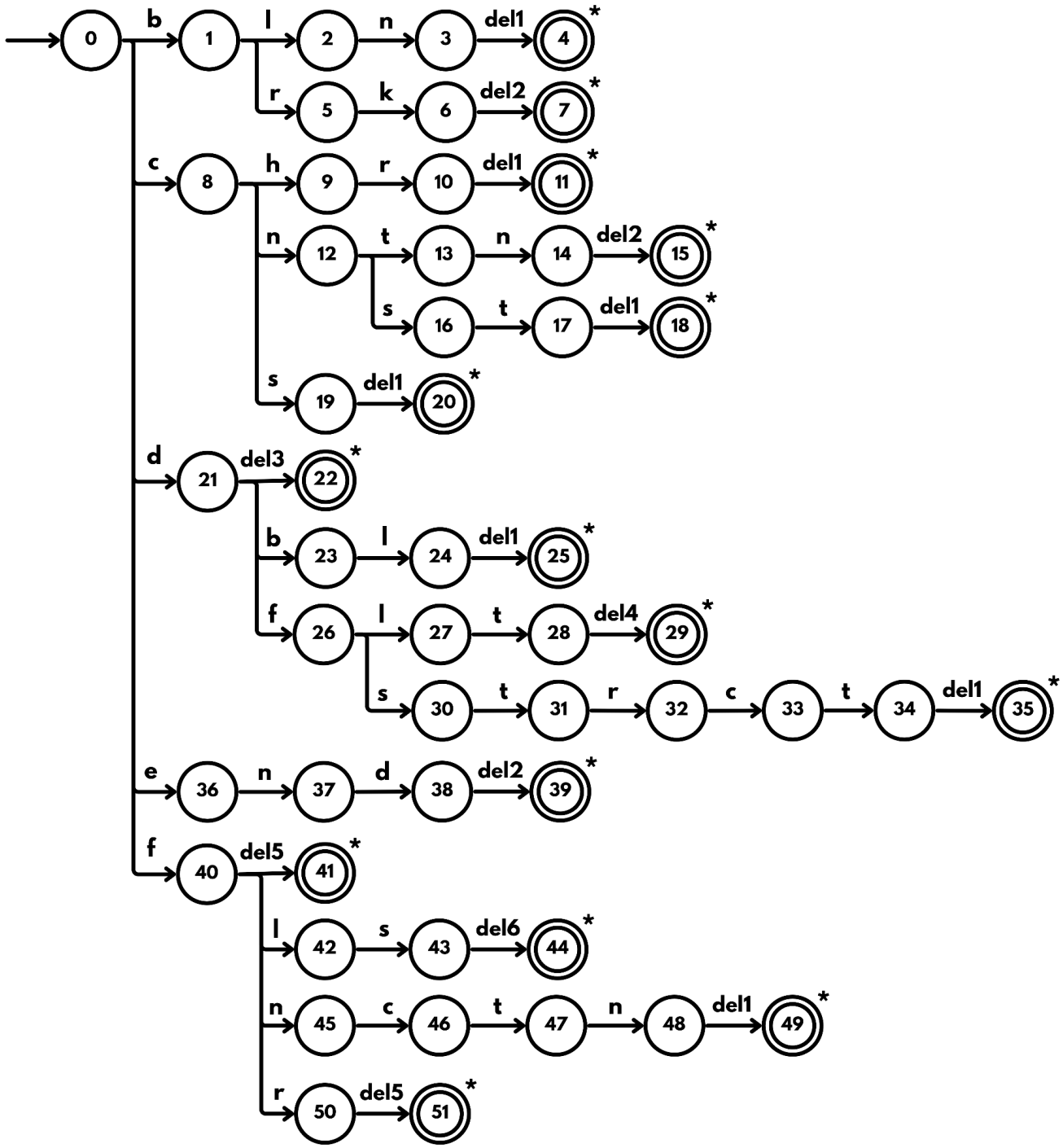
NAME	DEFINITION
zero	{0}
digit	{‘1’, ‘2’, ‘3’, ‘4’, ‘5’, ‘6’, ‘7’, ‘8’, ‘9’}
digitzero	{‘1’, ‘2’, ‘3’, ‘4’, ‘5’, ‘6’, ‘7’, ‘8’, ‘9’, ‘0’}

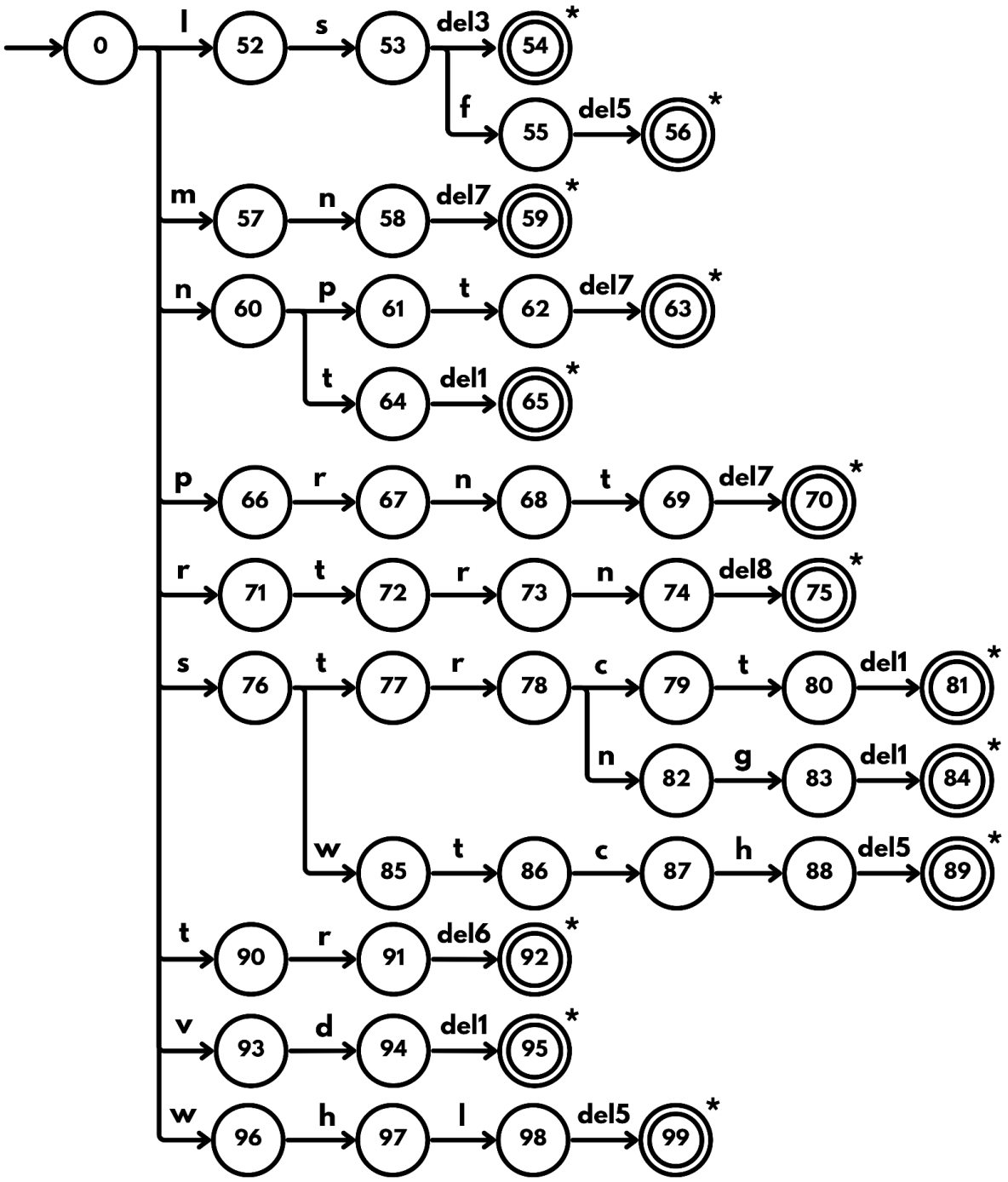
lowalpha	{a, b, c, d, e, f, g, h, i, j, k, l, m, n, o, p, q, r, s, t, u, v, w, x, y, z}
upalpha	{A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, P, Q, R, S, T, U, V, W, X, Y, Z}
alpha	{lowalpha, upalpha}
curse	{whitespace, digitzero, alpha, !, “, #, $, %, &, (,), *, +, ‘, -, ., /, :, <, +, >, ?, @, [, \,], ^, _, {, |, }, ~}
newline	{\n}
tab	{\t}
del1	{ whitespace }
del2	{ ‘;’ }
del3	{whitespace, ‘{‘ }
del4	{ ‘:’ }
del5	{ whitespace, ‘(’ }
del6	{ whitespace, ‘;’, ‘,’, '=', '>', '<', '!' }
del7	{ ‘(‘ }
del8	{ whitespace, ‘;’ }
del9	{ whitespace, alpha, ‘(’, ‘;’, ‘)’, ‘,’ }
del10	{ whitespace, ‘;’, ‘)’ }
del11	{ whitespace, ‘\n’ }
del12	{ alpha, digitzero, ‘]’, ‘~’ }
del13	{ whitespace, ‘;’, ‘)’, ‘[‘ }
del14	{ whitespace, ‘\n’, ‘ “ ‘, ‘ ’ ’, ‘{’, alpha, digitzero }
del15	{ whitespace, ‘\n’, ‘;’, ‘}’, ‘,’ }
del16	{ alpha, digitzero, ‘)’, ‘ “ ‘, ‘!’, ‘(’}
del17	{ whitespace ‘}’, ‘;’, ‘,‘, '+', '-', '*', '/', '%', '=' , '>', '<', '!', '&', '|' }
del18	{ whitespace, ‘;’, ‘{‘, ‘)’, '&', '|', '+', '-', '*', '/', '%' }
del19	{ whitespace, ‘;’, ‘ , ‘, ‘}’, ‘)’ , '=', '>', '<', '!' }

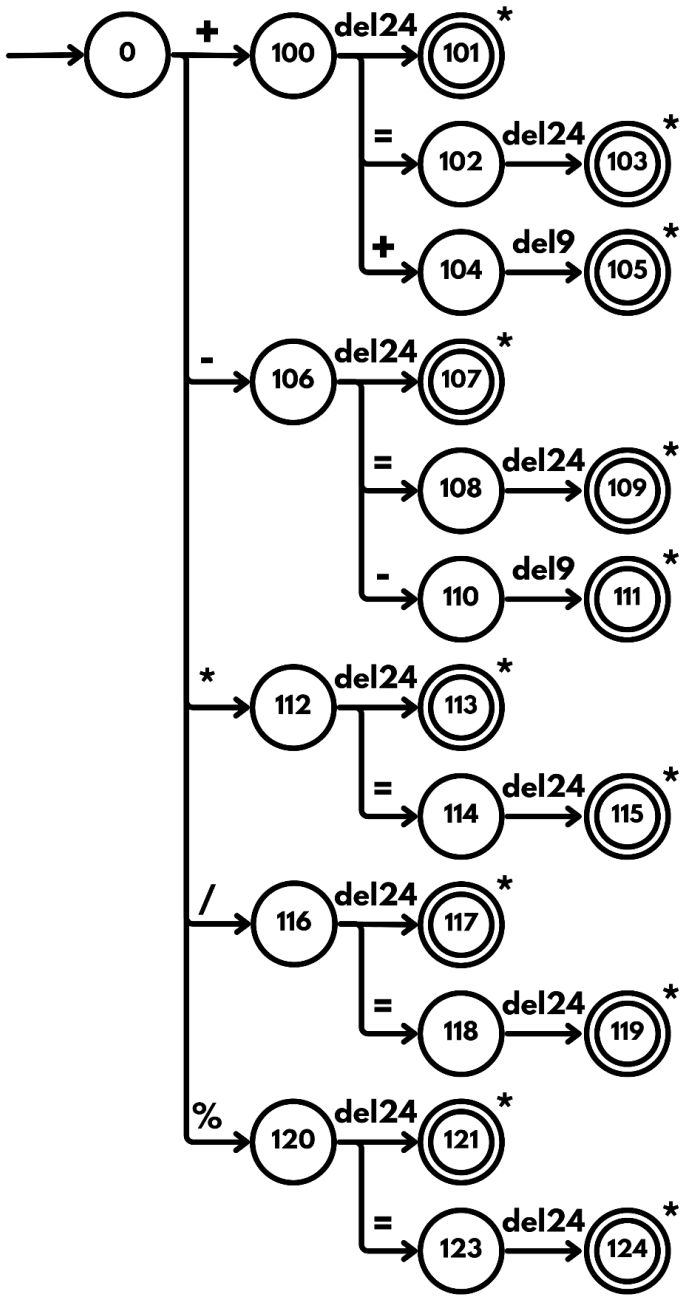
del20	{ whitespace, alpha, digizero, ‘“‘, ‘ ’ ’, ‘{‘ }
del21	{ digit }
del22	{ whitespace, ‘ , ‘, ‘;’, ‘(‘, ‘)’, ‘{‘, ‘[‘, ‘]’, ‘:’, '+', '-', '*', '/', '%' , '>', '<' }
del23	{ whitespace, ‘;’, ‘ , ’, ‘}’, ‘]’, ‘)’ ‘:’, '+', '-', '*', '/', '%', '=' , '>', '<', '!', '&', '|' }
del24	{ whitespace, digitzero, alpha, ‘~’, ‘(‘ }
del25	{ whitespace, digitzero, alpha, ‘~’, ‘ “ ‘, “ ‘ “, }
del26	{ whitespace, ';', ',', '}', ')', '=', '>', '<', '!', ':' }
del27	{ whitespace, ‘ “ ‘ }
del28	{ whitespace, digitzero, alpha, ‘~’, ‘ “ ‘, “ ‘ “, ‘+’, ‘!’ }

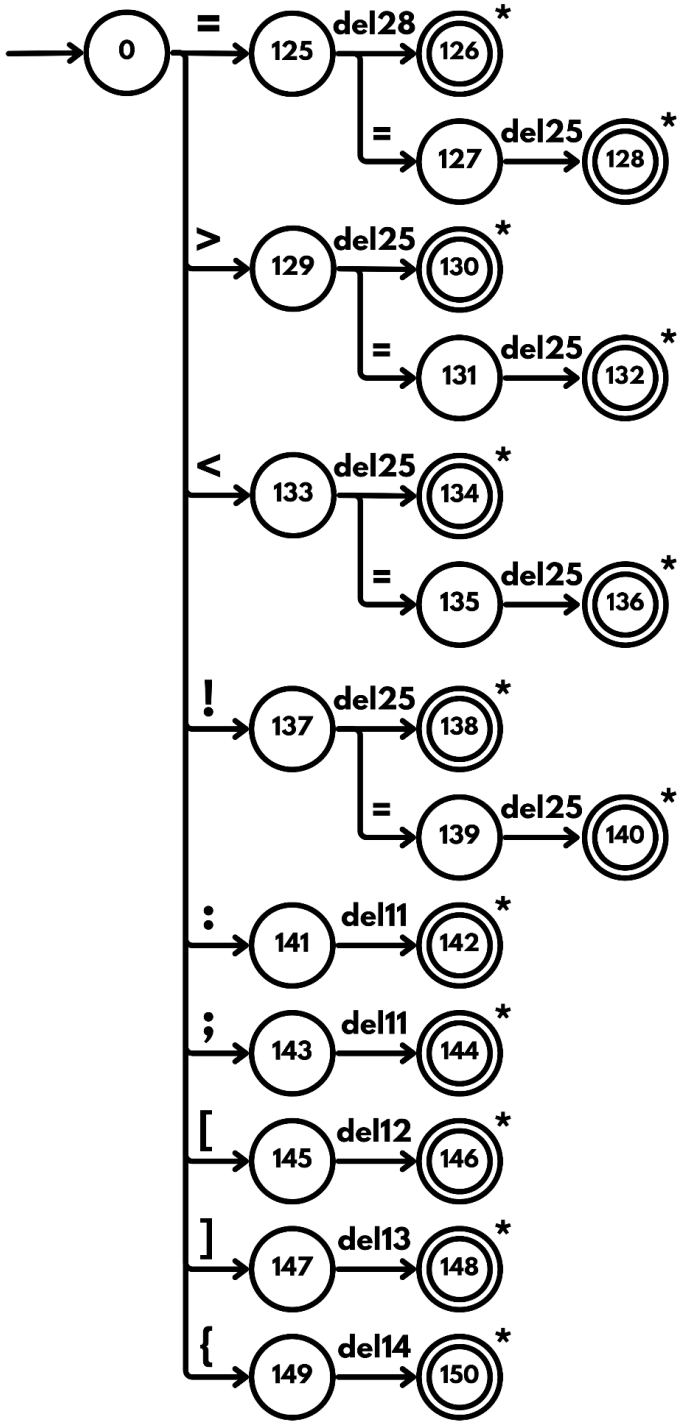
VIII. Transition Diagram

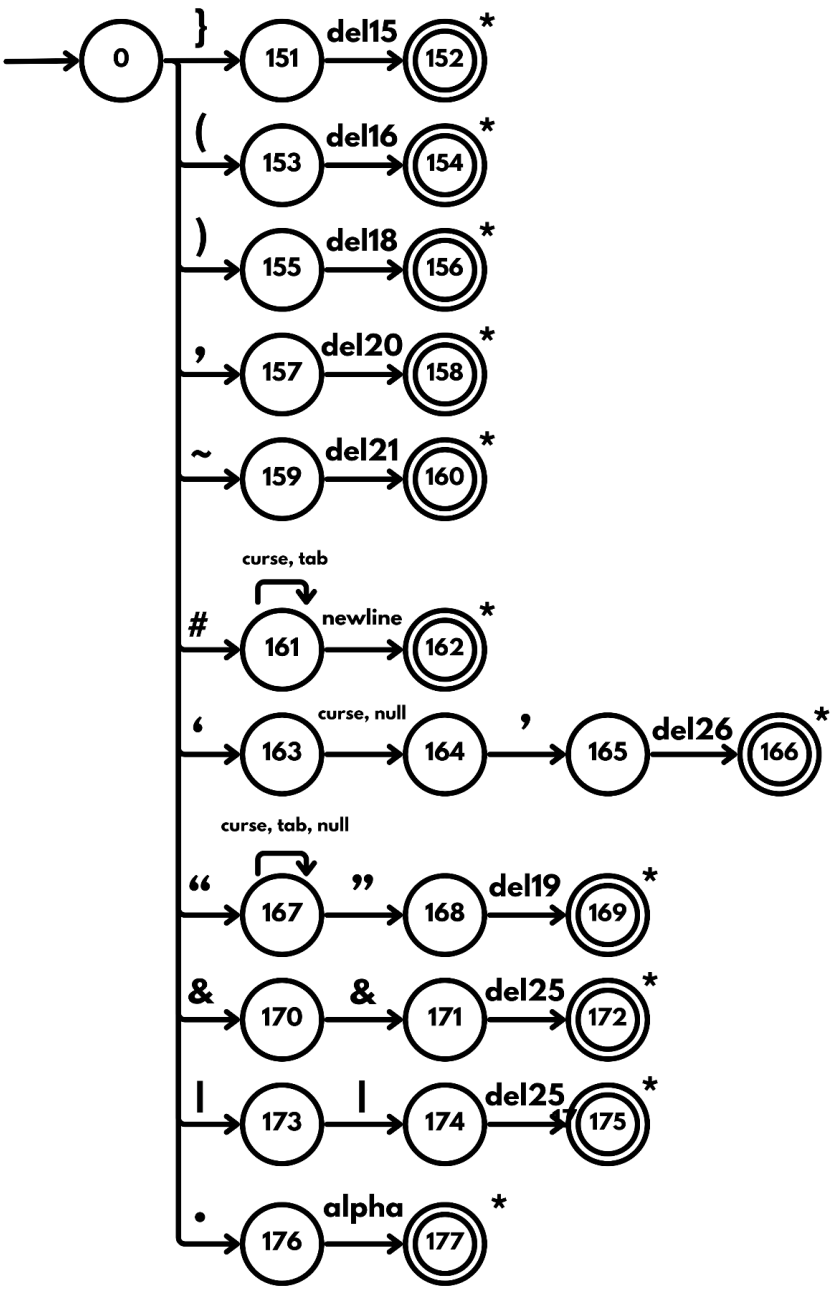
TRANSITION DIAGRAM - RESERVED WORDS



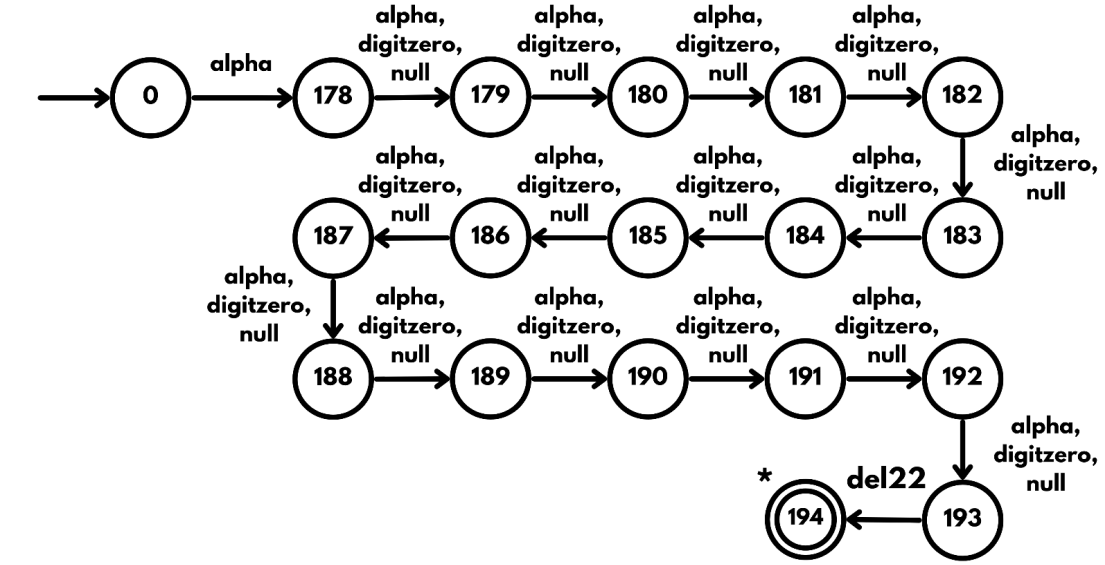

0
l
52
s
53
del3
54
*
f
55
del5
56
*
m
57
n
58
del7
59
*
n
60
p
61
t
62
del7
63
*
t
64
del1
65
*
p
66
r
67
n
68
t
69
del7
70
*
r
71
t
72
r
73
n
74
del8
75
*
s
76
t
77
r
78
c
79
t
80
del1
81
*
n
82
g
83
del1
84
*
w
85
t
86
c
87
h
88
del5
89
*
t
90
r
91
del6
92
*
v
93
d
94
del1
95
*
w
96
h
97
l
98
del5
99
*

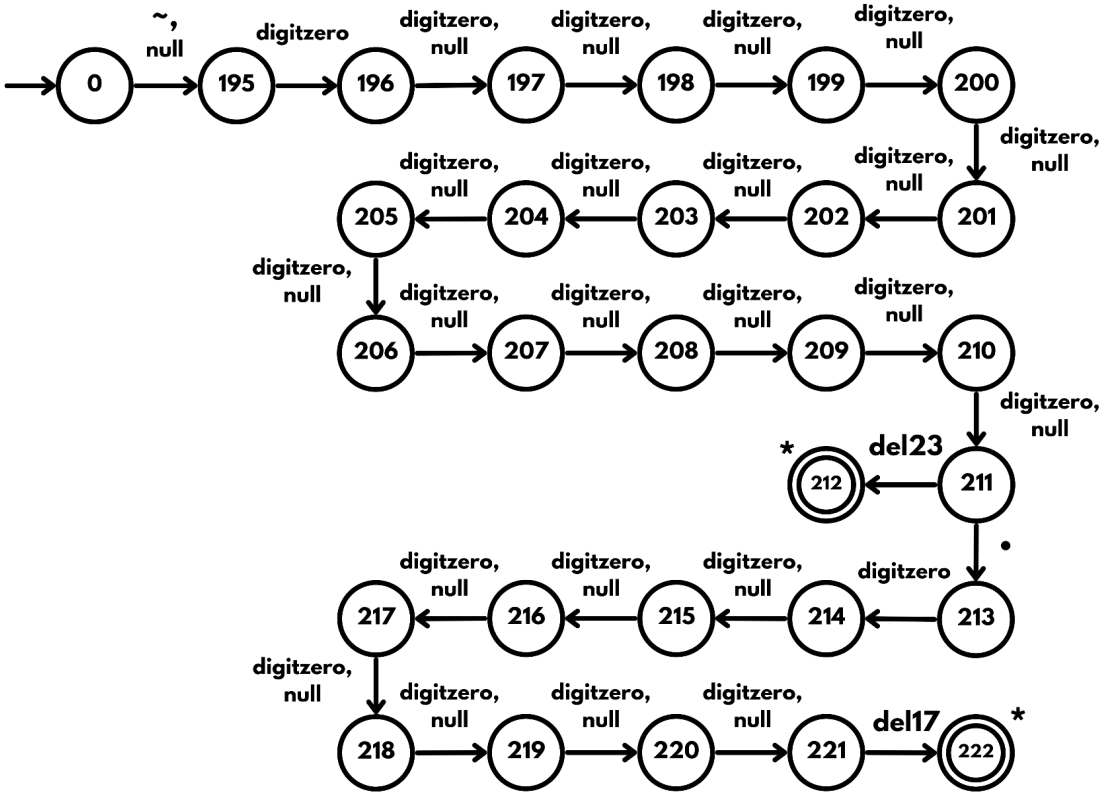

0
+
100
del24
101
*
=
102
del24
103
*
+
104
del9
105
*
-
106
del24
107
*
=
108
del24
109
*
-
110
del9
111
*
*
112
del24
113
*
=
114
del24
115
*
/
116
del24
117
*
=
118
del24
119
*
%
120
del24
121
*
=
123
del24
124
*


0
=
125
del28
126
*
=
127
del25
128
*
>
129
del25
130
*
=
131
del25
132
*
<
133
del25
134
*
=
135
del25
136
*
!
137
del25
138
*
=
139
del25
140
*
:
141
del11
142
*
;
143
del11
144
*
[
145
del12
146
*
]
147
del13
148
*
{
149
del14
150
*



TRANSITION DIAGRAM - ALPHA AND DIGIT


0
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
alpha
alpha, digitzero, null
del22
*


0
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
~, null
digitzero
digitzero, null
del23
del17
*
•

IX. Context Free Grammar

	Production		**Production Set**
1.	<program>	->	<global_declaration> <function_definition> mn() { <declaration> <statement> end; }
2.	<global_declaration>	->	<strct_declaration><global_declaration>
3.	<global_declaration>	->	<var_dec> ; <global_declaration>
4.	<global_declaration>	->	<cnst_dec> ; <global_declaration>
5.	<global_declaration>	->	dfstrct id id <strct_init_next> ; <global_declaration>
6.	<global_declaration>	->	λ
7.	<declaration>	->	<var_dec> ; <declaration>
8.	<declaration>	->	<cnst_dec> ; <declaration>
9.	<declaration>	->	dfstrct id id <strct_init_next> ; <declaration>
10.	<declaration>	->	λ
11.	<var_dec>	->	nt id <nt_value><nt_value_next>
12.	<var_dec>	->	dbl id <dbl_value><dbl_value_next>
13.	<var_dec>	->	bln id <bln_value><bln_value_next>
14.	<var_dec>	->	chr id <chr_value><chr_value_next>
15.	<var dec>	->	strng id <str_value><str_value_next>
16.	<nt_value>	->	= <arithmetic_nt>
17.	<nt_value>	->	<nt_array_dec>
18.	<nt_value>	->	λ
19.	<nt_value_next>	->	, id <nt_value><nt_value_next>
20.	<nt_value_next>	->	λ
21.	<arithmetic_nt>	->	<nt_math_val><arithmetic_nt_next>
22.	<nt_math_val>	->	<ntliterals>
23.	<nt_math_val>	->	(<arithmetic_nt>)

24.	<nt_math_val>	->	<unary_entities>
25.	<ntliterals>	->	intlit
26.	<ntliterals>	->	~ntlit
27.	<unary_entities>	->	<unary_inc/dec> id
28.	<unary_entities>	->	id <unary_next>
29.	<unary_next>	->	<unary_inc/dec>
30.	<unary_next>	->	<id_entity_next>
31.	<unary_next>	->	λ
32.	<unary_inc/dec>	->	++
33.	<unary_inc/dec>	->	--
34.	<arithmetic_nt_next>	->	<arithmetic_operator> <nt_math_val> <arithmetic_nt_next>
35.	<arithmetic_nt_next>	->	λ
36.	<arithmetic_operator>	->	+
37.	<arithmetic_operator>	->	-
38.	<arithmetic_operator>	->	*
39.	<arithmetic_operator>	->	/
40.	<arithmetic_operator>	->	%
41.	<entities>	->	id <id_entity_next>
42.	<id_entity_next>	->	<id_next>
43.	<id_entity_next>	->	<function_call>
44.	<id_entity_next>	->	λ
45.	<id_build>	->	id<id_next>
46.	<id_next>	->	<array_access>
47.	<id_next>	->	<strct_elem_access>
48.	<id_next>	->	λ

49.	<array_access>	->	[<index>] <index_next>
50.	<index>	->	<arithmetic_nt>
51.	<index_next>	->	[<index>]
52.	<index_next>	->	λ
53.	<strct_elem_access>	->	.id
54.	<function_call>	->	(<arguments>)
55.	<arguments>	->	<args_value> <args_next>
56.	<arguments>	->	λ
57.	<args_value>	->	<expression>
58.	<args_next>	->	, <args_value> <args_next>
59.	<args_next>	->	λ
60.	<expression>	->	<rel_arith_logic_expr>
61.	<rel_arith_logic_expr>	->	<operands><operand_next>
62.	<operands>	->	(<rel_arith_logic_expr>)
63.	<operands>	->	<math_value>
64.	<operands>	->	chrlit
65.	<operands>	->	!<operands>
66.	<operands>	->	strnglit
67.	<operands>	->	blnlit
68.	<operand_next>	->	<arithmetic_operator><rel_arith_logic_expr>
69.	<operand_next>	->	<relational_operator><rel_arith_logic_expr>
70.	<operand_next>	->	<logical_operator><rel_arith_logic_expr>
71.	<operand_next>	->	λ
72.	<logical_operator>	->	&&
73.	<logical_operator>	->	||
74.	<relational_operator>	->	==

75.	<relational_operator>	->	!=
76.	<relational_operator>	->	<=
77.	<relational_operator>	->	>=
78.	<relational_operator>	->	<
79.	<relational_operator>	->	>
80.	<arithmetic>	->	<math_value><arithmetic_next>
81.	<math_value>	->	<dblliterals>
82.	<math_value>	->	<ntliterals>
83.	<math_value>	->	<unary_entities>
84.	<dblliterals>	->	dbllit
85.	<dblliterals>	->	~dbllit
86.	<arithmetic_next>	->	<arithmetic_operator><arithmetic>
87.	<arithmetic_next>	->	λ
88.	<nt_array_dec>	->	[<array_size>] <nt_array_initializer>
89.	<array_size>	->	ntlit
90.	<array_size>	->	id
91.	<nt_array_initializer>	->	= { <nt_array_elem> }
92.	<nt_array_initializer>	->	[<array_size>] <nt_2d_init>
93.	<nt_array_initializer>	->	λ
94.	<nt_array_elem>	->	<nt_array_elem_val><nt_array_init_next>
95.	<nt_array_elem_val>	->	<ntliterals>
96.	<nt_array_init_next>	->	, <nt_array_elem_val><nt_array_init_next>
97.	<nt_array_init_next>	->	λ
98.	<nt_2d_init>	->	= { <nt_array2d_elem> }
99.	<nt_2d_init>	->	λ
100.	<nt_array2d_elem>	->	{ <nt_array_elem> } <nt_array_elem_next>

101.	<nt_array_elem_next>	->	, <nt_array2d_elem>
102.	<nt_array_elem_next>	->	λ
103.	<dbl_value>	->	= <arithmetic>
104.	<dbl_value>	->	<dbl_array_dec>
105.	<dbl_value>	->	λ
106.	<dbl_value_next>	->	, id <dbl_value> <dbl_value_next>
107.	<dbl_value_next>	->	λ
108.	<dbl_array_dec>	->	[<array_size>] <dbl_array_initializer>
109.	<dbl_array_initializer>	->	= { <dbl_array_elem> }
110.	<dbl_array_initializer>	->	[<array_size>] <dbl_2d_init>
111.	<dbl_array_initializer>	->	λ
112.	<dbl_array_elem>	->	<dbl_array_elem_val> <dbl_array_init_next>
113.	<dbl_array_elem_val>	->	<dblliterals>
114.	<dbl_array_init_next>	->	, <dbl_array_elem_val> <dbl_array_init_next>
115.	<dbl_array_init_next>	->	λ
116.	<dbl_2d_init>	->	= { <dbl_array2d_elem> }
117.	<dbl_2d_init>	->	λ
118.	<dbl_array2d_elem>	->	{ <dbl_array_elem> } <dbl_array_elem_next>
119.	<dbl_array_elem_next>	->	, <dbl_array2d_elem>
120.	<dbl_array_elem_next>	->	λ
121.	<bln_value>	->	= <equals_bln_val>
122.	<bln_value>	->	<bln_array_dec>
123.	<bln_value>	->	λ
124.	<equals_bln_val>	->	<rel_arith_logic_expr>
125.	<bln_value_next>	->	, id <bln_value><bln_value_next>
126.	<bln_value_next>	->	λ

127.	<bln_array_dec>	->	[<array_size>] <bln_array_initializer>
128.	<bln_array_initializer>	->	= { <bln_array_elem> }
129.	<bln_array_initializer>	->	[<array_size>] <bln_2d_init>
130.	<bln_array_initializer>	->	λ
131.	<bln_array_elem>	->	<bln_array_elem_val> <bln_array_init_next>
132.	<bln_array_elem_val>	->	blnlit
133.	<bln_array_init_next>	->	, <bln_array_elem_val> <bln_array_init_next>
134.	<bln_array_init_next>	->	λ
135.	<bln_2d_init>	->	= { <bln_array2d_elem> }
136.	<bln_2d_init>	->	λ
137.	<bln_array2d_elem>	->	{ <bln_array_elem> } <bln_array_elem_next>
138.	<bln_array_elem_next>	->	, <bln_array2d_elem>
139.	<bln_array_elem_next>	->	λ
140.	<chr_value>	->	= <equals_chr_val>
141.	<chr_value>	->	<chr_array_dec>
142.	<chr_value>	->	λ
143.	<equals_chr_val>	->	<entities>
144.	<equals_chr_val>	->	chrlit
145.	<chr_value_next>	->	, id <chr_value> <chr_value_next>
146.	<chr_value_next>	->	λ
147.	<chr_array_dec>	->	[<array_size>] <chr_array_initializer>
148.	<chr_array_initializer>	->	= { <chr_array_elem> }
149.	<chr_array_initializer>	->	[<array_size>] <chr_2d_init>
150.	<chr_array_initializer>	->	λ
151.	<chr_array_elem>	->	<chr_array_elem_val> <chr_array_init_next>
152.	<chr_array_elem_val>	->	chrlit

153.	<chr_array_init_next>	->	, <chr_array_elem_val> <chr_array_init_next>
154.	<chr_array_init_next>	->	λ
155.	<chr_2d_init>	->	={ <chr_array2d_elem> }
156.	<chr_2d_init>	->	λ
157.	<chr_array2d_elem>	->	{ <chr_array_elem> } <chr_array_elem_next>
158.	<chr_array_elem_next>	->	, <chr_array2d_elem>
159.	<chr_array_elem_next>	->	λ
160.	<str_value>	->	= <equals_str_val>
161.	<str_value>	->	<str_array_dec>
162.	<str_value>	->	λ
163.	<equals_str_val>	->	<entities>
164.	<equals_str_val>	->	strnglit
165.	<str_value_next>	->	, id <str_value> <str_value_next>
166.	<str_value_next>	->	λ
167.	<str_array_dec>	->	[<array_size>] <str_array_initializer>
168.	<str_array_initializer>	->	= { <str_array_elem> }
169.	<str_array_initializer>	->	[<array_size>] <str_2d_init>
170.	<str_array_initializer>	->	λ
171.	<str_array_elem>	->	<str_array_elem_val> <str_array_init_next>
172.	<str_array_elem_val>	->	strnglit
173.	<str_array_init_next>	->	, <str_array_elem_val> <str_array_init_next>
174.	<str_array_init_next>	->	λ
175.	<str_2d_init>	->	= { <str_array2d_elem> }
176.	<str_2d_init>	->	λ
177.	<str_array2d_elem>	->	{ <str_array_elem> } <str_array_elem_next>
178.	<str_array_elem_next>	->	, <str_array2d_elem>

179.	<str_array_elem_next>	->	λ
180.	<cnst_dec>	->	cnst <cnst_dec_data_type>
181.	<cnst_dec_data_type>	->	nt id <nt_cnst_initializer>
182.	<cnst_dec_data_type>	->	dbl id <dbl_cnst_initializer>
183.	<cnst_dec_data_type>	->	bln id <bln_cnst_initializer>
184.	<cnst_dec_data_type>	->	chr id <chr_cnst_initializer>
185.	<cnst_dec_data_type>	->	strng id <str_cnst_initializer>
186.	<nt_cnst_initializer>	->	= <nt_cnst_init_val>
187.	<nt_cnst_initializer>	->	<cnst_nt_array_dec>
188.	<nt_cnst_init_val>	->	<ntliterals> <nt_cnst_next>
189.	<nt_cnst_next>	->	, id <nt_cnst_initializer>
190.	<nt_cnst_next>	->	λ
191.	<cnst_nt_array_dec>	->	[<cnst_array_size>] <cnst_nt_array_initializer>
192.	<cnst_array_size>	->	<ntliterals>
193.	<cnst_nt_array_initializer>	->	= { <nt_array_elem> }
194.	<cnst_nt_array_initializer>	->	[<array_size>] <nt_2d_init>
195.	<dbl_cnst_initializer>	->	= <dbl_cnst_init_val>
196.	<dbl_cnst_initializer>	->	<cnst_dbl_array_dec>
197.	<dbl_cnst_init_val>	->	<dblliterals><dbl_cnst_next>
198.	<dbl_cnst_next>	->	, id <dbl_cnst_initializer>
199.	<dbl_cnst_next>	->	λ
200.	<cnst_dbl_array_dec>	->	[<cnst_array_size>] <cnst_dbl_array_initializer>
201.	<cnst_dbl_array_initializer>	->	= { <dbl_array_elem> }
202.	<cnst_dbl_array_initializer>	->	[<array_size>] <dbl_2d_init>
203.	<bln_cnst_initializer>	->	= <bln_cnst_init_val>
204.	<bln_cnst_initializer>	->	<cnst_bln_array_dec>

205.	<bln_cnst_init_val>	->	blnlit <bln_cnst_next>
206.	<bln_cnst_next>	->	, id <bln_cnst_initializer>
207.	<bln_cnst_next>	->	λ
208.	<cnst_bln_array_dec>	->	[<cnst_array_size>] <cnst_bln_array_initializer>
209.	<cnst_bln_array_initializer>	->	={ <bln_array_elem> }
210.	<cnst_bln_array_initializer>	->	[<array_size>] <bln_2d_init>
211.	<chr_cnst_initializer>	->	= <chr_cnst_init_val>
212.	<chr_cnst_initializer>	->	<cnst_chr_array_dec>
213.	<chr_cnst_init_val>	->	chrlit <chr_cnst_next>
214.	<chr_cnst_next>	->	, id <chr_cnst_initializer>
215.	<chr_cnst_next>	->	λ
216.	<cnst_chr_array_dec>	->	[<cnst_array_size>] <cnst_chr_array_initializer>
217.	<cnst_chr_array_initializer>	->	= { <chr_array_elem>}
218.	<cnst_chr_array_initializer>	->	[<array_size>] <chr_2d_init>
219.	<str_cnst_initializer>	->	= <str_cnst_init_val>
220.	<str_cnst_initializer>	->	<cnst_str_array_dec>
221.	<str_cnst_init_val>	->	strnglit <str_cnst_next>
222.	<str_cnst_next>	->	, id <str_cnst_initializer>
223.	<str_cnst_next>	->	λ
224.	<cnst_str_array_dec>	->	[<cnst_array_size>] <cnst_str_array_initializer>
225.	<cnst_str_array_initializer>	->	= { <str_array_elem> }
226.	<cnst_str_array_initializer>	->	[<array_size>] <str_2d_init>
227.	<strct_init_next>	->	, id <strct_init_next>
228.	<strct_init_next>	->	λ
229.	<strct_declaration>	->	strct id { <strct_body> };
230.	<strct_body>	->	<strct_var_dec> ; <var_dec_next>

231.	<strct_var_dec>	->	nt id
232.	<strct_var_dec>	->	dbl id
233.	<strct_var_dec>	->	bln id
234.	<strct_var_dec>	->	chr id
235.	<strct_var_dec>	->	strng id
236.	<var_dec_next>	->	<strct_body>
237.	<var_dec_next>	->	λ
238.	<function_definition>	->	fnctn <func_type> id (<parameter>) { <func_var_dec> <statement> } <function_definition>
239.	<function_definition>	->	λ
240.	<func_type>	->	<data_type>
241.	<func_type>	->	vd
242.	<data_type>	->	nt
243.	<data_type>	->	dbl
244.	<data_type>	->	bln
245.	<data_type>	->	chr
246.	<data_type>	->	strng
247.	<parameter>	->	<data_type> id <parameter_next>
248.	<parameter>	->	λ
249.	<parameter_next>	->	, <parameter>
250.	<parameter_next>	->	λ
251.	<func_var_dec>	->	<var_dec> ; <func_var_dec>
252.	<func_var_dec>	->	λ
253.	<statement>	->	<id_build_statements> <statement>
254.	<statement>	->	<unary_pre_statement> <statement>
255.	<statement>	->	<output_statement> <statement>

256.	<statement>	->	<conditional_statement> <statement>
257.	<statement>	->	<fr_statement> <statement>
258.	<statement>	->	<d-whl_statement> <statement>
259.	<statement>	->	<whl_statement> <statement>
260.	<statement>	->	<rtrn_statement> <statement>
261.	<statement>	->	<control_statements> <statement>
262.	<statement>	->	λ
263.	<id_build_statements>	->	id <id_build_statement_next>
264.	<id_build_statement_next>	->	<id_next> <assignment_statement>
265.	<id_build_statement_next>	->	<assignment_statement>
266.	<id_build_statement_next>	->	<function_call>;
267.	<id_build_statement_next>	->	<unary_inc/dec>;
268.	<assignment_statement>	->	<assign_operator> <assignment_value>
269.	<assignment_statement>	->	= <assignment_value_input>
270.	<assign_operator>	->	+=
271.	<assign_operator>	->	-=
272.	<assign_operator>	->	*=
273.	<assign_operator>	->	/=
274.	<assignment_value>	->	<expression>;
275.	<assignment_value_input>	->	<assignment_value>
276.	<assignment_value_input>	->	<input_statement_value>
277.	<input_statement_value>	->	npt (strnglit) ;
278.	<unary_pre_statement>	->	<unary_inc/dec> id ;
279.	<output_statement>	->	prnt (<output_value> <output_next>) ;
280.	<output_value>	->	<expression>
281.	<output_next>	->	, <output_value> <output_next>

282.	<output_next>	->	λ
283.	<conditional_statement>	->	f (<condition>) { <statement> <lsf_statement> <ls_statement> } <lsf_statement> <ls_statement>
284.	<conditional_statement>	->	swtch (id) { <swtch_body> }
285.	<condition>	->	<rel_arith_logic_expr>
286.	<lsf_statement>	->	lsf (<condition>) { <statement> } <lsf_statement>
287.	<lsf_statement>	->	λ
288.	<ls_statement>	->	ls { <statement> }
289.	<ls_statement>	->	λ
290.	<swtch_body>	->	<cs><dflt>
291.	<cs>	->	cs <swtch_value> : <statement> <cs_next>
292.	<swtch_value>	->	<ntliterals>
293.	<swtch_value>	->	chrlit
294.	<cs_next>	->	<cs> <cs_next>
295.	<cs_next>	->	λ
296.	<dflt>	->	dflt : <statement>
297.	<dflt>	->	λ
298.	<fr_statement>	->	fr (<initialization> ; <fr_condi> ; <inc/dec>){ <statement> }
299.	<initialization>	->	<id_build> = <arithmetic_nt>
300.	<fr_condi>	->	<condition>
301.	<inc/dec>	->	<fr_unary>
302.	<inc/dec>	->	id<inc/dec_next>
303.	<inc/dec_next>	->	<assign_operator> <inc/dec_value>
304.	<inc/dec_next>	->	<unary_inc/dec>
305.	<fr_unary>	->	<unary_inc/dec> id

306.	<inc/dec_value>	->	<id_build>
307.	<inc/dec_value>	->	<ntliterals>
308.	<d-whl_statement>	->	d { <statement> } whl (<condition>) ;
309.	<whl_statement>	->	whl (<condition>) { <statement> }
310.	<rtrn_statement>	->	rtrn <rtrn_value> ;
311.	<rtrn_value>	->	<expression>
312.	<rrn_value>	->	λ
313.	<control_statements>	->	brk ;
314.	<control_statements>	->	cntn ;

X. First Set

1.	<program>	->	{ nt, dbl, bln, chr, strng, cnst, dfstrct, strct, fnctn, mn }
2.	<global_declaration>	->	{ nt, dbl, bln, chr, strng, cnst, dfstrct, strct, λ }
3.	<declaration>	->	{ nt, dbl, bln, chr, strng, cnst, dfstrct, λ }
4.	<var_dec>	->	{ nt, dbl, bln, chr, strng }
5.	<nt_value>	->	{ =, [, λ }
6.	<nt_value_next>	->	{ **,**, λ }
7.	<arithmetic_nt>	->	{ ntlit, ~ntlit, (, ++, --, id }
8.	<nt_math_val>	->	{ ntlit, ~ntlit, (, ++, --, id }
9.	<ntliterals>	->	{ ntlit, ~ntlit }
10.	<unary_entities>	->	{ ++, --, id }
11.	<unary_next>	->	{ ++, --, [, (, **.** , λ }
12.	<unary_inc/dec>	->	{ ++, -- }
13.	<arithmetic_nt_next>	->	{ +, -, *, /, %, λ }
14.	<arithmetic_operator>	->	{ +, -, *, /, % }
15.	<entities>	->	{ id }
16.	<id_entity_next>	->	{ [, **.**, (, λ }
17.	<id_build>	->	{ id }
18.	<id_next>	->	{ [, **.**, λ}
19.	<array_access>	->	{ [}
20.	<index>	->	{ ntlit, ~ntlit, (, ++, --, id }
21.	<index_next>	->	{ [, λ }
22.	<strct_elem_access>	->	{ **.** }
23.	<function_call>	->	{ (}
24.	<arguments>	->	{ (, dbllit, ~dbllit, ntlit, ~ntlit, ++, --, id, chrlit, !, strnglit, blnlit, λ }

25.	<args_value>	->	{ (, dbllit, ~dbllit, ntlit, ~ntlit, ++, --, id, chrlit, !, strnglit, blnlit }
26.	<args_next>	->	{ ,, λ }
27.	<expression>	->	{ (, dbllit, ~dbllit, ntlit, ~ntlit, ++, --, id, chrlit, !, strnglit, blnlit }
28.	<rel_arith_logic_expr>	->	{ (, dbllit, ~dbllit, ntlit, ~ntlit, ++, --, id, chrlit, !, strnglit, blnlit }
29.	<operands>	->	{ (, dbllit, ~dbllit, ntlit, ~ntlit, ++, --, id, chrlit, !, strnglit, blnlit }
30.	<operand_next>	->	{ +, -, *, /, %, ==, !=, <=, >=, <, >, &&, ||, λ }
31.	<logical_operator>	->	{ &&, || }
32.	<relational_operator>	->	{ ==, !=, <=, >=, <, > }
33.	<arithmetic>	->	{ dbllit, ~dbllit, ntlit, ~ntlit, ++, --, id }
34.	<math_value>	->	{ dbllit, ~dbllit, ntlit, ~ntlit, ++, --, id }
35.	<dblliterals>	->	{ dbllit, ~dbllit }
36.	<arithmetic_next>	->	{ +, -, *, /, %, λ }
37.	<nt_array_dec>	->	{ [}
38.	<array_size>	->	{ ntlit, id }
39.	<nt_array_initializer>	->	{ =, [, λ}
40.	<nt_array_elem>	->	{ ntlit, ~ntlit }
41.	<nt_array_elem_val>	->	{ ntlit, ~ntlit }
42.	<nt_array_init_next>	->	{ ,, λ }
43.	<nt_2d_init>	->	{ =, λ }
44.	<nt_array2d_elem>	->	{ { }
45.	<nt_array_elem_next>	->	{ ,, λ}
46.	<dbl_value>	->	{ =, [, λ }
47.	<dbl_value_next>	->	{ ,, λ }
48.	<dbl_array_dec>	->	{ [}

49.	<dbl_array_initializer>	->	{ =, [, λ}
50.	<dbl_array_elem>	->	{ dbllit, ~dbllit }
51.	<dbl_array_elem_val>	->	{ dbllit, ~dbllit }
52.	<dbl_array_init_next>	->	{ **,**, λ}
53.	<dbl_2d_init>	->	{ =, λ}
54.	<dbl_array2d_elem>	->	{ { }
55.	<dbl_array_elem_next>	->	{ **,**, λ }
56.	<bln_value>	->	{ =, [, λ}
57.	<equals_bln_val>	->	{ id, dbllit, ~dbllit, ntlit, ~ntlit, ++, --, id, chrlit, !, strnglit, blnlit }
58.	<bln_value_next>	->	{ **,**, λ }
59.	<bln_array_dec>	->	{ [}
60.	<bln_array_initializer>	->	{ =, [, λ}
61.	<bln_array_elem>	->	{ blnlit }
62.	<bln_array_elem_val>	->	{ blnlit }
63.	<bln_array_init_next>	->	{ **,**, λ }
64.	<bln_2d_init>	->	{ =, λ }
65.	<bln_array2d_elem>	->	{ { }
66.	<bln_array_elem_next>	->	{ **,**, λ }
67.	<chr_value>	->	{ =, [, λ }
68.	<equals_chr_val>	->	{ id, chrlit }
69.	<chr_value_next>	->	{ **,**, λ }
70.	<chr_array_dec>	->	{ [}
71.	<chr_array_initializer>	->	{ =, [, λ }
72.	<chr_array_elem>	->	{ chrlit }
73.	<chr_array_elem_val>	->	{ chrlit }

74.	<chr_array_init_next>	->	{ **,**, λ }
75.	<chr_2d_init>	->	{ =, λ }
76.	<chr_array2d_elem>	->	{ { }
77.	<chr_array_elem_next>	->	{ **,**, λ }
78.	<str_value>	->	{ =, [, λ }
79.	<equals_str_val>	->	{ id, strnglit }
80.	<str_value_next>	->	{ **,**, λ }
81.	<str_array_dec>	->	{ [}
82.	<str_array_initializer>	->	{ =, [, λ }
83.	<str_array_elem>	->	{ strnglit }
84.	<str_array_elem_val>	->	{ strnglit }
85.	<str_array_init_next>	->	{ **,**, λ }
86.	<str_2d_init>	->	{ =, λ }
87.	<str_array2d_elem>	->	{ { }
88.	<str_array_elem_next>	->	{ **,**, λ }
89.	<cnst_dec>	->	{ cnst }
90.	<cnst_dec_data_type>	->	{ nt, dbl, chr, bln, strng }
91.	<nt_cnst_initializer>	->	{ =, [}
92.	<nt_cnst_init_val>	->	{ ntlit, ~ntlit }
93.	<nt_cnst_next>	->	{ **,**, λ }
94.	<cnst_nt_array_dec>	->	{ [}
95.	<cnst_array_size>	->	{ ntlit, ~ntlit }
96.	<cnst_nt_array_initializer>	->	{ =, [}
97.	<dbl_cnst_initializer>	->	{ =, [}
98.	<dbl_cnst_init_val>	->	{ dbllit, ~dbllit }
99.	<dbl_cnst_next>	->	{ **,**, λ }

100.	<cnst_dbl_array_dec>	->	{ [}
101.	<cnst_dbl_array_initializer>	->	{ =, [}
102.	<bln_cnst_initializer>	->	{ =, [}
103.	<bln_cnst_init_val>	->	{ blnlit }
104.	<bln_cnst_next>	->	{ **,**, λ }
105.	<cnst_bln_array_dec>	->	{ [}
106.	<cnst_bln_array_initializer>	->	{ =, [}
107.	<chr_cnst_initializer>	->	{ =, [}
108.	<chr_cnst_init_val>	->	{ chrlit }
109.	<chr_cnst_next>	->	{ **,**, λ }
110.	<cnst_chr_array_dec>	->	{ [}
111.	<cnst_chr_array_initializer>	->	{ =, [}
112.	<str_cnst_initializer>	->	{ =, [}
113.	<str_cnst_init_val>	->	{ strnglit }
114.	<str_cnst_next>	->	{ **,**, λ }
115.	<cnst_str_array_dec>	->	{ [}
116.	<cnst_str_array_initializer>	->	{ =, [}
117.	<strct_init_next>	->	{ **,**, λ }
118.	<strct_declaration>	->	{ strct }
119.	<strct_body>	->	{ nt, dbl, bln, chr, strng }
120.	<strct_var_dec>	->	{ nt, dbl, bln, chr, strng }
121.	<var_dec_next>	->	{ ;, λ }
122.	<function_definition>	->	{ fnctn, λ }
123.	<func_type>	->	{ nt, dbl, bln, chr, strng, vd }
124.	<data_type>	->	{ nt, dbl, bln, chr, strng }
125.	<parameter>	->	{ nt, dbl, bln, chr, strng, λ }

126.	<parameter_next>	->	{ **,**, λ }
127.	<func_var_dec>	->	{ nt, dbl, bln, chr, strng, λ }
128.	<statement>	->	{ id, ++, --, prnt, f, swtch, fr, d, whl, rtrn, brk, cntn, λ }
129.	<id_build_statements>	->	{ id }
130.	<id_build_statement_next>	->	{ [, **.**, +=, -=, *=, /=, =, (, ++, -- }
131.	<assignment_statement>	->	{ +=, -=, *=, /=, = }
132.	<assign_operator>	->	{ +=, -=, *=, /= }
133.	<assignment_value>	->	{ (, dbllit, ~dbllit, ntlit, ~ntlit, ++, --, id, chrlit, !, strnglit, blnlit }
134.	<assignment_value_input>	->	{ (, dbllit, ~dbllit, ntlit, ~ntlit, ++, --, id, chrlit, !, strnglit, blnlit, npt }
135.	<input_statement_value>	->	{ npt }
136.	<unary_pre_statement>	->	{ ++, -- }
137.	<output_statement>	->	{ prnt }
138.	<output_value>	->	{ (, dbllit, ~dbllit, ntlit, ~ntlit, ++, --, id, chrlit, !, strnglit, blnlit}
139.	<output_next>	->	{ **,**, λ }
140.	<conditional_statement>	->	{ f, swtch }
141.	<condition>	->	{ (, dbllit, ~dbllit, ntlit, ~ntlit, ++, --, id, chrlit, !, strnglit, blnlit }
142.	<lsf_statement>	->	{ lsf, λ }
143.	<ls_statement>	->	{ ls, λ }
144.	<swtch_body>	->	{ cs }
145.	<cs>	->	{ cs }
146.	<swtch_value>	->	{ ntlit, ~ntlit, chrlit }
147.	<cs_next>	->	{ cs, λ }
148.	<dflt>	->	{ dflt, λ }
149.	<fr_statement>	->	{ fr }

150.	<initialization>	->	{ id }
151.	<fr_condi>	->	{ (, dbllit, ~dbllit, ntlit, ~ntlit, ++, --, id, chrlit, !, strnglit, blnlit, npt }
152.	<inc/dec>	->	{ ++, --, id }
153.	<inc/dec_next>	->	{ +=, -=, *=, /=, ++, -- }
154.	<fr_unary>	->	{ ++, -- }
155.	<inc/dec_value>	->	{ id, ntlit, ~ntlit }
156.	<d-whl_statement>	->	{ d }
157.	<whl_statement>	->	{ whl }
158.	<rtrn_statement>	->	{ rtrn }
159.	<rtrn_value>	->	{ (, dbllit, ~dbllit, ntlit, ~ntlit, ++, --, id, chrlit, !, strnglit, blnlit, λ }
160.	<control_statements>	->	{ brk, cntn }

XI. Follow Set

1.	<program>	->	{ $ }
2.	<global_declaration>	->	{ fnctn, mn }
3.	<declaration>	->	{ id, ++, --, prnt, f, swtch, fr, d, whl, rtrn, brk, cntn, end }
4.	<var_dec>	->	{ ; }
5.	<nt_value>	->	{ ,, ; }
6.	<nt_value_next>	->	{ ; }
7.	<arithmetic_nt>	->	{ ,,),], ; }
8.	<nt_math_val>	->	{ +, -, *, /, %, ,,),], ; }
9.	<ntliterals>	->	{ ,, +, -, *, /, %,), ==, !=, <=, >=, <, >, &&, ||, ;,] }
10.	<unary_entities>	->	{ +, -, *, /, %, ,, ;,), &&, ||, ==, !=, <=, >=, <, > }
11.	<unary_next>	->	{ +, -, *, /, %, ,, ;,), &&, ||, ==, !=, <=, >=, <, >,] }
12.	<unary_inc/dec>	->	{ id, +, -, *, /, %, ,, ;,), &&, ||, ==, !=, <=, >=, <, >,) }
13.	<arithmetic_nt_next>	->	{ ,,),], ; }
14.	<arithmetic_operator>	->	{ ntlit, ~ntlit, (, ++, --, id, (, dbllit, ~dbllit }
15.	<entities>	->	{ ,, ; }
16.	<id_entity_next>	->	{ +, -, *, /, %, ,,),], ;, &&, ||, ==, !=, <=, >=, <, > }
17.	<id_build>	->	{ =,) }
18.	<id_next>	->	{ +, -, *, /, %, ,,),], ;, ,, =, +=, -=, *=, /=, &&, ||, ==, !=, <=, >=, <, > }
19.	<array_access>	->	{ +, -, *, /, %, ,,),], ;, ,, =, +=, -=, *=, /=, &&, ||, ==, !=, <=, >=, <, > }
20.	<index>	->	{] }
21.	<index_next>	->	{ +, -, *, /, %, ,,),], ;, ,, =, +=, -=, *=, /=, &&, ||, ==, !=, <=, >=, <, > }
22.	<strct_elem_access>	->	{ +, -, *, /, %, ,,),], ;, ,, =, +=, -=, *=, /=, &&, || }
23.	<function_call>	->	{ +, -, *, /, %, ,,),], ;, &&, ||, !, ,, id, ++, --, prnt, f, swtch, fr, d, whl, rtrn, brk, cntn, end }

24.	<arguments>	->	{) }
25.	<args_value>	->	{ , }
26.	<args_next>	->	{) }
27.	<expression>	->	{ ,, ;,) }
28.	<rel_arith_logic_expr>	->	{ ,, ;,) }
29.	<operands>	->	{ &&, ||, ==, !=, <=, >=, <, >, +, -, *, /, %, ,, ;,) }
30.	<operand_next>	->	{ ,, ;,) }
31.	<logical_operator>	->	{ (, dbllit, ~dbllit, ntlit, ~ntlit, ++, --, id, chrlit, !, strnglit, blnlit }
32.	<relational_operator>	->	{ (, dbllit, ~dbllit, ntlit, ~ntlit, ++, --, id, chrlit, !, strnglit, blnlit }
33.	<arithmetic>	->	{ ,, ; }
34.	<math_value>	->	{ +, -, *, /, %, ,, ;,), &&, ||, ==, !=, <=, >=, <, > }
35.	<dbllliterals>	->	{ }, +, -, *, /, %, ,, ;,), &&, ||, ==, !=, <=, >=, <, > }
36.	<arithmetic_next>	->	{ ,, ; }
37.	<nt_array_dec>	->	{ ,, ; }
38.	<array_size>	->	{] }
39.	<nt_array_initializer>	->	{ ,, ; }
40.	<nt_array_elem>	->	{ } }
41.	<nt_array_elem_val>	->	{ ,, } }
42.	<nt_array_init_next>	->	{ } }
43.	<nt_2d_init>	->	{ ,, ; }
44.	<nt_array2d_elem>	->	{ } }
45.	<nt_array_elem_next>	->	{ } }
46.	<dbl_value>	->	{ ,, ; }
47.	<dbl_value_next>	->	{ ; }
48.	<dbl_array_dec>	->	{ ,, ; }

49.	<dbl_array_initializer>	->	{ „ ; }
50.	<dbl_array_elem>	->	{ } }
51.	<dbl_array_elem_val>	->	{ „ } }
52.	<dbl_array_init_next>	->	{ } }
53.	<dbl_2d_init>	->	{ „ ; }
54.	<dbl_array2d_elem>	->	{ } }
55.	<dbl_array_elem_next>	->	{ } }
56.	<bln_value>	->	{ „ ; }
57.	<equals_bln_val>	->	{ „ ; }
58.	<bln_value_next>	->	{ ; }
59.	<bln_array_dec>	->	{ „ ; }
60.	<bln_array_initializer>	->	{ „ ; }
61.	<bln_array_elem>	->	{ } }
62.	<bln_array_elem_val>	->	{ „ } }
63.	<bln_array_init_next>	->	{ } }
64.	<bln_2d_init>	->	{ „ ; }
65.	<bln_array2d_elem>	->	{ } }
66.	<bln_array_elem_next>	->	{ } }
67.	<chr_value>	->	{ „ ; }
68.	<equals_chr_val>	->	{ „ ; }
69.	<chr_value_next>	->	{ ; }
70.	<chr_array_dec>	->	{ „ ; }
71.	<chr_array_initializer>	->	{ „ ; }
72.	<chr_array_elem>	->	{ } }
73.	<chr_array_elem_val>	->	{ „ } }
74.	<chr_array_init_next>	->	{ } }

75.	<chr_2d_init>	->	{ „ ; }
76.	<chr_array2d_elem>	->	{ } }
77.	<chr_array_elem_next>	->	{ } }
78.	<str_value>	->	{ „ ; }
79.	<equals_str_val>	->	{ „ ; }
80.	<str_value_next>	->	{ ; }
81.	<str_array_dec>	->	{ „ ; }
82.	<str_array_initializer>	->	{ „ ; }
83.	<str_array_elem>	->	{ } }
84.	<str_array_elem_val>	->	{ „ } }
85.	<str_array_init_next>	->	{ } }
86.	<str_2d_init>	->	{ „ ; }
87.	<str_array2d_elem>	->	{ } }
88.	<str_array_elem_next>	->	{ } }
89.	<cnst_dec>	->	{ ; }
90.	<cnst_dec_data_type>	->	{ ; }
91.	<nt_cnst_initializer>	->	{ ; }
92.	<nt_cnst_init_val>	->	{ ; }
93.	<nt_cnst_next>	->	{ ; }
94.	<cnst_nt_array_dec>	->	{ ; }
95.	<cnst_array_size>	->	{] }
96.	<cnst_nt_array_initializer>	->	{ ; }
97.	<dbl_cnst_initializer>	->	{ ; }
98.	<dbl_cnst_init_val>	->	{ ; }
99.	<dbl_cnst_next>	->	{ ; }
100.	<cnst_dbl_array_dec>	->	{ ; }

101.	<cnst_dbl_array_initializer>	->	{ ; }
102.	<bln_cnst_initializer>	->	{ ; }
103.	<bln_cnst_init_val>	->	{ ; }
104.	<bln_cnst_next>	->	{ ; }
105.	<cnst_bln_array_dec>	->	{ ; }
106.	<cnst_bln_array_initializer>	->	{ ; }
107.	<chr_cnst_initializer>	->	{ ; }
108.	<chr_cnst_init_val>	->	{ ; }
109.	<chr_cnst_next>	->	{ ; }
110.	<cnst_chr_array_dec>	->	{ ; }
111.	<cnst_chr_array_initializer>	->	{ ; }
112.	<str_cnst_initializer>	->	{ ; }
113.	<str_cnst_init_val>	->	{ ; }
114.	<str_cnst_next>	->	{ ; }
115.	<cnst_str_array_dec>	->	{ ; }
116.	<cnst_str_array_initializer>	->	{ ; }
117.	<strct_init_next>	->	{ ; }
118.	<strct_declaration>	->	{ nt, dbl, bln, chr, strng, cnst, strct, fnctn, mn }
119.	<strct_body>	->	{ } }
120.	<strct_var_dec>	->	{ ; }
121.	<var_dec_next>	->	{ } }
122.	<function_definition>	->	{ mn }
123.	<func_type>	->	{ id }
124.	<data_type>	->	{ id }
125.	<parameter>	->	{) }
126.	<parameter_next>	->	{) }

127.	<func_var_dec>	->	{ id, ++, --, prnt, f, swtch, fr, d, whl, rtrn, brk, cntn, } }
128.	<statement>	->	{ end, }, lsf, ls, cs, dflt }
129.	<id_build_statements>	->	{ id, ++, --, prnt, f, swtch, fr, d, whl, rtrn, brk, cntn, end }
130.	<id_build_statement_next>	->	{ id, ++, --, prnt, f, swtch, fr, d, whl, rtrn, brk, cntn, end }
131.	<assignment_statement>	->	{ id, ++, --, prnt, f, swtch, fr, d, whl, rtrn, brk, cntn, end }
132.	<assign_operator>	->	{ (, dbllit, ~dbllit, ntlit, ~ntlit, ++, --, id, chrlit, !, strnglit, blnlit }
133.	<assignment_value>	->	{ id, ++, --, prnt, f, swtch, fr, d, whl, rtrn, brk, cntn, end }
134.	<assignment_value_input>	->	{ id, ++, --, prnt, f, swtch, fr, d, whl, rtrn, brk, cntn, end }
135.	<input_statement_value>	->	{ id, ++, --, prnt, f, swtch, fr, d, whl, rtrn, brk, cntn, end }
136.	<unary_pre_statement>	->	{ id, ++, --, prnt, f, swtch, fr, d, whl, rtrn, brk, cntn, end }
137.	<output_statement>	->	{ id, ++, --, prnt, f, swtch, fr, d, whl, rtrn, brk, cntn, end }
138.	<output_value>	->	{ ,,) }
139.	<output_next>	->	{) }
140.	<conditional_statement>	->	{ id, ++, --, prnt, f, swtch, fr, d, whl, rtrn, brk, cntn, end }
141.	<condition>	->	{), ; }
142.	<lsf_statement>	->	{ ls, }, id, ++, --, prnt, f, swtch, fr, d, whl, rtrn, brk, cntn, end }
143.	<ls_statement>	->	{ }, id, ++, --, prnt, f, swtch, fr, d, whl, rtrn, brk, cntn, end }
144.	<swtch_body>	->	{ } }
145.	<cs>	->	{ dflt, } }
146.	<swtch_value>	->	{ : }
147.	<cs_next>	->	{ dflt, } }
148.	<dflt>	->	{ } }
149.	<fr_statement>	->	{ id, ++, --, prnt, f, swtch, fr, d, whl, rtrn, brk, cntn, end }
150.	<initialization>	->	{ ; }
151.	<fr_condi>	->	{ ; }

152.	<inc/dec>	->	{) }
153.	<inc/dec_next>	->	{) }
154.	<fr_unary>	->	{) }
155.	<inc/dec_value>	->	{) }
156.	<d-whl_statement>	->	{ id, ++, --, prnt, f, swtch, fr, d, whl, rtrn, brk, cntn, end }
157.	<whl_statement>	->	{ id, ++, --, prnt, f, swtch, fr, d, whl, rtrn, brk, cntn, end }
158.	<rtrn_statement>	->	{ id, ++, --, prnt, f, swtch, fr, d, whl, rtrn, brk, cntn, end }
159.	<rtrn_value>	->	{ ; }
160.	<control_statements>	->	{ id, ++, --, prnt, f, swtch, fr, d, whl, rtrn, brk, cntn, end }

XII. Predict Set

1.	first (<global_declaration> <function_definition> mn(){ <declaration> <statement> end; })	first(<global_declaration>)	{ nt, dbl, bln, chr, strng, cnst, dfstrct, strct, fnctn, mn }
2.	first(<strct_declaration><global_declaration>)	first(<strct_declaration>)	{ strct }
3.	first(<var_dec> ; <global_declaration>)	first(<var_dec>)	{ nt, dbl, bln, chr, strng }
4.	first(<cnst_dec> ; <global_declaration>)	first(<cnst_dec>)	{ cnst }
5.	first(dftstrct id id <strct_init_next> ; <global_declaration>)	first(dfstrct)	{ dfstrct }
6.	first(λ) u follow(<global_declaration>)	follow(<global_declaration>)	{ fnctn, mn }
7.	first(<var_dec> ; <declaration>)	first(<var_dec>)	{ nt, dbl, bln, chr, strng }
8.	first(<cnst_dec> ; <declaration>)	first(<cnst_dec>)	{ cnst }
9.	first(dfstrct id id <strct_init_next> ; <declaration>)	first(dfstrct)	{ dfstrct }
10.	first(λ) u follow(<declaration>)	follow(<declaration>)	{ id, ++, --, prnt, f, swtch, fr, d, whl, rtrn, brk, cntn, end }
11.	first(nt id <nt_value> <nt_value_next>)	first(nt)	{ nt }
12.	first(dbl id <dbl_value> <dbl_value_next>)	first(dbl)	{ dbl }
13.	first(bln id <bln_value> <bln_value_next>)	first(bln)	{ bln }
14.	first(chr id <chr_value> <chr_value_next>)	first(chr)	{ chr }
15.	first(strng id <str_value> <str_value_next>)	first(strng)	{ strng }

16.	first(= <arithmetic_nt>)	first(=)	{ = }
17.	first(<nt_array_dec>)	first(<nt_array_dec>)	{ [}
18.	first(λ) u follow(<nt_value>)	follow(<nt_value>)	{ **,**, ; }
19.	first(, id <nt_value> <nt_value_next>)	first(,)	{ **,** }
20.	first(λ) u follow(<nt_value_next>)	follow(<nt_value_next>)	{ ; }
21.	first(<nt_math_val> <arithmetic_nt_next>)	first(<nt_math_val>)	{ ntlit, ~ntlit, (, ++, --, id }
22.	first(<ntliterals>)	first(<ntliterals>)	{ ntlit, ~ntlit }
23.	first((<arithmetic_nt>))	first(( )	{ (}
24.	first(<unary_entities>)	first(<unary_entities>)	{ ++, --, id }
25.	first(intlit)	first(intlit)	{ intlit }
26.	first(~intlit)	first(~intlit)	{ ~intlit }
27.	first(<unary_inc/dec> id)	first(<unary_inc/dec>)	{++, --}
28.	first(id <unary_next>)	first(id)	{ id }
29.	first(<unary_inc/dec>)	first(<unary_inc/dec>)	{ ++, -- }
30.	first(<id_entity_next>)	first(<id_entity_next>)	{ [, **.**, (, λ }
31.	first(λ) u follow(<unary_next>)	follow(<unary_next>)	{ +, -, *, /, %, **,**, ;,), &&, ||, !, ==, !=, <=, >=, <, > }
32.	first(++)	first(++)	{++}
33.	first(--)	first(--)	{--}
34.	first(<arithmetic_operator> <nt_math_val> <arithmetic_nt_next>)	first(<arithmetic_operator>)	{+, -, *, /, %}
35.	first(λ) u follow(<arithmetic_nt_next>)	follow(<arithmetic_nt_next>)	{ **,**,),], ; }
36.	first(+)	first(+)	{+}
37.	first(-)	first(-)	{-}

38.	first(*)	first(*)	{*}
39.	first(/)	first(/)	{/}
40.	first(%)	first(%)	{%}
41.	first(id <id_entity_next>)	first(id)	{id}
42.	first(<id_next>)	first(<id_next>)	{ [, ., λ}
43.	first(<function_call>)	first(<function_call>)	{ (}
44.	first(λ) u follow(<id_entity_next>)	follow(<id_entity_next>)	{ +, -, *, /, %, ,,),], ;, &&, || }
45.	first(id<id_next>)	first(id)	{id}
46.	first(<array_access>)	first(<array_access>)	{ [}
47.	first(<strct_elem_access>)	first(<strct_elem_access>)	{ . }
48.	first(λ) u follow(<id_next>)	follow(<id_next>)	{ +, -, *, /, %, ,,),], ;, ,, =, +=, -=, *=, /=, &&, || }
49.	first([<index>] <index_next>)	first([)	{ [}
50.	first(<arithmetic_nt>)	first(<arithmetic_nt>)	{ ntlit, ~ntlit, (, ++, --, id }
51.	first([<index>])	first([)	{ [}
52.	first(λ) u follow(<index_next>)	follow(<index_next>)	{ +, -, *, /, %, ,,),], ;, ,, =, +=, -=, *=, /=, &&, || }
53.	first(.id)	first(.)	{ . }
54.	first((<arguments>);)	first(()	{ (}
55.	first(<args_value> <args_next>)	first(<args_value>)	{ (, dbllit, ~dbllit, ntlit, ~ntlit, ++, --, id, chrlit, !, strnglit, blnlit }
56.	first(λ) u follow(<arguments>)	follow(<arguments>)	{) }
57.	first(<expression>)	first(<expression>)	{ (, dbllit, ~dbllit, ntlit, ~ntlit, ++, --, id, chrlit, !, strnglit, blnlit }
58.	first(, <args_value> <args_next>)	first(,)	{ , }
59.	first(λ) u follow(<args_next>)	follow(<args_next>)	{) }

60.	first(<rel_arith_logic_expr>)	first(<rel_arith_logic_expr>)	{ (, dbllit, ~dbllit, ntlit, ~ntlit, ++, --, id, chrlit, !, strnglit, blnlit }
61.	first(<operands> <operand_next>)	first(<operands>)	{ (, dbllit, ~dbllit, ntlit, ~ntlit, ++, --, id, chrlit, !, strnglit, blnlit }
62.	first((<rel_arith_logic_expr>))	first(()	{ (}
63.	first(<math_value>)	first(<math_value>)	{ dbllit, ~dbllit, ntlit, ~ntlit, ++, --, id }
64.	first(chrlit)	first(chrlit)	{ chrlit }
65.	first(!<operands>)	first(!)	{ ! }
66.	first(strnglit)	first(strnglit)	{ strnglit }
67.	first(blnlit)	first(blnlit)	{ blnlit }
68.	first(<arithmetic_operator> <rel_arith_logic_expr>)	follow(<arithmetic_operator>)	{ +, -, *, /, % }
69.	first(<relational_operator> <rel_arith_logic_expr>)	first(<relational_operator>)	{ ==, !=, <=, >=, <, > }
70.	first(<logical_operator> <rel_arith_logic_expr>)	first(<logical_operator>)	{ &&, || }
71.	first(λ) u follow(<operand_next>)	follow(<operand_next>)	{ **,**, ;,) }
72.	first(&&)	first(&&)	{ && }
73.	first(||)	first(||)	{ || }
74.	first(==)	first(==)	{ == }
75.	first(!=)	first(!=)	{ !=}
76.	first(<=)	first(<=)	{ <= }
77.	first(>=)	first(>=)	{ >= }
78.	first(<)	first(<)	{ < }
79.	first(>)	first(>)	{ > }
80.	first(<math_value> <arithmetic_next>)	first(<math_value>)	{ dbllit, ~dbllit, ntlit, ~ntlit, ++, --, id }

81.	first(<dblliterals>)	first(<dblliterals>)	{ dbllit, ~dbllit }
82.	first(<ntliterals>)	first(<ntliterals>)	{ ntlit, ~ntlit }
83.	first(<unary_entities>)	first(<unary_entities>)	{ ++, --, id }
84.	first(dbllit)	first(dbllit)	{ dbllit }
85.	first(~dbllit)	first(~dbllit)	{ ~dbllit }
86.	first(<arithmetic_operator> <arithmetic>)	first(<arithmetic_operator>)	{+, -, *, /, %}
87.	first(λ) u follow(<arithmetic_next>)	follow(<arithmetic_next>)	{ **,**, ; }
88.	first([<array_size>] <nt_array_initializer>)	first([)	{ [}
89.	first(ntlit)	first(ntlit)	{ ntlit }
90.	first(id)	first(id)	{ id}
91.	first(={<nt_array_elem>})	first(=)	{ = }
92.	first([<array_size>] <nt_2d_init>)	first([)	{ [}
93.	first(λ) u follow(<nt_array_initializer>)	follow(<nt_array_initializer>)	{ **,**, ; }
94.	first(<nt_array_elem_val> <nt_array_init_next>)	first(<nt_array_elem_val>)	{ ntlit, ~ntlit }
95.	first(<ntliterals>)	first(<ntliterals>)	{ ntlit, ~ntlit }
96.	first(, <nt_array_elem_val> <nt_array_init_next>)	first(,)	{ , }
97.	first(λ) u follow(<nt_array_init_next>)	follow(<nt_array_init_next>)	{ } }
98.	first(={<nt_array2d_elem>})	first(=)	{ = }
99.	first(λ) u follow(<nt_2d_init>)	follow(<nt_2d_init>)	{ **,**, ; }
100.	first({<nt_array_elem>} <nt_array_elem_next>)	first({)	{ { }
101.	first(, <nt_array2d_elem>)	first(,)	{,}

102.	first(λ) u follow(<nt_array_elem_next>)	follow(<nt_array_elem_next>)	{ } }
103.	first(=<arithmetic>)	firs(=)	{ = }
104.	first(<dbl_array_dec>)	first(<dbl_array_dec>)	{ [}
105.	first(λ) u follow(<dbl_value>)	follow(<dbl_value>)	{ **,**, ; }
106.	first(, id<dbl_value> <dbl_value_next>)	first(,)	{ **,** }
107.	first(λ) u follow(<dbl_value_next>)	follow(<dbl_value_next>)	{ ; }
108.	first([<array_size>] <dbl_array_initializer>)	first([)	{ [}
109.	first(= {dbl_array_elem})	first(=)	{ = }
110.	firs([<array_size>] <dbl_2d_init>)	first([)	{ [}
111.	first(λ) u follow(<dbl_array_initializer>)	follow(<dbl_array_initializer>)	{ **,**, ; }
112.	first(<dbl_array_elem_val> <dbl_array_init_next>)	first(<dbl_array_elem_val>)	{ dbllit, ~dbllit }
113.	first(<dbllliterals>)	first(<dbllliterals>)	{ dbllit, ~dbllit }
114.	first(, <dbl_array_elem_val> <dbl_array_init_next>)	first(,)	{ **,** }
115.	first(λ) u follow(<dbl_array_init_next>)	follow(<dbl_array_init_next>)	{ } }
116.	first(={<dbl_array2d_elem>})	first(=)	{ = }
117.	first(λ) u follow(<dbl_2d_init>)	follow(<dbl_2d_init>)	{ **,**, ; }
118.	first({<dbl_array_elem> } <dbl_array_elem_next>)	first({)	{ { }
119.	first(, <dbl_array2d_elem>)	first(,)	{ **,** }
120.	first(λ) u follow(<dbl_array_elem_next>)	follow(<dbl_array_elem_next >)	{ } }
121.	first(=<equals_bln_val>)	first(=)	{ = }

122.	first(<bln_array_dec>)	first(<bln_array_dec>)	{ [}
123.	first(λ) u follow(<bln_value>)	follow(<bln_value>)	{ **,**, ; }
124.	first(<rel_arith_logic_expr>;)	first(<rel_arith_logic_expr>)	{ dbllit, ~dbllit, ntlit, ~ntlit, ++, --, id, chrlit, !, strnglit, blnlit }
125.	first(, id <bln_value> <bln_value_next>)	first(,)	{ **,** }
126.	first(λ) U follow(<bln_value_next>)	follow(<bln_value_next>)	{ ; }
127.	first([<array_size>] <bln_array_initializer>)	first([)	{ [}
128.	first(= {<bln_array_elem>})	first(=)	{ = }
129.	first([<array_size>] <bln_2d_init>)	first([)	{ [}
130.	first(λ) U follow(<bln_array_initializer>)	follow(<bln_array_initializer>)	{ **,**, ; }
131.	first(<bln_array_elem_val> <bln_array_init_next>)	first(<bln_array_elem_val>)	{ blnlit }
132.	first(blnlit)	first(blnlit)	{ blnlit }
133.	first(, <bln_array_elem_val> <bln_array_init_next>)	first(,)	{ **,** }
134.	first(λ) U follow(<bln_array_init_next>)	follow(<bln_array_init_next>)	{ } }
135.	first(={<bln_array2d_elem>})	first(=)	{ = }
136.	first(λ) U follow(<bln_2d_init>)	follow(<bln_2d_init>)	{ **,**, ; }
137.	First{ <bln_array_elem> } <bln_array_elem_next>)	first({)	{ { }
138.	first(, <bln_array2d_elem>)	first(,)	{ **,** }
139.	First(λ) U Follow(<bln_array_elem_next>)	Follow(<bln_array_elem_next >)	{ } }
140.	First(= <equals_chr_val>)	First(=)	{ = }
141.	First(<chr_array_dec>)	First(<chr_array_dec>)	{ [}

142.	First(λ) U Follow(<chr_value>)	Follow(<chr_value>)	{ **,**, ; }
143.	First(<entities>)	First(<entities>)	{ id }
144.	First(chrlit)	First(chrlit)	{ chrlit }
145.	First(, id <chr_value> <chr_value_next>)	First(,)	{ **,** }
146.	First(λ) U Follow (<chr_value_next>)	Follow(<chr_value_next>)	{ ; }
147.	First([<array_size>] <chr_array_initializer>)	First([)	{ [}
148.	First(= { <chr_array_elem> })	First(=)	{ = }
149.	First([<array_size>] <chr_2d_init>)	First([)	{ [}
150.	First(λ) U Follow(<chr_array_initializer>)	Follow(<chr_array_initializer>)	{ **,**, ; }
151.	First(<chr_array_elem_val> <chr_array_init_next>)	First(<chr_array_elem_val>)	{ chrlit }
152.	First(chrlit)	First(chrlit)	{ chrlit }
153.	First(, <chr_array_elem_val> <chr_array_init_next>)	First(,)	{ **,** }
154.	First(λ) U Follow(<chr_array_init_next>)	Follow(<chr_array_init_next>)	{ } }
155.	First(={ <chr_array2d_elem> })	First(=)	{ = }
156.	First(λ) U Follow(<chr_2d_init>)	Follow(<chr_2d_init>)	{ **,**, ; }
157.	First({ <chr_array_elem> } <chr_array_elem_next>)	First({)	{ { }
158.	First(, <chr_array2d_elem>)	First(,)	{ **,** }
159.	First(λ) U Follow(<chr_array_elem_next>)	Follow(<chr_array_elem_next >)	{ } }
160.	First(= <equals_str_val>)	First(=)	{ = }
161.	First(<str_array_dec>)	First(<str_array_dec>)	{ [}

162.	First(λ) U Follow(<str_value>)	Follow(<str_value>)	{ **,**, ; }
163.	First(<entities>)	First(<entities>)	{ id }
164.	first(strnglit)	first(strnglit)	{ strnglit }
165.	first(, id <str_value> <str_value_next>)	first(,)	{ , }
166.	first(λ) U follow(<str_value_next>)	follow(<str_value_next>)	{ ; }
167.	first([<array_size>] <str_array_initializer>)	first([)	{ [}
168.	first(= {<str_array_elem>})	first(=)	{ = }
169.	first([<array_size>] <str_2d_init>)	first([)	{ [}
170.	first(λ) U follow(<str_array_initializer>)	follow(<str_array_initializer>)	{ **,**, ; }
171.	first(<str_array_elem_val> <str_array_init_next>)	first(<str_array_elem_val>)	{ strnglit }
172.	first(strnglit)	first(strnglit)	{ strnglit }
173.	first(, <str_array_elem_val> <str_array_init_next>)	first(,)	{ , }
174.	first(λ) U follow(<str_array_init_next>)	follow(<str_array_init_next>)	{ } }
175.	first(={<str_array2d_elem>})	first(=)	{ = }
176.	first(λ) U follow(<str_2d_init>)	follow(<str_2d_init>)	{ **,**, ; }
177.	first({<str_array_elem>} <str_array_elem_next>)	first({)	{ { }
178.	first(, <str_array2d_elem>)	first(,)	{ , }
179.	first(λ) U follow(<str_array_elem_next>)	follow(<str_array_elem_next>)	{ } }
180.	first(cnst <cnst_dec_data_type>;)	first(cnst)	{ cnst }
181.	first(nt id <nt_cnst_initializer>)	first(nt)	{ nt }

182.	first(dbl id <dbl_cnst_initializer>)	first(dbl)	{ dbl }
183.	first(bln id <bln_cnst_initializer>)	first(bln)	{ bln }
184.	first(chr id <chr_cnst_initializer>)	first(chr)	{ chr }
185.	first(strng id <str_cnst_initializer>)	first(strng)	{ strng }
186.	first(=<nt_cnst_init_val>)	first(=)	{ = }
187.	first(<cnst_nt_array_dec>)	first(<cnst_nt_array_dec>)	{ [}
188.	first(<ntliterals> <nt_cnst_next>)	first(<ntliterals>)	{ ntlit, ~ntlit }
189.	first(, id<nt_cnst_initializer>)	first(,)	{ , }
190.	first(λ) U follow(<nt_cnst_next>)	follow(<nt_cnst_next>)	{ ; }
191.	first([<cnst_array_size>] <cnst_nt_array_initializer>)	first([)	{ [}
192.	first(<ntliterals>)	first(<ntliterals>)	{ ntlit, ~ntlit }
193.	first(={<nt_array_elem>})	first(=)	{ = }
194.	first([<array_size>] <nt_2d_init>)	first([)	{ [}
195.	First(= <dbl_cnst_init_val>)	First(=)	{ = }
196.	First(<cnst_dbl_array_dec>)	First(<cnst_dbl_array_dec>)	{ [}
197.	First(<dblliterals> <dbl_cnst_next>)	First(<dblliterals>)	{ dbllit, ~dbllit }
198.	First(, id<dbl_cnst_initializer>)	First(,)	{ , }
199.	First(λ) U Follow(<dbl_cnst_next>)	Follow(<dbl_cnst_next>)	{ ; }
200.	First([<cnst_array_size>] <cnst_dbl_array_initializer>)	First([)	{ [}
201.	First(={<dbl_array_elem>})	First(=)	{ = }

202.	First([<array_size>] <dbl_2d_init>)	First([)	{ [}
203.	First(= <bln_cnst_init_val>)	First(=)	{ = }
204.	First(<cnst_bln_array_dec>)	First(<cnst_bln_array_dec>)	{ [}
205.	First(blnlit<bln_cnst_next>)	First(blnlit)	{ blnlit }
206.	First(, id <bln_cnst_initializer>)	First(,)	{ , }
207.	First(λ) U Follow(<bln_cnst_next>)	Follow(<bln_cnst_next>)	{ ; }
208.	First([<cnst_array_size>] <cnst_bln_array_initializer>)	First([)	{ [}
209.	First(={<bln_array_elem>})	First(=)	{ = }
210.	First([<array_size>] <bln_2d_init>)	First([)=	{ [}
211.	First(= <chr_cnst_init_val>)	First(=)	{ = }
212.	First(<cnst_chr_array_dec>)	First(<cnst_chr_array_dec>)	{ [}
213.	First(chrlit<chr_cnst_next>)	First(chrlit)	{ chrlit }
214.	First(, id<chr_cnst_initializer>)	First(,)	{ , }
215.	First(λ) U Follow(<chr_cnst_next>)	Follow(<chr_cnst_next>)	{ ; }
216.	First([<cnst_array_size>] <cnst_chr_array_initializer>)	First([)	{ [}
217.	First(={<chr_array_elem>})	First(=)	{ = }
218.	First([<array_size>] <chr_2d_init>)	First([)	{ [}
219.	First(= <str_cnst_init_val>)	First(=)	{ = }
220.	First(<cnst_str_array_dec>)	First(<cnst_str_array_dec>)	{ [}
221.	First(strnglit<str_cnst_next>)	First(strnglit)	{ strnglit }
222.	First(, id<str_cnst_initializer>)	First(,)	{ , }
223.	First(λ) U Follow(<str_cnst_next>)	Follow(<str_cnst_next>)	{ ; }

224.	First([<cnst_array_size>] <cnst_str_array_initializer>)	First([)	{ [}
225.	First(={<str_array_elem>})=	First(=)	{ = }
226.	First([<array_size>] <str_2d_init>)	First([)	{ [}
227.	First(, id <strct_init_next>)	First(,)	{ , }
228.	First(λ) U Follow(<strct_init_next>)	Follow(<strct_init_next>)	{ ; }
229.	First(strct id {<strct_body>};)	First(strct)	{ strct }
230.	First(<strct_var_dec> ; <var_dec_next>)	First(<strct_var_dec>)	{ nt, dbl, bln, chr, strng }
231.	First(nt id)	First(nt)	{ nt }
232.	First(dbl id)	First(dbl)	{ dbl }
233.	First(bln id)	First(bln)	{ bln }
234.	First(chr id)	First(chr)	{ chr }
235.	First(strng id)	First(strng)	{ strng }
236.	First(<strct_body>)	First(<strct_body>)	{ nt, dbl, bln, chr, strng }
237.	First(λ) U Follow(<var_dec_next>)	Follow(<var_dec_next>)	{ } }
238.	First(fnctn <func_type> id (<parameter>) { <func_var_dec> <statement> } <function_definition>)	First(fnctn)	{ fnctn }
239.	First(λ) U Follow(<function_definition>)	Follow(<function_definition>)	{ mn }
240.	First(<data_type>)	First(<data_type>)	{ nt, dbl, bln, chr, strng }
241.	First(vd)	First(vd)	{ vd }
242.	First(nt)	First(nt)	{ nt }
243.	First(dbl)	First(dbl)	{ dbl }
244.	First(bln)	First(bln)	{ bln }

245.	First(chr)	First(chr)	{ chr }
246.	First(strng)	First(strng)	{ strng }
247.	First(<data_type> id <parameter_next>)	First(<data_type>)	{ nt, dbl, bln, chr, strng }
248.	First(λ) U Follow(<parameter>)	Follow(<parameter>)	{) }
249.	First(, <parameter>)	First(,)	{ , }
250.	First(λ) U Follow(<parameter_next>)	Follow(<parameter_next>)	{) }
251.	First(<var_dec> ; <func_var_dec>)	First(<var_dec>)	{ nt, dbl, bln, chr, strng }
252.	First(λ) U Follow(<func_var_dec>)	Follow(<func_var_dec>)	{ id, ++, --, prnt, f, swtch, fr, d, whl, rtrn, brk, cntn, } }
253.	First(<id_build_statements> <statement>)	First(<id_build_statements>)=	{ id }
254.	First(<unary_pre_statement> <statement>)	First(<unary_pre_statement>)	{ ++, -- }
255.	First(<output_statement> <statement>)	First(<output_statement>)	{ prnt }
256.	First(<conditional_statement> <statement>)	First(<conditional_statement>) =	{ f, swtch }
257.	First(<fr_statement> <statement>)	First(<fr_statement>)	{ fr }
258.	First(<d-whl_statement> <statement>)	First(<d-whl_statement>)	{ d }
259.	First(<whl_statement> <statement>)	First(<whl_statement>)	{ whl }
260.	First(<rtrn_statement> <statement>)	First(<rtrn_statement>)	{ rtrn }
261.	First(<control_statements> <statement>)	First(<control_statements>)	{ brk, cntn }
262.	First(λ) U Follow (<statement>)	Follow(<statement>)	{ end, }, lsf, ls, cs, dflt }
263.	First(id	First(id)	{ id }

	<id_build_statement_next>)		
264.	First(<id_next> <assignment_statement>)	First(<id_next>)	{ [, ., λ}
265.	First(<assignment_statement>)	First(<assignment_statement>)	{ +=, -=, *=, /=, = }
266.	First(<function_call>)	First(<function_call>)	{ (}
267.	First(<unary_inc/dec>)	First(<unary_inc/dec>)	{++, --}
268.	First(<assign_operator> <assignment_value>)	First(<assign_operator>)	{ +=, -=, *=, /= }
269.	First(= <assignment_value_input>)	First(=)	{ = }
270.	First(+=)	First(+=)	{ += }
271.	First(-=)	First(-=)	{ -=}
272.	First(*=)	First(*=)	{ *= }
273.	First(/=)	First(/=)	{ /= }
274.	First(<expression>;)	First(<expression>)	{ (, dbllit, ~dbllit, ntlit, ~ntlit, ++, --, id, chrlit, !, strnglit, blnlit }
275.	First(<assignment_value>)	First(<assignment_value>)	{ (, dbllit, ~dbllit, ntlit, ~ntlit, ++, --, id, chrlit, !, strnglit, blnlit }
276.	First(<input_statement_value>)	First(<input_statement_value>)	{ npt }
277.	First(npt(strnglit);)	First(npt)	{ npt }
278.	First(<unary_inc/dec>id;)=	First(<unary_inc/dec>)	{++, --}
279.	First(prnt(<output_value> <output_next>);)	First(prnt)	{ prnt }
280.	First(<expression>)	First(<expression>)	{ (, dbllit, ~dbllit, ntlit, ~ntlit, ++, --, id, chrlit, !, strnglit, blnlit } }
281.	First(, <output_value> <output_next>)	First(,)	{ , }
282.	First(λ) U	Follow(<output_next>)	{) }

	Follow(<output_next>)		
283.	First(f(<condition>){<statement> <lsf_statement> <ls_statement>} <lsf_statement> <ls_statement>)	First(f)	{ f }
284.	First(swtch(id) {<swtch_body>})	First(swtch)	{ swtch }
285.	First(<rel_arith_logic_expr>)	First(<rel_arith_logic_expr>)	{ (, dbllit, ~dbllit, ntlit, ~ntlit, ++, --, id, chrlit, !, strnglit, blnlit }
286.	First(lsf(<condition>) { <statement> } <lsf_statement>)	First(lsf)	{ lsf }
287.	First(λ) U Follow(<lsf_statement>)	Follow(<lsf_statement>)	{ ls, }, id, ++, --, prnt, f, swtch, fr, d, whl, rtrn, brk, cntn }
288.	First(ls { <statement> })	First(ls)	{ ls }
289.	First(λ) U Follow(<ls_statement>)	Follow(<ls_statement>)	{ }, id, ++, --, prnt, f, swtch, fr, d, whl, rtrn, brk, cntn, end }
290.	First(<cs><dflt>)	First(<cs>)	{ cs }
291.	First(cs <swtch_value>: <statement><cs_next>)	First(cs)	{ cs }
292.	First(<ntliterals>)	First(<ntliterals>)	{ ntlit, ~ntlit }
293.	First(chrlit)	First(chrlit)	{ chrlit }
294.	First(<cs><cs_next>)	First(<cs>)	{ cs }
295.	First(λ) U Follow(<cs_next>)	Follow(<cs_next>)	{ dflt, } }
296.	First(dflt: <statement>)	First(dflt)	{ dflt }
297.	First(λ) U Follow(<dflt>)	Follow(<dflt>)	{ } }
298.	First(fr(<initialization>; <fr_condi>;<inc/dec>){ <statement>})	First(fr)	{ fr }
299.	First(<id_build> = <arithmetic_nt>)	First(<id_build>)	{ id }

300.	First(<condition>)	First(<condition>)	{ (, dbllit, ~dbllit, ntlit, ~ntlit, ++, --, id, chrlit, !, strnglit, blnlit, npt }
301.	First(<fr_unary>)	First(<fr_unary>)	{ ++, -- }
302.	First(id<inc/dec_next>)	First(id)	{ id }
303.	First(<assign_operator> <inc/dec_value>)	First(<assign_operator>)	{ +=, -=, *=, /= }
304.	First(<unary_inc/dec>)	First(<unary_inc/dec>)	{ ++, -- }
305.	First(<unary_inc/dec> id)	First(<unary_inc/dec>)	{ ++, -- }
306.	First(<id_build>)	First(<id_build>)	{ id }
307.	First(<ntliterals>)	First(<ntliterals>)	{ntlit, ~ntlit}
308.	First(d{ <statement> } whl(<condition>);)	First(d)	{ d }
309.	First(whl(<condition>) {<statement>})	First(whl)	{ whl }
310.	First(rtrn <rtrn_value>;)	First(rtrn)	{ rtrn }
311.	First(<expression>)	First(<expression>)	{ (, dbllit, ~dbllit, ntlit, ~ntlit, ++, --, id, chrlit, !, strnglit, blnlit }
312.	First(brk;)	First(brk)	{ brk }
313.	First(cntn;)	First(cntn)	{ cntn }

XIII. Lexical Test Script

No.	Sample Script	Expected	Actual	Remarks
1	mn()	No Lexical Error	No Lexical Error	**PASSED**
2	mn-	Lexical Error: Invalid Delimiter	Lexical Error: Invalid Delimiter	**PASSED**
3	mn	No Lexical Error	No Lexical Error	**PASSED**
4	main	No Lexical Error (id)	No Lexical Error (id)	**PASSED**
5	MN	No Lexical Error (id)	No Lexical Error (id)	**PASSED**
6	thismn	No Lexical Error (id)	No Lexical Error (id)	**PASSED**
7	$mn	Lexical Error: Illegal Character ($)	Lexical Error: Illegal Character ($)	**PASSED**
8	mn↵	No Lexical Error	No Lexical Error	**PASSED**
9	rtrn;	No Lexical Error	No Lexical Error	**PASSED**
10	rtrn	No Lexical Error	No Lexical Error	**PASSED**
11	rtrn()	Lexical Error: Invalid Delimiter	Lexical Error: Invalid Delimiter	**PASSED**
12	rtrn0	No Lexical Error (id)	No Lexical Error (id)	**PASSED**
13	rtrn_	No Lexical Error (id)	No Lexical Error (id)	**PASSED**
14	RTRN	No Lexical Error (id)	No Lexical Error (id)	**PASSED**
15	111rtrn	Lexical Error: Identifiers cannot start with a number.	Lexical Error: Identifiers cannot start with a number.	**PASSED**
16	aartrn	No Lexical Error (id)	No Lexical Error (id)	**PASSED**
17	$rtrn	Lexical Error: Illegal Character ($)	Lexical Error: Illegal Character ($)	**PASSED**
18	rtrn{	Lexical Error: Invalid Delimiter	Lexical Error: Invalid Delimiter	**PASSED**
19	rtrn!	Lexical Error: Invalid Delimiter	Lexical Error: Invalid Delimiter	**PASSED**
20	nt	No Lexical Error	No Lexical Error	**PASSED**
21	nt;	Lexical Error: Expected an identifier after 'nt'	Lexical Error: Expected an identifier after 'nt'	**PASSED**
22	nt_	No Lexical Error (id)	No Lexical Error	**PASSED**
23	nt%	Lexical Error: Invalid Delimiter	Lexical Error: Invalid Delimiter	**PASSED**
24	nt(	Lexical Error: Invalid Delimiter	Lexical Error: Invalid Delimiter	**PASSED**
25	$nt	Lexical Error: Unexpected Character ($)	Lexical Error: Unexpected Character ($)	**PASSED**

26	thisnt	No Lexical Error (id)	No Lexical Error (id)	**PASSED**
27	NT	No Lexical Error (id)	No Lexical Error (id)	**PASSED**
28	dbl	No Lexical Error	No Lexical Error	**PASSED**
29	dbl;	Lexical Error: Expected an identifier after 'dbl'	Lexical Error: Expected an identifier after 'dbl'	**PASSED**
30	dbl_	No Lexical Error (id)	No Lexical Error (id)	**PASSED**
31	dbl%	Lexical Error: Invalid Delimiter	Lexical Error Invalid Delimiter	**PASSED**
32	dbl(	Lexical Error: Invalid Delimiter	Lexical Error: Invalid Delimiter	**PASSED**
33	dbl[	Lexical Error: Invalid Delimiter	Lexical Error: Invalid Delimiter	**PASSED**
34	dblA	No Lexical Error (id)	No Lexical Error (id)	**PASSED**
35	dbl3	No Lexical Error (id)	No Lexical Error (id)	**PASSED**
36	dbl}	Lexical Error: Invalid Delimiter	Lexical Error: Invalid Delimiter	**PASSED**
37	dbl,	Lexical Error: Invalid Delimiter	Lexical Error: Invalid Delimiter	**PASSED**
38	222dbl	Lexical Error: Identifiers cannot start with a number.	Lexical Error: Identifiers cannot start with a number.	**PASSED**
39	bbdbl	No Lexical Error (id)	No Lexical Error (id)	**PASSED**
40	$dbl	Lexical Error: Illegal Character ($)	Lexical Error: Illegal Character ($)	**PASSED**
41	dbl/	Lexical Error: Invalid Delimiter	Lexical Error Invalid Delimiter	**PASSED**
42	DBL	No Lexical Error (id)	No Lexical Error (id)	**PASSED**
43	chr	No Lexical Error	No Lexical Error	**PASSED**
44	chr;	Lexical Error: Expected an identifier after 'chr"	Lexical Error: Expected an identifier after 'chr"	**PASSED**
45	chr_	No Lexical Error (id)	No Lexical Error (id)	**PASSED**
46	chr%	Lexical Error: Invalid Delimiter	Lexical Error: Invalid Delimiter	**PASSED**
47	chr(	Lexical Error: Invalid Delimiter	Lexical Error: Invalid Delimiter	**PASSED**
48	chr[	Lexical Error: Invalid Delimiter	Lexical Error: Invalid Delimiter	**PASSED**
49	chrB	No Lexical Error (id)	No Lexical Error (id)	**PASSED**
50	chr3	No Lexical Error (id)	No Lexical Error (id)	**PASSED**
51	chr}	Lexical Error: Invalid	Lexical Error: Invalid	**PASSED**

		Delimiter	Delimiter	
52	chr,	Lexical Error: Invalid Delimiter	Lexical Error: Invalid Delimiter	**PASSED**
53	123_chr	Lexical Error: Identifiers cannot start with a number	Lexical Error: Identifiers cannot start with a number	**PASSED**
54	this_is_chr	No Lexical Error	No Lexical Error	**PASSED**
55	$chr	Lexical Error:Unexpected Character "c"	Lexical Error:Unexpected Character "c"	**PASSED**
56	/chr	Lexical Error:Unexpected Character "c"	Lexical Error:Unexpected Character "c"	**PASSED**
57	;chr	Lexical Error:Unexpected Character "c"	Lexical Error:Unexpected Character "c"	**PASSED**
58	CHR	No Lexical Error (id)	No Lexical Error (id)	**PASSED**
59	_chr	Lexical Error: Identifiers cannot start with a ' _ '	Lexical Error: Identifiers cannot start with a ' _ '	**PASSED**
60	bln	No Lexical Error	No Lexical Error	**PASSED**
61	bln;	Lexical Error: Invalid Delimiter	Lexical Error: Invalid Delimiter	**PASSED**
62	bln_	No Lexical Error (id)	No Lexical Error (id)	**PASSED**
63	bln*	Lexical Error: Invalid Delimiter	Lexical Error: Invalid Delimiter	**PASSED**
64	bln@	Lexical Error: Invalid Delimiter	Lexical Error: Invalid Delimiter	**PASSED**
65	bln(	Lexical Error: Invalid Delimiter	Lexical Error: Invalid Delimiter	**PASSED**
66	bln[	Lexical Error: Invalid Delimiter	Lexical Error: Invalid Delimiter	**PASSED**
67	blnG	No Lexical Error (id)	No Lexical Error (id)	**PASSED**
68	bln789	No Lexical Error (id)	No Lexical Error (id)	**PASSED**
69	bln}	Lexical Error: Invalid Delimiter	Lexical Error: Invalid Delimiter	**PASSED**
70	bln,	Lexical Error: Invalid Delimiter	Lexical Error: Invalid Delimiter	**PASSED**
71	bln`	Lexical Error: Invalid Delimiter	Lexical Error: Invalid Delimiter	**PASSED**
72	321_bln	Lexical Error: Identifiers cannot start with a number	Lexical Error: Identifiers cannot start with a number	**PASSED**
73	A3B_bln	No Lexical Error (id)	No Lexical Error (id)	**PASSED**

74	$bln	Lexical Error:Unexpected Character "b"	Lexical Error:Unexpected Character "b"	**PASSED**
75	\bln	Lexical Error:Unexpected Character "b"	Lexical Error:Unexpected Character "b"	**PASSED**
76	;bln	Lexical Error:Unexpected Character "b"	Lexical Error:Unexpected Character "b"	**PASSED**
77	BLN	No Lexical Error (id)	No Lexical Error (id)	**PASSED**
78	___bln	Lexical Error: Identifiers cannot start with a ' _ "	Lexical Error: Identifiers cannot start with a ' _ '	**PASSED**
79	strng	No Lexical Error	No Lexical Error	**PASSED**
80	strng;	Lexical Error: Expected an identifier after "strng"	Lexical Error: Expected an identifier after "strng"	**PASSED**
81	strng_	No Lexical Error (id)	No Lexical Error (id)	**PASSED**
82	strng+	Lexical Error: Invalid Delimiter	Lexical Error: Invalid Delimiter	**PASSED**
83	strng~	Lexical Error: Invalid Delimiter	Lexical Error: Invalid Delimiter	**PASSED**
84	strng(	Lexical Error: Invalid Delimiter	Lexical Error: Invalid Delimiter	**PASSED**
85	strng[	Lexical Error: Invalid Delimiter	Lexical Error: Invalid Delimiter	**PASSED**
86	strnglit	No Lexical Error	No Lexical Error	**PASSED**
87	strng_892B	No Lexical Error	No Lexical Error	**PASSED**
88	strng}	Lexical Error: Invalid Delimiter	Lexical Error: Invalid Delimiter	**PASSED**
89	strng,	Lexical Error: Invalid Delimiter	Lexical Error: Invalid Delimiter	**PASSED**
90	7strng	Lexical Error: Identifiers cannot start with a number	Lexical Error: Identifiers cannot start with a number	**PASSED**
91	string	No Lexical Error	No Lexical Error	**PASSED**
92	Strng	No Lexical Error (id)	No Lexical Error (id)	**PASSED**
93	STRNG	No Lexical Error (id)	No Lexical Error (id)	**PASSED**
94	+strng	Lexical Error:Unexpected Character "s"	Lexical Error:Unexpected Character "s"	**PASSED**
95	_strng	Lexical Error: Identifiers cannot start with a " _ "	Lexical Error: Identifiers cannot start with a ' _ '	**PASSED**

96	names[	No Lexical Error	No Lexical Error	**PASSED**
97	32[	Lexical Error: Invalid Delimiter	Lexical Error: Invalid Delimiter	**PASSED**
98	932B[	Lexical Error: Identifiers cannot start with a number	Lexical Error: Identifiers cannot start with a number	**PASSED**
99	AB3[	No Lexical Error	No Lexical Error	**PASSED**
100	cnst	No Lexical Error	No Lexical Error	**PASSED**

XIV. Syntax Test Script

No.	TEST ITEM	EXPECTED	ACTUAL	REMARKS
1	nt num = 12;	No Syntax Error	No Syntax Error	PASSED
2	nt num, dig;	No Syntax Error	No Syntax Error	PASSED
3	nt a = 4 + 5;	No Syntax Error	No Syntax Error	PASSED
4	nt b = 10 - 3;	No Syntax Error	No Syntax Error	PASSED
5	nt c = 15 * 2;	No Syntax Error	No Syntax Error	PASSED
6	nt d = 20 / 4;	No Syntax Error	No Syntax Error	PASSED
7	nt e = 3 % 9;	No Syntax Error	No Syntax Error	PASSED
8	nt x = 10 + z;	No Syntax Error	No Syntax Error	PASSED
9	nt x = num + num2;	No Syntax Error	No Syntax Error	PASSED
10	nt a = 3, b = 4, c; c = a+b;	No Syntax Error	No Syntax Error	PASSED
11	nt x; x += 10;	No Syntax Error	No Syntax Error	PASSED
12	nt x; x -= 20;	No Syntax Error	No Syntax Error	PASSED
13	nt x; x *= 30;	No Syntax Error	No Syntax Error	PASSED
14	nt x; x /= 40;	No Syntax Error	No Syntax Error	PASSED
15	dbl score = 84.5;	No Syntax Error	No Syntax Error	PASSED
16	dbl pi = 3,14;	Syntax Error	Syntax Error, Expected ' . '	PASSED
17	dbl result = (10.5 / 2 * 3;	Syntax Error	Syntax Error, Expected ') '	PASSED
18	chr choice = 'C';	No Syntax Error	No Syntax Error	PASSED
19	chr letter = ‘A’; prnt(“The first letter is “ , letter);	No Syntax Error	No Syntax Error	PASSED
20	chr pick 'A';	Syntax Error	Syntax Error	PASSED

21	strng name = "Dan";	No Syntax Error	No Syntax Error	PASSED
22	strng name = "Carl " James";	Syntax Error	Syntax Error	PASSED
23	strng name = "Carl ;	Syntax Error	Syntax Error, Expected ' " '	PASSED
24	bln cb = tr;	No Syntax Error	No Syntax Error	PASSED
25	bln cb = False;	Syntax Error	Syntax Error, Expected: [' tr ', ' fls ']	PASSED
26	bln cb = T;	Syntax Error	Syntax Error, Expected: [',', ';']	PASSED
27	prnt("Hello World");	No Syntax Error	No Syntax Error	PASSED
28	print ("Hello World");	Syntax Error	Syntax Error, Expected: [' prnt ']	PASSED
29	prnt(a+b);	No Syntax Error	No Syntax Error	PASSED

30	fnctn vd greet() {
	prnt("Hello");
	}
		No Syntax Error	No Syntax Error	PASSED

31	funtion vd hello() {
	prnt("Hello");
	}
		Syntax Error	Syntax Error,
			Expected: [' fnctn']
				PASSED

32	f (x == 10){ }	No Syntax Error	No Syntax Error	PASSED
33	f (x = 10){ }	Syntax Error	Syntax Error	PASSED

34	nt x = 5;
	nt y = 10;
	prnt(x > y);
		No Syntax Error	No Syntax Error	PASSED

35	nt a = 8;
	nt b = 8;
	prnt(a == b);
		No Syntax Error	No Syntax Error	PASSED

36	nt p = 4;
	nt q = 9;
	prnt(p != q);
		No Syntax Error	No Syntax Error	PASSED

37	dbl num1 = 7.5;
	dbl num2 = 7.5;
	prnt(num1 <= num2);
		No Syntax Error	No Syntax Error	PASSED

38	strng text1 = "Hi"; strng text2 = "Hello"; prnt(text1 > text2);	No Syntax Error	No Syntax Error	PASSED

39	bln x = tr;
	bln y = fls;
	prnt(x && y);
		No Syntax Error	No Syntax Error	PASSED

40	bln a = fls; prnt(!a);	No Syntax Error	No Syntax Error	PASSED
41	nt num = 7;	No Syntax Error	No Syntax Error	PASSED

	prnt((num > 5) && (num < 10));			

42	for (nt j = 1; j <= 5; j = j + 1)
	{
	prnt(j);
	}
		No Syntax Error	No Syntax Error	PASSED

43	nt k = 5;
	while (k > 0)
	{
	prnt(k);
	k = k - 1;
	}
		No Syntax Error	No Syntax Error	PASSED

44	nt res = (5 + 3) * 2; prnt(res);	No Syntax Error	No Syntax Error	PASSED
45	prnt((2 + 3) * (4 - 1));	No Syntax Error	No Syntax Error	PASSED

46	nt x = 3;
	if ((x > 2) && (x < 5))
	{
	prnt("Valid");
	}
		No Syntax Error	No Syntax Error	PASSED

47	nt a = (5 + 2; prnt(a);	Syntax Error Missing closing)	Syntax Error: Unexpected symbol ';', expected ')'	PASSED

48	fnctn add(nt x, nt y)
	{
	rtrn x + y;
	}
		No Syntax Error	No Syntax Error	PASSED

49	fnctn check()
	{
	prnt("Checking");
	}
		Syntax Error
		Missing return; statement
			Syntax Error: No valid rule for <declaration> with lookahead 'fnctn', expected one of: ['cnst', 'strct', 'nt', 'dbl', 'bln', 'chr', 'strng', 'id', '++', '--', 'prnt', 'f', 'swtch', 'fr', 'd', 'whl', 'rtrn', 'brk', 'cntn']	PASSED

50	fnctn sum(nt a, nt b)
	{
	rtrn a + b;
	}
	sum(5, 3);
		No Syntax Error	No Syntax Error	PASSED

XV. Semantic Test Script

No.	TEST ITEM	EXPECTED	ACTUAL	REMARKS
1	nt x; x += 10;	No semantic error	No semantic error	**PASSED**
2	nt y; y = x + 5;	Semantic error (x may be undeclared)	Semantic Error at line 3, column 5: Undefined variable 'x'	**PASSED**
3	nt math = (8 + 2) * (4 - 2);	No semantic error	No semantic error	**PASSED**
4	nt total = 15 % 4;	No semantic error	No semantic error	**PASSED**
5	nt num = 5.7;	Semantic error (Type mismatch: int cannot store float)	Semantic Error at line 2, column 8: Type mismatch: Cannot initialize 'nt' with dbl	**PASSED**
6	dbl n = 5 / 2.5;	Semantic error (Type mismatch: dbl cannot store nt)	Semantic Error at line 2, column 13: Type mismatch: Conso requires identical types for arithmetic operations ('nt' and 'dbl')	**PASSED**
7	dbl result = 7.5 * 2;	Semantic error (Type mismatch: dbl cannot store nt)	Semantic Error at line 2, column 20: Type mismatch: Conso requires identical types for arithmetic operations ('dbl' and 'nt')	**PASSED**
8	nt sum = 5 + 3 * 2;	No semantic error	No semantic error	**PASSED**
9	nt num1 = 5, num2; num2 = num1 + 5;	No semantic error	No semantic error	**PASSED**
10	nt y = 5; y = "abc";	Semantic error (Reassigning different type)	Semantic Error at line 3, column 1: Type mismatch: Cannot assign strng to nt	**PASSED**
11	dbl pi = 3.14; pi = 3.1415;	No semantic error	No semantic error	**PASSED**
12	nt arr[2] = {1,2,3};	Semantic error (Too many elements in array initialization)	Semantic Error at line 2, column 19: Array initialization has 3 elements, but array size is 2	**PASSED**
13	nt num[5] = {1,2,3,4,5}; prnt(num[3]);	No semantic error	No semantic error	**PASSED**

14	nt matrix[2][2] = {{1, 2}, {3, 4}};	No semantic error	No semantic error	**PASSED**
15	nt list[2]; list[1] = 5.5;	Semantic error (Type mismatch: int cannot store float)	Semantic Error at line 3, column 14: Type mismatch: Cannot assign 'dbl' to array element of type 'nt'	**PASSED**
16	nt arr[3] = {1,2,3}; prnt(arr[5]);	Semantic error (Index out of bounds)	Semantic Error at line 3, column 10: Array index 5 out of bounds (size 3)	**PASSED**
17	strng words[2] = {"Hello", "World"};	No semantic error	No semantic error	**PASSED**
18	bln flag = 1;	Semantic error (Type mismatch: boolean cannot be assigned an integer)	Semantic Error at line 2, column 10: Type mismatch: Cannot initialize 'bln' with nt	**PASSED**
19	bln flag = tr;	No semantic error	No semantic error	**PASSED**
20	bln flag = "true";	Semantic error (Type mismatch: assigning string to boolean)	Semantic Error at line 2, column 10: Type mismatch: Cannot initialize 'bln' with strng	**PASSED**
21	nt x = tr + fls;	Semantic error (Boolean cannot be used in arithmetic operations)	Semantic Error at line 2, column 11: Type mismatch: Cannot apply '+' to 'bln'	**PASSED**
22	bln not = !(fls);	No semantic error	No semantic error	**PASSED**
23	bln valid = (5 > 3);	No semantic error	No semantic error	**PASSED**

24	bln isTrue = tr;
	f (isTrue) {
	prnt("True");
	}
		No semantic error	No semantic error	**PASSED**

25	nt a = 10;
	strng valid = "This is valid";
	f(a){
	prnt(valid);
	}
		Semantic error (Condition must evaluate to boolean)	Semantic Error at line 4, column 2: Condition in 'f' statement must be of type 'bln', got 'nt'	**PASSED**

26	nt count = 5;
	whl(count > 0) {
	count -= 1;
	}
		No semantic error	No semantic error	**PASSED**

27	whl("yes") {
	prnt("Loop");
	}
		Semantic error (Invalid condition type)	Semantic Error at line 2, column 4: Condition in 'whl' statement must be of	**PASSED**

			type 'bln', got 'strng'	

28	nt age = 18;
	f(age >= 18) {
	prnt("Adult");
	}
		No semantic error	No semantic error	**PASSED**

29	nt a = 1;
	dbl b = 2.2;
	f (a >= b){
	}
		Semantic error (Implicit conversion not allowed: nt and dbl)	Semantic error	**PASSED**

30	nt p = 0; nt q = 0;
	f (p && q == 0) {
	}
		Semantic error (Invalid logical expression in relational context)	Semantic error	**PASSED**

31	fnctn nt add (nt a, nt b) {
	rtrn a + b;
	}
		No semantic error	No semantic error	**PASSED**

32	mn(){
	nt num = 2, num2;
	dbl frac = 2.31, frac2;
	bln flag = tr, flag2;
	chr char = 'c', char2;
	strng greet = "Hello" ` "World", greet2;
	
	flag2 = num > num;
	flag2 = "Hello" == "World";
	flag2 = 'c' == 'h';
	
	prnt('c' + 'h');
	prnt(5 > "What");
	prnt("Hello" > "World");
	prnt(num + frac);
	end;
	}
		No semantic error	No semantic error	**PASSED**

33	mn(){
	nt num = 2, num2;
	dbl frac = 2.31, frac2;
	bln flag = tr, flag2;
	chr char = 'c', char2;
	strng greet = "Hello" ` "World", greet2;
	
	prnt('c' + 'h');
	prnt(5 > "What");
	prnt("Hello" > "World");
	prnt(num + frac);
	end;
	}
		Semantic error	Semantic error	**PASSED**

34	fnctn nt Test(){
	nt id = 3;
	rtrn 1;
		No semantic error	No Semantic error	**PASSED**

	}
	
	mn(){
	nt arr[2] = { 2 };
	prnt("I am ", 2, 3, 4, 5);
	f (arr[1] > 5){
	prnt("Hello");
	}
	end;
	}			

35	fnctn nt Test(){
	nt id = 3;
	rtrn 1;
	}
	
	mn(){
	nt arr[2] = { 2 }, expr;
	prnt("I am ", 2, 3, 4, 5);
	expr = arr[2];
	end;
	}
		No semantic error	No semantic error	**PASSED**

36	fnctn nt Test(){
	nt id = 3;
	rtrn 1;
	}
	
	mn(){
	nt arr[2][5];
	nt sum;
	sum = ((5 + 3) / 5); #Unexpected token 'ntlit' in expression
	end;
	}
		No semantic error	No Semantic error	**PASSED**

37	fnctn nt Test(){
	nt id = 3;
	rtrn 1;
	}
	
	mn(){
	nt arr[2][5];
	nt sum;
	sum = ((5 - 3) * (sum + 2));
	#Unexpected closing parenthesis
	end;
	}
		No semantic error	Semantic error	**PASSED**

38	nt arr[2][2] = {{2}, {3}};
	dbl sum;
	
	fnctn dbl Test(){
	rtrn 3.0;
	}
		No semantic error	No Semantic error	**PASSED**

	mn(){
	sum = arr[1][1] + Test();
	end;
	}			

39	nt arr[2][2] = {{2}, {3}};
	dbl sum;
	nt num;
	bln flag;
	
	fnctn dbl Test(){
	rtrn 3.0;
	}
	
	mn(){
	num = ((3 + 5));
	num = ((3 + 5) / 6);
	num = ((3 + 5) * (5 - 6));
	sum = (arr[1][1] + Test()) / 6;
	sum = ((arr[1][1] + Test()) / 6);
	sum = ((arr[1][1] + Test()) / 6) +
	num + 3 + 2 + (2 - (3 + 5 * (2)) +
	7);
	prnt(((arr[1][1] + Test()) / 6) +
	num);
	prnt((3 + 5));
	prnt((((3 + 5))));
	
	f((3 > 5) && ("Hello" ==
	"World")){
	prnt("Test");
	}
	
	f(((3 > 5) && ("Hello" ==
	"World")) || flag){
	prnt("Test");
	}
	
	end;
	}
		No semantic error	No Semantic error	**PASSED**

40	mn(){
	nt num = 1;
	swtch(num) {
	cs 1:
	prnt("One");
	cntn;
	brk;
	cs 2:
	prnt("Two");
	brk;
	dflt:
	prnt("Other");
	brk;
		Semantic error	Semantic error	**PASSED**

	}
	end;
	}			

41	fnctn nt factorial(nt n) {
	f (n == 0) {
	rtrn 1;
	}
	rtrn n * factorial(n - 1);
	}
	
	mn(){
	nt input;
	input = npt("Enter a number for factorial: ");
	prnt(factorial(5));
	end;
	}
		No Semantic error	No Semantic error	**PASSED**

42	fnctn nt factorial(nt n) {
	f (n == 0) {
	rtrn 3 - 5;
	}
	rtrn n;
	}
	
	mn(){
	nt n;
	f (n == 0) {
	rtrn 3 - 5;
	}
	end;
	}
		No Semantic error	No Semantic error	**PASSED**

43	mn(){
	cnst nt num = 3, num2 = ~2;
	cnst dbl frac = 3.12, frac2 = ~3.14;
	cnst chr char = 'c', char2 = 'd';
	cnst bln flag = tr, flag2 = fls;
	cnst strng greet = "Hello", greet2 = "World";
	
	num = 7;
	
	end;
	}
		Semantic error	Semantic error	**PASSED**

44	mn(){
	nt num = 2, num2;
	dbl frac = 2.31, frac2;
	bln flag = tr, flag2;
	chr char = 'c', char2;
	strng greet = "Hello" ` "World", greet2;
	flag2 = num > num;
		Semantic error	Semantic error	**PASSED**

	flag2 = "Hello" == "World";
	flag2 = 'c' == 'h';
	prnt('c' + 'h');
	prnt(5 > "What");
	prnt("Hello" > "World");
	prnt(num + frac);
	end;
	}			

45	mn(){
	chr counter;
	nt i;
	fr(i = 0; i > 5; i+=counter){
	prnt("hello");
	}
	end;
	}
		Semantic error	Semantic error	**PASSED**

46	mn(){
	nt arr[2] = { 2 }, expr;
	expr = 3;
	end;
	}
		No Semantic error	No Semantic error	**PASSED**

47	fnctn nt add(nt a, nt b) {
	rtrn a + b;
	}
	
	mn(){
	bln id;
	id = !!!!!id;
	end;
	}
		No Semantic error	No Semantic error	**PASSED**

48	fnctn nt add(nt a, nt b) {
	rtrn a + b;
	}
	
	mn(){
	nt arr[2], expr;
	bln id;
	expr = 3;
	end;
	}
		No Semantic error	No Semantic error	**PASSED**

49	mn(){
	bln flag, flag2 = tr;
	flag = !!!!flag2;
	end;
	}
		No Semantic error	No Semantic error	**PASSED**

50	dbl frac[2] = { 3.2 }, frac2[2][1] = {{3.3},{2.5}}; bln flag[2] = { tr, fls }, flag2[2][1] = {{tr},{tr}}; chr letter[2] = { 'c' }, letter2[2][1]	Semantic error	Semantic error	**PASSED**

```
= {{'d'},{'c'}};
strng name[1] = { "Hello "};


mn(){
prnt(num[1] + num2[1][0]);
prnt(frac[1] * num2[1][0]);
prnt((flag2[1][0] == tr) && (5 >
num[1]));
prnt(letter[1] == letter2[1][0]);
prnt(name[0] != "hello");
end;
}
```

XVI. 50 problems with Source Code

No.	Machine Problems	Source Code

1.	Write a program that solves the area of a circle with a radius of 10	mn() {
		nt radius = 10;
		dbl pi = 3.14159265;
		dbl area;
		
		area = pi * radius * radius;
		prnt("Area of the circle: ", area);
		end;
		}

2.	Write a program that converts 420 Philippine Pesos to Japanese Yen, given that 1 Philippine Peso = 2.68 Japanese Yen. Display the equivalent amount in Yen.	mn() {
		nt php = 420;
		dbl rate = 2.68;
		dbl jpy;
		
		jpy = php * rate;
		prnt("Japanese Yen: ", jpy);
		end;
		}

3.	Create a program that prints the subject code, class time, and professor of the Compiler Design course.	mn() {
		strng code = "CSC 0321";
		strng time = "10:00 AM - 11:30 AM";
		strng prof = "Prof. Martinez";
		
		prnt("Subject Code: ", code, " \n");
		prnt("Time: ", time, " \n");
		prnt("Professor: ", prof, " \n");
		end;
		}

4.	Create a program that prints the name "Alvin A. Alday" in surname-first format (i.e., "Alday, Alvin A.").	mn() {
		strng _fName = "Alvin", _mInitial = "A.", _lName = "Alday";
		
		prnt("Name: ", _fName, " ", _mInitial, " ", _lName);
		prnt("\nSurnameFirst: ", _lName, ", ", _fName, " ", _mInitial);
		end;
		}

5.	Create a program that computes the volume of a sphere with a radius of 7.	mn() {
		nt radius = 7;
		dbl pi = 3.14159265;
		dbl volume;
		
		volume = (4.0 / 3.0) * pi * radius * radius * radius;
		prnt("Volume of the sphere: ", volume);
		end;
		}

6.	Write a program that calculates the number of bullets required to kill 125 dragons, given that 1 bullet takes ¼ of a dragon's life.	mn() {
		nt dragons = 125;
		dbl bulletPower = 0.25;
		dbl bullets;

		bullets = dragons / bulletPower;
		prnt("Bullets needed: ", bullets);
		end;
		}

7.	Create a program that reads a number from input and prints its square.	mn() {
		nt num;
		nt square;
		
		num = npt("Enter a number: ");
		
		square = num * num;
		prnt("Square of the number: ", square);
		end;
		}

8.	Create a program that solves for the trapezoid area given the base1 = 118, base2 = 80, and height = 50.	mn() {
		nt base1 = 118;
		nt base2 = 80;
		nt height = 50;
		dbl area;
		
		area = 0.5 * (base1 + base2) * height;
		prnt("Area of the trapezoid: ", area);
		end;
		}

9.	Build a program that prompts the user to enter their name and a number N, then prints "[Name] will be [N + 10] years old in 10 years."	mn() {
		strng name;
		nt age;
		nt futureAge;
		
		name = npt("Enter your Name: ");
		age = npt("Enter your number Age: ");
		
		futureAge = age + 10;
		
		prnt(name, " will be ", futureAge, " years old in 10 years.");
		end;
		}

10.	Build a program that prints the number of doormats that can be made with 1000m of cloth, given that a doormat needs 200m of cloth to be made.	mn() {
		nt totalCloth = 1000;
		nt clothPerMat = 200;
		nt doormats;
		
		doormats = totalCloth / clothPerMat;
		prnt("Number of doormats: ", doormats);
		end;
		}

11.	Write a program that enters a string and prints the same number of asterisks as there are characters in the input.	mn() {
		strng input;
		nt count;
		nt i = 0;

		input = npt("Enter a string: ");
		count = npt("How many characters is your string?: ");
		
		whl (i < count) {
		prnt("*");
		i = i + 1;
		}
		
		end;
		}

12.	Build a program that will display the number of "*" depending on the number entered.	mn() {
		nt count;
		nt i = 0;
		
		count = npt("Enter a number: ");
		
		whl (i < count) {
		prnt("*");
		i = i + 1;
		}
		
		end;
		}

13.	Create a program that shows the number position of a letter in the alphabet.	mn() {
		chr alphabet[26] = { 'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j',
		'k', 'l', 'm', 'n', 'o', 'p', 'q', 'r', 's', 't',
		'u', 'v', 'w', 'x', 'y', 'z' };
		chr input;
		nt i = 0;
		nt position = ~1;
		
		input = npt("Enter a letter (lowercase): ");
		
		whl (i < 26) {
		f (input == alphabet[i]) {
		position = i + 1;
		brk;
		}
		i = i + 1;
		}
		
		prnt("Position in the alphabet: ", position);
		end;
		}

14.	Take a number as input, store it in a list, and ask the user if they want to add again. Repeat until they say no, then display all numbers.	mn() {
		nt nums[100];
		chr again;
		nt i = 0;
		nt j;
		nt temp;
		
		whl (tr) {
		temp = npt("Enter a number: ");

		nums[i] = temp;
		i = i + 1;
		
		again = npt("Add again? (y/n): ");
		f (again == 'n') {
		brk;
		}
		}
		
		j = 0;
		prnt("You entered: ");
		whl (j < i) {
		prnt("\n",nums[j]);
		j = j + 1;
		}
		
		end;
		}

15.	Solve for the total area that a round umbrella can cover with a radius of 3.5 meters.	mn() {
		dbl radius = 3.5;
		dbl pi = 3.14159265;
		dbl area;
		
		area = pi * radius * radius;
		prnt("Area covered by the umbrella: ", area);
		end;
		}

16.	Create a program that enters the height and weight of a person and calculates the BMI.	mn() {
		dbl height;
		dbl weight;
		dbl bmi;
		dbl heightInMeters;
		
		height = npt("Enter height in centimeters: ");
		weight = npt("Enter weight in kilograms: ");
		
		heightInMeters = height / 100.0;
		bmi = weight / (heightInMeters * heightInMeters);
		
		prnt("Your BMI is: ", bmi);
		end;
		}

17.	Create a program that asks for age, then prints whether the inputted age is eligible to vote or not.	mn() {
		nt age;
		
		age = npt("Enter your age: ");
		
		f (age >= 18) {
		prnt("You are eligible to vote.");
		}
		ls {
		prnt("You are not eligible to vote.");
		}

		end; }

18.	Build a program that asks for a number between 1–7 and prints the day it corresponds to in the week.	mn() {
		nt day;
		
		day = npt("Enter a number (1-7): ");
		
		swtch (day) {
		cs 1:
		prnt("Sunday");
		brk;
		cs 2:
		prnt("Monday");
		brk;
		cs 3:
		prnt("Tuesday");
		brk;
		cs 4:
		prnt("Wednesday");
		brk;
		cs 5:
		prnt("Thursday");
		brk;
		cs 6:
		prnt("Friday");
		brk;
		cs 7:
		prnt("Saturday");
		brk;
		dflt:
		prnt("Invalid number. Please enter 1 to 7.");
		brk;
		}
		
		end;
		}

19.	Build a program that asks for the user's GWA and tells if they are a Dean's Lister or not.	mn() {
		dbl gwa;
		
		gwa = npt("Enter your GWA: ");
		
		f (gwa <= 1.75) {
		prnt("You are a Dean's Lister!");
		}
		ls {
		prnt("You are not a Dean's Lister.");
		}
		
		end;
		}

20.	Write a program that checks the password's strength.	mn() { chr password[100];

		nt i = 0;
		nt length;
		
		password = npt("Enter your password: ");
		
		whl (password[i] != '\0') {
		i = i + 1;
		}
		
		length = i;
		
		f (length >= 12) {
		prnt("Strength: Strong");
		}
		lsf (length >= 8) {
		prnt("Strength: Moderate");
		}
		ls {
		prnt("Strength: Weak");
		}
		
		end;
		}

21.	Write a program that checks if the input number is greater than, less than, or equal to the other inputted number.	mn() {
		nt num1;
		nt num2;
		
		num1 = npt("Enter the first number: ");
		num2 = npt("Enter the second number: ");
		
		f (num1 > num2) {
		prnt("First number is greater than second number.");
		}
		lsf (num1 < num2) {
		prnt("First number is less than second number.");
		}
		ls {
		prnt("Both numbers are equal.");
		}
		
		end;
		}

22.	Write a program that tells if the student passed or failed the class according to the GWA he inputted.	mn() {
		dbl gwa;
		
		gwa = npt("Enter your GWA: ");
		
		f (gwa <= 3.00) {
		prnt("You passed the class.");
		}
		ls {
		prnt("You failed the class.");
		}

		end; }

23.	Read temperature in centigrade and display a suitable message according to the temperature.	mn() {
		dbl temp;
		
		temp = npt("Enter temperature in Celsius: ");
		
		f (temp < 0) {
		prnt("Freezing weather.");
		}
		lsf (temp < 10) {
		prnt("Very cold weather.");
		}
		lsf (temp < 20) {
		prnt("Cold weather.");
		}
		lsf (temp < 30) {
		prnt("Normal in temperature.");
		}
		lsf (temp < 40) {
		prnt("It's hot.");
		}
		ls {
		prnt("It's very hot.");
		}
		
		end;
		}

24.	Enter your MBTI and print the Disney Princess that has the same personality as you.	mn() {
		strng mbti;
		
		mbti = npt("Enter your MBTI type (UPPERCASE, e.g., INFP, ESTJ):
		");
		
		f (mbti == "INFP") {
		prnt("You are like Belle from Beauty and the Beast.");
		}
		lsf (mbti == "ENFP") {
		prnt("You are like Anna from Frozen.");
		}
		lsf (mbti == "ISFJ") {
		prnt("You are like Cinderella.");
		}
		lsf (mbti == "ESFJ") {
		prnt("You are like Snow White.");
		}
		lsf (mbti == "INTJ") {
		prnt("You are like Elsa from Frozen.");
		}
		lsf (mbti == "ENTP") {
		prnt("You are like Rapunzel.");
		}

		lsf (mbti == "ISTJ") {
		prnt("You are like Tiana from The Princess and the Frog.");
		}
		lsf (mbti == "INFJ") {
		prnt("You are like Pocahontas.");
		}
		ls {
		prnt("No matching Disney Princess for that MBTI type.");
		}
		
		end;
		}

25.	Build a program that reads a word and identifies if the word is all vowels, all consonants, or a mix of both. Also, print the number of each letter.	mn() {
		chr word[100];
		nt i = 0;
		nt vowels = 0;
		nt consonants = 0;
		nt total;
		
		word = npt("Enter a word (lowercase letters only): ");
		
		whl (word[i] != '\0') {
		f (word[i] == 'a') {
		vowels = vowels + 1;
		}
		lsf (word[i] == 'e') {
		vowels = vowels + 1;
		}
		lsf (word[i] == 'i') {
		vowels = vowels + 1;
		}
		lsf (word[i] == 'o') {
		vowels = vowels + 1;
		}
		lsf (word[i] == 'u') {
		vowels = vowels + 1;
		}
		ls {
		consonants = consonants + 1;
		}
		i = i + 1;
		}
		
		total = vowels + consonants;
		
		f (vowels > 0) {
		f (consonants == 0) {
		prnt("The word is all vowels.");
		}
		lsf (consonants > 0) {
		prnt("The word is a mix of vowels and consonants.");
		}
		}

		lsf (vowels == 0) {
		f (consonants > 0) {
		prnt("The word is all consonants.");
		}
		ls {
		prnt("No letters were counted.");
		}
		}
		
		prnt("\nTotal letters: ", total);
		prnt("\nVowels: ", vowels);
		prnt("\nConsonants: ", consonants);
		
		end;
		}

26.	From the range 100, create a program that will sum up all odd and even numbers.	mn() {
		nt i = 1;
		nt sumOdd = 0;
		nt sumEven = 0;
		
		whl (i <= 100) {
		f (i % 2 == 0) {
		sumEven = sumEven + i;
		}
		ls {
		sumOdd = sumOdd + i;
		}
		i = i + 1;
		}
		
		prnt("Sum of even numbers: ", sumEven);
		prnt("Sum of odd numbers: ", sumOdd);
		
		end;
		}

27.	Make a program that takes the input of the user from 1–12 and prints out the corresponding month.	mn() {
		nt month;
		
		month = npt("Enter a number (1-12): ");
		
		swtch (month) {
		cs 1:
		prnt("January");
		brk;
		cs 2:
		prnt("February");
		brk;
		cs 3:
		prnt("March");
		brk;
		cs 4:
		prnt("April");
		brk;

		cs 5:
		prnt("May");
		brk;
		cs 6:
		prnt("June");
		brk;
		cs 7:
		prnt("July");
		brk;
		cs 8:
		prnt("August");
		brk;
		cs 9:
		prnt("September");
		brk;
		cs 10:
		prnt("October");
		brk;
		cs 11:
		prnt("November");
		brk;
		cs 12:
		prnt("December");
		brk;
		dflt:
		prnt("Invalid input. Please enter a number from 1 to 12.");
		brk;
		}
		
		end;
		}

28.	Write a program that asks for a number ranging from 3 to 8 and prints the name of a polygon with the corresponding number of sides.	mn() {
		nt sides;
		
		sides = npt("Enter a number (3-8): ");
		
		swtch (sides) {
		cs 3:
		prnt("Triangle");
		brk;
		cs 4:
		prnt("Quadrilateral");
		brk;
		cs 5:
		prnt("Pentagon");
		brk;
		cs 6:
		prnt("Hexagon");
		brk;
		cs 7:
		prnt("Heptagon");
		brk;
		cs 8:

		prnt("Octagon");
		brk;
		dflt:
		prnt("Invalid input. Please enter a number from 3 to 8.");
		brk;
		}
		
		end;
		}

29.	Write a program that enters three numbers and finds the largest among them.	mn() {
		nt num1;
		nt num2;
		nt num3;
		
		num1 = npt("Enter first number: ");
		num2 = npt("Enter second number: ");
		num3 = npt("Enter third number: ");
		
		f (num1 >= num2) {
		f (num1 >= num3) {
		prnt("The largest number is: ", num1);
		}
		ls {
		prnt("The largest number is: ", num3);
		}
		}
		ls {
		f (num2 >= num3) {
		prnt("The largest number is: ", num2);
		}
		ls {
		prnt("The largest number is: ", num3);
		}
		}
		
		end;
		}

30.	Create a program that loops 10 times and prints the sum of each number and its square.	mn() {
		nt i = 1;
		nt square;
		nt sum;
		
		whl (i <= 10) {
		square = i * i;
		sum = i + square;
		prnt("\nNumber: ", i);
		prnt("\nSum of number and its square: ", sum);
		i = i + 1;
		}
		
		end;
		}

31.	Build a program that adds up all	mn() {

	the numbers between 1 and a given positive integer n, then prints the result.	nt n;
		nt i = 1;
		nt sum = 0;
		
		n = npt("Enter a positive integer: ");
		
		whl (i <= n) {
		sum = sum + i;
		i = i + 1;
		}
		
		prnt("The sum from 1 to ", n);
		prnt(" is: ", sum);
		
		end;
		}

32.	Create a program that makes a pyramid pattern using asterisks based on the number of rows entered by the user.	mn() {
		nt rows;
		nt i = 1;
		nt j;
		nt k;
		nt space;
		
		rows = npt("Enter number of rows: ");
		
		whl (i <= rows) {
		space = rows - i;
		j = 1;
		
		whl (j <= space) {
		prnt(" ");
		j = j + 1;
		}
		
		k = 1;
		whl (k <= (2 * i - 1)) {
		prnt("*");
		k = k + 1;
		}
		
		prnt("\n");
		i = i + 1;
		}
		
		end;
		}

33.	Simulate a number guessing game by asking the user to guess a fixed number until they get it right.	mn() {
		nt number = 42;
		nt guess;
		
		prnt("Guess the number (1-100): ");
		
		whl (tr) {

		guess = npt("Enter your guess: ");
		
		f (guess > number) {
		prnt("Too high, try again.");
		}
		lsf (guess < number) {
		prnt("Too low, try again.");
		}
		ls {
		prnt("Correct! You guessed the number.");
		brk;
		}
		}
		
		end;
		}

34.	Print the Fibonacci series of n terms where n is input by the user.	mn() {
		nt n;
		nt a = 0;
		nt b = 1;
		nt next;
		nt i = 1;
		
		n = npt("Enter number of Fibonacci terms: ");
		
		prnt("Fibonacci Series: ");
		
		whl (i <= n) {
		prnt(" ",a);
		next = a + b;
		a = b;
		b = next;
		i = i + 1;
		}
		
		end;
		}

35.	Write a program that reads a word and counts how many times the letter 'e' appears.	mn() {
		chr word[20];
		nt i = 0;
		nt count = 0;
		
		word = npt("Enter a word: ");
		
		whl (word[i] != '\0') {
		f (word[i] == 'e') {
		count = count + 1;
		}
		i = i + 1;
		}
		
		prnt("Number of e's: ", count);

		end; }

36.	Write a program that asks for a word and displays it three times.	mn() {
		strng word;
		nt i = 1;
		
		word = npt("Enter a word: ");
		
		whl (i <= 3) {
		prnt(word, " ");
		i = i + 1;
		}
		
		end;
		}

37.	Print all odd numbers from 0 to 100.	mn() {
		nt i = 1;
		
		whl (i <= 100) {
		prnt(" ", i);
		i = i + 2;
		}
		
		end;
		}

38.	Write a function that returns a string message: "Hello, thanks for using our compiler!".	fnctn strng greet() {
		rtrn "Hello, thanks for using our compiler!";
		}
		
		mn() {
		strng message;
		
		message = greet();
		prnt(message);
		
		end;
		}

39.	Write a program that converts USD to Philippine Peso, given that a dollar is equivalent to 58.94 pesos.	mn() {
		dbl usd;
		dbl php;
		
		usd = npt("Enter amount in USD: ");
		
		php = usd * 58.94;
		
		prnt("Equivalent in PHP: ", php);
		
		end;
		}

40.	Write a program that accepts an integer as input and outputs the integer's square.	mn() { nt num; nt square;

		num = npt("Enter an integer: ");
		square = num * num;
		
		prnt("The square is: ", square);
		
		end;
		}

41.	Create a program that accepts a lowercase string and counts how many vowels and consonants it contains.	mn() {
		chr word[100];
		nt i = 0;
		nt vowels = 0;
		nt consonants = 0;
		
		word = npt("Enter a lowercase word: ");
		
		whl (word[i] != '\0') {
		# Vowel check
		f (word[i] == 'a') {
		vowels = vowels + 1;
		}
		lsf (word[i] == 'e') {
		vowels = vowels + 1;
		}
		lsf (word[i] == 'i') {
		vowels = vowels + 1;
		}
		lsf (word[i] == 'o') {
		vowels = vowels + 1;
		}
		lsf (word[i] == 'u') {
		vowels = vowels + 1;
		}
		# Consonant check — explicit list (non-vowels only)
		lsf (word[i] == 'b') { consonants = consonants + 1; }
		lsf (word[i] == 'c') { consonants = consonants + 1; }
		lsf (word[i] == 'd') { consonants = consonants + 1; }
		lsf (word[i] == 'f') { consonants = consonants + 1; }
		lsf (word[i] == 'g') { consonants = consonants + 1; }
		lsf (word[i] == 'h') { consonants = consonants + 1; }
		lsf (word[i] == 'j') { consonants = consonants + 1; }
		lsf (word[i] == 'k') { consonants = consonants + 1; }
		lsf (word[i] == 'l') { consonants = consonants + 1; }
		lsf (word[i] == 'm') { consonants = consonants + 1; }
		lsf (word[i] == 'n') { consonants = consonants + 1; }
		lsf (word[i] == 'p') { consonants = consonants + 1; }
		lsf (word[i] == 'q') { consonants = consonants + 1; }
		lsf (word[i] == 'r') { consonants = consonants + 1; }
		lsf (word[i] == 's') { consonants = consonants + 1; }
		lsf (word[i] == 't') { consonants = consonants + 1; }
		lsf (word[i] == 'v') { consonants = consonants + 1; }
		lsf (word[i] == 'w') { consonants = consonants + 1; }
		lsf (word[i] == 'x') { consonants = consonants + 1; }

		lsf (word[i] == 'y') { consonants = consonants + 1; }
		lsf (word[i] == 'z') { consonants = consonants + 1; }
		
		i = i + 1;
		}
		
		prnt("Number of vowels: ", vowels);
		prnt("\nNumber of consonants: ", consonants);
		
		end;
		}

42.	Write a program that asks for a base and exponent and computes the result.	mn() {
		nt base;
		nt exponent;
		nt result = 1;
		nt i = 1;
		
		base = npt("Enter base: ");
		exponent = npt("Enter exponent: ");
		
		whl (i <= exponent) {
		result = result * base;
		i = i + 1;
		}
		
		prnt("Result: ", result);
		
		end;
		}

43.	Build a program that asks for 10 grades and computes the average, then tells if the student passed.	mn() {
		nt grades[10];
		nt i = 0;
		nt sum = 0;
		nt avg;
		nt input;
		
		whl (i < 10) {
		prnt("Grade number ", i + 1, ",");
		input = npt(" Enter grade: ");
		grades[i] = input;
		sum = sum + input;
		i = i + 1;
		}
		
		avg = sum / 10;
		
		prnt("Average: ", avg);
		
		f (avg >= 75) {
		prnt("\nStudent passed.");
		}
		ls {
		prnt("\nStudent failed.");

		}
		
		end;
		}

44.	Write a program that converts seconds to hours, minutes, and seconds.	mn() {
		nt totalSeconds;
		nt hours;
		nt minutes;
		nt seconds;
		
		totalSeconds = npt("Enter total seconds: ");
		
		hours = totalSeconds / 3600;
		minutes = (totalSeconds % 3600) / 60;
		seconds = totalSeconds % 60;
		
		prnt("\nHours: ", hours);
		prnt("\nMinutes: ", minutes);
		prnt("\nSeconds: ", seconds);
		
		end;
		}

45.	Write a program that takes a name and prints it vertically, one letter per line.	mn() {
		chr name[100];
		nt i = 0;
		chr letter;
		
		name = npt("Enter your name: ");
		
		whl (name[i] != '\0') {
		letter = name[i];
		prnt(letter,"\n");
		i = i + 1;
		}
		
		end;
		}

46.	Ask for a number 1–7 and print the day of the week.	mn() {
		nt day;
		
		day = npt("Enter a number (1-7: ");
		
		swtch (day) {
		cs 1:
		prnt("Sunday");
		brk;
		cs 2:
		prnt("Monday");
		brk;
		cs 3:
		prnt("Tuesday");
		brk;
		cs 4:

		prnt("Wednesday");
		brk;
		cs 5:
		prnt("Thursday");
		brk;
		cs 6:
		prnt("Friday");
		brk;
		cs 7:
		prnt("Saturday");
		brk;
		dflt:
		prnt("Invalid input. Please enter a number from 1 to 7.");
		brk;
		}
		
		end;
		}

47.	Create a program that converts celsius to Farenheit and Kelvin	mn(){
		dbl cel, far, kel;
		
		cel = npt("Enter temperature in Celsius: ");
		
		#Celsius to Farenheit
		far = (cel * 9/5) + 32;
		#Celsius to kelvin
		kel = cel + 273.15 ;
		
		prnt(cel,"Celsius is",far, "in Farenheit and", kel,"in Kelvin");
		
		end;
		}

48.	Create a program that identify if a number is a Harshad number or not	mn(){
		nt temp, dig, num, sum, no;
		
		num = npt("Enter a number: ");
		no = num; # for output purposes
		whl (no!=0) {
		dig = no % 10;# Get the last digit of the number
		sum += dig;#Add the digit to the sum
		no/=10; #Remove the last digit from the number
		}
		temp = num%sum;
		f(temp == 0){
		prnt(num, "is a Harshad number");
		} ls {
		prnt(num, "is not a Harshad number");
		}
		
		end;
		}

49.	Create a program that identify id a number is divisible by 3 and 5	mn(){ nt num;

		num = npt("Enter a number: ");
		
		f(((num % 3)==0)&&((num % 5)==0)){
		prnt(num,"is divisible by 3 and 5");
		} ls {
		prnt(num,"is not divisible by 3 and 5");
		}
		end;
		}

50.	Create a program that identify if a number is a Strong number or not	fnctn nt factorial(nt n) {
		f (n == 0) {
		rtrn 1;
		}
		rtrn n * factorial(n - 1);
		}
		
		mn(){
		nt num, orig, sum = 0, digit;
		
		num = npt("Enter a number: ");
		orig = num;
		
		whl (num != 0) {
		digit = num % 10; # Get the last digit
		sum += factorial(digit); # Add the factorial of the digit to sum
		num /= 10; # Remove the last digit
		}
		
		f (sum == orig) {
		prnt(orig, "is a Strong Number.");
		} ls {
		prnt(orig, "is not a Strong Number.");
		}
		
		end;
		}

XVII. Source Code

Lexical Analyzer

```
class LexerError(Exception):
    """Custom exception for lexical errors."""
    def __init__(self, message, line, column):
        super().__init__(message)
        self.message = message
        self.line = line
        self.column = column

    def __str__(self):
        return f"Lexical Error at line {self.line}, column {self.column}: {self.message}"

# --- Token Types ---
TT_NPT = 'npt'
TT_PRNT = 'prnt'
TT_INT = 'nt'
TT_DOUBLE = 'dbl'
TT_STRING = 'strng'
TT_BOOL = 'bln'
TT_CHAR = 'chr'
TT_F = 'f'
TT_LS = 'ls'
TT_LSF = 'lsf'
TT_SWTCH = 'swtch'
TT_FR = 'fr'
TT_WHl = 'whl'
TT_D = 'd'
TT_MN = 'mn'
TT_CS = 'cs'
TT_DFLT = 'dflt'
TT_BRK = 'brk'
TT_CNST = 'cnst'
TT_TR = 'tr'
TT_FLS = 'fls'
TT_FNCTN = 'fnctn'
TT_RTRN = 'rtrn'
TT_END = 'end'
TT_NULL = 'null'
TT_CNTN = 'cntn'
TT_STRCT = 'strct'
TT_DFSTRCT = 'dfstrct'
TT_VD = 'vd'
TT_IDENTIFIER = 'id'

# Operators and Delimiters
TT_PLUS = '+'
TT_MINUS = '-'
TT_MUL = '*'
TT_DIV = '/'
TT_MOD = '%'
TT_EXP = '**'
TT_EQ = '='
TT_EQTO = '==' #
TT_PLUSEQ = '+='
TT_MINUSEQ = '-='
TT_MULTIEQ = '*='
TT_DIVEQ = '/='
TT_MODEQ = '%='
TT_CONCAT = '`'
TT_LPAREN = '('
TT_RPAREN = ')'
TT_SEMICOLON = ';'
TT_COMMA = ','
TT_COLON = ':'
TT_BLOCK_START = '{'
TT_BLOCK_END = '}'
TT_LT = '<'
TT_GT = '>'
TT_LTEQ = '<='
TT_GTEQ = '>='
TT_NOTEQ = '!='
TT_EOF = 'EOF'
TT_AND = '&&'
TT_OR = '||'
TT_NOT = '!'
TT_INCREMENT = '++'
TT_DECREMENT = '--'
TT_LSQBR = '['
TT_RSQBR = ']'
TT_NEGATIVE = '~'
TT_INTEGERLIT = 'ntlit'
TT_DOUBLELIT = 'dbllit'
TT_STRINGLIT = 'strnglit'
TT_CHARLIT = 'chrlit'
TT_STRCTACCESS = '.'


# --- Character Sets (Based on RE1 & RE2) ---
ZERO = {'0'}
DIGIT = {'1', '2', '3', '4', '5', '6', '7', '8', '9'}
DIGITZERO = DIGIT | ZERO
LOWALPHA = set("abcdefghijklmnopqrstuvwxyz")
UPALPHA = set("ABCDEFGHIJKLMNOPQRSTUVWXYZ")
ALPHA = LOWALPHA | UPALPHA
ALPHA_NUMERIC = ALPHA | DIGITZERO
NEWLINE = {'\n'}
TAB = {'\t'}
WHITESPACE = {' ', '\t', '\n'}

# Delimiter sets based on RE1.png, RE2.png and referenced in TDs
DEL1 = WHITESPACE
DEL2 = {';'}
DEL3 = WHITESPACE | {'{'}
DEL4 = set()
DEL5 = WHITESPACE | {'('}
DEL6 = WHITESPACE | {',', ';', '=', '>', '<', '!'}
DEL7 = {'('}
DEL8 = WHITESPACE | {';'}
DEL9 = WHITESPACE | ALPHA | {'(', ',', ';', ')'}
DEL11 = WHITESPACE | NEWLINE
DEL12 = ALPHA | DIGITZERO | {']', '~'}
DEL13 = WHITESPACE | {';', '{', ')', ']', '<', '>', '=', '|', '&', '+', '-', '/', '*', '%', ',', '\n'}
DEL14 = WHITESPACE | NEWLINE | {"", """, '{'} | ALPHA | DIGITZERO
DEL15 = WHITESPACE | NEWLINE | {'}'}
DEL16 = ALPHA | DIGITZERO | {"", '(', ')'}
DEL17 = WHITESPACE | {';', ',', '}', ')', '+', '-', '*', '/', '%', '=', '>', '<', '!', '&', '|', '['}
DEL18 = WHITESPACE | {';', '{', ')', '&', '|', '+', '-', '*', '/', '%'}
DEL19 = WHITESPACE | {',', '}', ')', '=', '>', '<', '!'}
DEL20 = WHITESPACE | ALPHA | DIGITZERO | {"", """, '{'}
DEL21_for_tilde_op = WHITESPACE | ALPHA | DIGITZERO | {'(', ';', '{', '}'}
DEL22 = WHITESPACE | {',', '}', ']', ')', ':', '+', '-', '*', '/', '%', '=', '>', '<', '!', '&', '|'}
DEL23 = WHITESPACE | {';', ',', '}', ')', '=', '>', '<', '!', '&', '|'}
DEL24 = WHITESPACE | DIGITZERO | ALPHA | {'~', '('} #
DEL25 = WHITESPACE | DIGITZERO | ALPHA | {'~', "", """}
DEL26 = WHITESPACE | {';', ',', '}', ')', '=', '>', '<', '!', ':'}
DEL27 = WHITESPACE | {""}
```

```
DEL28 = WHITESPACE |
DIGITZERO | ALPHA | {"", """, '{'}

IDENTIFIER_CHARS =
ALPHA_NUMERIC | {'_'}
CHAR_CONTENT_CHARS =
{chr(i) for i in range(32, 127) if
chr(i) not in {"'", '\\'}}
STRING_CONTENT_CHARS =
{chr(i) for i in range(32, 127) if
chr(i) not in {'"', '\\'}}


class Token:
    def __init__(self, type,
value=None, line=0, column=0):
        self.type = type; self.value =
value; self.line = line; self.column =
column
    def __repr__(self):
        return f'Token({self.type},
{repr(self.value)}, line {self.line},
col {self.column})'

class Lexer:
    def __init__(self, text):
        self.text = text; self.pos = -1;
self.current_char = None
        self.line = 1; self.column = 0;
self.advance()

    def advance(self):
        if self.current_char == '\n':
self.line += 1; self.column = 0
        self.pos += 1
        if self.pos < len(self.text):
self.current_char =
self.text[self.pos]; self.column += 1
        else: self.current_char = None

    def peek(self, offset=1):
        peek_pos = self.pos + offset
        return self.text[peek_pos] if
peek_pos < len(self.text) else None

    def step_back(self):
        if self.pos < 0: return
        self.pos -= 1
        if self.pos >=0:
            self.current_char =
self.text[self.pos]
            if self.current_char != '\n':
self.column -=1
            if self.column < 1 and
self.current_char != '\n' :
self.column =1
        else: self.current_char = None;
self.pos = -1; self.column = 0

    def make_tokens(self):
        tokens = []; errors = []; state =
0; lexeme = ""
        start_line = 1; start_column = 0
        keywords = { # Keyword
mapping
            "bln": TT_BOOL, "brk":
TT_BRK, "chr": TT_CHAR, "cst":
"TT_CST_DIAGRAM", # TD1 'cst'
            "cntn": TT_CNTN, "d":
TT_D, "dbl": TT_DOUBLE, "dflt":
TT_DFLT,
            "dfstrct": TT_DFSTRCT,
"end": TT_END, "f": TT_F, "fls":
TT_FLS,
            "fnctn": TT_FNCTN, "fr":
TT_FR, "ls": TT_LS, "lsf":
TT_LSF, "mn": TT_MN,
            "npt": TT_NPT, "nt":
TT_INT, "prnt": TT_PRNT, "rtrn":
TT_RTRN,
            "strct": TT_STRCT, "strng":
TT_STRING, "swtch":
TT_SWTCH,
            "tr": TT_TR, "vd": TT_VD,
"whl": TT_WHl,
            "nll": TT_NULL, "cnst":
TT_CNST, "cs": TT_CS # User's
original keywords
        }

        while self.current_char is not
None or state != 0:
            if state == 0: # Initial state
dispatch
                lexeme = ""; start_line =
self.line; start_column =
self.column
                if self.current_char is
None: break
                if self.current_char in
WHITESPACE: self.advance();
continue
                if self.current_char == '#':
state = 161; self.advance(); continue
# TD5 Comment


                if self.current_char in
ALPHA: # Keywords & Identifiers
(TD1, TD2, TD6)
                    if self.current_char ==
'b': state = 1; lexeme +=
self.current_char; self.advance();
continue
                    elif self.current_char
== 'c': state = 8; lexeme +=
self.current_char; self.advance();
continue
                    elif self.current_char
== 'd': state = 21; lexeme +=
self.current_char; self.advance();
continue
                    elif self.current_char
== 'e': state = 36; lexeme +=
self.current_char; self.advance();
continue
                    elif self.current_char
== 'f': state = 40; lexeme +=
self.current_char; self.advance();
continue
                    elif self.current_char
== 'l': state = 52; lexeme +=
self.current_char; self.advance();
continue
                    elif self.current_char
== 'm': state = 57; lexeme +=
self.current_char; self.advance();
continue
                    elif self.current_char
== 'n': state = 60; lexeme +=
self.current_char; self.advance();
continue
                    elif self.current_char
== 'p': state = 66; lexeme +=
self.current_char; self.advance();
continue
                    elif self.current_char
== 'r': state = 71; lexeme +=
self.current_char; self.advance();
continue
                    elif self.current_char
== 's': state = 76; lexeme +=
self.current_char; self.advance();
continue
                    elif self.current_char
== 't': state = 90; lexeme +=
self.current_char; self.advance();
continue
                    elif self.current_char
== 'v': state = 93; lexeme +=
self.current_char; self.advance();
continue
                    elif self.current_char
== 'w': state = 96; lexeme +=
self.current_char; self.advance();
continue
                    else: state = 180;
lexeme += self.current_char;
self.advance(); continue # TD6
Identifier

                elif self.current_char ==
'~': # Could be TD7 number or TD5
operator
                    if self.peek() in
DIGITZERO: # Check if part of a
number
                        state = 195; lexeme
+= self.current_char; self.advance();
continue # TD7 Number
                    else: # Standalone tilde
operator from TD5
```

```
                state = 159; lexeme
+= self.current_char; self.advance();
continue
            elif self.current_char in
DIGITZERO: # Number starting
with digit
                state = 195; lexeme +=
self.current_char; self.advance();
continue # TD7 Number

            elif self.current_char ==
"": state = 167; lexeme +=
self.current_char; self.advance();
continue # TD5 String
            elif self.current_char ==
"""": state = 163; lexeme +=
self.current_char; self.advance();
continue # TD5 Char

            elif self.current_char ==
'+': state = 100; lexeme +=
self.current_char; self.advance();
continue # TD3
            elif self.current_char ==
'-': state = 106; lexeme +=
self.current_char; self.advance();
continue # TD3
            elif self.current_char ==
'*': state = 112; lexeme +=
self.current_char; self.advance();
continue # TD3
            elif self.current_char ==
'/': state = 116; lexeme +=
self.current_char; self.advance();
continue # TD3
            elif self.current_char ==
'%': state = 120; lexeme +=
self.current_char; self.advance();
continue # TD3
            elif self.current_char ==
'=': state = 125; lexeme +=
self.current_char; self.advance();
continue # TD4
            elif self.current_char ==
'>': state = 129; lexeme +=
self.current_char; self.advance();
continue # TD4
            elif self.current_char ==
'<': state = 133; lexeme +=
self.current_char; self.advance();
continue # TD4
            elif self.current_char ==
'!': state = 137; lexeme +=
self.current_char; self.advance();
continue # TD4
            elif self.current_char ==
':': state = 141; lexeme +=
self.current_char; self.advance();
continue # TD4
            elif self.current_char ==
';': state = 143; lexeme +=
self.current_char; self.advance();
continue # TD4
            elif self.current_char ==
'[': state = 145; lexeme +=
self.current_char; self.advance();
continue # TD4
            elif self.current_char ==
']': state = 147; lexeme +=
self.current_char; self.advance();
continue # TD4
            elif self.current_char ==
'{': state = 149; lexeme +=
self.current_char; self.advance();
continue # TD4
            elif self.current_char ==
'}': state = 151; lexeme +=
self.current_char; self.advance();
continue # TD5
            elif self.current_char ==
'(': state = 153; lexeme +=
self.current_char; self.advance();
continue # TD5
            elif self.current_char ==
')': state = 155; lexeme +=
self.current_char; self.advance();
continue # TD5
            elif self.current_char ==
',': state = 157; lexeme +=
self.current_char; self.advance();
continue # TD5
            elif self.current_char ==
'&': state = 170; lexeme +=
self.current_char; self.advance();
continue # TD5 &&
            elif self.current_char ==
'|': state = 173; lexeme +=
self.current_char; self.advance();
continue # TD5 ||
            elif self.current_char ==
'.': state = 176; lexeme +=
self.current_char; self.advance();
continue # TD5 .
            elif self.current_char ==
'`': state = 178; lexeme +=
self.current_char; self.advance();
continue # TD5 `

            else:

errors.append(LexerError(f"Illegal
character: '{self.current_char}'",
start_line, start_column))
                self.advance(); state =
0; continue

        # --- Keyword States (TD1
& TD2, States 1-99) ---
        elif state == 1: # lexeme "b"
            if self.current_char == 'l':
state = 2; lexeme +=
self.current_char; self.advance()
            elif self.current_char ==
'r': state = 5; lexeme +=
self.current_char; self.advance()
            elif self.current_char is
not None and self.current_char in
IDENTIFIER_CHARS: state = 180;
lexeme += self.current_char;
self.advance()
            else:
                if self.current_char is
None or self.current_char in
DEL22:
tokens.append(Token(TT_IDENTIF
IER, lexeme, start_line,
start_column)); state = 0;
                if self.current_char is
not None: self.step_back()
                else:
errors.append(LexerError(f"Invalid
char after 'b'", self.line,
self.column)); state = 0;
self.advance()
        elif state == 2: # lexeme "bl"
            if self.current_char == 'n':
state = 3; lexeme +=
self.current_char; self.advance()
            elif self.current_char is
not None and self.current_char in
IDENTIFIER_CHARS: state = 180;
lexeme += self.current_char;
self.advance()
            else:
                if self.current_char is
None or self.current_char in
DEL22:
tokens.append(Token(TT_IDENTIF
IER, lexeme, start_line,
start_column)); state = 0;
                if self.current_char is
not None: self.step_back()
                else:
errors.append(LexerError(f"Invalid
char after 'bl'", self.line,
self.column)); state = 0;
self.advance()
        elif state == 3: # lexeme
"bln"
            if self.current_char is
None or self.current_char in DEL1:
state = 4; # Delimiter check
            elif self.current_char is
not None and self.current_char in
IDENTIFIER_CHARS: state = 180;
lexeme += self.current_char;
self.advance()
            else:
errors.append(LexerError(f"Invalid
delimiter for 'bln'", self.line,
self.column)); state = 0;
self.advance()
```

```
        elif state == 4:
tokens.append(Token(TT_BOOL,
lexeme, start_line, start_column));
state = 0;
        if self.current_char is not
None: self.step_back() # Step back
after tokenizing

        elif state == 5: # lexeme "br"
            if self.current_char == 'k':
state = 6; lexeme +=
self.current_char; self.advance()
            elif self.current_char is
not None and self.current_char in
IDENTIFIER_CHARS: state = 180;
lexeme += self.current_char;
self.advance()
            else:
                if self.current_char is
None or self.current_char in
DEL22:
tokens.append(Token(TT_IDENTIF
IER, lexeme, start_line,
start_column)); state = 0;
                if self.current_char is
not None: self.step_back()
                else:
errors.append(LexerError(f"Invalid
char after 'br'", self.line,
self.column)); state = 0;
self.advance()
        elif state == 6: # lexeme
"brk"
            if self.current_char is
None or self.current_char in DEL2:
state = 7;
            elif self.current_char is
not None and self.current_char in
IDENTIFIER_CHARS: state = 180;
lexeme += self.current_char;
self.advance()
            else:
errors.append(LexerError(f"Invalid
delimiter for 'brk'", self.line,
self.column)); state = 0;
self.advance()
        elif state == 7:
tokens.append(Token(TT_BRK,
lexeme, start_line, start_column));
state = 0;
        if self.current_char is not
None: self.step_back()

        elif state == 8: # lexeme "c"
            if self.current_char == 'h':
state = 9; lexeme +=
self.current_char; self.advance()   #
for chr
            elif self.current_char ==
'n': state = 12; lexeme +=
self.current_char; self.advance() #
for cntn (TD1) or cnst (user code)
            elif self.current_char ==
's': state = 16; lexeme +=
self.current_char; self.advance() #
for cst (TD1) or cs (user code)
            elif self.current_char is
not None and self.current_char in
IDENTIFIER_CHARS: state = 180;
lexeme += self.current_char;
self.advance()
            else:
                if self.current_char is
None or self.current_char in
DEL22:
tokens.append(Token(TT_IDENTIF
IER, lexeme, start_line,
start_column)); state = 0;
                if self.current_char is
not None: self.step_back()
                else:
errors.append(LexerError(f"Invalid
char after 'c'", self.line,
self.column)); state = 0;
self.advance()
        elif state == 9: # lexeme
"ch"
            if self.current_char == 'r':
state = 10; lexeme +=
self.current_char; self.advance()
            elif self.current_char is
not None and self.current_char in
IDENTIFIER_CHARS: state = 180;
lexeme += self.current_char;
self.advance()
            else:
                if self.current_char is
None or self.current_char in
DEL22:
tokens.append(Token(TT_IDENTIF
IER, lexeme, start_line,
start_column)); state = 0;
                if self.current_char is
not None: self.step_back()
                else:
errors.append(LexerError(f"Invalid
char after 'ch'", self.line,
self.column)); state = 0;
self.advance()
        elif state == 10: # lexeme
"chr"
            if self.current_char is
None or self.current_char in DEL1:
state = 11;
            elif self.current_char is
not None and self.current_char in
IDENTIFIER_CHARS: state = 180;
lexeme += self.current_char;
self.advance()
            else:
errors.append(LexerError(f"Invalid
delimiter for 'chr'", self.line,
self.column)); state = 0;
self.advance()
        elif state == 11:
tokens.append(Token(TT_CHAR,
lexeme, start_line, start_column));
state = 0;
        if self.current_char is not
None: self.step_back()

        elif state == 12: # lexeme
"cn"
            if self.current_char == 't':
state = 13; lexeme +=
self.current_char; self.advance() #
For TD1 'cntn'
            elif self.current_char ==
's': state = 701; lexeme +=
self.current_char; self.advance() #
For user 'cnst' -> new internal state
701
            elif self.current_char is
not None and self.current_char in
IDENTIFIER_CHARS: state = 180;
lexeme += self.current_char;
self.advance()
            else:
                if self.current_char is
None or self.current_char in
DEL22:
tokens.append(Token(TT_IDENTIF
IER, lexeme, start_line,
start_column)); state = 0;
                if self.current_char is
not None: self.step_back()
                else:
errors.append(LexerError(f"Invalid
char after 'cn'", self.line,
self.column)); state = 0;
self.advance()
        elif state == 13: # lexeme
"cnt" for cntn
            if self.current_char == 'n':
state = 14; lexeme +=
self.current_char; self.advance()
            elif self.current_char is
not None and self.current_char in
IDENTIFIER_CHARS: state = 180;
lexeme += self.current_char;
self.advance()
            else:
                if self.current_char is
None or self.current_char in
DEL22:
tokens.append(Token(TT_IDENTIF
IER, lexeme, start_line,
start_column)); state = 0;
                if self.current_char is
not None: self.step_back()
                else:
errors.append(LexerError(f"Invalid
```

```
char after 'cnt'", self.line,
self.column)); state = 0;
self.advance()
        elif state == 14: # lexeme
"cntn"
            if self.current_char is
None or self.current_char in DEL2:
state = 15;
            elif self.current_char is
not None and self.current_char in
IDENTIFIER_CHARS: state = 180;
lexeme += self.current_char;
self.advance()
            else:
errors.append(LexerError(f"Invalid
delimiter for 'cntn'", self.line,
self.column)); state = 0;
self.advance()
        elif state == 15:
tokens.append(Token(TT_CNTN,
lexeme, start_line, start_column));
state = 0;
        if self.current_char is not
None: self.step_back()

        elif state == 16: # lexeme
"cs"
            if self.current_char == 't':
state = 17; lexeme +=
self.current_char; self.advance() #
For TD1 'cst'
            elif self.current_char is
None or self.current_char in
DEL26: # For user 'cs' keyword

tokens.append(Token(TT_CS,
lexeme, start_line, start_column));
state = 0;
                if self.current_char is
not None: self.step_back()
            elif self.current_char is
not None and self.current_char in
IDENTIFIER_CHARS: state = 180;
lexeme += self.current_char;
self.advance()
            else:
                if self.current_char is
None or self.current_char in
DEL22:
tokens.append(Token(TT_IDENTIF
IER, lexeme, start_line,
start_column)); state = 0;
                if self.current_char is
not None: self.step_back()
                else:
errors.append(LexerError(f"Invalid
char or delimiter after 'cs'", self.line,
self.column)); state = 0;
self.advance()
        elif state == 17: # lexeme
"cst"
            if self.current_char is
None or self.current_char in DEL1:
state = 18;
            elif self.current_char is
not None and self.current_char in
IDENTIFIER_CHARS: state = 180;
lexeme += self.current_char;
self.advance()
            else:
errors.append(LexerError(f"Invalid
delimiter for 'cst'", self.line,
self.column)); state = 0;
self.advance()
        elif state == 18:
tokens.append(Token("TT_CST_DI
AGRAM", lexeme, start_line,
start_column)); state = 0;
        if self.current_char is not
None: self.step_back()

        # ... (States 21 to 99 for
other keywords from TD1 & TD2,
following similar pattern) ...
        # Example for 'd' and 'dbl'
        elif state == 21: # lexeme
"d"
            if self.current_char == 'b':
state = 23; lexeme +=
self.current_char; self.advance()  #
for dbl
            elif self.current_char ==
'f': state = 26; lexeme +=
self.current_char; self.advance()  #
for dflt
            elif self.current_char ==
's': state = 30; lexeme +=
self.current_char; self.advance()  #
for dfstrct
            elif self.current_char is
None or self.current_char in DEL3:
state = 22; # for 'd' keyword
            elif self.current_char is
not None and self.current_char in
IDENTIFIER_CHARS: state = 180;
lexeme += self.current_char;
self.advance()
            else:
                if self.current_char is
None or self.current_char in
DEL22:
tokens.append(Token(TT_IDENTIF
IER, lexeme, start_line,
start_column)); state = 0;
                if self.current_char is
not None: self.step_back()
                else:
errors.append(LexerError(f"Invalid
char after 'd'", self.line,
self.column)); state = 0;
self.advance()
        elif state == 22:
tokens.append(Token(TT_D,
lexeme, start_line, start_column));
state = 0;
        if self.current_char is not
None: self.step_back()
        elif state == 23: # lexeme
"db"
            if self.current_char == 'l':
state = 24; lexeme +=
self.current_char; self.advance()
            elif self.current_char is
not None and self.current_char in
IDENTIFIER_CHARS: state = 180;
lexeme += self.current_char;
self.advance()
            else:
                if self.current_char is
None or self.current_char in
DEL22:
tokens.append(Token(TT_IDENTIF
IER, lexeme, start_line,
start_column)); state = 0;
                if self.current_char is
not None: self.step_back()
                else:
errors.append(LexerError(f"Invalid
char after 'db'", self.line,
self.column)); state = 0;
self.advance()
        elif state == 24: # lexeme
"dbl"
            if self.current_char is
None or self.current_char in DEL1:
state = 25;
            elif self.current_char is
not None and self.current_char in
IDENTIFIER_CHARS: state = 180;
lexeme += self.current_char;
self.advance()
            else:
errors.append(LexerError(f"Invalid
delimiter for 'dbl'", self.line,
self.column)); state = 0;
self.advance()
        elif state == 25:
tokens.append(Token(TT_DOUBL
E, lexeme, start_line,
start_column)); state = 0;
        if self.current_char is not
None: self.step_back()

        # ... (dflt: 26-29, dfstrct:
30-35)
        elif state == 26: # dflt path
            if self.current_char == 'l':
state = 27; lexeme +=
self.current_char; self.advance()
            elif self.current_char is
not None and self.current_char in
IDENTIFIER_CHARS: state = 180;
```

```
lexeme += self.current_char;
self.advance()
            else:
                if self.current_char is
None or self.current_char in
DEL22:
tokens.append(Token(TT_IDENTIF
IER, lexeme, start_line,
start_column)); state = 0;
                if self.current_char is
not None: self.step_back()
                else:
errors.append(LexerError(f"Invalid
char after 'df'", self.line,
self.column)); state = 0;
self.advance()
        elif state == 27:
            if self.current_char == 't':
state = 28; lexeme +=
self.current_char; self.advance()
            elif self.current_char is
not None and self.current_char in
IDENTIFIER_CHARS: state = 180;
lexeme += self.current_char;
self.advance()
            else:
                if self.current_char is
None or self.current_char in
DEL22:
tokens.append(Token(TT_IDENTIF
IER, lexeme, start_line,
start_column)); state = 0;
                if self.current_char is
not None: self.step_back()
                else:
errors.append(LexerError(f"Invalid
char after 'dfl'", self.line,
self.column)); state = 0;
self.advance()
        elif state == 28:
            if self.current_char is
None or self.current_char in DEL4:
state = 29; # DEL4 is empty set, so
essentially means no specific char
needed
            elif self.current_char is
not None and self.current_char in
IDENTIFIER_CHARS: state = 180;
lexeme += self.current_char;
self.advance()
            else:
errors.append(LexerError(f"Invalid
delimiter for 'dflt'", self.line,
self.column)); state = 0;
self.advance()
        elif state == 29:
tokens.append(Token(TT_DFLT,
lexeme, start_line, start_column));
state = 0;
        if self.current_char is not
None: self.step_back()

        elif state == 30: # dfstrct
path
            if self.current_char == 't':
state = 31; lexeme +=
self.current_char; self.advance()
            # ... (fallback logic similar
to above)
            elif self.current_char is
not None and self.current_char in
IDENTIFIER_CHARS: state = 180;
lexeme += self.current_char;
self.advance()
            else:
                if self.current_char is
None or self.current_char in
DEL22:
tokens.append(Token(TT_IDENTIF
IER, lexeme, start_line,
start_column)); state = 0;
                if self.current_char is
not None: self.step_back()
                else:
errors.append(LexerError(f"Invalid
char after 'ds'", self.line,
self.column)); state = 0;
self.advance()
        elif state == 31:
            if self.current_char == 'r':
state = 32; lexeme +=
self.current_char; self.advance()
            elif self.current_char is
not None and self.current_char in
IDENTIFIER_CHARS: state = 180;
lexeme += self.current_char;
self.advance()
            else:
                if self.current_char is
None or self.current_char in
DEL22:
tokens.append(Token(TT_IDENTIF
IER, lexeme, start_line,
start_column)); state = 0;
                if self.current_char is
not None: self.step_back()
                else:
errors.append(LexerError(f"Invalid
char after 'dst'", self.line,
self.column)); state = 0;
self.advance()
        elif state == 32:
            if self.current_char == 'c':
state = 33; lexeme +=
self.current_char; self.advance()
            elif self.current_char is
not None and self.current_char in
IDENTIFIER_CHARS: state = 180;
lexeme += self.current_char;
self.advance()
            else:
                if self.current_char is
None or self.current_char in
DEL22:
tokens.append(Token(TT_IDENTIF
IER, lexeme, start_line,
start_column)); state = 0;
                if self.current_char is
not None: self.step_back()
                else:
errors.append(LexerError(f"Invalid
char after 'dstr'", self.line,
self.column)); state = 0;
self.advance()
        elif state == 33:
            if self.current_char == 't':
state = 34; lexeme +=
self.current_char; self.advance()
            elif self.current_char is
not None and self.current_char in
IDENTIFIER_CHARS: state = 180;
lexeme += self.current_char;
self.advance()
            else:
                if self.current_char is
None or self.current_char in
DEL22:
tokens.append(Token(TT_IDENTIF
IER, lexeme, start_line,
start_column)); state = 0;
                if self.current_char is
not None: self.step_back()
                else:
errors.append(LexerError(f"Invalid
char after 'dstrc'", self.line,
self.column)); state = 0;
self.advance()
        elif state == 34:
            if self.current_char is
None or self.current_char in DEL1:
state = 35;
            elif self.current_char is
not None and self.current_char in
IDENTIFIER_CHARS: state = 180;
lexeme += self.current_char;
self.advance()
            else:
errors.append(LexerError(f"Invalid
delimiter for 'dfstrct'", self.line,
self.column)); state = 0;
self.advance()
        elif state == 35:
tokens.append(Token(TT_DFSTRC
T, lexeme, start_line,
start_column)); state = 0;
        if self.current_char is not
None: self.step_back()

        # end: 36-39
        elif state == 36: # lexeme
"e"
```

```
            if self.current_char == 'n': state = 37; lexeme += self.current_char; self.advance()
            elif self.current_char is not None and self.current_char in IDENTIFIER_CHARS: state = 180; lexeme += self.current_char; self.advance()
            else:
                if self.current_char is None or self.current_char in DEL22: tokens.append(Token(TT_IDENTIFIER, lexeme, start_line, start_column)); state = 0;
                if self.current_char is not None: self.step_back()
                else: errors.append(LexerError(f"Invalid char after 'e'", self.line, self.column)); state = 0; self.advance()
        elif state == 37: # lexeme "en"
            if self.current_char == 'd': state = 38; lexeme += self.current_char; self.advance()
            elif self.current_char is not None and self.current_char in IDENTIFIER_CHARS: state = 180; lexeme += self.current_char; self.advance()
            else:
                if self.current_char is None or self.current_char in DEL22: tokens.append(Token(TT_IDENTIFIER, lexeme, start_line, start_column)); state = 0;
                if self.current_char is not None: self.step_back()
                else: errors.append(LexerError(f"Invalid char after 'en'", self.line, self.column)); state = 0; self.advance()
        elif state == 38: # lexeme "end"
            if self.current_char is None or self.current_char in DEL2: state = 39;
            elif self.current_char is not None and self.current_char in IDENTIFIER_CHARS: state = 180; lexeme += self.current_char; self.advance()
            else: errors.append(LexerError(f"Invalid delimiter for 'end'", self.line, self.column)); state = 0; self.advance()
        elif state == 39: tokens.append(Token(TT_END, lexeme, start_line, start_column)); state = 0;
            if self.current_char is not None: self.step_back()

        # f, fls, fnctn, fr: 40-51
        elif state == 40: # lexeme "f"
            if self.current_char == 'l': state = 42; lexeme += self.current_char; self.advance()  # for fls
            elif self.current_char == 'n': state = 45; lexeme += self.current_char; self.advance()  # for fnctn
            elif self.current_char == 'r': state = 50; lexeme += self.current_char; self.advance()  # for fr
            elif self.current_char is None or self.current_char in DEL5: state = 41; # for 'f' keyword
            elif self.current_char is not None and self.current_char in IDENTIFIER_CHARS: state = 180; lexeme += self.current_char; self.advance()
            else:
                if self.current_char is None or self.current_char in DEL22: tokens.append(Token(TT_IDENTIFIER, lexeme, start_line, start_column)); state = 0;
                if self.current_char is not None: self.step_back()
                else: errors.append(LexerError(f"Invalid char after 'f'", self.line, self.column)); state = 0; self.advance()
        elif state == 41: tokens.append(Token(TT_F, lexeme, start_line, start_column)); state = 0;
            if self.current_char is not None: self.step_back()
        elif state == 42: # fls path
            if self.current_char == 's': state = 43; lexeme += self.current_char; self.advance()
            elif self.current_char is not None and self.current_char in IDENTIFIER_CHARS: state = 180; lexeme += self.current_char; self.advance()
            else:
                if self.current_char is None or self.current_char in DEL22: tokens.append(Token(TT_IDENTIFIER, lexeme, start_line, start_column)); state = 0;
                if self.current_char is not None: self.step_back()
                else: errors.append(LexerError(f"Invalid char after 'fl'", self.line, self.column)); state = 0; self.advance()
        elif state == 43:
            if self.current_char is None or self.current_char in DEL6: state = 44;
            elif self.current_char is not None and self.current_char in IDENTIFIER_CHARS: state = 180; lexeme += self.current_char; self.advance()
            else: errors.append(LexerError(f"Invalid delimiter for 'fls'", self.line, self.column)); state = 0; self.advance()
        elif state == 44: tokens.append(Token(TT_FLS, lexeme, start_line, start_column)); state = 0;
            if self.current_char is not None: self.step_back()
        elif state == 45: # fnctn path
            if self.current_char == 'c': state = 46; lexeme += self.current_char; self.advance()
            elif self.current_char is not None and self.current_char in IDENTIFIER_CHARS: state = 180; lexeme += self.current_char; self.advance()
            else:
                if self.current_char is None or self.current_char in DEL22: tokens.append(Token(TT_IDENTIFIER, lexeme, start_line, start_column)); state = 0;
                if self.current_char is not None: self.step_back()
                else: errors.append(LexerError(f"Invalid char after 'fn'", self.line, self.column)); state = 0; self.advance()
        elif state == 46:
            if self.current_char == 't': state = 47; lexeme += self.current_char; self.advance()
            elif self.current_char is not None and self.current_char in IDENTIFIER_CHARS: state = 180;
```

```
lexeme += self.current_char; self.advance()
            else:
                if self.current_char is None or self.current_char in DEL22: tokens.append(Token(TT_IDENTIFIER, lexeme, start_line, start_column)); state = 0;
                if self.current_char is not None: self.step_back()
                else: errors.append(LexerError(f"Invalid char after 'fnc'", self.line, self.column)); state = 0; self.advance()
        elif state == 47:
            if self.current_char == 'n': state = 48; lexeme += self.current_char; self.advance()
            elif self.current_char is not None and self.current_char in IDENTIFIER_CHARS: state = 180; lexeme += self.current_char; self.advance()
            else:
                if self.current_char is None or self.current_char in DEL22: tokens.append(Token(TT_IDENTIFIER, lexeme, start_line, start_column)); state = 0;
                if self.current_char is not None: self.step_back()
                else: errors.append(LexerError(f"Invalid char after 'fnct'", self.line, self.column)); state = 0; self.advance()
        elif state == 48:
            if self.current_char is None or self.current_char in DEL1: state = 49;
            elif self.current_char is not None and self.current_char in IDENTIFIER_CHARS: state = 180; lexeme += self.current_char; self.advance()
            else: errors.append(LexerError(f"Invalid delimiter for 'fnctn'", self.line, self.column)); state = 0; self.advance()
        elif state == 49: tokens.append(Token(TT_FNCTN, lexeme, start_line, start_column)); state = 0;
        if self.current_char is not None: self.step_back()
        elif state == 50: # fr path
            if self.current_char is None or self.current_char in DEL5: state = 51;
            elif self.current_char is not None and self.current_char in IDENTIFIER_CHARS: state = 180; lexeme += self.current_char; self.advance() # 'fr' could be start of ID
            else: errors.append(LexerError(f"Invalid delimiter for 'fr'", self.line, self.column)); state = 0; self.advance()
        elif state == 51: tokens.append(Token(TT_FR, lexeme, start_line, start_column)); state = 0;
        if self.current_char is not None: self.step_back()

        # ls, lsf: 52-56
        elif state == 52: # lexeme "l"
            if self.current_char == 's': state = 53; lexeme += self.current_char; self.advance()
            elif self.current_char is not None and self.current_char in IDENTIFIER_CHARS: state = 180; lexeme += self.current_char; self.advance()
            else:
                if self.current_char is None or self.current_char in DEL22: tokens.append(Token(TT_IDENTIFIER, lexeme, start_line, start_column)); state = 0;
                if self.current_char is not None: self.step_back()
                else: errors.append(LexerError(f"Invalid char after 'l'", self.line, self.column)); state = 0; self.advance()
        elif state == 53: # lexeme "ls"
            if self.current_char == 'f': state = 55; lexeme += self.current_char; self.advance() # for lsf
            elif self.current_char is None or self.current_char in DEL3: state = 54; # for ls
            elif self.current_char is not None and self.current_char in IDENTIFIER_CHARS: state = 180; lexeme += self.current_char; self.advance()
            else: errors.append(LexerError(f"Invalid char or delimiter for 'ls'/'lsf'", self.line, self.column)); state = 0; self.advance()
        elif state == 54: tokens.append(Token(TT_LS, lexeme, start_line, start_column)); state = 0;
        if self.current_char is not None: self.step_back()
        elif state == 55: # lexeme "lsf"
            if self.current_char is None or self.current_char in DEL5: state = 56;
            elif self.current_char is not None and self.current_char in IDENTIFIER_CHARS: state = 180; lexeme += self.current_char; self.advance()
            else: errors.append(LexerError(f"Invalid delimiter for 'lsf'", self.line, self.column)); state = 0; self.advance()
        elif state == 56: tokens.append(Token(TT_LSF, lexeme, start_line, start_column)); state = 0;
        if self.current_char is not None: self.step_back()

        # mn: 57-59
        elif state == 57: # lexeme "m"
            if self.current_char == 'n': state = 58; lexeme += self.current_char; self.advance()
            elif self.current_char is not None and self.current_char in IDENTIFIER_CHARS: state = 180; lexeme += self.current_char; self.advance()
            else:
                if self.current_char is None or self.current_char in DEL22: tokens.append(Token(TT_IDENTIFIER, lexeme, start_line, start_column)); state = 0;
                if self.current_char is not None: self.step_back()
                else: errors.append(LexerError(f"Invalid char after 'm'", self.line, self.column)); state = 0; self.advance()
        elif state == 58: # lexeme "mn"
            if self.current_char is None or self.current_char in DEL7: state = 59;
```

```
            elif self.current_char is
not None and self.current_char in
IDENTIFIER_CHARS: state = 180;
lexeme += self.current_char;
self.advance()
            else:
errors.append(LexerError(f"Invalid
delimiter for 'mn'", self.line,
self.column)); state = 0;
self.advance()
        elif state == 59:
tokens.append(Token(TT_MN,
lexeme, start_line, start_column));
state = 0;
        if self.current_char is not
None: self.step_back()

        # npt, nt: 60-65
        elif state == 60: # lexeme
"n"
            if self.current_char == 'p':
state = 61; lexeme +=
self.current_char; self.advance() #
for npt
            elif self.current_char ==
't': state = 64; lexeme +=
self.current_char; self.advance() #
for nt
            # Add path for 'nll' if it
starts with 'n' and is distinct
            elif self.current_char == 'l'
and self.peek() == 'l': # Check for
'nll'
                state = 704; lexeme +=
self.current_char; self.advance() #
Go to internal state for 'nl' part of
'nll'
            elif self.current_char is
not None and self.current_char in
IDENTIFIER_CHARS: state = 180;
lexeme += self.current_char;
self.advance()
            else:
                if self.current_char is
None or self.current_char in
DEL22:
tokens.append(Token(TT_IDENTIF
IER, lexeme, start_line,
start_column)); state = 0;
                if self.current_char is
not None: self.step_back()
                else:
errors.append(LexerError(f"Invalid
char after 'n'", self.line,
self.column)); state = 0;
self.advance()
        elif state == 61: # npt path
            if self.current_char == 't':
state = 62; lexeme +=
self.current_char; self.advance()
            elif self.current_char is
not None and self.current_char in
IDENTIFIER_CHARS: state = 180;
lexeme += self.current_char;
self.advance()
            else:
                if self.current_char is
None or self.current_char in
DEL22:
tokens.append(Token(TT_IDENTIF
IER, lexeme, start_line,
start_column)); state = 0;
                if self.current_char is
not None: self.step_back()
                else:
errors.append(LexerError(f"Invalid
char after 'np'", self.line,
self.column)); state = 0;
self.advance()
        elif state == 62:
            if self.current_char is
None or self.current_char in DEL7:
state = 63;
            elif self.current_char is
not None and self.current_char in
IDENTIFIER_CHARS: state = 180;
lexeme += self.current_char;
self.advance()
            else:
errors.append(LexerError(f"Invalid
delimiter for 'npt'", self.line,
self.column)); state = 0;
self.advance()
        elif state == 63:
tokens.append(Token(TT_NPT,
lexeme, start_line, start_column));
state = 0;
        if self.current_char is not
None: self.step_back()
        elif state == 64: # nt path
            if self.current_char is
None or self.current_char in DEL1:
state = 65;
            elif self.current_char is
not None and self.current_char in
IDENTIFIER_CHARS: state = 180;
lexeme += self.current_char;
self.advance()
            else:
errors.append(LexerError(f"Invalid
delimiter for 'nt'", self.line,
self.column)); state = 0;
self.advance()
        elif state == 65:
tokens.append(Token(TT_INT,
lexeme, start_line, start_column));
state = 0;
        if self.current_char is not
None: self.step_back()

        # prnt: 66-70
        elif state == 66: # lexeme
"p"
            if self.current_char == 'r':
state = 67; lexeme +=
self.current_char; self.advance()
            elif self.current_char is
not None and self.current_char in
IDENTIFIER_CHARS: state = 180;
lexeme += self.current_char;
self.advance()
            else:
                if self.current_char is
None or self.current_char in
DEL22:
tokens.append(Token(TT_IDENTIF
IER, lexeme, start_line,
start_column)); state = 0;
                if self.current_char is
not None: self.step_back()
                else:
errors.append(LexerError(f"Invalid
char after 'p'", self.line,
self.column)); state = 0;
self.advance()
        elif state == 67: # lexeme
"pr"
            if self.current_char == 'n':
state = 68; lexeme +=
self.current_char; self.advance()
            elif self.current_char is
not None and self.current_char in
IDENTIFIER_CHARS: state = 180;
lexeme += self.current_char;
self.advance()
            else:
                if self.current_char is
None or self.current_char in
DEL22:
tokens.append(Token(TT_IDENTIF
IER, lexeme, start_line,
start_column)); state = 0;
                if self.current_char is
not None: self.step_back()
                else:
errors.append(LexerError(f"Invalid
char after 'pr'", self.line,
self.column)); state = 0;
self.advance()
        elif state == 68: # lexeme
"prn"
            if self.current_char == 't':
state = 69; lexeme +=
self.current_char; self.advance()
            elif self.current_char is
not None and self.current_char in
IDENTIFIER_CHARS: state = 180;
lexeme += self.current_char;
self.advance()
            else:
                if self.current_char is
None or self.current_char in
```

```
DEL22:
tokens.append(Token(TT_IDENTIF
IER, lexeme, start_line,
start_column)); state = 0;
                if self.current_char is
not None: self.step_back()
                else:
errors.append(LexerError(f"Invalid
char after 'prn'", self.line,
self.column)); state = 0;
self.advance()
            elif state == 69: # lexeme
"prnt"
                if self.current_char is
None or self.current_char in DEL7:
state = 70;
                elif self.current_char is
not None and self.current_char in
IDENTIFIER_CHARS: state = 180;
lexeme += self.current_char;
self.advance()
                else:
errors.append(LexerError(f"Invalid
delimiter for 'prnt'", self.line,
self.column)); state = 0;
self.advance()
            elif state == 70:
tokens.append(Token(TT_PRNT,
lexeme, start_line, start_column));
state = 0;
            if self.current_char is not
None: self.step_back()

            # rtrn: 71-75
            elif state == 71: # lexeme "r"
                if self.current_char == 't':
state = 72; lexeme +=
self.current_char; self.advance()
                elif self.current_char is
not None and self.current_char in
IDENTIFIER_CHARS: state = 180;
lexeme += self.current_char;
self.advance()
                else:
                    if self.current_char is
None or self.current_char in
DEL22:
tokens.append(Token(TT_IDENTIF
IER, lexeme, start_line,
start_column)); state = 0;
                if self.current_char is
not None: self.step_back()
                else:
errors.append(LexerError(f"Invalid
char after 'r'", self.line,
self.column)); state = 0;
self.advance()
            elif state == 72: # lexeme
"rt"
                if self.current_char == 'r':
state = 73; lexeme +=
self.current_char; self.advance()
                elif self.current_char is
not None and self.current_char in
IDENTIFIER_CHARS: state = 180;
lexeme += self.current_char;
self.advance()
                else:
                    if self.current_char is
None or self.current_char in
DEL22:
tokens.append(Token(TT_IDENTIF
IER, lexeme, start_line,
start_column)); state = 0;
                if self.current_char is
not None: self.step_back()
                else:
errors.append(LexerError(f"Invalid
char after 'rt'", self.line,
self.column)); state = 0;
self.advance()
            elif state == 73: # lexeme
"rtr"
                if self.current_char == 'n':
state = 74; lexeme +=
self.current_char; self.advance()
                elif self.current_char is
not None and self.current_char in
IDENTIFIER_CHARS: state = 180;
lexeme += self.current_char;
self.advance()
                else:
                    if self.current_char is
None or self.current_char in
DEL22:
tokens.append(Token(TT_IDENTIF
IER, lexeme, start_line,
start_column)); state = 0;
                if self.current_char is
not None: self.step_back()
                else:
errors.append(LexerError(f"Invalid
char after 'rtr'", self.line,
self.column)); state = 0;
self.advance()
            elif state == 74: # lexeme
"rtrn"
                if self.current_char is
None or self.current_char in DEL8:
state = 75;
                elif self.current_char is
not None and self.current_char in
IDENTIFIER_CHARS: state = 180;
lexeme += self.current_char;
self.advance()
                else:
errors.append(LexerError(f"Invalid
delimiter for 'rtrn'", self.line,
self.column)); state = 0;
self.advance()
            elif state == 75:
tokens.append(Token(TT_RTRN,
lexeme, start_line, start_column));
state = 0;
            if self.current_char is not
None: self.step_back()

            # strct, strng, swtch: 76-89
            elif state == 76: # lexeme "s"
                if self.current_char == 't':
state = 77; lexeme +=
self.current_char; self.advance() #
for strct or strng
                elif self.current_char ==
'w': state = 85; lexeme +=
self.current_char; self.advance() #
for swtch
                elif self.current_char is
not None and self.current_char in
IDENTIFIER_CHARS: state = 180;
lexeme += self.current_char;
self.advance()
                else:
                    if self.current_char is
None or self.current_char in
DEL22:
tokens.append(Token(TT_IDENTIF
IER, lexeme, start_line,
start_column)); state = 0;
                if self.current_char is
not None: self.step_back()
                else:
errors.append(LexerError(f"Invalid
char after 's'", self.line,
self.column)); state = 0;
self.advance()
            elif state == 77: # lexeme
"st"
                if self.current_char == 'r':
state = 78; lexeme +=
self.current_char; self.advance()
                elif self.current_char is
not None and self.current_char in
IDENTIFIER_CHARS: state = 180;
lexeme += self.current_char;
self.advance()
                else:
                    if self.current_char is
None or self.current_char in
DEL22:
tokens.append(Token(TT_IDENTIF
IER, lexeme, start_line,
start_column)); state = 0;
                if self.current_char is
not None: self.step_back()
                else:
errors.append(LexerError(f"Invalid
char after 'st'", self.line,
self.column)); state = 0;
self.advance()
```

```
        elif state == 78: # lexeme "str"
            if self.current_char == 'c': state = 79; lexeme += self.current_char; self.advance() # for strct
            elif self.current_char == 'n': state = 82; lexeme += self.current_char; self.advance() # for strng
            elif self.current_char is not None and self.current_char in IDENTIFIER_CHARS: state = 180; lexeme += self.current_char; self.advance()
            else:
                if self.current_char is None or self.current_char in DEL22: tokens.append(Token(TT_IDENTIFIER, lexeme, start_line, start_column)); state = 0;
                if self.current_char is not None: self.step_back()
                else: errors.append(LexerError(f"Invalid char after 'str'", self.line, self.column)); state = 0; self.advance()
        elif state == 79: # strct path
            if self.current_char == 't': state = 80; lexeme += self.current_char; self.advance()
            elif self.current_char is not None and self.current_char in IDENTIFIER_CHARS: state = 180; lexeme += self.current_char; self.advance()
            else:
                if self.current_char is None or self.current_char in DEL22: tokens.append(Token(TT_IDENTIFIER, lexeme, start_line, start_column)); state = 0;
                if self.current_char is not None: self.step_back()
                else: errors.append(LexerError(f"Invalid char after 'strc'", self.line, self.column)); state = 0; self.advance()
        elif state == 80:
            if self.current_char is None or self.current_char in DEL1: state = 81;
            elif self.current_char is not None and self.current_char in IDENTIFIER_CHARS: state = 180; lexeme += self.current_char; self.advance()
            else: errors.append(LexerError(f"Invalid delimiter for 'strct'", self.line, self.column)); state = 0; self.advance()
        elif state == 81: tokens.append(Token(TT_STRCT, lexeme, start_line, start_column)); state = 0;
            if self.current_char is not None: self.step_back()
        elif state == 82: # strng path
            if self.current_char == 'g': state = 83; lexeme += self.current_char; self.advance()
            elif self.current_char is not None and self.current_char in IDENTIFIER_CHARS: state = 180; lexeme += self.current_char; self.advance()
            else:
                if self.current_char is None or self.current_char in DEL22: tokens.append(Token(TT_IDENTIFIER, lexeme, start_line, start_column)); state = 0;
                if self.current_char is not None: self.step_back()
                else: errors.append(LexerError(f"Invalid char after 'strn'", self.line, self.column)); state = 0; self.advance()
        elif state == 83:
            if self.current_char is None or self.current_char in DEL1: state = 84;
            elif self.current_char is not None and self.current_char in IDENTIFIER_CHARS: state = 180; lexeme += self.current_char; self.advance()
            else: errors.append(LexerError(f"Invalid delimiter for 'strng'", self.line, self.column)); state = 0; self.advance()
        elif state == 84: tokens.append(Token(TT_STRING, lexeme, start_line, start_column)); state = 0;
            if self.current_char is not None: self.step_back()
        elif state == 85: # swtch path
            if self.current_char == 't': state = 86; lexeme += self.current_char; self.advance()
            elif self.current_char is not None and self.current_char in IDENTIFIER_CHARS: state = 180; lexeme += self.current_char; self.advance()
            else:
                if self.current_char is None or self.current_char in DEL22: tokens.append(Token(TT_IDENTIFIER, lexeme, start_line, start_column)); state = 0;
                if self.current_char is not None: self.step_back()
                else: errors.append(LexerError(f"Invalid char after 'sw'", self.line, self.column)); state = 0; self.advance()
        elif state == 86:
            if self.current_char == 'c': state = 87; lexeme += self.current_char; self.advance()
            elif self.current_char is not None and self.current_char in IDENTIFIER_CHARS: state = 180; lexeme += self.current_char; self.advance()
            else:
                if self.current_char is None or self.current_char in DEL22: tokens.append(Token(TT_IDENTIFIER, lexeme, start_line, start_column)); state = 0;
                if self.current_char is not None: self.step_back()
                else: errors.append(LexerError(f"Invalid char after 'swt'", self.line, self.column)); state = 0; self.advance()
        elif state == 87:
            if self.current_char == 'h': state = 88; lexeme += self.current_char; self.advance()
            elif self.current_char is not None and self.current_char in IDENTIFIER_CHARS: state = 180; lexeme += self.current_char; self.advance()
            else:
                if self.current_char is None or self.current_char in DEL22: tokens.append(Token(TT_IDENTIFIER, lexeme, start_line, start_column)); state = 0;
                if self.current_char is not None: self.step_back()
                else: errors.append(LexerError(f"Invalid char after 'swtc'", self.line,
```

```
self.column)); state = 0;
self.advance()
        elif state == 88:
            if self.current_char is
None or self.current_char in DEL5:
state = 89;
            elif self.current_char is
not None and self.current_char in
IDENTIFIER_CHARS: state = 180;
lexeme += self.current_char;
self.advance()
            else:
errors.append(LexerError(f"Invalid
delimiter for 'swtch'", self.line,
self.column)); state = 0;
self.advance()
        elif state == 89:
tokens.append(Token(TT_SWTCH,
lexeme, start_line, start_column));
state = 0;
        if self.current_char is not
None: self.step_back()

        # tr: 90-92
        elif state == 90: # lexeme "t"
            if self.current_char == 'r':
state = 91; lexeme +=
self.current_char; self.advance()
            elif self.current_char is
not None and self.current_char in
IDENTIFIER_CHARS: state = 180;
lexeme += self.current_char;
self.advance()
            else:
                if self.current_char is
None or self.current_char in
DEL22:
tokens.append(Token(TT_IDENTIF
IER, lexeme, start_line,
start_column)); state = 0;
                if self.current_char is
not None: self.step_back()
                else:
errors.append(LexerError(f"Invalid
char after 't'", self.line,
self.column)); state = 0;
self.advance()
        elif state == 91: # lexeme
"tr"
            if self.current_char is
None or self.current_char in DEL6:
state = 92;
            elif self.current_char is
not None and self.current_char in
IDENTIFIER_CHARS: state = 180;
lexeme += self.current_char;
self.advance()
            else:
errors.append(LexerError(f"Invalid
delimiter for 'tr'", self.line,
self.column)); state = 0;
self.advance()
        elif state == 92:
tokens.append(Token(TT_TR,
lexeme, start_line, start_column));
state = 0;
        if self.current_char is not
None: self.step_back()

        # vd: 93-95
        elif state == 93: # lexeme
"v"
            if self.current_char == 'd':
state = 94; lexeme +=
self.current_char; self.advance()
            elif self.current_char is
not None and self.current_char in
IDENTIFIER_CHARS: state = 180;
lexeme += self.current_char;
self.advance()
            else:
                if self.current_char is
None or self.current_char in
DEL22:
tokens.append(Token(TT_IDENTIF
IER, lexeme, start_line,
start_column)); state = 0;
                if self.current_char is
not None: self.step_back()
                else:
errors.append(LexerError(f"Invalid
char after 'v'", self.line,
self.column)); state = 0;
self.advance()
        elif state == 94: # lexeme
"vd"
            if self.current_char is
None or self.current_char in DEL1:
state = 95;
            elif self.current_char is
not None and self.current_char in
IDENTIFIER_CHARS: state = 180;
lexeme += self.current_char;
self.advance()
            else:
errors.append(LexerError(f"Invalid
delimiter for 'vd'", self.line,
self.column)); state = 0;
self.advance()
        elif state == 95:
tokens.append(Token(TT_VD,
lexeme, start_line, start_column));
state = 0;
        if self.current_char is not
None: self.step_back()

        # whl: 96-99
        elif state == 96: # lexeme
"w"
            if self.current_char == 'h':
state = 97; lexeme +=
self.current_char; self.advance()
            elif self.current_char is
not None and self.current_char in
IDENTIFIER_CHARS: state = 180;
lexeme += self.current_char;
self.advance()
            else:
                if self.current_char is
None or self.current_char in
DEL22:
tokens.append(Token(TT_IDENTIF
IER, lexeme, start_line,
start_column)); state = 0;
                if self.current_char is
not None: self.step_back()
                else:
errors.append(LexerError(f"Invalid
char after 'w'", self.line,
self.column)); state = 0;
self.advance()
        elif state == 97: # lexeme
"wh"
            if self.current_char == 'l':
state = 98; lexeme +=
self.current_char; self.advance()
            elif self.current_char is
not None and self.current_char in
IDENTIFIER_CHARS: state = 180;
lexeme += self.current_char;
self.advance()
            else:
                if self.current_char is
None or self.current_char in
DEL22:
tokens.append(Token(TT_IDENTIF
IER, lexeme, start_line,
start_column)); state = 0;
                if self.current_char is
not None: self.step_back()
                else:
errors.append(LexerError(f"Invalid
char after 'wh'", self.line,
self.column)); state = 0;
self.advance()
        elif state == 98: # lexeme
"whl"
            if self.current_char is
None or self.current_char in DEL5:
state = 99;
            elif self.current_char is
not None and self.current_char in
IDENTIFIER_CHARS: state = 180;
lexeme += self.current_char;
self.advance()
            else:
errors.append(LexerError(f"Invalid
delimiter for 'whl'", self.line,
self.column)); state = 0;
self.advance()
```

```
        elif state == 99:
tokens.append(Token(TT_WHl,
lexeme, start_line, start_column));
state = 0;
        if self.current_char is not
None: self.step_back()

        # --- Operator States (TD3:
100-124) ---
        elif state == 100: # lexeme is
"+"
            if self.current_char == '=':
state = 102; lexeme +=
self.current_char; self.advance()
            elif self.current_char ==
'+': state = 104; lexeme +=
self.current_char; self.advance()
            elif self.current_char is
None or self.current_char in
DEL24: state = 101;
            else:
errors.append(LexerError(f"Invalid
char after '+'", self.line,
self.column)); state = 0;
self.advance()
        elif state == 101:
tokens.append(Token(TT_PLUS,
lexeme, start_line, start_column));
state = 0;
        if self.current_char is not
None: self.step_back()
        elif state == 102: # lexeme is
"+="
            if self.current_char is
None or self.current_char in
DEL24: state = 103;
            else:
errors.append(LexerError(f"Invalid
char after '+='", self.line,
self.column)); state = 0;
self.advance()
        elif state == 103:
tokens.append(Token(TT_PLUSEQ,
lexeme, start_line, start_column));
state = 0;
        if self.current_char is not
None: self.step_back()
        elif state == 104: # lexeme is
"++"
            if self.current_char is
None or self.current_char in DEL9:
state = 105;
            else:
errors.append(LexerError(f"Invalid
char after '++'", self.line,
self.column)); state = 0;
self.advance()
        elif state == 105:
tokens.append(Token(TT_INCREM
ENT, lexeme, start_line,
start_column)); state = 0;
        if self.current_char is not
None: self.step_back()

        elif state == 106: # lexeme is
"-"
            if self.current_char == '=':
state = 108; lexeme +=
self.current_char; self.advance()
            elif self.current_char ==
'-': state = 110; lexeme +=
self.current_char; self.advance()
            elif self.current_char is
None or self.current_char in
DEL24: state = 107;
            else:
errors.append(LexerError(f"Invalid
char after '-'", self.line,
self.column)); state = 0;
self.advance()
        elif state == 107:
tokens.append(Token(TT_MINUS,
lexeme, start_line, start_column));
state = 0;
        if self.current_char is not
None: self.step_back()
        elif state == 108: # lexeme is
"-="
            if self.current_char is
None or self.current_char in
DEL24: state = 109;
            else:
errors.append(LexerError(f"Invalid
char after '-='", self.line,
self.column)); state = 0;
self.advance()
        elif state == 109:
tokens.append(Token(TT_MINUSE
Q, lexeme, start_line,
start_column)); state = 0;
        if self.current_char is not
None: self.step_back()
        elif state == 110: # lexeme is
"--"
            if self.current_char is
None or self.current_char in DEL9:
state = 111;
            else:
errors.append(LexerError(f"Invalid
char after '--'", self.line,
self.column)); state = 0;
self.advance()
        elif state == 111:
tokens.append(Token(TT_DECRE
MENT, lexeme, start_line,
start_column)); state = 0;
        if self.current_char is not
None: self.step_back()

        elif state == 112: # lexeme is
"*"
            if self.current_char == '=':
state = 114; lexeme +=
self.current_char; self.advance()
            elif self.current_char ==
'*': state = 800; lexeme +=
self.current_char; self.advance() #
TT_EXP
            elif self.current_char is
None or self.current_char in
DEL24: state = 113;
            else:
errors.append(LexerError(f"Invalid
char after '*'", self.line,
self.column)); state = 0;
self.advance()
        elif state == 113:
tokens.append(Token(TT_MUL,
lexeme, start_line, start_column));
state = 0;
        if self.current_char is not
None: self.step_back()
        elif state == 114: # lexeme is
"*="
            if self.current_char is
None or self.current_char in
DEL24: state = 115;
            else:
errors.append(LexerError(f"Invalid
char after '*='", self.line,
self.column)); state = 0;
self.advance()
        elif state == 115:
tokens.append(Token(TT_MULTIE
Q, lexeme, start_line,
start_column)); state = 0;
        if self.current_char is not
None: self.step_back()

        elif state == 116: # lexeme is
"/"
            if self.current_char == '=':
state = 118; lexeme +=
self.current_char; self.advance()
            elif self.current_char is
None or self.current_char in
DEL24: state = 117;
            else:
errors.append(LexerError(f"Invalid
char after '/'", self.line,
self.column)); state = 0;
self.advance()
        elif state == 117:
tokens.append(Token(TT_DIV,
lexeme, start_line, start_column));
state = 0;
        if self.current_char is not
None: self.step_back()
        elif state == 118: # lexeme is
"/="
```

```
            if self.current_char is
None or self.current_char in
DEL24: state = 119;
            else:
errors.append(LexerError(f"Invalid
char after '/='", self.line,
self.column)); state = 0;
self.advance()
        elif state == 119:
tokens.append(Token(TT_DIVEQ,
lexeme, start_line, start_column));
state = 0;
        if self.current_char is not
None: self.step_back()

        elif state == 120: # lexeme is
"%"
            if self.current_char == '=':
state = 123; lexeme +=
self.current_char; self.advance()
            elif self.current_char is
None or self.current_char in
DEL24: state = 121;
            else:
errors.append(LexerError(f"Invalid
char after '%'", self.line,
self.column)); state = 0;
self.advance()
        elif state == 121:
tokens.append(Token(TT_MOD,
lexeme, start_line, start_column));
state = 0;
        if self.current_char is not
None: self.step_back()
        elif state == 123: # lexeme is
"%="
            if self.current_char is
None or self.current_char in
DEL24: state = 124;
            else:
errors.append(LexerError(f"Invalid
char after '%='", self.line,
self.column)); state = 0;
self.advance()
        elif state == 124:
tokens.append(Token(TT_MODEQ,
lexeme, start_line, start_column));
state = 0;
        if self.current_char is not
None: self.step_back()

        # --- Operator States (TD4:
125-150) ---
        elif state == 125: # lexeme is
"="
            if self.current_char == '=':
state = 127; lexeme +=
self.current_char; self.advance()
            elif self.current_char is
None or self.current_char in
DEL28: state = 126;
            else:
errors.append(LexerError(f"Invalid
char after '='", self.line,
self.column)); state = 0;
self.advance()
        elif state == 126:
tokens.append(Token(TT_EQ,
lexeme, start_line, start_column));
state = 0;
        if self.current_char is not
None: self.step_back()
        elif state == 127: # lexeme is
"=="
            if self.current_char is
None or self.current_char in
DEL25: state = 128;
            else:
errors.append(LexerError(f"Invalid
char after '=='", self.line,
self.column)); state = 0;
self.advance()
        elif state == 128:
tokens.append(Token(TT_EQTO,
lexeme, start_line, start_column));
state = 0;
        if self.current_char is not
None: self.step_back()

        elif state == 129: # lexeme is
">"
            if self.current_char == '=':
state = 131; lexeme +=
self.current_char; self.advance()
            elif self.current_char is
None or self.current_char in
DEL25: state = 130;
            else:
errors.append(LexerError(f"Invalid
char after '>'", self.line,
self.column)); state = 0;
self.advance()
        elif state == 130:
tokens.append(Token(TT_GT,
lexeme, start_line, start_column));
state = 0;
        if self.current_char is not
None: self.step_back()
        elif state == 131: # lexeme is
">="
            if self.current_char is
None or self.current_char in
DEL25: state = 132;
            else:
errors.append(LexerError(f"Invalid
char after '>='", self.line,
self.column)); state = 0;
self.advance()
        elif state == 132:
tokens.append(Token(TT_GTEQ,
lexeme, start_line, start_column));
state = 0;
        if self.current_char is not
None: self.step_back()

        elif state == 133: # lexeme is
"<"
            if self.current_char == '=':
state = 135; lexeme +=
self.current_char; self.advance()
            elif self.current_char is
None or self.current_char in
DEL25: state = 134;
            else:
errors.append(LexerError(f"Invalid
char after '<'", self.line,
self.column)); state = 0;
self.advance()
        elif state == 134:
tokens.append(Token(TT_LT,
lexeme, start_line, start_column));
state = 0;
        if self.current_char is not
None: self.step_back()
        elif state == 135: # lexeme is
"<="
            if self.current_char is
None or self.current_char in
DEL25: state = 136;
            else:
errors.append(LexerError(f"Invalid
char after '<='", self.line,
self.column)); state = 0;
self.advance()
        elif state == 136:
tokens.append(Token(TT_LTEQ,
lexeme, start_line, start_column));
state = 0;
        if self.current_char is not
None: self.step_back()

        elif state == 137: # lexeme is
"!"
            if self.current_char == '=':
state = 139; lexeme +=
self.current_char; self.advance()
            elif self.current_char is
None or self.current_char in
DEL25: state = 138;
            else:
errors.append(LexerError(f"Invalid
char after '!'", self.line,
self.column)); state = 0;
self.advance()
        elif state == 138:
tokens.append(Token(TT_NOT,
lexeme, start_line, start_column));
state = 0;
        if self.current_char is not
None: self.step_back()
        elif state == 139: # lexeme is
"!="
```

```
            if self.current_char is
None or self.current_char in
DEL25: state = 140;
            else:
errors.append(LexerError(f"Invalid
char after '!='", self.line,
self.column)); state = 0;
self.advance()
        elif state == 140:
tokens.append(Token(TT_NOTEQ,
lexeme, start_line, start_column));
state = 0;
        if self.current_char is not
None: self.step_back()

        elif state == 141: # lexeme is
":"
            if self.current_char is
None or self.current_char in
DEL11: state = 142;
            else:
errors.append(LexerError(f"Invalid
char after ':'", self.line,
self.column)); state = 0;
self.advance()
        elif state == 142:
tokens.append(Token(TT_COLON,
lexeme, start_line, start_column));
state = 0;
        if self.current_char is not
None: self.step_back()

        elif state == 143: # lexeme is
";"
            if self.current_char is
None or self.current_char in
DEL11: state = 144;
            else:
errors.append(LexerError(f"Invalid
char after ';'", self.line,
self.column)); state = 0;
self.advance()
        elif state == 144:
tokens.append(Token(TT_SEMICO
LON, lexeme, start_line,
start_column)); state = 0;
        if self.current_char is not
None: self.step_back()

        elif state == 145: # lexeme is
"["
            if self.current_char is
None or self.current_char in
DEL12: state = 146;
            else:
errors.append(LexerError(f"Invalid
char after '['", self.line,
self.column)); state = 0;
self.advance()
        elif state == 146:
tokens.append(Token(TT_LSQBR,
lexeme, start_line, start_column));
state = 0;
        if self.current_char is not
None: self.step_back()

        elif state == 147: # lexeme is
"]"
            if self.current_char is
None or self.current_char in
DEL13: state = 148;
            else:
errors.append(LexerError(f"Invalid
char after ']'", self.line,
self.column)); state = 0;
self.advance()
        elif state == 148:
tokens.append(Token(TT_RSQBR,
lexeme, start_line, start_column));
state = 0;
        if self.current_char is not
None: self.step_back()

        elif state == 149: # lexeme is
"{"
            if self.current_char is
None or self.current_char in
DEL14: state = 150;
            else:
errors.append(LexerError(f"Invalid
char after '{{'", self.line,
self.column)); state = 0;
self.advance()
        elif state == 150:
tokens.append(Token(TT_BLOCK_
START, lexeme, start_line,
start_column)); state = 0;
        if self.current_char is not
None: self.step_back()

        # --- TD5 Specific States
(151-160) ---
        elif state == 151: # lexeme is
"}"
            if self.current_char is
None or self.current_char in
DEL15: state = 152
            else:
errors.append(LexerError(f"Invalid
char after '}}'", self.line,
self.column)); state = 0;
self.advance()
        elif state == 152:
tokens.append(Token(TT_BLOCK_
END, lexeme, start_line,
start_column)); state = 0;
        if self.current_char is not
None: self.step_back()

        elif state == 153: # lexeme is
"("
            if self.current_char is
None or self.current_char in
DEL16: state = 154
            else:
errors.append(LexerError(f"Invalid
char after '('", self.line,
self.column)); state = 0;
self.advance()
        elif state == 154:
tokens.append(Token(TT_LPAREN
, lexeme, start_line, start_column));
state = 0;
        if self.current_char is not
None: self.step_back()

        elif state == 155: # lexeme is
")"
            if self.current_char is
None or self.current_char in
DEL18: state = 156
            else:
errors.append(LexerError(f"Invalid
char after ')'", self.line,
self.column)); state = 0;
self.advance()
        elif state == 156:
tokens.append(Token(TT_RPAREN
, lexeme, start_line, start_column));
state = 0;
        if self.current_char is not
None: self.step_back()

        elif state == 157: # lexeme is
","
            if self.current_char is
None or self.current_char in
DEL20: state = 158
            else:
errors.append(LexerError(f"Invalid
char after ','", self.line,
self.column)); state = 0;
self.advance()
        elif state == 158:
tokens.append(Token(TT_COMMA
, lexeme, start_line, start_column));
state = 0;
        if self.current_char is not
None: self.step_back()

        elif state == 159: # lexeme is
"~" (standalone operator)
            if self.current_char is
None or self.current_char in
DEL21_for_tilde_op: state = 160 #
Using custom DEL for unary tilde
            else:
errors.append(LexerError(f"Invalid
char after '~' operator", self.line,
self.column)); state = 0;
self.advance()
```

```
        elif state == 160:
tokens.append(Token(TT_NEGATI
VE, lexeme, start_line,
start_column)); state = 0;
        if self.current_char is not
None: self.step_back()

        # --- Internal states for
user-defined keywords (if needed)
---
        elif state == 701: # lexeme
"cns" (internal state after 'cns' for
'cnst')
            if self.current_char == 't':
state = 702; lexeme +=
self.current_char; self.advance()
            elif self.current_char is
not None and self.current_char in
IDENTIFIER_CHARS: state = 180;
lexeme += self.current_char;
self.advance()
            else:
                if self.current_char is
None or self.current_char in
DEL22:
tokens.append(Token(TT_IDENTIF
IER, lexeme, start_line,
start_column)); state = 0;
                if self.current_char is
not None: self.step_back()
                else:
errors.append(LexerError(f"Invalid
char after 'cns'", self.line,
self.column)); state = 0;
self.advance()
        elif state == 702: # lexeme
"cnst"
            if self.current_char is
None or self.current_char in DEL1:
state = 703;
            elif self.current_char is
not None and self.current_char in
IDENTIFIER_CHARS: state = 180;
lexeme += self.current_char;
self.advance()
            else:
errors.append(LexerError(f"Invalid
delimiter for 'cnst'", self.line,
self.column)); state = 0;
self.advance()
        elif state == 703:
tokens.append(Token(TT_CNST,
lexeme, start_line, start_column));
state = 0;
        if self.current_char is not
None: self.step_back()

        elif state == 704: # lexeme
"nl" (internal state for 'nll')
            if self.current_char == 'l':
state = 705; lexeme +=
self.current_char; self.advance()
            elif self.current_char is
not None and self.current_char in
IDENTIFIER_CHARS: state = 180;
lexeme += self.current_char;
self.advance()
            else:
                if self.current_char is
None or self.current_char in
DEL22:
tokens.append(Token(TT_IDENTIF
IER, lexeme, start_line,
start_column)); state = 0;
                if self.current_char is
not None: self.step_back()
                else:
errors.append(LexerError(f"Invalid
char after 'nl'", self.line,
self.column)); state = 0;
self.advance()
        elif state == 705: # lexeme
"nll"
            if self.current_char is
None or self.current_char in
DEL22: state = 706; # DEL22 was
used in original user code for 'nll'
            elif self.current_char is
not None and self.current_char in
IDENTIFIER_CHARS: state = 180;
lexeme += self.current_char;
self.advance()
            else:
errors.append(LexerError(f"Invalid
delimiter for 'nll'", self.line,
self.column)); state = 0;
self.advance()
        elif state == 706:
tokens.append(Token(TT_NULL,
lexeme, start_line, start_column));
state = 0;
        if self.current_char is not
None: self.step_back()

        elif state == 800: # lexeme is
"**" (for TT_EXP)
            if self.current_char is
None or self.current_char in
DEL24: state = 801; # Assuming
DEL24 for TT_EXP
            else:
errors.append(LexerError(f"Invalid
char after '**'", self.line,
self.column)); state = 0;
self.advance()
        elif state == 801:
tokens.append(Token(TT_EXP,
lexeme, start_line, start_column));
state = 0;
        if self.current_char is not
None: self.step_back()


        # --- Comment, Char/String
Literal, Identifier, Number states ---
        # ... (States 161-222,
900-901 as previously defined and
corrected) ...
        # Note: step_back() is added
after tokenizing in most
keyword/operator final states
        # to allow the delimiter to be
processed as the start of the next
token if it's not whitespace.

        # '#' Comment
        elif state == 161:
            if self.current_char is not
None and self.current_char != '\n':
self.advance()
            elif self.current_char ==
'\n': state = 162
            elif self.current_char is
None: state = 162
        elif state == 162: state = 0; #
Newline will be consumed by
advance or it's EOF

        # "' Char Literal (TD5:
163-166)
        elif state == 163:
            if self.current_char == '\\':
state = 900; lexeme +=
self.current_char; self.advance()
            elif self.current_char is
not None and self.current_char in
CHAR_CONTENT_CHARS:
                state = 164; lexeme +=
self.current_char; self.advance()
            elif self.current_char ==
"'":
errors.append(LexerError("Empty
character literal", start_line,
start_column)); state = 0
            else:
errors.append(LexerError(f"Invalid
character in char literal:
{self.current_char}", start_line,
start_column)); state = 0;
            if self.current_char is not
None: self.advance()
        elif state == 900:
            valid_escapes = {"'": "'",
"\\": "\\", "n": "\n", "t": "\t", "0":
"\0"}
            if self.current_char is not
None and self.current_char in
valid_escapes:
```

```
                lexeme +=
self.current_char; state = 164;
self.advance()
            else:
errors.append(LexerError(f"Invalid
escape sequence:
\\{self.current_char}", start_line,
start_column)); state = 0;
            if self.current_char is not
None: self.advance()
        elif state == 164:
            if self.current_char == "'":
state = 165; lexeme +=
self.current_char; self.advance()
            else:
errors.append(LexerError(f"Unclose
d character literal, expected ' found
{self.current_char}", start_line,
start_column)); state = 0;
            if self.current_char is not
None: self.advance()
        elif state == 165:
            if self.current_char is
None or self.current_char in
DEL26: state = 166
            else:
errors.append(LexerError(f"Invalid
delimiter after char literal:
{self.current_char}", self.line,
self.column)); state = 0;
self.advance()
        elif state == 166:
            val = lexeme[1:-1];
actual_val = ""
            if len(val) == 2 and val[0]
== '\\':
                escape_map = {"'": "'",
"\\": "\\", "n": "\n", "t": "\t", "0":
"\0"}
                actual_val =
escape_map.get(val[1], val[1])
            elif len(val) == 1:
actual_val = val
            else:
errors.append(LexerError(f"Invalid
char literal content: {val}",
start_line, start_column));
            if not errors or actual_val:
tokens.append(Token(TT_CHARLI
T, actual_val, start_line,
start_column));
            state = 0;
            if self.current_char is not
None: self.step_back()

        elif state == 167:
            if self.current_char == "'":
state = 168; lexeme +=
self.current_char; self.advance()
            elif self.current_char ==
'\\': state = 901; lexeme +=
self.current_char; self.advance()
            elif self.current_char is
not None and self.current_char !=
'\n': lexeme += self.current_char;
self.advance()
            elif self.current_char ==
'\n':
errors.append(LexerError("Newline
in string literal", start_line,
start_column)); state = 0
            elif self.current_char is
None:
errors.append(LexerError("Untermi
nated string literal (EOF)",
start_line, start_column)); state = 0
            else:
errors.append(LexerError(f"Unexpe
cted char in string:
{self.current_char}", start_line,
start_column)); state = 0;
self.advance()
        elif state == 901:
            valid_escapes = {"'": "'",
'\\': '\\', 'n': '\n', 't': '\t', '0': '\0'}
            if self.current_char is not
None and self.current_char in
valid_escapes:
                lexeme +=
self.current_char; state = 167;
self.advance()
            else:
errors.append(LexerError(f"Invalid
string escape sequence:
\\{self.current_char}", start_line,
start_column)); state = 167;
            if self.current_char is not
None: self.advance()
        elif state == 168:
            if self.current_char is
None or self.current_char in
DEL19: state = 169
            else:
errors.append(LexerError(f"Invalid
delimiter after string literal:
{self.current_char}", self.line,
self.column)); state = 0;
self.advance()
        elif state == 169:
            raw_val = lexeme[1:-1];
actual_val = ""; i = 0
            while i < len(raw_val):
                if raw_val[i] == '\\':
                    if i + 1 <
len(raw_val):
                        esc_char =
raw_val[i+1]
                        escape_map = {"'":
"'", '\\': '\\', 'n': '\n', 't': '\t', '0': '\0'}
                        actual_val +=
escape_map.get(esc_char, esc_char)
                        i += 2
                    else: actual_val +=
raw_val[i]; i+=1
                else: actual_val +=
raw_val[i]; i+=1

tokens.append(Token(TT_STRING
LIT, actual_val, start_line,
start_column)); state = 0;
            if self.current_char is not
None: self.step_back()

        # TD5 &&, ||, ., `
        elif state == 170: # lexeme is
"&"
            if self.current_char == '&':
state = 171; lexeme +=
self.current_char; self.advance()
            else:
errors.append(LexerError("Expecte
d '&' for '&&', found single '&'",
self.line, self.column)); state = 0; #
Single '&' is error unless defined
elsewhere
        elif state == 171: # lexeme is
"&&"
            if self.current_char is
None or self.current_char in
DEL25: state = 172
            else:
errors.append(LexerError(f"Invalid
char after '&&'", self.line,
self.column)); state = 0;
self.advance()
        elif state == 172:
tokens.append(Token(TT_AND,
lexeme, start_line, start_column));
state = 0;
        if self.current_char is not
None: self.step_back()

        elif state == 173: # lexeme is
"|"
            if self.current_char == '|':
state = 174; lexeme +=
self.current_char; self.advance()
            else:
errors.append(LexerError("Expecte
d '|' for '||', found single '|'", self.line,
self.column)); state = 0;
        elif state == 174: # lexeme is
"||"
            if self.current_char is
None or self.current_char in
DEL25: state = 175
            else:
errors.append(LexerError(f"Invalid
char after '||'", self.line,
```

```
self.column)); state = 0;
self.advance()
        elif state == 175:
tokens.append(Token(TT_OR,
lexeme, start_line, start_column));
state = 0;
        if self.current_char is not
None: self.step_back()

        elif state == 176: # lexeme is
"."
            # TD5 shows transition on
alpha to 177 (final). This means '.' is
a token, and alpha starts next.

tokens.append(Token(TT_STRCTA
CCESS, lexeme, start_line,
start_column))
            state = 0 # Reset to
process the alpha (or other char)
            # No advance() here,
current_char is the char *after* '.',
which starts next token.
            # No step_back() needed
as '.' is a single char token here.
        # State 177 is implicit
tokenization of '.'

        elif state == 178: # lexeme is
"`"
            if self.current_char is
None or self.current_char in
DEL27: state = 179
            else:
errors.append(LexerError(f"Invalid
char after '`'", self.line,
self.column)); state = 0;
self.advance()
        elif state == 179:
tokens.append(Token(TT_CONCAT
, lexeme, start_line, start_column));
state = 0;
        if self.current_char is not
None: self.step_back()



        # --- Identifier State (TD6:
180) ---
        elif state == 180:
            if self.current_char is not
None and self.current_char in
IDENTIFIER_CHARS:
                if len(lexeme) < 25:
lexeme += self.current_char;
self.advance()
                else:

errors.append(LexerError(f"Identifi
er '{lexeme}{self.current_char}'
exceeds max length 25", start_line,
start_column))
                while
self.current_char is not None and
self.current_char in
IDENTIFIER_CHARS:
self.advance()
                state = 0
            elif self.current_char is
None or self.current_char in
DEL22:
                token_type =
keywords.get(lexeme,
TT_IDENTIFIER)

tokens.append(Token(token_type,
lexeme, start_line, start_column))
                state = 0
                if self.current_char is
not None: self.step_back() #
Delimiter might start next token
            else:

errors.append(LexerError(f"Invalid
char '{self.current_char}' in
identifier '{lexeme}'", self.line,
self.column)); state = 0;
self.advance()

        # --- Number Literal States
(TD7: 195-222) ---
        elif state == 195:
            if lexeme == "~" and not
(self.current_char in DIGITZERO):

errors.append(LexerError("Stray '~';
not a valid number prefix here.",
start_line, start_column)); state = 0;
continue
            elif self.current_char in
DIGITZERO: lexeme +=
self.current_char; state = 196;
self.advance()
            elif self.current_char ==
'.':
                if lexeme == "~" and
self.peek() not in DIGITZERO :
errors.append(LexerError("Digit
expected after '~.'", start_line,
start_column)); state = 0;
                else: lexeme +=
self.current_char; state = 213;
self.advance()
            elif self.current_char is
None or self.current_char in
DEL23:

tokens.append(Token(TT_INTEGE
RLIT, lexeme, start_line,
start_column)); state = 0;
                if self.current_char is
not None: self.step_back()
            else:
errors.append(LexerError(f"Invalid
char '{self.current_char}' starting
number '{lexeme}'", self.line,
self.column)); state = 0;
self.advance()
        elif state >= 196 and state
<= 209:
            if self.current_char in
DIGITZERO: lexeme +=
self.current_char; state += 1;
self.advance()
            elif self.current_char ==
'.': lexeme += self.current_char;
state = 213; self.advance()
            elif self.current_char is
None or self.current_char in
DEL23: state = 212;
            else:
errors.append(LexerError(f"Invalid
char '{self.current_char}' in integer
part '{lexeme}'", self.line,
self.column)); state = 0;
self.advance()
        elif state == 210:
            if self.current_char in
DIGITZERO: lexeme +=
self.current_char; state = 211;
self.advance()
            elif self.current_char ==
'.': lexeme += self.current_char;
state = 213; self.advance()
            elif self.current_char is
None or self.current_char in
DEL23: state = 212;
            else:
errors.append(LexerError(f"Invalid
char '{self.current_char}' in integer
part '{lexeme}'", self.line,
self.column)); state = 0;
self.advance()
        elif state == 211:
            if self.current_char == '.':
lexeme += self.current_char; state =
213; self.advance()
            elif self.current_char is
None or self.current_char in
DEL23: state = 212;
            else:
errors.append(LexerError(f"Expecte
d '.' or delimiter after integer part,
got '{self.current_char}'", self.line,
self.column)); state = 0;
self.advance()
        elif state == 212:
tokens.append(Token(TT_INTEGE
RLIT, lexeme, start_line,
start_column)); state = 0;
        if self.current_char is not
None: self.step_back()
        elif state == 213:
```

```
            if self.current_char in
DIGITZERO: lexeme +=
self.current_char; state = 214;
self.advance()
            elif self.current_char is
None or self.current_char in
DEL17:
                if lexeme.endswith('.'):
tokens.append(Token(TT_DOUBL
ELIT, lexeme, start_line,
start_column)); state = 0;
                if self.current_char is
not None: self.step_back()
                else:
errors.append(LexerError(f"Malfor
med double ending with dot:
'{lexeme}'", self.line, self.column));
state = 0; self.advance()
            else:
errors.append(LexerError(f"Digit
expected after '.', got
'{self.current_char}'", self.line,
self.column)); state = 0;
self.advance()
        elif state >= 214 and state
<= 220:
            if self.current_char in
DIGITZERO: lexeme +=
self.current_char; state += 1;
self.advance()
            elif self.current_char is
None or self.current_char in
DEL17: state = 222;
            else:
errors.append(LexerError(f"Invalid
char '{self.current_char}' in
fractional part '{lexeme}'", self.line,
self.column)); state = 0;
self.advance()
        elif state == 221:
            if self.current_char in
DIGITZERO:
                lexeme +=
self.current_char; self.advance()
                if self.current_char is
None or self.current_char in
DEL17: state = 222;
                else:
errors.append(LexerError(f"Expecte
d delimiter DEL17 after fractional
part, got '{self.current_char}'",
self.line, self.column)); state = 0
            elif self.current_char is
None or self.current_char in
DEL17: state = 222
            else:
errors.append(LexerError(f"Invalid
char '{self.current_char}' in
fractional part '{lexeme}'", self.line,
self.column)); state = 0;
self.advance()
        elif state == 222:
tokens.append(Token(TT_DOUBL
ELIT, lexeme, start_line,
start_column)); state = 0;
        if self.current_char is not
None: self.step_back()

        else: # Unhandled state
            if state != 0:

errors.append(LexerError(f"Lexer
in unhandled state {state} for
lexeme '{lexeme}' with char
'{self.current_char}'", self.line,
self.column))
                if self.current_char is
not None: self.advance()
                state = 0

    if not tokens or tokens[-1].type
!= TT_EOF:

tokens.append(Token(TT_EOF,
'EOF', self.line, self.column if
self.current_char else self.column))
    return tokens, errors
```

Syntax Analyzer

```
import string

state = []
lexeme = []
token = []
idens = []

arithmetic_operator = ['+', '-', '*', '/']
relational_operator = ['==', '!=', '>',
'<', '>=', '<=']
logical_operator = ['&&', '||', '!']
assignment_operator = ['=', '+=', '-=',
'*=', '/=', '%=']
number_operator =
arithmetic_operator +
relational_operator +
logical_operator
all_operators = number_operator +
assignment_operator

punc_symbols = ['=' , '+', '-', '*', '/',
'&&', '||', '!', '>', '<', '[', ']', '{', '}', '(',
')', ',', ';', ':', '#', '~', '.', '@', '$', '^', '_',
'?']

quote_symbols = ["", """]

alpha = list(string.ascii_letters)
digit = list(string.digits)
alphanumeric =
list(string.ascii_letters +
string.digits)
ascii_def = alphanumeric +
punc_symbols + quote_symbols
identifier = string.ascii_lowercase

#keywords
keywords = ['npt', 'prnt', 'nt', 'dbl',
'strng', 'bln', 'chr', 'f', 'ls', 'lsf', 'swtch',
'fr',
    'whl', 'd', 'mn', 'cs', 'dflt', 'brk',
'cnst', 'tr', 'fls', 'fnctn', 'rtrn', 'nll',
'cntn', 'strct', 'dfstrct', 'vd']

whitespace = [' ', '\t', '\n']

#parsing table
parsing_table = {
    "<program>": {},
    "<global_declaration>": {"strct":
["<strct_declaration>",
"<global_declaration>"]},
    "<declaration>": {"cnst":
["<cnst_dec>", ";",
"<declaration>"], "dfstrct":
["dfstrct", "id", "id",
"<strct_init_next>", ";",
"<declaration>"]},
    "<var_dec>": {"nt": ["nt", "id",
"<nt_value>", "<nt_value_next>"],
"dbl": ["dbl", "id", "<dbl_value>",
"<dbl_value_next>"],
            "bln": ["bln", "id",
"<bln_value>",
"<bln_value_next>"], "chr": ["chr",
"id", "<chr_value>",
"<chr_value_next>"],
            "strng": ["strng", "id",
"<str_value>",
"<str_value_next>"]},
    "<nt_value>": {"=": ["=",
"<arithmetic_nt>"], "[":
["<nt_array_dec>"]},
```

```
    "<nt_value_next>": {",": [",",
"id", "<nt_value>",
"<nt_value_next>"], ";": ["null"]},
    "<arithmetic_nt>": {},
    "<nt_math_val>": {"(": ["(",
"<arithmetic_nt>", ")"]},
    "<ntliterals>": {"ntlit": ["ntlit"],
"~ntlit": ["~ntlit"]},
    "<unary_entities>": {"id": ["id",
"<unary_next>"]},
    "<unary_next>": {},
    "<unary_inc/dec>": {"++":
["++"], "--": ["--"]},
    "<arithmetic_nt_next>": {},
    "<arithmetic_operator>": {"+":
["+"], "-": ["-"], "*": ["*"], "/": ["/"],
"%": ["%"]},
    "<entities>": {"id": ["id",
"<id_entity_next>"]},
    "<id_entity_next>": {"(":
["<function_call>"]},
    "<id_build>": {"id": ["id",
"<id_next>"]},
    "<id_next>": {"[":
["<array_access>"], ".":
["<strct_elem_access>"]},
    "<array_access>": {"[": ["[",
"<index>", "]", "<index_next>"]},
    "<index>": {},
    "<index_next>": {"[": ["[",
"<index>", "]"]},
    "<strct_elem_access>": {".": [".",
"id"]},
    "<function_call>": {"(": ["(",
"<arguments>", ")"]},
    "<arguments>": {},
    "<args_value>": {},
    "<args_next>": {",": [",",
"<args_value>", "<args_next>"],
")": ["null"]},
    "<expression>": {},
    "<rel_arith_logic_expr>": {},
    "<operands>": {"(": ["(",
"<rel_arith_logic_expr>", ")"]},
    "<operand_next>": {},
    "<logical_operator>": {"&&":
["&&"], "||": ["||"]},
    "<relational_operator>": {"==":
["=="], "!=": ["!="], "<=": ["<="],
">=": [">="], "<": ["<"], ">":
[">"]},
    "<arithmetic>": {},
    "<math_value>": {},
    "<dblliterals>": {"dbllit":
["dbllit"], "~dbllit": ["~dbllit"]},
    "<arithmetic_next>": {},
    "<strng_concat>": {},
    "<strng_concat_val>":
{"strnglit": ["strnglit"]},
    "<strng_concat_next>": {"`":
["`", "<strng_concat_val>",
"<strng_concat_next>"]},
    "<nt_array_dec>": {"[": ["[",
"<array_size>", "]",
"<nt_array_initializer>"]},
    "<array_size>": {"ntlit": ["ntlit"],
"id": ["id"]},
    "<nt_array_initializer>": {"=":
["=", "{", "<nt_array_elem>", "}"],
"[": ["[", "<array_size>", "]",
"<nt_2d_init>"]},
    "<nt_array_elem>": {},
    "<nt_array_elem_val>": {},
    "<nt_array_init_next>": {",": [",",
"<nt_array_elem_val>",
"<nt_array_init_next>"]},
    "<nt_2d_init>": {"=": ["=", "{",
"<nt_array2d_elem>", "}"]},
    "<nt_array2d_elem>": {"{": ["{",
"<nt_array_elem>", "}",
"<nt_array_elem_next>"]},
    "<nt_array_elem_next>": {",":
[",", "<nt_array2d_elem>"]},
    "<dbl_value>": {"=": ["=",
"<arithmetic>"], "[":
["<dbl_array_dec>"]},
    "<dbl_value_next>": {",": [",",
"id", "<dbl_value>",
"<dbl_value_next>"]},
    "<dbl_array_dec>": {"[": ["[",
"<array_size>", "]",
"<dbl_array_initializer>"]},
    "<dbl_array_initializer>": {"=":
["=", "{", "<dbl_array_elem>", "}"],
"[": ["[", "<array_size>", "]",
"<dbl_2d_init>"]},
    "<dbl_array_elem>": {},
    "<dbl_array_elem_val>": {},
    "<dbl_array_init_next>": {",":
[",", "<dbl_array_elem_val>",
"<dbl_array_init_next>"]},
    "<dbl_2d_init>": {"=": ["=", "{",
"<dbl_array2d_elem>", "}"]},
    "<dbl_array2d_elem>": {"{":
["{", "<dbl_array_elem>", "}",
"<dbl_array_elem_next>"]},
    "<dbl_array_elem_next>": {",":
[",", "<dbl_array2d_elem>"]},
    "<bln_value>": {"=": ["=",
"<equals_bln_val>"], "[":
["<bln_array_dec>"]},
    "<equals_bln_val>": {},
    "<bln_value_next>": {",": [",",
"id", "<bln_value>",
"<bln_value_next>"]},
    "<bln_array_dec>": {"[": ["[",
"<array_size>", "]",
"<bln_array_initializer>"]},
    "<bln_array_initializer>": {"=":
["=", "{", "<bln_array_elem>", "}"],
"[": ["[", "<array_size>", "]",
"<bln_2d_init>"]},
    "<bln_array_elem>": {},
    "<bln_array_elem_val>":
{"blnlit": ["blnlit"]},
    "<bln_array_init_next>": {",":
[",", "<bln_array_elem_val>",
"<bln_array_init_next>"]},
    "<bln_2d_init>": {"=": ["=", "{",
"<bln_array2d_elem>", "}"]},
    "<bln_array2d_elem>": {"{":
["{", "<bln_array_elem>", "}",
"<bln_array_elem_next>"]},
    "<bln_array_elem_next>": {",":
[",", "<bln_array2d_elem>"]},
    "<chr_value>": {"=": ["=",
"<equals_chr_val>"], "[":
["<chr_array_dec>"]},
    "<equals_chr_val>": {"id":
["<entities>"], "chrlit": ["chrlit"]},
    "<chr_value_next>": {",": [",",
"id", "<chr_value>",
"<chr_value_next>"]},
    "<chr_array_dec>": {"[": ["[",
"<array_size>", "]",
"<chr_array_initializer>"]},
    "<chr_array_initializer>": {"=":
["=", "{", "<chr_array_elem>", "}"],
"[": ["[", "<array_size>", "]",
"<chr_2d_init>"]},
    "<chr_array_elem>": {},
    "<chr_array_elem_val>":
{"chrlit": ["chrlit"]},
    "<chr_array_init_next>": {",":
[",", "<chr_array_elem_val>",
"<chr_array_init_next>"]},
    "<chr_2d_init>": {"=": ["=", "{",
"<chr_array2d_elem>", "}"]},
    "<chr_array2d_elem>": {"{":
["{", "<chr_array_elem>", "}",
"<chr_array_elem_next>"]},
    "<chr_array_elem_next>": {",":
[",", "<chr_array2d_elem>"]},
    "<str_value>": {"=": ["=",
"<equals_str_val>"], "[":
["<str_array_dec>"]},
    "<equals_str_val>": {"id":
["<entities>"], "strnglit":
["<strng_concat>"]},
    "<str_value_next>": {",": [",",
"id", "<str_value>",
"<str_value_next>"]},
    "<str_array_dec>": {"[": ["[",
"<array_size>", "]",
"<str_array_initializer>"]},
    "<str_array_initializer>": {"=":
["=", "{", "<str_array_elem>", "}"],
"[": ["[", "<array_size>", "]",
"<str_2d_init>"]},
```

```
    "<str_array_elem>": {"strnglit":
["<str_array_elem_val>",
"<str_array_init_next>"]},
    "<str_array_elem_val>":
{"strnglit": ["strnglit"]},
    "<str_array_init_next>": {",":
[",", "<str_array_elem_val>",
"<str_array_init_next>"]},
    "<str_2d_init>": {"=": ["=", "{",
"<str_array2d_elem>", "}"]},
    "<str_array2d_elem>": {"{": ["{",
"<str_array_elem>", "}",
"<str_array_elem_next>"]},
    "<str_array_elem_next>": {",":
[",", "<str_array2d_elem>"]},
    "<cnst_dec>": {"cnst": ["cnst",
"<cnst_dec_data_type>"]},
    "<cnst_dec_data_type>": {"nt":
["nt", "id", "<nt_cnst_initializer>"],
"dbl": ["dbl", "id",
"<dbl_cnst_initializer>"],
                "bln": ["bln", "id",
"<bln_cnst_initializer>"], "chr":
["chr", "id",
"<chr_cnst_initializer>"],
                "strng": ["strng",
"id", "<str_cnst_initializer>"]},
    "<nt_cnst_initializer>": {"=":
["=", "<nt_cnst_init_val>"], "[":
["<cnst_nt_array_dec>"]},
    "<nt_cnst_init_val>": {},
    "<nt_cnst_next>": {",": [",", "id",
"<nt_cnst_initializer>"]},
    "<cnst_nt_array_dec>": {"[":
["[", "<cnst_array_size>", "]",
"<cnst_nt_array_initializer>"]},
    "<cnst_array_size>": {},
    "<cnst_nt_array_initializer>":
{"=": ["=", "{", "<nt_array_elem>",
"}"], "[": ["[", "<array_size>", "]",
"<nt_2d_init>"]},
    "<dbl_cnst_initializer>": {"=":
["=", "<dbl_cnst_init_val>"], "[":
["<cnst_dbl_array_dec>"]},
    "<dbl_cnst_init_val>": {},
    "<dbl_cnst_next>": {",": [",",
"id", "<dbl_cnst_initializer>"]},
    "<cnst_dbl_array_dec>": {"[":
["[", "<cnst_array_size>", "]",
"<cnst_dbl_array_initializer>"]},
    "<cnst_dbl_array_initializer>":
{"=": ["=", "{",
"<dbl_array_elem>", "}"], "[": ["[",
"<array_size>", "]",
"<dbl_2d_init>"]},
    "<bln_cnst_initializer>": {"=":
["=", "<bln_cnst_init_val>"], "[":
["<cnst_bln_array_dec>"]},
    "<bln_cnst_init_val>": {"blnlit":
["blnlit", "<bln_cnst_next>"]},
    "<bln_cnst_next>": {",": [",",
"id", "<bln_cnst_initializer>"]},
    "<cnst_bln_array_dec>": {"[":
["[", "<cnst_array_size>", "]",
"<cnst_bln_array_initializer>"]},
    "<cnst_bln_array_initializer>":
{"=": ["=", "{",
"<bln_array_elem>", "}"], "[": ["[",
"<array_size>", "]",
"<bln_2d_init>"]},
    "<chr_cnst_initializer>": {"=":
["=", "<chr_cnst_init_val>"], "[":
["<cnst_chr_array_dec>"]},
    "<chr_cnst_init_val>": {"chrlit":
["chrlit", "<chr_cnst_next>"]},
    "<chr_cnst_next>": {",": [",",
"id", "<chr_cnst_initializer>"]},
    "<cnst_chr_array_dec>": {"[":
["[", "<cnst_array_size>", "]",
"<cnst_chr_array_initializer>"]},
    "<cnst_chr_array_initializer>":
{"=": ["=", "{",
"<chr_array_elem>", "}"], "[": ["[",
"<array_size>", "]",
"<chr_2d_init>"]},
    "<str_cnst_initializer>": {"=":
["=", "<str_cnst_init_val>"], "[":
["<cnst_str_array_dec>"]},
    "<str_cnst_init_val>": {"strnglit":
["strnglit", "<str_cnst_next>"]},
    "<str_cnst_next>": {",": [",", "id",
"<str_cnst_initializer>"]},
    "<cnst_str_array_dec>": {"[":
["[", "<cnst_array_size>", "]",
"<cnst_str_array_initializer>"]},
    "<cnst_str_array_initializer>":
{"=": ["=", "{", "<str_array_elem>",
"}"], "[": ["[", "<array_size>", "]",
"<str_2d_init>"]},
    "<strct_init_next>": {",": [",",
"id", "<strct_init_next>"]},
    "<strct_declaration>": {"strct":
["strct", "id", "{", "<strct_body>",
"}", ";"]},
    "<strct_body>": {},
    "<strct_var_dec>": {"nt": ["nt",
"id"], "dbl": ["dbl", "id"], "bln":
["bln", "id"],
                "chr": ["chr", "id"],
"strng": ["strng", "id"]},
    "<var_dec_next>": {},
    "<function_definition>":
{"fnctn": ["fnctn", "<func_type>",
"id", "(", "<parameter>", ")", "{",
"<func_var_dec>", "<statement>",
"}", "<function_definition>"]},
    "<func_type>": {"vd": ["vd"]},
    "<data_type>": {"nt": ["nt"],
"dbl": ["dbl"], "bln": ["bln"], "chr":
["chr"], "strng": ["strng"]},
    "<parameter>": {},
    "<parameter_next>": {",": [",",
"<parameter>"]},
    "<func_var_dec>": {},
    "<statement>": {"id":
["<id_build_statements>",
"<statement>"], "prnt":
["<output_statement>",
"<statement>"], "fr":
["<fr_statement>", "<statement>"],
                "d":
["<d-whl_statement>",
"<statement>"], "whl":
["<whl_statement>",
"<statement>"], "rtrn":
["<rtrn_statement>",
"<statement>"]},
    "<id_build_statements>": {"id":
["id",
"<id_build_statement_next>"]},
    "<id_build_statement_next>":
{"(": ["<function_call>", ";"]},
    "<assignment_statement>": {"=":
["=",
"<assignment_value_input>"]},
    "<assign_operator>": {"+=":
["+="], "-=": ["-="], "*=": ["*="],
"/=": ["/="]},
    "<assignment_value>": {},
    "<assignment_value_input>":
{"npt":
["<input_statement_value>"]},
    "<input_statement_value>":
{"npt": ["npt", "(", "strnglit", ")",
";"]},
    "<unary_pre_statement>": {},
    "<output_statement>": {"prnt":
["prnt", "(", "<output_value>",
"<output_next>", ")", ";"]},
    "<output_value>": {},
    "<output_next>": {",": [",",
"<output_value>",
"<output_next>"]},
    "<conditional_statement>": {"f":
["f", "(", "<condition>", ")", "{",
"<statement>", "<lsf_statement>",
"<ls_statement>", "}",
"<lsf_statement>",
"<ls_statement>"],
                "swtch":
["swtch", "(", "id", ")", "{",
"<swtch_body>", "}"]},
    "<condition>": {},
    "<lsf_statement>": {"lsf": ["lsf",
"(", "<condition>", ")", "{",
"<statement>", "}",
"<lsf_statement>"]},
    "<ls_statement>": {"ls": ["ls",
"{", "<statement>", "}"]},
    "<swtch_body>": {"cs": ["<cs>",
"<dflt>"]},
```

```
    "<cs>": {"cs": ["cs",
"<swtch_value>", ":",
"<statement>", "<cs_next>"]},
    "<swtch_value>": {"chrlit":
["chrlit"]},
    "<cs_next>": {"cs": ["<cs>",
"<cs_next>"]},
    "<dflt>": {"dflt": ["dflt", ":",
"<statement>"]},
    "<fr_statement>": {"fr": ["fr",
"(", "<initialization>", ";",
"<fr_condi>", ";", "<inc/dec>", ")",
"{", "<statement>", "}"]},
    "<initialization>": {"id":
["<id_build>", "=",
"<arithmetic_nt>"]},
    "<fr_condi>": {},
    "<inc/dec>": {"id": ["id",
"<inc/dec_next>"]},
    "<inc/dec_next>": {},
    "<fr_unary>": {},
    "<inc/dec_value>": {"id":
["<id_build>"]},
    "<d-whl_statement>": {"d": ["d",
"{", "<statement>", "}", "whl", "(",
"<condition>", ")", ";"]},
    "<whl_statement>": {"whl":
["whl", "(", "<condition>", ")", "{",
"<statement>", "}"]},
    "<rtrn_statement>": {"rtrn":
["rtrn", "<rtrn_value>", ";"]},
    "<rtrn_value>": {},
    "<control_statements>": {"brk":
["brk", ";"], "cntn": ["cntn", ";"]},
}

def add_set(set, production,
prod_set):
    for terminal in set:
        if terminal not in parsing_table:

parsing_table[production][terminal]
= []

parsing_table[production][terminal].
extend(prod_set)

def add_all_set():
    add_set(["nt", "dbl", "bln", "chr",
"strng", "cnst", "dfstrct", "strct",
"fnctn", "mn"], "<program>",
            ["<global_declaration>",
"<function_definition>", "mn", "(",
")", "{", "<declaration>",
"<statement>", "end", ";", "}"])
    add_set(["nt", "dbl", "bln", "chr",
"strng"], "<global_declaration>",
["<var_dec>", ";",
"<global_declaration>"])
    add_set(["cnst"],
"<global_declaration>",
["<cnst_dec>", ";",
"<global_declaration>"])
    add_set(["dfstrct"],
"<global_declaration>", ["dfstrct",
"id", "id", "<strct_init_next>", ";",
"<global_declaration>"])

    add_set(["fnctn", "mn"],
"<global_declaration>", ["null"])
    add_set(["nt", "dbl", "bln", "chr",
"strng"], "<declaration>",
["<var_dec>", ";", "<declaration>"])
    add_set(["id", "++", "--", "prnt",
"f", "swtch", "fr", "d", "whl", "rtrn",
"brk", "cntn", "end"],
"<declaration>", ["null"])
    add_set([",", ";"], "<nt_value>",
["null"])
    add_set(["ntlit", "~ntlit", "(",
"++", "--", "id"], "<arithmetic_nt>",
["<nt_math_val>",
"<arithmetic_nt_next>"])
    add_set(["ntlit", "~ntlit", ],
"<nt_math_val>", ["<ntliterals>"])
    add_set(["++", "--", "id"],
"<nt_math_val>",
["<unary_entities>"])
    add_set(["++", "--"],
"<unary_entities>",
["<unary_inc/dec>", "id"])
    add_set(["++", "--"],
"<unary_next>",
["<unary_inc/dec>"])
    add_set(["[", ".", "("],
"<unary_next>",
["<id_entity_next>"])
    add_set(["+", "-", "*", "/", "%",
",", ";", ")", "&&", "||", "==", "!=",
"<=", ">=", "<", ">", "]"],
"<unary_next>", ["null"])
    add_set(["+", "-", "*", "/", "%"],
"<arithmetic_nt_next>",
["<arithmetic_operator>",
"<nt_math_val>",
"<arithmetic_nt_next>"])
    add_set([",", ")", "]", ";"],
"<arithmetic_nt_next>", ["null"])
    add_set(["[", "."],
"<id_entity_next>", ["<id_next>"])
    add_set(["+", "-", "*", "/", "%",
",", ")", "]", ";", "&&", "||", "==",
"!=", "<=", ">=", "<", ">"],
"<id_entity_next>", ["null"])
    add_set(["+", "-", "*", "/", "%",
",", ")", "]", ";", "=", "+=", "-=",
"*=", "/=", "&&", "||", "==", "!=",
"<=", ">=", "<", ">"], "<id_next>",
["null"])
    add_set(["ntlit", "~ntlit" "(",
"++", "--", "id"],
"<index>",["<arithmetic_nt>"])
    add_set(["+", "-", "*", "/", "%",
",", ")", "]", ";", "=", "+=", "-=",
"*=", "/=", "&&", "||", "==", "!=",
"<=", ">=", "<", ">"],
"<index_next>", ["null"])
    add_set(["(", "dbllit", "~dbllit",
"ntlit", "~ntlit", "++", "--", "id",
"chrlit", "!", "strnglit", "blnlit"],
"<arguments>", ["<args_value>",
"<args_next>"])
    add_set([")"], "<arguments>",
["null"])
    add_set(["(", "dbllit", "~dbllit",
"ntlit", "~ntlit", "++", "--", "id",
"chrlit", "!", "strnglit", "blnlit"],
"<args_value>", ["<expression>"])
    add_set(["(", "dbllit", "~dbllit",
"ntlit", "~ntlit", "++", "--", "id",
"chrlit", "!", "strnglit", "blnlit"],
"<expression>",
["<rel_arith_logic_expr>"])
    add_set(["(", "dbllit", "~dbllit",
"ntlit", "~ntlit", "++", "--", "id",
"chrlit", "!", "strnglit", "blnlit"],
"<rel_arith_logic_expr>",
["<operands>", "<operand_next>"])
    add_set(["dbllit", "~dbllit",
"ntlit", "~ntlit", "++", "--", "id"],
"<operands>", ["<math_value>"])
    add_set(["chrlit"], "<operands>",
["chrlit"])
    add_set(["!"], "<operands>", ["!",
"<operands>"])
    add_set(["strnglit"],
"<operands>", ["<strng_concat>"])
    add_set(["blnlit"], "<operands>",
["blnlit"])
    add_set(["+", "-", "*", "/", "%"],
"<operand_next>",
["<arithmetic_operator>",
"<rel_arith_logic_expr>"])
    add_set(["==", "!=", "<=", ">=",
"<", ">"], "<operand_next>",
["<relational_operator>",
"<rel_arith_logic_expr>"])
    add_set(["&&", "||"],
"<operand_next>",
["<logical_operator>",
"<rel_arith_logic_expr>"])
    add_set([",", ";", ")" ],
"<operand_next>", ["null"])
    add_set(["dbllit", "~dbllit",
"ntlit", "~ntlit", "++", "--", "id"],
"<arithmetic>", ["<math_value>",
"<arithmetic_next>"])
    add_set(["dbllit", "~dbllit"],
"<math_value>", ["<dblliterals>"])
    add_set(["ntlit", "~ntlit"],
"<math_value>", ["<ntliterals>"])
```

```
    add_set(["++", "--", "id"],
"<math_value>",
["<unary_entities>"])
    add_set(["+", "-", "*", "/", "%"],
"<arithmetic_next>",
["<arithmetic_operator>",
"<arithmetic>"])
    add_set([",", ";", ")"],
"<arithmetic_next>", ["null"])
    add_set(["strnglit"],
"<strng_concat>",
["<strng_concat_val>",
"<strng_concat_next>"])
    add_set(["==", "!=", "<=", ">=",
"<", ">", "&&", "||", "+", "-", "*",
"/", "%", ",", ";", ")"],
"<strng_concat_next>", ["null"])
    add_set([",", ";"],
"<nt_array_initializer>", ["null"])
    add_set(["ntlit", "~ntlit"],
"<nt_array_elem>",
["<nt_array_elem_val>",
"<nt_array_init_next>"])
    add_set(["ntlit", "~ntlit"],
"<nt_array_elem_val>",
["<ntliterals>"])
    add_set(["}"],
"<nt_array_init_next>", ["null"])
    add_set([",", ";"], "<nt_2d_init>",
["null"])
    add_set(["}"],
"<nt_array_elem_next>", ["null"])
    add_set([",", ";"], "<dbl_value>",
["null"])
    add_set([";"],
"<dbl_value_next>", ["null"])
    add_set([",", ";"],
"<dbl_array_initializer>", ["null"])
    add_set(["dbllit", "~dbllit"],
"<dbl_array_elem>",
["<dbl_array_elem_val>",
"<dbl_array_init_next>"])
    add_set(["dbllit", "~dbllit"],
"<dbl_array_elem_val>",
["<dbllliterals>"])
    add_set(["}"],
"<dbl_array_init_next>", ["null"])
    add_set([",", ";"],
"<dbl_2d_init>", ["null"])
    add_set(["}"],
"<dbl_array_elem_next>", ["null"])
    add_set([",", ";"], "<bln_value>",
["null"])
    add_set(["(", "dbllit", "~dbllit",
"ntlit", "~ntlit", "++", "--", "id",
"chrlit", "!", "strnglit", "blnlit"],
"<equals_bln_val>",
["<rel_arith_logic_expr>"])
    add_set([";"],
"<bln_value_next>", ["null"])
    add_set([",", ";"],
"<bln_array_initializer>", ["null"])
    add_set(["blnlit"],
"<bln_array_elem>",
["<bln_array_elem_val>",
"<bln_array_init_next>"])
    add_set(["}"],
"<bln_array_init_next>", ["null"])
    add_set([",", ";"],
"<bln_2d_init>", ["null"])
    add_set(["}"],
"<bln_array_elem_next>", ["null"])
    add_set([",", ";"], "<chr_value>",
["null"])
    add_set([";"],
"<chr_value_next>", ["null"])
    add_set([",", ";"],
"<chr_array_initializer>", ["null"])
    add_set(["chrlit"],
"<chr_array_elem>",
["<chr_array_elem_val>",
"<chr_array_init_next>"])
    add_set(["}"],
"<chr_array_init_next>", ["null"])
    add_set([",", ";"],
"<chr_2d_init>", ["null"])
    add_set(["}"],
"<chr_array_elem_next>", ["null"])
    add_set([",", ";"], "<str_value>",
["null"])
    add_set([";"],
"<str_value_next>", ["null"])
    add_set([",", ";"],
"<str_array_initializer>", ["null"])
    add_set(["}"],
"<str_array_init_next>", ["null"])
    add_set([",", ";"],
"<str_2d_init>", ["null"])
    add_set(["}"],
"<str_array_elem_next>", ["null"])
    add_set(["ntlit", "~ntlit"],
"<nt_cnst_init_val>",
["<ntliterals>", "<nt_cnst_next>"])
    add_set([";"], "<nt_cnst_next>",
["null"])
    add_set(["ntlit", "~ntlit"],
"<cnst_array_size>",
["<ntliterals>"])
    add_set(["dbllit", "~dbllit"],
"<dbl_cnst_init_val>",
["<dbllliterals>",
"<dbl_cnst_next>"])
    add_set([";"], "<dbl_cnst_next>",
["null"])
    add_set([";"], "<bln_cnst_next>",
["null"])
    add_set([";"], "<chr_cnst_next>",
["null"])
    add_set([";"], "<str_cnst_next>",
["null"])
    add_set([";"], "<strct_init_next>",
["null"])
    add_set(["nt", "dbl", "bln", "chr",
"strng"], "<strct_body>",
["<strct_var_dec>", ";",
"<var_dec_next>"])
    add_set(["nt", "dbl", "bln", "chr",
"strng"], "<var_dec_next>",
["<strct_body>"])
    add_set(["}"], "<var_dec_next>",
["null"])
    add_set(["mn"],
"<function_definition>", ["null"])
    add_set(["nt", "dbl", "bln", "chr",
"strng"], "<func_type>",
["<data_type>"])
    add_set(["nt", "dbl", "bln", "chr",
"strng"], "<parameter>",
["<data_type>", "id",
"<parameter_next>"])
    add_set([")"], "<parameter>",
["null"])
    add_set([")"],
"<parameter_next>", ["null"])
    add_set(["nt", "dbl", "bln", "chr",
"strng"], "<func_var_dec>",
["<var_dec>", ";",
"<func_var_dec>"])
    add_set(["id", "++", "--", "prnt",
"f", "swtch", "fr", "d", "whl", "rtrn",
"brk", "cntn" "}"],
"<func_var_dec>", ["null"])
    add_set(["++", "--"],
"<statement>",
["<unary_pre_statement>",
"<statement>"])
    add_set(["f", "swtch"],
"<statement>",
["<conditional_statement>",
"<statement>"])
    add_set(["brk", "cntn"],
"<statement>",
["<control_statements>",
"<statement>"])
    add_set(["end", "}", "lsf", "ls",
"cs", "dflt"], "<statement>",
["null"])

    add_set(["[", "."],
"<id_build_statement_next>",
["<id_next>",
"<assignment_statement>"]) # di ko
alam pano lalagay yung null sa first
set ng <id_next>
    add_set(["+=", "-=", "*=", "/=",
"="], "<id_build_statement_next>",
["<assignment_statement>"])
    add_set(["++", "--"],
"<id_build_statement_next>",
["<unary_inc/dec>", ";"])
```

```
    add_set(["+=", "-=", "*=", "/="],
"<assignment_statement>",
["<assign_operator>",
"<assignment_value>"])
    add_set(["(", "dbllit", "~dbllit",
"ntlit", "~ntlit", "++", "--", "id",
"chrlit", "!", "strnglit", "blnlit"],
"<assignment_value>",
["<expression>", ";"])
    add_set(["(", "dbllit", "~dbllit",
"ntlit", "~ntlit", "++", "--", "id",
"chrlit", "!", "strnglit", "blnlit"],
"<assignment_value_input>",
["<assignment_value>"])
    add_set(["++", "--"],
"<unary_pre_statement>",
["<unary_inc/dec>", "id", ";"])
    add_set(["(", "dbllit", "~dbllit",
"ntlit", "~ntlit", "++", "--", "id",
"chrlit", "!", "strnglit", "blnlit"],
"<output_value>",
["<expression>"])
    add_set([")"], "<output_next>",
["null"])
    add_set(["(", "dbllit", "~dbllit",
"ntlit", "~ntlit", "++", "--", "id",
"chrlit", "!", "strnglit", "blnlit"],
"<condition>",
["<rel_arith_logic_expr>"])
    add_set(["ls", "}", "id", "++",
"--", "prnt", "f", "swtch", "fr", "d",
"whl", "rtrn", "brk", "cntn", "end"],
"<lsf_statement>", ["null"])
    add_set(["}", "id", "++", "--",
"prnt", "f", "swtch", "fr", "d", "whl",
"rtrn", "brk", "cntn", "end"],
"<ls_statement>", ["null"])
    add_set(["ntlit", "~ntlit"],
"<swtch_value>", ["<ntliterals>"])
    add_set(["dflt", "}"], "<cs>",
["null"])
    add_set(["dflt", "}"],
"<cs_next>", ["null"])
    add_set(["}"], "<dflt>", ["null"])
    add_set(["(", "dbllit", "~dbllit",
"ntlit", "~ntlit", "++", "--", "id",
"chrlit", "!", "strnglit", "blnlit"],
"<fr_condi>", ["<condition>"])
    add_set(["++", "--"], "<inc/dec>",
["<fr_unary>"])
    add_set(["+=", "-=", "*=", "/="],
"<inc/dec_next>",
["<assign_operator>",
"<inc/dec_value>"])
    add_set(["++", "--"],
"<inc/dec_next>",
["<unary_inc/dec>"])
    add_set(["++", "--"],
"<fr_unary>", ["<unary_inc/dec>",
"id"])
    add_set(["ntlit", "~ntlit"],
"<inc/dec_value>", ["<ntliterals>"])
    add_set(["(", "dbllit", "~dbllit",
"ntlit", "~ntlit", "++", "--", "id",
"chrlit", "!", "strnglit", "blnlit"],
"<rtrn_value>", ["<expression>"])
    add_set([";"], "<rtrn_value>",
["null"])

from definitions import *

class ParserError(Exception):
    def __init__(self, message,
line=None, column=None):
        super().__init__(message)
        self.message = message
        self.line = line if line is not
None else None
        self.column = column

    def __str__(self):
        if self.line is not None and
self.column is not None:
            return f"{self.message}
(Line {self.line}, Column
{self.column})"
        return self.message

def parse(token_list=None):
    add_all_set()

    # Use provided token list or fall
back to global
    tokens_to_parse = token_list if
token_list is not None else token

    if not tokens_to_parse:
        raise ParserError("Syntax
Error: No tokens provided.")

    stack = ["<program>"]
    current_token_index = 0
    log_messages = []
    error_message = []
    syntax_valid = False  # Add this
flag

    def get_lookahead():
        """Safely retrieve the current
token and its position."""
        if current_token_index <
len(tokens_to_parse):  # FIXED:
Use tokens_to_parse instead of
token
            token_data =
tokens_to_parse[current_token_inde
x]

            # Handle Token objects
            if hasattr(token_data, 'type')
and hasattr(token_data, 'line') and
hasattr(token_data, 'column'):
                curr_token =
token_data.type
                line = token_data.line
                column =
token_data.column
            # Handle token tuples with
various formats
            elif isinstance(token_data,
tuple):
                if len(token_data) == 3:
                    curr_token, line,
column = token_data
                elif len(token_data) == 4:
                    curr_token,
token_value, line, column =
token_data
                else:
                    return "$", None, None
            else:
                # Fallback for other
formats
                return "$", None, None

            if isinstance(curr_token, str)
and curr_token.startswith("id"):
                return "id", line, column
            return curr_token, line,
column
        return "$", None, None

    try:
        while stack:
            top = stack.pop()
            lookahead, line, column =
get_lookahead()

            # Debugging logs
            print(f"Current Index:
{current_token_index}")

log_messages.append(f"Stack Top:
{top}, Lookahead: {lookahead}
(Line {line-1 if line is not None else
line}, Column {column})")

            if lookahead == "$" and top
!= "$":
                raise ParserError("Syntax
Error: Unexpected end of input.",
line, column)

            if top == lookahead:

log_messages.append(f"Matched:
{lookahead} (Line {line-1 if line is
not None else line}, Column
{column})")  # Debug
                current_token_index += 1
```

```
        elif top in parsing_table:
            rule = parsing_table[top].get(lookahead)
            if rule:
                if rule == ["null"]:  # Handle `null` (epsilon) productions

log_messages.append(f"Skipping {top} (Epsilon Production)")
                else:

log_messages.append(f"Applying Rule: {top} -> {' '.join(rule)}")  # Debug

stack.extend(reversed(rule))
            else:
                expected_tokens = list(parsing_table[top].keys())
                raise ParserError(f"Syntax Error: Unexpected token '{lookahead}', expected one of: {expected_tokens}", line, column)
            else:
                raise ParserError(f"Syntax Error: Unexpected symbol '{lookahead}', expected '{top}'", line, column)

        # Check if we parsed all tokens
        if not stack and current_token_index < len(tokens_to_parse) and tokens_to_parse[current_token_index][0] == "EOF":

error_message.append("Input accepted: Syntactically correct.")
            syntax_valid = True  # Set flag to True when syntax is correct
        else:
            raise ParserError("Input rejected: Syntax Error - Unexpected tokens remaining.")

    except ParserError as e:
        error_message.append(str(e))
        syntax_valid = False  # Ensure flag is False on errors
    except Exception as e:

error_message.append(f"Unexpected error: {str(e)}")
        syntax_valid = False  # Ensure flag is False on errors

    return log_messages, error_message, syntax_valid  # Return the flag
```

Semantic Analyzer

```
from lexer import Lexer

class SemanticError(Exception):
    def __init__(self, message, line=None, column=None):
        super().__init__(message)
        self.message = message
        self.line = line
        self.column = column

    def __str__(self):
        if self.line is not None and self.column is not None:
            return f"Semantic Error at line {self.line}, column {self.column}: {self.message}"
        return f"Semantic Error: {self.message}"

class Symbol:
    def __init__(self, name, type, data_type=None, initialized=False, is_constant=False,
                 is_array=False, array_dimensions=None, array_sizes=None, line=None, column=None):
        self.name = name
        self.type = type  # 'variable', 'function', 'struct'
        self.data_type = data_type  # 'nt', 'dbl', 'bln', 'chr', 'strng'
        self.initialized = initialized
        self.is_constant = is_constant
        self.is_array = is_array
        self.array_dimensions = array_dimensions  # 1 for 1D, 2 for 2D
        self.array_sizes = array_sizes # [size] for 1D, [size1, size2] for 2D
        self.line = line
        self.column = column
        # New attribute to track initialized members of struct instances
        self.initialized_members = set() if type == 'struct_instance' else None

    def __str__(self):
        status = "initialized" if self.initialized else "uninitialized"
        constant = "constant " if self.is_constant else ""
        array_info = ""
        if self.is_array:
            if self.array_dimensions == 1:
                array_info = f"[{self.array_sizes[0]}]"
            elif self.array_dimensions == 2:
                array_info = f"[{self.array_sizes[0]}][{self.array_sizes[1]}]"

        location = f"(line {self.line}, col {self.column})" if self.line and self.column else ""
        return f"{self.name}: {constant}{self.type} {self.data_type}{array_info} ({status}) {location}"

class FunctionSymbol(Symbol):
    def __init__(self, name, return_type, parameters=None, line=None, column=None):
        super().__init__(name, 'function', return_type, True, False, False, None, None, line, column)
        self.parameters = parameters or []
        self.has_return_statement = False  # Track if function has a return statement
        # ADDED: To store start index of function body tokens (index AFTER '{')
        self.body_start_index = None

    def __str__(self):
```

```
        params = [f"{param.data_type} {param.name}" for param in self.parameters]
        params_str = ", ".join(params)
        location = f"(line {self.line}, col {self.column})" if self.line and self.column else ""
        # Indicate if body start index is set (for debugging)
        body_info = f" [Body Start Idx: {self.body_start_index}]" if self.body_start_index is not None else ""
        return f"{self.name}: {self.type} returns {self.data_type} ({params_str}) {location}{body_info}"

# Add this class after FunctionSymbol
class StructSymbol(Symbol):
    def __init__(self, name, members=None, line=None, column=None):
        super().__init__(name, 'struct', None, True, False, False, None, None, line, column)
        self.members = members or {} # Dict of member name -> Symbol

    def __str__(self):
        member_str = ", ".join([f"{symbol.data_type} {name}" for name, symbol in self.members.items()])
        location = f"(line {self.line}, col {self.column})" if self.line and self.column else ""
        return f"{self.name}: {self.type} members: {{{member_str}}} {location}"

class SymbolTable:
    def __init__(self, parent=None, scope_name="global"):
        self.symbols = {}
        self.parent = parent
        self.scope_name = scope_name

    def insert(self, name, symbol):
        if name in self.symbols:
            return False
        self.symbols[name] = symbol
        return True

    def lookup(self, name):
        # First check in current scope
        if name in self.symbols:
            return self.symbols[name]
        # If not found in current scope and this is a function scope,
        # check directly in global scope (skip any intermediate scopes)
        if self.scope_name.startswith("function "):
            # We're in a function scope, check global scope directly
            global_scope = None

            # Find the global scope
            temp_scope = self
            while temp_scope.parent:
                temp_scope = temp_scope.parent
            global_scope = temp_scope

            # Check in global scope
            if name in global_scope.symbols:
                print(f"Found '{name}' in global scope from function scope '{self.scope_name}'")
                return global_scope.symbols[name]

        # Normal parent lookup
        if self.parent:
            return self.parent.lookup(name)

        return None

    def print_table(self, indent=0):
        """Print the contents of the symbol table"""
        indent_str = "  " * indent
        print(f"{indent_str}=== Symbol Table: {self.scope_name} ===")

        if not self.symbols:
            print(f"{indent_str} (empty)")
            return

        # Print all symbols in current scope
        for name, symbol in sorted(self.symbols.items()):
            print(f"{indent_str} {symbol}")

class SemanticAnalyzer:
    def __init__(self):
        self.current_token_index = 0
        self.token_stream = []
        self.current_scope = None
        self.global_scope = SymbolTable(scope_name="global")
        self.function_scopes = {}  # Map function name to its SymbolTable

        # Define valid data types
        self.data_types = {'nt', 'dbl', 'bln', 'chr', 'strng'}

        # Define valid operators
        self.arithmetic_operators = {'+', '-', '*', '/', '%'}
        self.logical_operators = {'&&', '||'}
        self.relational_operators = {'<', '>', '<=', '>=', '==', '!='}
        self.equality_operators = {'==', '!='}  # Subset of relational that works for all types
        self.assignment_operators = {'='}
        self.unary_operators = {'!'}
        self.string_concat_operator = {'`'}

        # Define built-in functions
        self.built_in_functions = {'prnt', 'scan', 'len', 'npt'}

        # Add struct-related keywords
        self.struct_keywords = {'strct', 'dfstrct'}

        # Add these tracking flags
        self.in_loop = False
        self.in_switch = False
        self.in_case_block = False  # New flag to track if we're inside a case/default block

    def print_symbol_tables(self):
        """Print all symbol tables in the current scope hierarchy, including function scopes"""
        print("\nSymbol Table Contents:")

print("====================")

        # Print global scope hierarchy
        current = self.current_scope
        scopes = []
        while current:
            scopes.insert(0, current)
            current = current.parent
        for i, scope in enumerate(scopes):
            scope.print_table(indent=i)
```

```
    # Print all function scopes (including main)
    if hasattr(self, "function_scopes"):
        for func_name, func_scope in self.function_scopes.items():
            print(f"\n=== Symbol Table: {func_scope.scope_name} ===")

func_scope.print_table(indent=1)

  # --- Modified collect_declarations ---
  def collect_declarations(self):
    """First pass: Collect struct definitions and function signatures (including main)."""
    print("Collecting declarations (Pass 1)...")
    original_index = self.current_token_index # Save entry index (usually 0)
    temp_index = 0 # Use a temporary index to scan the entire stream

    while temp_index < len(self.token_stream):
        # Set current_token_index temporarily for analysis functions
        self.current_token_index = temp_index
        token_type, token_value, line, column = self.get_current_token()

        if token_type == 'fnctn':
            func_name = "unknown"
            if temp_index + 2 < len(self.token_stream):
                name_token = self.token_stream[temp_index + 2]
                if name_token[0] == 'id': func_name = name_token[1]
            print(f"Pass 1: Found 'fnctn' for '{func_name}' at index {temp_index}.")

self.analyze_function_declaration(is_first_pass=True)
            temp_index = self.current_token_index
            print(f"Pass 1: After skipping '{func_name}', temp_index is now {temp_index}.")
            continue

        elif token_type == 'strct':
            struct_name = "unknown"
            if temp_index + 1 < len(self.token_stream):
                name_token = self.token_stream[temp_index + 1]
                if name_token[0] == 'id': struct_name = name_token[1]
            print(f"Pass 1: Found 'strct' for '{struct_name}' at index {temp_index}.")

self.analyze_struct_declaration()
            temp_index = self.current_token_index
            print(f"Pass 1: After processing '{struct_name}', temp_index is now {temp_index}.")
            continue

        # --- ADDED: Handle 'mn' signature collection in Pass 1 ---
        elif token_type == 'mn':
            print(f"Pass 1: Found 'mn' function declaration at index {temp_index}.")
            mn_line, mn_col = line, column
            # --- Start: Skipping mn() { signature ---
            # We need to advance self.current_token_index to find body start
            # and then update temp_index to after the skipped body
            saved_scan_index = self.current_token_index # Save index at 'mn'
            self.advance() # Past 'mn'

            # Check syntax: mn() {
            if self.current_token_index >= len(self.token_stream) or self.get_current_token()[0] != '(':
                raise SemanticError("Expected '(' after 'mn'", self.get_current_token()[2], self.get_current_token()[3])
            self.advance() # Past '('
            if self.current_token_index >= len(self.token_stream) or self.get_current_token()[0] != ')':
                raise SemanticError("Expected ')' after '(' in 'mn'", self.get_current_token()[2], self.get_current_token()[3])
            self.advance() # Past ')'
            if self.current_token_index >= len(self.token_stream) or self.get_current_token()[0] != '{':
                raise SemanticError("Expected '{' after 'mn()'", self.get_current_token()[2], self.get_current_token()[3])
            body_start_index = self.current_token_index + 1 # Index after '{'
            # --- End: Skipping mn() { signature ---

            # Create symbol for mn
            main_symbol = FunctionSymbol(
                name="mn",
                return_type="vd",
                parameters=[],
                line=mn_line,
                column=mn_col
            )

main_symbol.body_start_index = body_start_index

            # Insert into global scope
            if not self.global_scope.insert("mn", main_symbol):
                # Check if already declared (should only happen once)
                existing = self.global_scope.lookup("mn")
                if existing and existing.type == 'function':
                    raise SemanticError("'mn' function can only be declared once", mn_line, mn_col)
                else: # If 'mn' exists but isn't a function (e.g., variable)
                    raise SemanticError("Symbol 'mn' already declared with a different type", mn_line, mn_col)
            print(f"Pass 1: Added 'mn' symbol to global scope.")

            # Create and store scope for mn
            if "mn" not in self.function_scopes:
                main_scope = SymbolTable(parent=self.global_scope, scope_name="function mn")

self.function_scopes["mn"] = main_scope
                print(f"Pass 1: Created scope for 'mn'.")
            else:
```

```
                # This case should
ideally not happen if mn can only be
declared once
                print(f"Warning: Scope
for 'mn' already existed in
self.function_scopes during Pass
1.")
                pass # Or raise error?

            # Skip body
(self.current_token_index is at '{')
            self.advance() # Past '{'
            brace_count = 1
            while brace_count > 0 and
self.current_token_index <
len(self.token_stream):
                t_type =
self.get_current_token()[0]
                if t_type == '{':
brace_count += 1
                elif t_type == '}':
brace_count -= 1
                if brace_count > 0:
self.advance() # Avoid advancing
past the final '}'

            # Now at final '}' or end
of stream
            if
self.current_token_index <
len(self.token_stream) and
self.get_current_token()[0] == '}':
                self.advance() # Past '}'
            else: # Reached end of
stream without closing brace
                raise
SemanticError("Unclosed body for
'mn'", line, column) # Use line/col
of '{' maybe?

            # Update the main loop
index (temp_index) to where we
ended up
            temp_index =
self.current_token_index
            print(f"Pass 1: After
skipping 'mn' body, temp_index is
now {temp_index}.")
            continue # Skip default
temp_index increment
        # --- End 'mn' handling ---

        # If not a function or struct
or main, just move to the next token
        temp_index += 1

    # Restore the original token
index before starting pass 2
    self.current_token_index =
original_index
    print("Finished collecting
declarations (Pass 1).")

  def
skip_function_declaration(self):
    """Skip past a function
declaration and body"""
    self.advance()  # Skip 'fnctn'

    # Skip return type if present
    token_type, token_value, line,
column = self.get_current_token()
    if token_type == 'vd' or
token_type in self.data_types:
        self.advance()

    # Skip function name
    self.advance()

    # Skip opening parenthesis
    self.advance()

    # Skip parameters
    paren_count = 1
    while paren_count > 0 and
self.current_token_index <
len(self.token_stream):
        token_type =
self.get_current_token()[0]
        if token_type == '(':
            paren_count += 1
        elif token_type == ')':
            paren_count -= 1
        self.advance()

    # Skip to opening brace of
function body
    while self.current_token_index
< len(self.token_stream) and
self.get_current_token()[0] != '{':
        self.advance()

    self.advance()  # Skip opening
brace

    # Skip function body
    brace_count = 1
    while brace_count > 0 and
self.current_token_index <
len(self.token_stream):
        token_type =
self.get_current_token()[0]
        if token_type == '{':
            brace_count += 1
        elif token_type == '}':
            brace_count -= 1
        self.advance()

  def skip_struct_declaration(self):
    """Skip past a struct
declaration"""
    self.advance()  # Skip 'strct'

    # Skip struct name
    self.advance()

    # Skip to opening brace
    while self.current_token_index
< len(self.token_stream) and
self.get_current_token()[0] != '{':
        self.advance()

    self.advance()  # Skip opening
brace

    # Skip struct body
    brace_count = 1
    while brace_count > 0 and
self.current_token_index <
len(self.token_stream):
        token_type =
self.get_current_token()[0]
        if token_type == '{':
            brace_count += 1
        elif token_type == '}':
            brace_count -= 1
        self.advance()

    # Skip semicolon after closing
brace
    if self.get_current_token()[0]
== ';':
        self.advance()

  # --- Modified
analyze_function_declaration ---
  def
analyze_function_declaration(self,
is_first_pass=True):
    """
    Analyzes 'fnctn' declaration.
    Pass 1: Collects signature,
creates scope, stores body start
index, skips body.
    Pass 2: Retrieves
symbol/scope, analyzes body.
    """
    # Ensure we're starting with
'fnctn' keyword
    token_type, _, line_fn, col_fn =
self.get_current_token() # Line/col
of 'fnctn'
    if token_type != 'fnctn':
        raise
SemanticError(f"Expected 'fnctn'
keyword, got '{token_type}'",
line_fn, col_fn)

    self.advance()  # Move past
'fnctn'

    # Check for return type
```

```
        token_type, _, line_ret, col_ret
= self.get_current_token()
        return_type = None
        if token_type == 'vd':
            return_type = 'vd'
            self.advance()
        elif token_type in
self.data_types:
            return_type = token_type
            self.advance()
        else:
            raise
SemanticError(f"Expected return
type or 'vd', got '{token_type}'",
line_ret, col_ret)

        # Get function name
        token_type, func_name,
line_name, col_name =
self.get_current_token()
        if token_type != 'id':
            raise
SemanticError(f"Expected function
name, got '{token_type}'",
line_name, col_name)
        # --- CRITICAL: 'mn' should
not be declared using 'fnctn' ---
        if func_name == 'mn':
            raise SemanticError(f"'mn' is
a reserved keyword and cannot be
used as a function name here. Use
'mn()' syntax.", line_name,
col_name)
        # --- End Check ---

        self.advance()  # Move past
function name

        # Process opening parenthesis
for parameters
        token_type, _, line_paren1,
col_paren1 =
self.get_current_token()
        if token_type != '(':
            raise
SemanticError(f"Expected '(' after
function name, got '{token_type}'",
line_paren1, col_paren1)
        self.advance()

        # Parse parameters
        parameters = []
        if self.get_current_token()[0]
!= ')':

self.parse_parameter(parameters)
            while
self.get_current_token()[0] == ',':
                self.advance()

self.parse_parameter(parameters)

        # Check for closing parenthesis
        token_type, _, line_paren2,
col_paren2 =
self.get_current_token()
        if token_type != ')':
            raise
SemanticError(f"Expected ')' after
parameters, got '{token_type}'",
line_paren2, col_paren2)
        self.advance()

        # --- Symbol and Scope
Handling (Pass 1 vs Pass 2) ---
        func_symbol = None
        function_scope = None

        if is_first_pass:
            # Pass 1: Create symbol, add
to global scope, create function
scope
            func_symbol =
FunctionSymbol(
                name=func_name,
                return_type=return_type,
                parameters=parameters,
                line=line_name, # Use
line/col of function name for
symbol
                column=col_name
            )
            if not
self.global_scope.insert(func_name,
func_symbol):
                existing =
self.global_scope.lookup(func_nam
e)
                if existing and
existing.type == 'function':
                    raise
SemanticError(f"Function
'{func_name}' already declared",
line_name, col_name)
                else:
                    raise
SemanticError(f"Symbol
'{func_name}' already declared with
a different type", line_name,
col_name)
            else:
                print(f"Pass 1: Added
function '{func_name}' symbol to
global scope.")

            # Create and store function
scope
            function_scope =
SymbolTable(parent=self.global_sc
ope, scope_name=f"function
{func_name}")
            for param in parameters:
                if not
function_scope.insert(param.name,
param):
                    param_line, param_col
= param.line, param.column
                    raise
SemanticError(f"Duplicate
parameter name '{param.name}' in
function '{func_name}'",
param_line, param_col)

self.function_scopes[func_name] =
function_scope
            print(f"Pass 1: Created scope
for function '{func_name}'.")

        else:
            # Pass 2: Retrieve existing
symbol and scope
            func_symbol =
self.global_scope.lookup(func_nam
e)
            if not func_symbol or
func_symbol.type != 'function':
                raise
SemanticError(f"Internal Error:
Function symbol for '{func_name}'
not found during pass 2",
line_name, col_name)
            if func_name not in
self.function_scopes:
                raise
SemanticError(f"Internal Error:
Function scope for '{func_name}'
not found during pass 2",
line_name, col_name)
            function_scope =
self.function_scopes[func_name]
            print(f"Pass 2: Retrieved
symbol and scope for function
'{func_name}'.")


        # Process opening brace for
function body
        token_type, _, brace_line,
brace_column =
self.get_current_token()
        if token_type != '{':
            raise
SemanticError(f"Expected '{{' to
start function body, got
'{token_type}'", brace_line,
brace_column)

        if is_first_pass:
            # Pass 1: Store body start
index and skip the body

func_symbol.body_start_index =
```

```
self.current_token_index + 1 # Store index AFTER '{'
        print(f"Pass 1: Stored body start index {func_symbol.body_start_index} for '{func_name}'. Skipping body.")
        self.advance() # Move past '{'
        brace_count = 1
        while brace_count > 0 and self.current_token_index < len(self.token_stream):
            token_type = self.get_current_token()[0]
            if token_type == '{': brace_count += 1
            elif token_type == '}': brace_count -= 1
            if brace_count > 0: self.advance()
        if self.get_current_token()[0] == '}': self.advance() # Move past the final '}'
        else: raise SemanticError(f"Unclosed function body for '{func_name}'", brace_line, brace_column)
        print(f"Pass 1: Finished skipping body for '{func_name}'. Current index: {self.current_token_index}")

    else:
        # Pass 2: Analyze the body now
        print(f"Pass 2: Analyzing body for '{func_name}' starting at index {self.current_token_index + 1}.")
        self.advance() # Move past '{'
        old_scope = self.current_scope
        self.current_scope = function_scope # Switch to function's scope
        has_return = self.analyze_function_body(return_type)
        if return_type != 'vd' and not has_return:
            raise SemanticError(f"Function '{func_name}' with return type '{return_type}' must have a return statement",
                                func_symbol.line, func_symbol.column)
        func_symbol.has_return_statement = has_return
        self.current_scope = old_scope
        print(f"Pass 2: Finished analyzing body for '{func_name}'. Current index: {self.current_token_index}")

    def parse_parameter(self, parameters):
        """Parse a single function parameter and add it to the parameters list"""
        # Get parameter type
        token_type, token_value, line, column = self.get_current_token()
        if token_type not in self.data_types:
            raise SemanticError(f"Expected parameter type, got '{token_type}'", line, column)

        param_type = token_type
        self.advance()  # Move past parameter type

        # Get parameter name
        token_type, token_value, line, column = self.get_current_token()
        if token_type != 'id':
            raise SemanticError(f"Expected parameter name, got '{token_type}'", line, column)

        param_name = token_value

        # Create parameter symbol
        param_symbol = Symbol(
            name=param_name,
            type='variable',
            data_type=param_type,
            initialized=True,  # Parameters are considered initialized
            line=line,
            column=column
        )

        parameters.append(param_symbol)
        self.advance()  # Move past parameter name

    def analyze_function_body(self, return_type):
        """Analyze the function body and check for return statements"""
        has_return = False

        # Save body start position for second pass if needed
        body_start = self.current_token_index

        # Save old context and ensure we're not in a loop or switch context
        old_in_loop = self.in_loop
        old_in_switch = self.in_switch
        self.in_loop = False
        self.in_switch = False

        # Process function body until we find closing brace
        while self.current_token_index < len(self.token_stream):
            token_type, token_value, line, column = self.get_current_token()

            # Debug output
            print(f"Function body token: {token_type} '{token_value}' at line {line}, column {column}")

            # Check for end of function
            if token_type == '}':
                self.advance()  # Move past '}'
                break

            # Check for direct break or continue at function level
            if token_type == 'brk':
                raise SemanticError("Break statement cannot be used directly in a function body", line, column)

            if token_type == 'cntn':
                raise SemanticError("Continue statement cannot be used directly in a function body", line, column)

            # Check for struct member access (identifier followed by dot)
            if token_type == 'id':
                next_token = self.peek_next_token()
                if next_token and next_token[0] == '.':
                    print(f"Found struct member access pattern, routing to analyze_struct_member_access for {token_value}.{next_token[1]}")
                    self.analyze_struct_member_access()
```

```
                continue
            # Check for
increment/decrement operators
            elif next_token and
next_token[0] in ['++', '--']:

self.analyze_increment_operation()
                continue
            # Check for shortcut
assignments
            elif next_token and
next_token[0] in ['+=', '-=', '*=', '/=',
'%=']:
                print(f"Found shortcut
assignment {token_value}
{next_token[0]}")

self.analyze_assignment()
                continue

        # Check for return statement
        if token_type == 'rtrn':
            has_return = True

self.analyze_return_statement(retur
n_type, line, column)
            continue

        # Check for print statement
        if token_type == 'prnt':
            print("Found print
statement in function body - calling
analyze_print_statement()")  #
Debug

self.analyze_print_statement()
            continue

        # Handle conditional
statements
        if token_type == 'f':

self.analyze_if_statement()
            continue

        # Handle switch statements
        if token_type == 'swtch':

self.analyze_switch_statement()
            continue

        # Handle for loops
        if token_type == 'fr':
            self.analyze_for_loop()
            continue

        # Handle while loops
        if token_type == 'whl':
            self.analyze_while_loop()
            continue

        # Handle do-while loops
        if token_type == 'd':

self.analyze_do_while_loop()
            continue

        # Handle other statements
using existing methods
        if token_type == 'cnst':

self.analyze_constant_declaration()
        elif token_type == 'dfstrct':

self.analyze_struct_instantiation()
        elif token_type in
self.data_types:
            start_pos =
self.current_token_index
            self.advance()
            if
self.current_token_index >=
len(self.token_stream) or
self.token_stream[self.current_token
_index][0] != 'id':
                raise
SemanticError(f"Expected an
identifier after '{token_type}'", line,
column)

            next_token =
self.peek_next_token()
            if next_token and
next_token[0] == '[':

self.current_token_index = start_pos

self.analyze_array_declaration()
            else:

self.current_token_index = start_pos

self.analyze_variable_declaration()
        elif token_type == 'id' and
self.peek_next_token()[0] == '(':

self.analyze_function_call()
        elif token_type == 'id' and
self.peek_next_token()[0] == '[':

self.analyze_array_access()

            # Skip to end of statement
            while
self.current_token_index <
len(self.token_stream) and
self.get_current_token()[0] != ';':
                self.advance()
            self.advance()  # Move
past semicolon
        elif token_type == 'id':
            next_token =
self.peek_next_token()
            if next_token and
next_token[0] == '=':

self.analyze_assignment()
            else:

self.check_variable_usage(token_va
lue, line, column)
                self.advance()
        else:
            self.advance()

    # If we haven't found a return
but need one (non-void function),
    # do a more thorough scan of
the function body
    if not has_return and
return_type != 'vd':
        print("No return found in
first pass, performing second linear
scan")
        # Save current position
        current_pos =
self.current_token_index

        # Reset to start of function
body
        self.current_token_index =
body_start

        # Scan for any return
statements in the function body
        brace_level = 1  # We start
inside the function body
        while
self.current_token_index <
len(self.token_stream):
            token =
self.token_stream[self.current_token
_index]
            token_type, token_value,
token_line, token_column = token

            # Track brace level to stay
within function
            if token_type == '{':
                brace_level += 1
            elif token_type == '}':
                brace_level -= 1
                if brace_level == 0:
                    # We've reached the
end of the function
                    break

            # Check for any return
statements
            if token_type == 'rtrn':
                print(f"Found return
statement in second pass at line
```

```
{token_line}, column
{token_column}")
                has_return = True
                break

            self.current_token_index
+= 1

        # Restore original position
        self.current_token_index =
current_pos

    # Restore old context
    self.in_loop = old_in_loop
    self.in_switch = old_in_switch

    print(f"Function body analysis
complete. Has return:
{has_return}")
    return has_return

  def
analyze_return_statement(self,
function_return_type, line, column):
    """Analyze a return statement
and ensure it matches the function's
return type"""
    self.advance()  # Move past
'rtrn'

    # --- REMOVED CHECK
FOR MAIN FUNCTION ---
    # # Check if we're in the main
function
    # current_function = None
    # temp_scope =
self.current_scope
    # while temp_scope:
    #     if
temp_scope.scope_name.startswith(
"function "):
    #         current_function =
temp_scope.scope_name[len("functi
on "):]
    #         break
    #     temp_scope =
temp_scope.parent
    #
    # # No return statements
allowed in main function
    # if current_function == "mn":
    #     raise
SemanticError("Return statements
are not allowed in the main
function", line, column)
    # --- END REMOVED
CHECK ---

    # Get the next token to
determine if there's a return
expression
    token_type, token_value,
token_line, token_column =
self.get_current_token()

    # For void functions, return
should not have a value
    if function_return_type == 'vd':
        # --- ADDED: Check if we
are in 'mn' function, which now
allows return 0 ---
        is_main_function = False
        temp_scope =
self.current_scope
        while temp_scope:
            if
temp_scope.scope_name ==
"function mn":
                is_main_function =
True
                break
            temp_scope =
temp_scope.parent

        if is_main_function:
            # Allow 'rtrn 0;' in main
            if token_type == 'ntlit' and
token_value == '0':
                self.advance() # Past 0
                if
self.get_current_token()[0] == ';':
                    self.advance() # Past
;
                    return # Valid return
0; in main
                else:
                    raise
SemanticError("Expected ';' after
'rtrn 0' in main function",
self.get_current_token()[2],
self.get_current_token()[3])
            elif token_type == ';':
                # Allow 'rtrn;' in main
(implicitly returns 0 later)
                self.advance() # Past ;
                return
            else:
                raise
SemanticError(f"Main function
('mn') can only return '0' or have no
return value ('rtrn;'), got
'{token_value}'", token_line,
token_column)
        # --- END ADDED CHECK
for 'mn' ---
        elif token_type != ';': #
Original check for other void
functions
            raise
SemanticError(f"Void function
cannot return a value", token_line,
token_column)
        self.advance()  # Move past
semicolon for non-main void
functions
        return

    # For non-void functions (and
potentially main now), there must
be a return value
    if token_type == ';':
        raise
SemanticError(f"Function with
return type '{function_return_type}'
must return a value", token_line,
token_column)

    # Find the end of the return
expression (semicolon)
    start_pos =
self.current_token_index

    while self.current_token_index
< len(self.token_stream) and
self.token_stream[self.current_token
_index][0] != ';':
        self.advance()

    if self.current_token_index >=
len(self.token_stream):
        raise
SemanticError("Unexpected end of
file inside return statement", line,
column)

    end_pos =
self.current_token_index
    self.current_token_index =
start_pos

    # Analyze the expression
    expr_type =
self.analyze_expression(end_pos)

    # --- MODIFIED CHECK for
'mn' ---
    is_main_function = False
    temp_scope =
self.current_scope
    while temp_scope:
        if temp_scope.scope_name
== "function mn":
            is_main_function = True
            break
        temp_scope =
temp_scope.parent

    if is_main_function:
        # In 'mn', only allow
returning an integer literal 0
        if not (expr_type == 'nt' and
self.token_stream[start_pos][0] ==
'ntlit' and
```

```
self.token_stream[start_pos][1] == '0' and start_pos + 1 == end_pos):
                raise SemanticError(f"Main function ('mn') can only return the integer literal '0', got expression of type '{expr_type}'", token_line, token_column)
        # --- END MODIFIED CHECK ---
        # Check if return type matches function return type (for non-main functions)
        elif expr_type != function_return_type:
            raise SemanticError(f"Return type mismatch: expected '{function_return_type}', got '{expr_type}'", token_line, token_column)

        # Move past the semicolon
        if self.get_current_token()[0] == ';':
            self.advance()

    def analyze_function_call(self):
        """Analyze a function call and its arguments"""
        token_type, func_name, line, column = self.get_current_token()

        # Debug output
        print(f"Analyzing function call for '{func_name}' at line {line}, column {column}")

        # Check if this is a built-in function
        if func_name in self.built_in_functions:
            # Handle built-in functions
            print(f"  '{func_name}' is a built-in function")
            self.advance()  # Move past function name
            self.advance()  # Move past opening parenthesis

            # Process arguments and move past closing parenthesis
            arguments = self.parse_function_arguments()

            # For npt function, return appropriate data type
            if func_name == 'npt':
                print("  Processing 'npt' input function")
                # npt function with a string prompt returns a string by default
                # The actual type conversion happens at runtime
                result_type = 'strng'

                # Check if there's exactly one argument of type string
                if len(arguments) != 1 or arguments[0] != 'strng':
                    raise SemanticError(f"Input function 'npt' requires a single string prompt", line, column)

                # Check for semicolon after function call
                if self.get_current_token()[0] == ';':
                    self.advance()  # Move past semicolon

                return result_type

            # Check for semicolon after function call
            if self.get_current_token()[0] == ';':
                self.advance()  # Move past semicolon

            return None if func_name != 'len' else 'nt' # len() returns an integer
        else:
            # First look in the global scope directly for functions
            func_symbol = self.global_scope.symbols.get(func_name)

            # If not found, try the regular lookup which includes current scope and parent scopes
            if not func_symbol:
                func_symbol = self.current_scope.lookup(func_name)

            # Debug output - check what's in global scope
            print(f"  Global scope functions: {[name for name, sym in self.global_scope.symbols.items() if sym.type == 'function']}")

            if not func_symbol:
                raise SemanticError(f"Undefined function '{func_name}'", line, column)

            if func_symbol.type != 'function':
                raise SemanticError(f"'{func_name}' is not a function", line, column)

            print(f"  Found function '{func_name}' with return type '{func_symbol.data_type}'")

            self.advance()  # Move past function name
            self.advance()  # Move past opening parenthesis

            # Parse arguments
            arguments = self.parse_function_arguments()

            # Check number of arguments against parameters
            if len(arguments) != len(func_symbol.parameters):
                raise SemanticError(
                    f"Function '{func_symbol.name}' expects {len(func_symbol.parameters)} arguments, got {len(arguments)}",
                    line, column
                )

            # Check argument types against parameter types
            for i, (arg_type, param) in enumerate(zip(arguments, func_symbol.parameters)):
                if arg_type != param.data_type:
                    raise SemanticError(
                        f"Argument {i+1} type mismatch: expected '{param.data_type}', got '{arg_type}'",
                        line, column
                    )

            # Check for semicolon after function call
            if self.get_current_token()[0] == ';':
                self.advance()  # Move past semicolon

            return func_symbol.data_type  # Return the function's return type
```

```
    def
parse_function_arguments(self):
        """Parse function arguments
and return a list of their types"""
        argument_types = []

        # If no arguments (empty
parentheses)
        if self.get_current_token()[0]
== ')':
            self.advance()  # Move past
closing parenthesis
            return argument_types

        # Process arguments
        while True:
            # Find the end of this
argument
            arg_start =
self.current_token_index
            paren_level = 0
            bracket_level = 0

            # Continue until we find a
comma or closing parenthesis at the
top level
            while
self.current_token_index <
len(self.token_stream):
                token_type =
self.token_stream[self.current_token
_index][0]

                if token_type == '(':
                    paren_level += 1
                elif token_type == ')':
                    if paren_level == 0:
                        # End of all
arguments
                        break
                    paren_level -= 1
                elif token_type == '[':
                    bracket_level += 1
                elif token_type == ']':
                    bracket_level -= 1
                elif token_type == ',' and
paren_level == 0 and bracket_level
== 0:
                    # End of current
argument
                    break

                self.current_token_index
+= 1

            # Reset to the start of the
argument
            arg_end =
self.current_token_index
            self.current_token_index =
arg_start

            # Parse the current argument
expression
            arg_type =
self.analyze_expression(arg_end)

argument_types.append(arg_type)

            # Now at the end of the
argument
            self.current_token_index =
arg_end

            # Check for comma or
closing parenthesis
            if self.get_current_token()[0]
== ',':
                self.advance()  # Move
past comma
            elif
self.get_current_token()[0] == ')':
                self.advance()  # Move
past closing parenthesis
                break
            else:
                raise SemanticError(
                    f"Expected ',' or ')' after
function argument, got
'{self.get_current_token()[0]}'",

self.get_current_token()[2],
self.get_current_token()[3]
                )

        return argument_types

    def
parse_function_argument(self):
        """Parse a single function
argument and return its type"""
        arg_start =
self.current_token_index

        # Handle special cases like
literals, identifiers, etc.
        token_type, token_value, line,
column = self.get_current_token()

        # Literal values
        if token_type in ['ntlit', '~ntlit']:
            self.advance()
            return 'nt'
        elif token_type in ['dbllit',
'~dbllit']:
            self.advance()
            return 'dbl'
        elif token_type in ['true', 'false',
'blnlit']:
            self.advance()
            return 'bln'
        elif token_type == 'chrlit':
            self.advance()
            return 'chr'
        elif token_type == 'strnglit':
            self.advance()
            return 'strng'
        elif token_type == 'id':
            # Could be a variable or a
function call
            symbol =
self.current_scope.lookup(token_val
ue)
            if not symbol:
                raise
SemanticError(f"Undefined
identifier '{token_value}'", line,
column)

            # Check if it's a function call
            self.advance()
            if self.get_current_token()[0]
== '(':
                # It's a function call
within an argument
                self.current_token_index
= arg_start  # Go back to the start
                return
self.analyze_function_call()   #
Process the function call and return
its type
            else:
                # It's a variable reference
                return symbol.data_type
        elif token_type == '(':
            # Expression in parentheses
            self.advance()  # Skip
opening parenthesis

            # Find the matching closing
parenthesis
            paren_level = 1
            expr_start =
self.current_token_index

            while paren_level > 0:
                self.advance()
                if
self.current_token_index >=
len(self.token_stream):
                    raise
SemanticError("Unclosed
parenthesis", line, column)

                current_token =
self.get_current_token()[0]
                if current_token == '(':
                    paren_level += 1
                elif current_token == ')':
                    paren_level -= 1

            # Analyze the expression
within the parentheses
```

```
            expr_end =
self.current_token_index
            self.current_token_index =
expr_start
            expr_type =
self.analyze_expression(expr_end)

            self.advance()  # Skip
closing parenthesis
            return expr_type
        else:
            # IMPROVED: Handle
general expressions with proper
nesting
            # Find the end of this
argument considering nested
parentheses/brackets
            paren_level = 0
            bracket_level = 0

            # Continue until we find a
comma or closing parenthesis at the
top level
            while
self.current_token_index <
len(self.token_stream):
                current_token =
self.get_current_token()[0]

                if current_token == '(':
                    paren_level += 1
                elif current_token == ')':
                    if paren_level == 0:
                        # End of all
arguments
                        break
                    paren_level -= 1
                elif current_token == '[':
                    bracket_level += 1
                elif current_token == ']':
                    bracket_level -= 1
                elif current_token == ','
and paren_level == 0 and
bracket_level == 0:
                    # End of current
argument
                    break

                self.advance()

            # Analyze the expression
            expr_end =
self.current_token_index
            self.current_token_index =
arg_start
            return
self.analyze_expression(expr_end)

    def
analyze_struct_declaration(self):
        """Analyze a struct
declaration"""
        # Ensure we're starting with
'strct' keyword
        token_type, token_value, line,
column = self.get_current_token()
        if token_type != 'strct':
            raise
SemanticError(f"Expected 'strct'
keyword, got '{token_type}'", line,
column)

        self.advance()  # Move past
'strct'

        # Get struct name
        token_type, token_value, line,
column = self.get_current_token()
        if token_type != 'id':
            raise
SemanticError(f"Expected struct
name, got '{token_type}'", line,
column)

        struct_name = token_value

        # Check for duplicate
declaration
        if
self.current_scope.lookup(struct_na
me):
            raise SemanticError(f"Struct
'{struct_name}' already declared",
line, column)

        self.advance()  # Move past
struct name

        # Process opening brace
        token_type, token_value, line,
column = self.get_current_token()
        if token_type != '{':
            raise
SemanticError(f"Expected '{' after
struct name, got '{token_type}'",
line, column)

        self.advance()  # Move past '{'

        # Process struct members
        members = {}

        while self.current_token_index
< len(self.token_stream):
            token_type, token_value,
line, column =
self.get_current_token()

            # Check for end of struct
            if token_type == '}':
                break

            # Parse member data type
            if token_type not in
self.data_types:
                raise
SemanticError(f"Expected a data
type for struct member, got
'{token_type}'", line, column)

            member_type = token_type
            self.advance()  # Move past
data type

            # Parse member name
            token_type, token_value,
line, column =
self.get_current_token()
            if token_type != 'id':
                raise
SemanticError(f"Expected member
name, got '{token_type}'", line,
column)

            member_name =
token_value

            # Check for duplicate
member
            if member_name in
members:
                raise
SemanticError(f"Duplicate member
'{member_name}' in struct
'{struct_name}'", line, column)

            # Create member symbol
            member_symbol = Symbol(
                name=member_name,
                type='variable',
                data_type=member_type,
                initialized=False,  #
Members start uninitialized
                line=line,
                column=column
            )

            members[member_name] =
member_symbol

            self.advance()  # Move past
member name

            # Check for semicolon
            token_type, token_value,
line, column =
self.get_current_token()
            if token_type != ';':
                raise
SemanticError(f"Expected ';' after
struct member, got '{token_type}'",
line, column)
```

```
        self.advance()  # Move past semicolon

        # Check for closing brace
        token_type, token_value, line, column = self.get_current_token()
        if token_type != '}':
            raise SemanticError(f"Expected '}}' at end of struct, got '{token_type}'", line, column)

        self.advance()  # Move past '}'

        # Check for semicolon after struct declaration
        token_type, token_value, line, column = self.get_current_token()
        if token_type != ';':
            raise SemanticError(f"Expected ';' after struct declaration, got '{token_type}'", line, column)

        self.advance()  # Move past semicolon

        # Create struct symbol
        struct_symbol = StructSymbol(
            name=struct_name,
            members=members,
            line=line,
            column=column
        )

        # Add to global scope (structs are global only)
        self.global_scope.insert(struct_name, struct_symbol)

        print(f"Added struct '{struct_name}' to global scope with {len(members)} members")

    def analyze_struct_instantiation(self):
        """Analyze a struct instance declaration with dfstrct"""
        # Ensure we're starting with 'dfstrct' keyword
        token_type, token_value, line, column = self.get_current_token()
        if token_type != 'dfstrct':
            raise SemanticError(f"Expected 'dfstrct' keyword, got '{token_type}'", line, column)
        self.advance()  # Move past 'dfstrct'

        # Get struct type name
        token_type, token_value, line, column = self.get_current_token()
        if token_type != 'id':
            raise SemanticError(f"Expected struct type name, got '{token_type}'", line, column)

        struct_type_name = token_value

        # Look up the struct type
        struct_symbol = self.global_scope.lookup(struct_type_name)
        if not struct_symbol or struct_symbol.type != 'struct':
            raise SemanticError(f"Undefined struct type '{struct_type_name}'", line, column)

        self.advance()  # Move past struct type name

        # Process one or more instance declarations
        while self.current_token_index < len(self.token_stream):
            token_type, token_value, line, column = self.get_current_token()

            # Check for end of declaration
            if token_type == ';':
                self.advance()  # Move past semicolon
                break

            # Parse instance name
            if token_type != 'id':
                raise SemanticError(f"Expected struct instance name, got '{token_type}'", line, column)

            instance_name = token_value

            # Check for duplicate declaration - FIXED to work in current scope
            if self.current_scope.lookup(instance_name):
                raise SemanticError(f"Variable '{instance_name}' already declared", line, column)

            instance_symbol = Symbol(
                name=instance_name,
                type='struct_instance',
                data_type=struct_type_name,
                initialized=True,
                line=line,
                column=column
            )

            instance_symbol.initialized_members = set()  # Initialize empty set for tracking initialized members

            # Add to current scope
            self.current_scope.insert(instance_name, instance_symbol)

            # Add this debug output:
            print(f"Added struct instance '{instance_name}' of type '{struct_type_name}' to scope '{self.current_scope.scope_name}'")

            self.advance()  # Move past instance name

            # Check for comma (for multiple declarations) or semicolon (end of declaration)
            token_type, token_value, line, column = self.get_current_token()
            if token_type == ',':
                self.advance()  # Move past comma
            elif token_type == ';':
                self.advance()  # Move past semicolon
                break
            else:
                raise SemanticError(f"Expected ',' or ';' after struct instance, got '{token_type}'", line, column)

    def analyze_struct_member_access(self):
        """
        Analyze a struct member access (reading or assignment LHS) and return the member type.
```

```
        This function only analyzes the `id.id` part, not the assignment or RHS.
        """
        # Get struct instance name
        token_type, instance_name, line, column = self.get_current_token()

        # Debug output
        print(f"Analyzing struct member access for '{instance_name}' at line {line}, column {column}")

        # Check if variable exists
        instance_symbol = self.current_scope.lookup(instance_name)
        print(f"Looking for struct instance '{instance_name}' in scope '{self.current_scope.scope_name}': {'Found' if instance_symbol else 'Not Found'}")
        if not instance_symbol:
            raise SemanticError(f"Undefined variable '{instance_name}'", line, column)

        # Check that it's a struct instance
        if instance_symbol.type != 'struct_instance':
            raise SemanticError(f"Variable '{instance_name}' is not a struct instance", line, column)

        # Get the struct type
        struct_type_name = instance_symbol.data_type

        # Look up the struct type in global scope (structs are only declared globally)
        struct_symbol = self.global_scope.lookup(struct_type_name)

        # Debug
        print(f"Looking for struct '{struct_type_name}' in global scope")
        print(f"Global scope contains: {[name for name in self.global_scope.symbols.keys()]}")

        if not struct_symbol:
            raise SemanticError(f"Undefined struct type '{struct_type_name}'", line, column)

        if struct_symbol.type != 'struct':
            raise SemanticError(f"Type '{struct_type_name}' is not a struct", line, column)

        self.advance()  # Move past instance name

        # Process dot operator
        token_type, token_value, line, column = self.get_current_token()
        if token_type != '.':
            raise SemanticError(f"Expected '.' for struct member access, got '{token_type}'", line, column)

        self.advance()  # Move past '.'

        # Get member name
        token_type, member_name, line, column = self.get_current_token()
        if token_type != 'id':
            raise SemanticError(f"Expected member name, got '{token_type}'", line, column)

        # Check if member exists in the struct
        struct_symbol = self.global_scope.lookup(instance_symbol.data_type)
        if not hasattr(struct_symbol, 'members') or member_name not in struct_symbol.members:
            raise SemanticError(f"Struct '{instance_symbol.data_type}' has no member '{member_name}'", line, column)

        # --- REMOVED: Assignment handling logic from here ---
        # The assignment logic will be handled by analyze_assignment

        self.advance()  # Move past member name

        # Get the member type for expression typing
        member_symbol = struct_symbol.members[member_name]
        print(f"Finished analyzing struct member access for '{instance_name}.{member_name}', member type: {member_symbol.data_type}")
        return member_symbol.data_type # Return the type of the member

    # --- Modified analyze_main_function ---
    def analyze_main_function(self):
        """Analyze the main function BODY ('mn'). Assumes called during Pass 2."""
        # Assumes current token is 'mn'
        mn_line, mn_col = self.get_current_token()[2], self.get_current_token()[3]

        # --- Retrieve symbol and scope created in Pass 1 ---
        main_symbol = self.global_scope.lookup("mn")
        if not main_symbol or main_symbol.type != 'function':
            raise SemanticError("Internal Error: Main function 'mn' symbol not found or invalid during pass 2", mn_line, mn_col)

        if "mn" not in self.function_scopes:
            raise SemanticError("Internal Error: Main function scope for 'mn' not found during pass 2", mn_line, mn_col)
        main_scope = self.function_scopes["mn"]
        # --- End retrieval ---

        # --- Skip signature part mn() ---
        self.advance() # Past 'mn'
        if self.current_token_index >= len(self.token_stream) or self.get_current_token()[0] != '(':
            raise SemanticError("Expected '(' after 'mn'", self.get_current_token()[2], self.get_current_token()[3])
        self.advance() # Past '('
        if self.current_token_index >= len(self.token_stream) or self.get_current_token()[0] != ')':
            raise SemanticError("Expected ')' after '(' in 'mn'", self.get_current_token()[2], self.get_current_token()[3])
        self.advance() # Past ')'
```

```
        # --- End skipping signature ---

        # --- Analyze Body ---
        old_scope = self.current_scope
        self.current_scope =
main_scope # Switch to main's
scope

        # Process opening brace
        token_type, _, brace_line,
brace_column =
self.get_current_token()
        if token_type != '{':
            raise
SemanticError(f"Expected '{{' to
start main function body, got
'{token_type}'", brace_line,
brace_column)
        self.advance()  # Move past '{'

        print(f"Pass 2: Starting to
process main function body in scope
'{self.current_scope.scope_name}'")

        # Save old context flags
        old_in_loop = self.in_loop
        old_in_switch = self.in_switch
        self.in_loop = False
        self.in_switch = False

        # Process main function body

self.analyze_function_body("vd") #
Pass "vd" as return type

        print(f"Pass 2: Finished
processing main function body,
returning to scope
'{old_scope.scope_name}'")

        # Restore old context flags
        self.in_loop = old_in_loop
        self.in_switch = old_in_switch

        # Restore outer scope
        self.current_scope = old_scope
        # analyze_function_body
should leave us positioned after '}'

    # --- Modified analyze method ---
    def analyze(self, tokens):
        """Main entry point for
semantic analysis (Handles Pass 1
and Pass 2)."""
        self.token_stream = tokens
        self.current_scope =
self.global_scope
        self.function_scopes = {} #
Initialize/reset function scopes map
        errors = []
        try:
            # --- Pass 1: Collect
Declarations ---
            self.current_token_index = 0
            self.collect_declarations() #
Populates global scope with
signatures, stores body indices

            # --- Debug Print Global
Scope After Pass 1 ---
            print("\n--- Global Scope
Contents After Pass 1 ---")
            if hasattr(self,
'global_scope') and
self.global_scope:
                for name, symbol in
self.global_scope.symbols.items():
                    print(f"  {name}:
{symbol}")
            else:
                print("  Global scope not
available.")

print("----------------------------------
----")
            # --- End Debug Print ---


            # --- Pass 2: Full Analysis ---
            self.current_token_index = 0
            self.current_scope =
self.global_scope # Ensure we start
in global scope for pass 2
            print("\nStarting full
analysis (Pass 2)...")

            while
self.current_token_index <
len(self.token_stream):
                start_of_loop_index =
self.current_token_index
                token_type, token_value,
line, column =
self.token_stream[self.current_token
_index]
                scope_name =
self.current_scope.scope_name if
self.current_scope else "None"
                print(f"Pass 2 - Index:
{self.current_token_index}, Token:
{token_type} ('{token_value}'),
Scope: {scope_name}")

                if token_type == 'strct':
                    print(f"Pass 2: Skipping
struct definition
'{self.peek_next_token()[1]}'.")

self.skip_struct_declaration()
                print(f"Pass 2: After
skipping struct, index is
{self.current_token_index}")
                continue

            # --- SIMPLIFIED 'fnctn'
handling in Pass 2 ---
            # Handles ONLY
user-defined functions declared with
'fnctn'
            elif token_type == 'fnctn':
                func_name =
"unknown"
                name_token_idx =
self.current_token_index + 2
                if name_token_idx <
len(self.token_stream):
                    func_name_token =
self.token_stream[name_token_idx]
                    if
func_name_token[0] == 'id':
func_name = func_name_token[1]
                print(f"Pass 2:
Analyzing body of function
'{func_name}'.")

self.analyze_function_declaration(is
_first_pass=False)
                print(f"Pass 2: After
analyzing '{func_name}' body,
index is
{self.current_token_index}")
                continue
            # --- End simplified 'fnctn'
handling ---

            # Handle 'mn' function
analysis trigger
            elif token_type == 'mn':
                print(f"Pass 2:
Analyzing 'mn' function.")

self.analyze_main_function() #
Analyzes the body
                print(f"Pass 2: After
analyzing 'mn', index is
{self.current_token_index}")
                continue

            # Handle global
declarations (struct instances,
variables, constants)
            elif token_type ==
'dfstrct':
                print(f"Pass 2:
Analyzing struct instantiation.")

self.analyze_struct_instantiation()
                print(f"Pass 2: After
dfstrct, index is
{self.current_token_index}")
```

```
                continue
            elif token_type == 'cnst':
                print(f"Pass 2:
Analyzing constant declaration.")

self.analyze_constant_declaration()
                print(f"Pass 2: After
cnst, index is
{self.current_token_index}")
                continue
            elif token_type in
self.data_types:
                print(f"Pass 2:
Analyzing variable/array declaration
starting with '{token_type}'.")
                start_pos =
self.current_token_index
                # Look ahead logic to
differentiate var/array
                self.advance() # Past
type
                next_is_id =
self.current_token_index <
len(self.token_stream) and
self.token_stream[self.current_token
_index][0] == 'id'
                if not next_is_id: raise
SemanticError(f"Expected an
identifier after '{token_type}'", line,
column)
                next_token_after_id =
self.peek_next_token()
                is_array =
next_token_after_id and
next_token_after_id[0] == '['
                # Reset and analyze

self.current_token_index = start_pos
                if is_array:
self.analyze_array_declaration()
                else:
self.analyze_variable_declaration()
                print(f"Pass 2: After
var/array decl, index is
{self.current_token_index}")
                continue


            # Handle potential
top-level statements
            elif token_type == 'id':
                next_token =
self.peek_next_token()
                handled = False
                if next_token:
                    if next_token[0] ==
'.':
                        print(f"Pass 2:
Analyzing top-level struct member
access/assignment.")
self.analyze_struct_member_access(
) # Returns type, ignore here
                        handled = True
                    # ... (other id
handlers: '(', '[', '=', '++', '--', '+=',
etc.) ...
                    elif next_token[0] ==
'(':
                        print(f"Pass 2:
Analyzing top-level function call.")

self.analyze_function_call() #
Returns type, ignore here
                        handled = True
                    elif next_token[0] ==
'[':
                        print(f"Pass 2:
Analyzing top-level array
access/assignment.")

self.analyze_array_access() #
Returns type, ignore here
                        if
self.current_token_index <
len(self.token_stream) and
self.get_current_token()[0] == ';':
self.advance()
                        handled = True
                    elif next_token[0] ==
'=':
                        print(f"Pass 2:
Analyzing top-level assignment.")

self.analyze_assignment()
                        handled = True
                    elif next_token[0] in
['++', '--']:
                        print(f"Pass 2:
Analyzing top-level
increment/decrement.")

self.analyze_increment_operation()
                        handled = True
                    elif next_token[0] in
['+=', '-=', '*=', '/=', '%=']:
                        print(f"Pass 2:
Analyzing top-level shortcut
assignment.")

self.analyze_assignment() #
Handles shortcut ops too
                        handled = True


                if handled:
                    print(f"Pass 2: After
top-level ID statement, index is
{self.current_token_index}")
                    continue
                else:
                    print(f"Pass 2:
Advancing past unhandled ID
'{token_value}'.")
                    self.advance() #
Advance if ID wasn't part of a
handled statement

            elif token_type == 'prnt':
                print(f"Pass 2:
Analyzing top-level print
statement.")

self.analyze_print_statement()
                print(f"Pass 2: After
top-level prnt, index is
{self.current_token_index}")
                continue

            # If the token wasn't
handled by any case above,
advance.
            if
self.current_token_index ==
start_of_loop_index:
                print(f"Pass 2:
Advancing past unhandled token
{token_type} ('{token_value}') to
prevent loop.")
                self.advance()

        # --- End of Pass 2 ---
        print("\nFinished full
analysis (Pass 2).")
        self.print_symbol_tables()
        return True, errors

    except SemanticError as e:
        errors.append(str(e))
        print("\n--- Symbol Tables
after Error ---")
        self.print_symbol_tables()

print("------------------------------")
        return False, errors
    except Exception as e:
        errors.append(f"Unexpected
Internal Error during analysis: {e}")
        print("\n--- Symbol Tables
after Internal Error ---")
        self.print_symbol_tables()

print("------------------------------")
        import traceback
        traceback.print_exc()
        return False, errors

# --- Helper:
skip_struct_declaration ---
def skip_struct_declaration(self):
    """Advances the token index
past a struct definition."""
```

```
        # Assumes current token is
'strct'
        if self.get_current_token()[0]
!= 'strct': return # Should not happen
        self.advance() # Skip 'strct'
        if self.get_current_token()[0]
!= 'id': return # Skip name (error if
not ID)
        self.advance()
        if self.get_current_token()[0]
!= '{': return # Skip '{' (error if not
'{')
        self.advance()
        brace_count = 1
        while brace_count > 0 and
self.current_token_index <
len(self.token_stream):
            token_type =
self.get_current_token()[0]
            if token_type == '{':
                brace_count += 1
            elif token_type == '}':
                brace_count -= 1
            # Advance unconditionally
inside the loop
            if brace_count > 0: # Avoid
advancing past the final '}'
                self.advance()

        # Now
self.current_token_index is at the
final '}'
        if self.get_current_token()[0]
== '}':
            self.advance() # Move past
'}'
        # Skip potential semicolon
after '}'
        if self.current_token_index <
len(self.token_stream) and
self.get_current_token()[0] == ';':
            self.advance()


    # --- Helper:
skip_function_declaration ---
    def skip_struct_declaration(self):
        """Advances the token index
past a struct definition."""
        # Assumes current token is
'strct'
        if self.get_current_token()[0]
!= 'strct': return # Should not happen
        self.advance() # Skip 'strct'
        if self.get_current_token()[0]
!= 'id': return # Skip name (error if
not ID)
        self.advance()
        if self.get_current_token()[0]
!= '{': return # Skip '{' (error if not
'{')
        self.advance()
        brace_count = 1
        while brace_count > 0 and
self.current_token_index <
len(self.token_stream):
            token_type =
self.get_current_token()[0]
            if token_type == '{':
                brace_count += 1
            elif token_type == '}':
                brace_count -= 1
            # Advance unconditionally
inside the loop
            if brace_count > 0: # Avoid
advancing past the final '}'
                self.advance()

        # Now
self.current_token_index is at the
final '}'
        if self.current_token_index <
len(self.token_stream) and
self.get_current_token()[0] == '}':
            self.advance() # Move past
'}'
        # Skip potential semicolon
after '}'
        if self.current_token_index <
len(self.token_stream) and
self.get_current_token()[0] == ';':
            self.advance()

    def is_in_function_body(self):
        """Check if current position is
inside a function body"""
        # Start from current position
and look backward for function
keyword
        pos = self.current_token_index

        # Go backwards until we find a
function start or beginning of file
        while pos > 0:
            token_type =
self.token_stream[pos][0]

            # If we find a function, we're
inside it
            if token_type in ['fnctn',
'mn']:
                # Check if we've passed
the opening brace
                brace_pos = pos
                while brace_pos <
self.current_token_index:
                    if
self.token_stream[brace_pos][0] ==
'{':
                        return True
                    brace_pos += 1
                return False

            # If we find the end of a
previous function, we're not in a
function
            if token_type == '}':
                # Check if it's the end of a
function
                # This is simplified and
might need refinement
                return False

            pos -= 1

        return False

    def
skip_function_declaration(self):
        """Advances the token index
past a 'fnctn' definition (signature
and body)."""
        # Assumes current token is
'fnctn'
        if self.get_current_token()[0]
!= 'fnctn': return
        self.advance() # Skip 'fnctn'
        # Skip return type/vd
        if self.get_current_token()[0]
in self.data_types or
self.get_current_token()[0] == 'vd':
            self.advance()
        else: return # Error in syntax
        # Skip function name
        if self.get_current_token()[0]
== 'id':
            self.advance()
        else: return # Error in syntax
        # Skip parameters (...)
        if self.get_current_token()[0]
!= '(': return
        self.advance() # Skip '('
        paren_count = 1
        while paren_count > 0 and
self.current_token_index <
len(self.token_stream):
            token_type =
self.get_current_token()[0]
            if token_type == '(':
paren_count += 1
            elif token_type == ')':
paren_count -= 1
            if paren_count > 0:
self.advance()
        if self.current_token_index <
len(self.token_stream) and
self.get_current_token()[0] == ')':
            self.advance() # Move past
')'
        else: return # Error in syntax

        # Skip body {...}
```

```
        if self.current_token_index >= len(self.token_stream) or self.get_current_token()[0] != '{':
            return
        self.advance() # Skip '{'
        brace_count = 1
        while brace_count > 0 and self.current_token_index < len(self.token_stream):
            token_type = self.get_current_token()[0]
            if token_type == '{':
                brace_count += 1
            elif token_type == '}':
                brace_count -= 1
            if brace_count > 0:
                self.advance()
        if self.current_token_index < len(self.token_stream) and self.get_current_token()[0] == '}':
            self.advance() # Move past '}'

    def analyze_constant_declaration(self):
        """Analyze a constant declaration"""
        # At this point, the current token should be 'cnst'
        self.advance()  # Move past 'cnst'

        # Get the data type
        data_type_token, data_type_value, data_line, data_column = self.get_current_token()
        if data_type_token not in self.data_types:
            raise SemanticError(f"Expected data type after 'cnst', got {data_type_token}", data_line, data_column)

        self.advance()  # Move past data type

        # Get the variable name
        var_token_type, var_name, name_line, name_column = self.get_current_token()
        if var_token_type != 'id':
            raise SemanticError(f"Expected an identifier, got {var_token_type}", name_line, name_column)

        # Check for duplicate declaration
        if self.current_scope.lookup(var_name) and self.current_scope.symbols.get(var_name):
            raise SemanticError(f"Variable '{var_name}' already declared", name_line, name_column)

        # Look ahead to see if this is an array declaration
        next_token = self.peek_next_token()
        if next_token and next_token[0] == '[':
            # Go back to the cnst token
            self.current_token_index -= 2  # Back to cnst

            self.analyze_array_declaration()
            return

        # Otherwise, handle as a regular constant
        self.advance()  # Move past variable name

        # Constants must be initialized
        token_type, token_value, line, column = self.get_current_token()
        if token_type != '=':
            raise SemanticError(f"Constants must be initialized", line, column)

        self.advance()  # Move past '='

        # Save starting position for expression analysis
        start_pos = self.current_token_index

        # Skip ahead to find the end of the expression (semicolon or comma)
        while (self.current_token_index < len(self.token_stream) and
               self.token_stream[self.current_token_index][0] not in [';', ',']):
            self.advance()

        # Reset position to start of expression
        end_pos = self.current_token_index
        self.current_token_index = start_pos

        # Analyze the expression
        expr_type = self.analyze_expression(end_pos)

        # Check if the expression type matches the variable type
        if data_type_token != expr_type:
            raise SemanticError(
                f"Type mismatch: Cannot initialize constant '{data_type_token}' with {expr_type}",
                line, column
            )

        # Create and add the constant symbol
        symbol = Symbol(
            name=var_name,
            type='variable',
            data_type=data_type_token,
            initialized=True,
            is_constant=True,
            line=name_line,
            column=name_column
        )

        self.current_scope.insert(var_name, symbol)

        # Current position is now at the end of the expression
        token_type, token_value, line, column = self.get_current_token()

        # Handle multiple declarations
        if token_type == ',':
            self.advance()  # Move past comma

            self.analyze_constant_declaration_continuation(data_type_token)
        elif token_type == ';':
            self.advance()  # Move past semicolon

    def analyze_constant_declaration_continuation(self, data_type):
        """Handle the continuation of a constant declaration (after a comma)"""
        # Get the next variable name
        var_token_type, var_name, name_line, name_column = self.get_current_token()
        if var_token_type != 'id':
            raise SemanticError(f"Expected an identifier, got {var_token_type}", name_line, name_column)
```

```
        # Check for duplicate declaration
        if self.current_scope.lookup(var_name) and self.current_scope.symbols.get(var_name):
            raise SemanticError(f"Variable '{var_name}' already declared", name_line, name_column)

        # Look ahead to see if this is an array declaration
        next_token = self.peek_next_token()
        if next_token and next_token[0] == '[':
            # Cannot handle array declaration in the middle of a constant declaration list
            raise SemanticError(f"Array declarations must be separate from variable declarations", name_line, name_column)

        self.advance()  # Move past variable name

        # Constants must be initialized
        token_type, token_value, line, column = self.get_current_token()
        if token_type != '=':
            raise SemanticError(f"Constants must be initialized", line, column)

        self.advance()  # Move past '='

        # Save starting position for expression analysis
        start_pos = self.current_token_index

        # Skip ahead to find the end of the expression (semicolon or comma)
        while (self.current_token_index < len(self.token_stream) and
               self.token_stream[self.current_token_index][0] not in [';', ',']):
            self.advance()

        # Reset position to start of expression
        end_pos = self.current_token_index
        self.current_token_index = start_pos

        # Analyze the expression
        expr_type = self.analyze_expression(end_pos)

        # Check if the expression type matches the variable type
        if data_type != expr_type:
            raise SemanticError(
                f"Type mismatch: Cannot initialize constant '{data_type}' with {expr_type}",
                line, column
            )

        # Create and add the constant symbol
        symbol = Symbol(
            name=var_name,
            type='variable',
            data_type=data_type,
            initialized=True,
            is_constant=True,
            line=name_line,
            column=name_column
        )

        self.current_scope.insert(var_name, symbol)

        # Current position is now at the end of the expression
        token_type, token_value, line, column = self.get_current_token()

        # Handle multiple declarations
        if token_type == ',':
            self.advance()  # Move past comma

            self.analyze_constant_declaration_continuation(data_type)
        elif token_type == ';':
            self.advance()  # Move past semicolon


    def analyze_variable_declaration(self):
        """Analyze a variable declaration and possible initialization for multiple variables"""
        data_type, type_value, type_line, type_column = self.token_stream[self.current_token_index]
        self.advance()  # Move past the data type

        # Continue parsing declarations until we hit a semicolon or end of statement
        while self.current_token_index < len(self.token_stream):
            # Get variable name
            var_token_type, var_name, name_line, name_column = self.get_current_token()

            if var_token_type != 'id':
                raise SemanticError(f"Expected an identifier, got {var_token_type}", name_line, name_column)

            # Check for duplicate declaration in current scope
            if self.current_scope.lookup(var_name) and self.current_scope.symbols.get(var_name):
                raise SemanticError(f"Variable '{var_name}' already declared", name_line, name_column)

            # Create symbol
            symbol = Symbol(
                name=var_name,
                type='variable',
                data_type=data_type,
                initialized=False,
                line=name_line,
                column=name_column
            )

            self.advance()  # Move past variable name

            # Check for initialization
            token_type, token_value, line, column = self.get_current_token()

            if token_type == '=':
                self.advance()  # Move past '='

                # Save starting position for expression analysis
                start_pos = self.current_token_index

                # Skip ahead to find the end of the expression (semicolon or comma)
```

```
            while
(self.current_token_index <
len(self.token_stream) and

self.token_stream[self.current_token
_index][0] not in [';', ',']):
                self.advance()

            # Reset position to start of
expression
            end_pos =
self.current_token_index
            self.current_token_index
= start_pos

            # Analyze the expression
            expr_type =
self.analyze_expression(end_pos)

            # Check if the expression
type matches the variable type
            if data_type != expr_type:
                # Special case for
boolean variables being assigned
relational expressions
                if data_type == 'bln'
and expr_type == 'bln':
                    # This is fine,
expression results in boolean and
variable is boolean
                    pass
                else:
                    raise SemanticError(
                        f"Type mismatch:
Cannot initialize '{data_type}' with
{expr_type}",
                        line, column
                    )

            symbol.initialized = True

            # Current position is now
at the end of the expression

        # Add the symbol to the
symbol table

self.current_scope.insert(var_name,
symbol)

        # Check for comma or end
of declaration
        token_type, token_value,
line, column =
self.get_current_token()

        if token_type == ',':
            self.advance()  # Move
past the comma
            continue
        elif token_type == ';':
            self.advance()  # Move
past the semicolon
            break
        else:
            # For languages that don't
require semicolons, or end of line
tokens
            # You might want
additional logic here
            break
    def
analyze_array_declaration(self):
        """Analyze an array
declaration and possible
initialization"""
        is_constant = False

        # Check if this is a constant
array
        token_type, token_value, line,
column = self.get_current_token()
        if token_type == 'cnst':
            is_constant = True
            self.advance()  # Move past
'cnst'

        data_type, type_value,
type_line, type_column =
self.get_current_token()
        self.advance()  # Move past the
data type

        # Process all array declarations
and variables in this declaration

self.process_array_or_variable(data
_type, is_constant)

    def
process_array_or_variable(self,
data_type, is_constant=False):
        """Process an array or variable
declaration, possibly followed by
more declarations"""
        # Get variable name
        var_token_type, var_name,
name_line, name_column =
self.get_current_token()

        if var_token_type != 'id':
            raise
SemanticError(f"Expected an
identifier, got {var_token_type}",
name_line, name_column)

        # Check for duplicate
declaration in current scope
        if
self.current_scope.lookup(var_name
) and
self.current_scope.symbols.get(var_
name):
            raise
SemanticError(f"Variable
'{var_name}' already declared",
name_line, name_column)

        self.advance()  # Move past
variable name

        # Check if this is an array
declaration (followed by '[')
        if self.get_current_token()[0]
== '[':
            # Process array dimensions
and sizes
            array_dimensions = 0
            array_sizes = []

            # First dimension
            array_dimensions += 1
            self.advance()  # Move past
'['

            # Parse size expression
            size1_token_type,
size1_value, size1_line,
size1_column =
self.get_current_token()

            if size1_token_type not in
['ntlit', 'id']:
                raise
SemanticError(f"Array size must be
an integer constant or variable",
size1_line, size1_column)

            # If the size is a variable,
check that it's declared and of type
'nt'
            if size1_token_type == 'id':
                size_symbol =
self.current_scope.lookup(size1_val
ue)
                if not size_symbol:
                    raise
SemanticError(f"Undefined variable
'{size1_value}' used as array size",
size1_line, size1_column)
                if size_symbol.data_type
!= 'nt':
                    raise
SemanticError(f"Array size must be
of type 'nt', got
'{size_symbol.data_type}'",
size1_line, size1_column)
                if not
size_symbol.initialized:
                    raise
SemanticError(f"Uninitialized
variable '{size1_value}' used as
```

```
array size", size1_line,
size1_column)

array_sizes.append(size1_value)  #
Store the variable name
        else:
            # It's a literal, make sure
it's positive
            size1 = int(size1_value)
            if size1 <= 0:
                raise
SemanticError(f"Array size must be
positive, got {size1}", size1_line,
size1_column)
            array_sizes.append(size1)

        self.advance()  # Move past
size

        if self.get_current_token()[0]
!= ']':
            raise
SemanticError(f"Expected ']', got
{self.get_current_token()[0]}",

self.get_current_token()[2],
self.get_current_token()[3])

        self.advance()  # Move past
']'

        # Check for second
dimension [size2]
        if self.get_current_token()[0]
== '[':
            array_dimensions += 1
            self.advance()  # Move
past '['

            # Parse size expression
            size2_token_type,
size2_value, size2_line,
size2_column =
self.get_current_token()

            if size2_token_type not in
['ntlit', 'id']:
                raise
SemanticError(f"Array size must be
an integer constant or variable",
size2_line, size2_column)

            # If the size is a variable,
check that it's declared and of type
'nt'
            if size2_token_type ==
'id':
                size_symbol =
self.current_scope.lookup(size2_val
ue)
                if not size_symbol:
                    raise
SemanticError(f"Undefined variable
'{size2_value}' used as array size",
size2_line, size2_column)
                if
size_symbol.data_type != 'nt':
                    raise
SemanticError(f"Array size must be
of type 'nt', got
'{size_symbol.data_type}'",
size2_line, size2_column)
                if not
size_symbol.initialized:
                    raise
SemanticError(f"Uninitialized
variable '{size2_value}' used as
array size", size2_line,
size2_column)

array_sizes.append(size2_value)  #
Store the variable name
            else:
                # It's a literal, make
sure it's positive
                size2 = int(size2_value)
                if size2 <= 0:
                    raise
SemanticError(f"Array size must be
positive, got {size2}", size2_line,
size2_column)

array_sizes.append(size2)

            self.advance()  # Move
past size

            if
self.get_current_token()[0] != ']':
                raise
SemanticError(f"Expected ']', got
{self.get_current_token()[0]}",

self.get_current_token()[2],
self.get_current_token()[3])

            self.advance()  # Move
past ']'

        # Create array symbol
        symbol = Symbol(
            name=var_name,
            type='variable',
            data_type=data_type,
            initialized=False,
            is_constant=is_constant,
            is_array=True,

array_dimensions=array_dimension
s,
            array_sizes=array_sizes,
            line=name_line,
            column=name_column
        )

        # Check for initialization
        if self.get_current_token()[0]
== '=':
            self.advance()  # Move
past '='

            # Handle array
initialization
            if
self.get_current_token()[0] != '{':
                raise
SemanticError(f"Expected '{{' for
array initialization, got
{self.get_current_token()[0]}",

self.get_current_token()[2],
self.get_current_token()[3])

            # Process array
initialization
            if array_dimensions == 1:

self.validate_1d_array_init(symbol,
data_type, array_sizes[0])
            elif array_dimensions ==
2:

self.validate_2d_array_init(symbol,
data_type, array_sizes[0],
array_sizes[1])

            symbol.initialized = True
        elif is_constant:
            # Constants must be
initialized
            raise
SemanticError(f"Constant array
'{var_name}' must be initialized",
name_line, name_column)

        # Add the array symbol to
the symbol table

self.current_scope.insert(var_name,
symbol)
        print(f"Added array
'{var_name}' of type '{data_type}'
with dimensions
{array_dimensions} to scope
'{self.current_scope.scope_name}'")

    else:
        # This is a regular variable,
not an array
        # Create symbol
        symbol = Symbol(
            name=var_name,
            type='variable',
```

```
            data_type=data_type,
            initialized=False,
            is_constant=is_constant,
            line=name_line,
            column=name_column
        )

        # Check for initialization
        if self.get_current_token()[0]
== '=':
            self.advance()  # Move
past '='

            # Save starting position
for expression analysis
            start_pos =
self.current_token_index

            # Skip ahead to find the
end of the expression (semicolon or
comma)
            while
(self.current_token_index <
len(self.token_stream) and

self.token_stream[self.current_token
_index][0] not in [';', ',']):
                self.advance()

            # Reset position to start of
expression
            end_pos =
self.current_token_index
            self.current_token_index
= start_pos

            # Analyze the expression
            expr_type =
self.analyze_expression(end_pos)

            # Check if the expression
type matches the variable type
            if data_type != expr_type:
                # Special case for
boolean variables being assigned
relational expressions
                if data_type == 'bln'
and expr_type == 'bln':
                    # This is fine,
expression results in boolean and
variable is boolean
                    pass
                else:
                    raise SemanticError(
                        f"Type mismatch:
Cannot initialize '{data_type}' with
{expr_type}",
                        name_line,
name_column
                    )
            symbol.initialized = True

            # Move to end of
expression
            self.current_token_index
= end_pos

            # Add the variable symbol to
the symbol table

self.current_scope.insert(var_name,
symbol)
            print(f"Added variable
'{var_name}' of type '{data_type}'
to scope
'{self.current_scope.scope_name}'")

        # Check for comma or
semicolon
        if self.get_current_token()[0]
== ',':
            self.advance()  # Move past
comma

self.process_array_or_variable(data
_type, is_constant)  # Process next
variable or array
        elif self.get_current_token()[0]
== ';':
            self.advance()  # Move past
semicolon
        else:
            raise
SemanticError(f"Expected ',' or ';',
got {self.get_current_token()[0]}",

self.get_current_token()[2],
self.get_current_token()[3])

    def validate_1d_array_init(self,
symbol, data_type, size):
        """Validate 1D array
initialization"""
        self.advance()  # Move past '{'

        element_count = 0

        # Process elements until we see
'}'
        while self.current_token_index
< len(self.token_stream) and
self.get_current_token()[0] != '}':
            # Get element
            elem_type, elem_value,
elem_line, elem_column =
self.get_current_token()

            # Validate element type
            if data_type == 'nt' and
elem_type not in ['ntlit', '~ntlit', 'id']:
                raise
SemanticError(f"Type mismatch:
Array element must be 'nt', got
'{elem_type}'",
                              elem_line,
elem_column)
            elif data_type == 'dbl' and
elem_type not in ['dbllit', '~dbllit',
'ntlit', '~ntlit', 'id']:
                raise
SemanticError(f"Type mismatch:
Array element must be 'dbl', got
'{elem_type}'",
                              elem_line,
elem_column)
            elif data_type == 'bln' and
elem_type not in ['true', 'false',
'blnlit', 'id']:
                raise
SemanticError(f"Type mismatch:
Array element must be 'bln', got
'{elem_type}'",
                              elem_line,
elem_column)
            elif data_type == 'chr' and
elem_type not in ['chrlit', 'id']:
                raise
SemanticError(f"Type mismatch:
Array element must be 'chr', got
'{elem_type}'",
                              elem_line,
elem_column)
            elif data_type == 'strng' and
elem_type not in ['strnglit', 'id']:
                raise
SemanticError(f"Type mismatch:
Array element must be 'strng', got
'{elem_type}'",
                              elem_line,
elem_column)

            # If element is an identifier,
check if it's declared and of the right
type
            if elem_type == 'id':
                elem_symbol =
self.current_scope.lookup(elem_val
ue)
                if not elem_symbol:
                    raise
SemanticError(f"Undefined variable
'{elem_value}' used as array
element",
                                  elem_line,
elem_column)
                if elem_symbol.data_type
!= data_type:
                    raise
SemanticError(f"Type mismatch:
Array element must be
```

```
'{data_type}', got
'{elem_symbol.data_type}'",
                        elem_line,
elem_column)
            if not
elem_symbol.initialized:
                raise
SemanticError(f"Uninitialized
variable '{elem_value}' used as
array element",
                        elem_line,
elem_column)

        element_count += 1
        self.advance()  # Move past
element

        # Skip comma if present
        if self.get_current_token()[0]
== ',':
            self.advance()

    # Check if we've gone over the
array size
    if isinstance(size, int) and
element_count > size:
        raise SemanticError(f"Array
initialization has {element_count}
elements, but array size is {size}",

self.get_current_token()[2],
self.get_current_token()[3])

    # For constant arrays, ensure
all elements are initialized
    if symbol.is_constant and
isinstance(size, int) and
element_count < size:
        raise
SemanticError(f"Constant array
must initialize all {size} elements,
but only {element_count}
provided",

self.get_current_token()[2],
self.get_current_token()[3])

    self.advance()  # Move past '}'

  def validate_2d_array_init(self,
symbol, data_type, rows, cols):
    """Validate 2D array
initialization"""
    self.advance()  # Move past '{'

    row_count = 0

    # Process rows until we see '}'
    while self.current_token_index
< len(self.token_stream) and
self.get_current_token()[0] != '}':
        if self.get_current_token()[0]
!= '{':
            raise
SemanticError(f"Expected '{{' for
2D array row, got
{self.get_current_token()[0]}",

self.get_current_token()[2],
self.get_current_token()[3])

        self.advance()  # Move past
'{'

        col_count = 0

        # Process elements in this
row
        while
self.current_token_index <
len(self.token_stream) and
self.get_current_token()[0] != '}':
            # Get element
            elem_type, elem_value,
elem_line, elem_column =
self.get_current_token()

            # Validate element type
(same as 1D array validation)
            if data_type == 'nt' and
elem_type not in ['ntlit', '~ntlit', 'id']:
                raise
SemanticError(f"Type mismatch:
Array element must be 'nt', got
'{elem_type}'",
                        elem_line,
elem_column)
            elif data_type == 'dbl' and
elem_type not in ['dbllit', '~dbllit',
'ntlit', '~ntlit', 'id']:
                raise
SemanticError(f"Type mismatch:
Array element must be 'dbl', got
'{elem_type}'",
                        elem_line,
elem_column)
            elif data_type == 'bln' and
elem_type not in ['true', 'false',
'blnlit', 'id']:
                raise
SemanticError(f"Type mismatch:
Array element must be 'bln', got
'{elem_type}'",
                        elem_line,
elem_column)
            elif data_type == 'chr' and
elem_type not in ['chrlit', 'id']:
                raise
SemanticError(f"Type mismatch:
Array element must be 'chr', got
'{elem_type}'",
                        elem_line,
elem_column)
            elif data_type == 'strng'
and elem_type not in ['strnglit', 'id']:
                raise
SemanticError(f"Type mismatch:
Array element must be 'strng', got
'{elem_type}'",
                        elem_line,
elem_column)

            # If element is an
identifier, check its type
            if elem_type == 'id':
                elem_symbol =
self.current_scope.lookup(elem_val
ue)
                if not elem_symbol:
                    raise
SemanticError(f"Undefined variable
'{elem_value}' used as array
element",
                                elem_line,
elem_column)
                if
elem_symbol.data_type !=
data_type:
                    raise
SemanticError(f"Type mismatch:
Array element must be
'{data_type}', got
'{elem_symbol.data_type}'",
                                elem_line,
elem_column)
                if not
elem_symbol.initialized:
                    raise
SemanticError(f"Uninitialized
variable '{elem_value}' used as
array element",
                                elem_line,
elem_column)

            col_count += 1
            self.advance()  # Move
past element

            # Skip comma if present
            if
self.get_current_token()[0] == ',':
                self.advance()

        # Check if we've gone over
the columns size
        if isinstance(cols, int) and
col_count > cols:
            raise
SemanticError(f"Row {row_count}
has {col_count} elements, but array
column size is {cols}",
```

```
self.get_current_token()[2],
self.get_current_token()[3])

            # For constant arrays, ensure
all elements in each row are
initialized
            if symbol.is_constant and
isinstance(cols, int) and col_count <
cols:
                raise
SemanticError(f"Constant array row
must initialize all {cols} elements,
but only {col_count} provided",

self.get_current_token()[2],
self.get_current_token()[3])

            self.advance()  # Move past
'}' for this row
            row_count += 1

            # Skip comma if present
            if self.get_current_token()[0]
== ',':
                self.advance()

        # Check if we've gone over the
rows size
        if isinstance(rows, int) and
row_count > rows:
            raise SemanticError(f"Array
initialization has {row_count} rows,
but array row size is {rows}",

self.get_current_token()[2],
self.get_current_token()[3])

        # For constant arrays, ensure
all rows are initialized
        if symbol.is_constant and
isinstance(rows, int) and row_count
< rows:
            raise
SemanticError(f"Constant array
must initialize all {rows} rows, but
only {row_count} provided",

self.get_current_token()[2],
self.get_current_token()[3])

        self.advance()  # Move past '}'

    def analyze_array_access(self):
        """
        Analyze array element access
(reading or assignment LHS) and
return the element type.
        This function only analyzes the
`id[...]` part, not the assignment or
RHS.
        """
        var_token_type, var_name,
line, column =
self.get_current_token()

        # Check if variable exists
        symbol =
self.current_scope.lookup(var_name
)
        if not symbol:
            raise
SemanticError(f"Undefined variable
'{var_name}'", line, column)

        # Check that it's an array
        if not symbol.is_array:
            raise
SemanticError(f"Variable
'{var_name}' is not an array", line,
column)

        self.advance()  # Move past
array name

        # Process array index(es)
        # First dimension
        if self.get_current_token()[0]
!= '[':
            raise
SemanticError(f"Expected '[', got
{self.get_current_token()[0]}",

self.get_current_token()[2],
self.get_current_token()[3])

        self.advance()  # Move past '['

        # Save the current index token
for bounds checking
        index1_token_type,
index1_value, index1_line,
index1_column =
self.get_current_token()

        # Find the end of this index
expression (should be the closing
bracket)
        start_pos =
self.current_token_index
        bracket_level = 1

        while bracket_level > 0 and
self.current_token_index <
len(self.token_stream):
            # Need to check for nested
brackets within the index expression
            current_token =
self.token_stream[self.current_token
_index][0]
            if current_token == '[':
                bracket_level += 1
            elif current_token == ']':
                bracket_level -= 1

            if bracket_level > 0: #
Advance only if still inside brackets
                self.current_token_index
+= 1

            if self.current_token_index
>= len(self.token_stream):
                raise
SemanticError("Unclosed bracket in
array access", line, column)


        # Now
self.current_token_index is at the
closing ']'
        end_pos =
self.current_token_index
        self.current_token_index =
start_pos # Reset position to analyze
the expression

        # Analyze the index expression
- it must evaluate to type nt
        index1_type =
self.analyze_expression(end_pos)
        if index1_type != 'nt':
            raise SemanticError(f"Array
index must be of type 'nt', got
'{index1_type}'",
                                index1_line,
index1_column)

        # Check bounds if index is a
literal and size is known
        if index1_token_type == 'ntlit'
and
isinstance(symbol.array_sizes[0],
int):
            index1 = int(index1_value)
            if index1 < 0 or index1 >=
symbol.array_sizes[0]:
                raise
SemanticError(f"Array index
{index1} out of bounds (size
{symbol.array_sizes[0]})",
                                index1_line,
index1_column)

        # Move past the closing
bracket
        if self.get_current_token()[0]
!= ']':
            # This should not happen if
the loop above worked correctly, but
as a safeguard
            raise
SemanticError(f"Expected ']', got
```

```
{self.get_current_token()[0]} after index expression",

self.get_current_token()[2], self.get_current_token()[3])
        self.advance()  # Move past ']'

        # Check for second dimension if this is a 2D array
        if symbol.array_dimensions == 2:
            if self.get_current_token()[0] != '[':
                raise SemanticError(f"Expected second dimension '[' for 2D array, got {self.get_current_token()[0]}",

self.get_current_token()[2], self.get_current_token()[3])

            self.advance()  # Move past '['

            # Save the current index token for bounds checking
            index2_token_type, index2_value, index2_line, index2_column = self.get_current_token()

            # Find the end of this index expression (should be the closing bracket)
            start_pos = self.current_token_index
            bracket_level = 1

            while bracket_level > 0 and self.current_token_index < len(self.token_stream):
                # Need to check for nested brackets within the index expression
                current_token = self.token_stream[self.current_token_index][0]
                if current_token == '[':
                    bracket_level += 1
                elif current_token == ']':
                    bracket_level -= 1

                if bracket_level > 0: # Advance only if still inside brackets

self.current_token_index += 1

                if self.current_token_index >= len(self.token_stream):
                    raise SemanticError("Unclosed bracket in 2D array access", line, column)

            # Now self.current_token_index is at the closing ']'
            end_pos = self.current_token_index
            self.current_token_index = start_pos # Reset position to analyze the expression

            # Validate index expression is of type nt
            index2_type = self.analyze_expression(end_pos)
            if index2_type != 'nt':
                raise SemanticError(f"Array index must be of type 'nt', got '{index2_type}'",
                                    index2_line, index2_column)

            # Check bounds if index is a literal and size is known
            if index2_token_type == 'ntlit' and isinstance(symbol.array_sizes[1], int):
                index2 = int(index2_value)
                if index2 < 0 or index2 >= symbol.array_sizes[1]:
                    raise SemanticError(f"Array index {index2} out of bounds (size {symbol.array_sizes[1]})",
                                        index2_line, index2_column)

            # Move past the closing bracket
            if self.get_current_token()[0] != ']':
                # This should not happen if the loop above worked correctly, but as a safeguard
                raise SemanticError(f"Expected ']', got {self.get_current_token()[0]} after 2nd index expression",

self.get_current_token()[2], self.get_current_token()[3])
            self.advance()  # Move past ']'

        # Return the data type of the array element
        # The caller (analyze_assignment) will handle the '=' and the RHS
        print(f"Finished analyzing array access for '{var_name}[...]', element type: {symbol.data_type}")
        return symbol.data_type # Return the type of the element


    def analyze_expression(self, end_pos):
        """Analyze an expression and determine its resulting type using strict type rules"""
        # Add debug output
        print(f"Analyzing expression from position {self.current_token_index} to {end_pos}")

        # Save current position
        original_pos = self.current_token_index

        # Debug - show the tokens being analyzed
        expr_tokens = self.token_stream[original_pos:end_pos]
        expr_str = " ".join([f"{t[0]}('{t[1]}')" for t in expr_tokens])
        print(f"Expression tokens: {expr_str}")

        # Check if this is a simple function call expression
        if (self.current_token_index < len(self.token_stream) and

self.token_stream[self.current_token_index][0] == 'id'):

            func_name = self.token_stream[self.current_token_index][1]
            next_pos = self.current_token_index + 1

            # Check if next token is opening parenthesis (function call)
            if (next_pos < len(self.token_stream) and

self.token_stream[next_pos][0] == '('):

                # This might be a function call - use our specialized function call parser
```

```
            return_type =
self.analyze_function_call()

            # If we're at or beyond the
end_pos, we're done
            if
self.current_token_index >=
end_pos:
                return return_type

            # Otherwise reset position
and fall back to regular expression
parsing
            self.current_token_index
= original_pos

        # Check if next token is dot
(struct member access)
        elif (next_pos <
len(self.token_stream) and

self.token_stream[next_pos][0] ==
'.'):
            print(f"Found potential
struct member access:
{func_name}.{self.token_stream[ne
xt_pos+1][1]}")

            # This is a struct member
access - use our specialized struct
member access parser
            member_type =
self.analyze_struct_member_access(
)

            # If we're at or beyond the
end_pos, we're done
            if
self.current_token_index >=
end_pos:
                return member_type

            # Otherwise reset position
and fall back to regular expression
parsing
            self.current_token_index
= original_pos

        # NEW: Check if next token
is square bracket (array access)
        elif (next_pos <
len(self.token_stream) and

self.token_stream[next_pos][0] ==
'['):
            print(f"Found potential
array access: {func_name}[...]")

            # This is an array access -
we need to handle it specially
            element_type =
self.analyze_array_element()

            # If we're at or beyond the
end_pos, we're done
            if
self.current_token_index >=
end_pos:
                return element_type

            # Otherwise reset position
and fall back to regular expression
parsing
            self.current_token_index
= original_pos

        # Regular expression parsing
        result_type, has_relational =
self._parse_expression(end_pos)
        print(f"Expression result type:
{result_type}, has relational:
{has_relational}")
        return result_type

    def _parse_expression(self,
end_pos):
        """
        Parse an expression and return
its type and if it contains relational
operators.
        Returns a tuple (type,
has_relational_op)

        UPDATED to handle nested
parentheses correctly.
        """
        # For complex expressions, we
need to track:
        # 1. The current operand type
we're working with
        # 2. The current operator (if
any)
        # 3. Whether we're expecting
an operand or operator

        current_type = None
        current_operator = None
        expecting_operand = True

        # If this is a relational
expression, the result is boolean
        contains_relational_op = False

        # Debug output
        print(f"Parsing expression
from position
{self.current_token_index} to
{end_pos}")
        # Print tokens being parsed
        expr_tokens =
self.token_stream[self.current_token
_index:end_pos]
        expr_str = "
".join([f"{t[0]}('{t[1]}')" for t in
expr_tokens])
        print(f"Expression tokens:
{expr_str}")

        while self.current_token_index
< end_pos:
            token_type, token_value,
line, column =
self.get_current_token()
            print(f"Processing token:
{token_type} '{token_value}'")

            # Handle opening
parenthesis - parse subexpression
            if token_type == '(':
                self.advance()  # Skip
over the opening parenthesis

                # Parse the subexpression
- find matching closing parenthesis
first
                subexpr_start =
self.current_token_index
                paren_level = 1
                subexpr_end = None

                # Find matching closing
parenthesis
                temp_pos =
self.current_token_index
                while paren_level > 0 and
temp_pos < end_pos:
                    temp_token_type =
self.token_stream[temp_pos][0]
                    if temp_token_type ==
'(':
                        paren_level += 1
                    elif temp_token_type
== ')':
                        paren_level -= 1
                        if paren_level == 0:
                            subexpr_end =
temp_pos
                    temp_pos += 1

                if subexpr_end is None:
                    raise
SemanticError("Unclosed
parenthesis", line, column)

                # Recursively parse the
subexpression
                subexpr_type,
subexpr_relational =
self._parse_expression(subexpr_end
)
```

```
            print(f"Subexpression
type: {subexpr_type}, relational:
{subexpr_relational}")

            # Move to position after
the closing parenthesis
            self.current_token_index
= subexpr_end + 1

            # Update current state
            if current_operator:
                # Check compatibility
with operator
                if current_operator in
self.arithmetic_operators:
                    # Arithmetic
operators require numeric operands,
allow mixing nt and dbl
                    if current_type not in
['nt', 'dbl'] or subexpr_type not in
['nt', 'dbl']:
                        print(f"ERROR:
Cannot apply '{current_operator}' to
'{current_type}' and
'{subexpr_type}'")
                        raise
SemanticError(
                            f"Type
mismatch: Cannot apply
'{current_operator}' to
'{current_type}' and
'{subexpr_type}'",
                            line, column
                        )

                    # If we mix nt and
dbl, the result is dbl
                    if current_type ==
'dbl' or subexpr_type == 'dbl':
                        current_type =
'dbl'
                    else:
                        # Both are nt,
result remains nt
                        current_type = 'nt'

                elif current_operator in
self.logical_operators:
                    # Logical operators
require boolean operands
                    if current_type !=
'bln' or subexpr_type != 'bln':
                        print(f"ERROR:
Cannot apply '{current_operator}' to
'{current_type}' and
'{subexpr_type}'")
                        raise
SemanticError(
                            f"Type
mismatch: Cannot apply
'{current_operator}' to
'{current_type}' and
'{subexpr_type}'",
                            line, column
                        )
                    # Result is boolean
                    current_type = 'bln'
                elif current_operator in
self.relational_operators:
                    # Relational
operators behavior depends on
specific operator
                    if current_operator in
self.equality_operators:
                        # == and != work
for all types, but both operands must
be same type
                        # However, we'll
allow comparing nt with dbl
                        if current_type !=
subexpr_type:
                            # Allow
comparing nt with dbl
                            if not
((current_type == 'nt' and
subexpr_type == 'dbl') or
                                (current_type
== 'dbl' and subexpr_type == 'nt')):

print(f"ERROR: Cannot compare
'{current_type}' with
'{subexpr_type}'")
                                raise
SemanticError(
                                    f"Type
mismatch: Cannot compare
'{current_type}' with
'{subexpr_type}'",
                                    line,
column
                                )
                    else:
                        # <, >, <=, >= only
work for numeric types - now allow
mixing nt and dbl
                        if current_type not
in ['nt', 'dbl'] or subexpr_type not in
['nt', 'dbl']:
                            print(f"ERROR:
Cannot apply '{current_operator}' to
'{current_type}' and
'{subexpr_type}'")
                            raise
SemanticError(
                                f"Type
mismatch: Cannot apply
'{current_operator}' to
'{current_type}' and
'{subexpr_type}'",
                                line, column
                            )
                    # Result is boolean
                    current_type = 'bln'

                contains_relational_op = True
                elif current_operator in
self.string_concat_operator:
                    # String
concatenation requires string
operands
                    if current_type !=
'strng' or subexpr_type != 'strng':
                        print(f"ERROR:
Cannot concatenate '{current_type}'
with '{subexpr_type}'")
                        raise
SemanticError(
                            f"Type
mismatch: Cannot concatenate
'{current_type}' with
'{subexpr_type}'",
                            line, column
                        )
                    # Result is string
                    current_type = 'strng'

                # Reset operator after
applying it
                current_operator =
None
                expecting_operand =
False
            else:
                # This is the first
operand
                current_type =
subexpr_type
                contains_relational_op
= contains_relational_op or
subexpr_relational
                expecting_operand =
False

            # Update relational
operator flag
            contains_relational_op =
contains_relational_op or
subexpr_relational

            continue

        # Handle operands (values,
variables, function calls, struct
member access)
        if expecting_operand:
            operand_type = None

            # Handle
pre-increment/decrement operators
            if token_type in ['++', '--']:
                print(f"Found
pre-{token_type} operator")
```

```
            pre_op = token_type
            self.advance()  # Move past the operator

            # Must be followed by an identifier
            if self.get_current_token()[0] != 'id':
                print(f"ERROR: {pre_op} must be followed by an identifier")
                raise SemanticError(f"{pre_op} operator must be followed by an identifier", line, column)

            var_name = self.get_current_token()[1]
            var_line, var_column = self.get_current_token()[2], self.get_current_token()[3]

            # Check if variable exists
            symbol = self.current_scope.lookup(var_name)
            if not symbol:
                print(f"ERROR: Undefined variable '{var_name}'")
                raise SemanticError(f"Undefined variable '{var_name}'", var_line, var_column)

            # Check if variable is of type 'nt'
            if symbol.data_type != 'nt':
                print(f"ERROR: {pre_op} operator can only be applied to 'nt' variables, not '{symbol.data_type}'")
                raise SemanticError(f"{pre_op} operator can only be applied to 'nt' variables, not '{symbol.data_type}'",
                                    var_line, var_column)

            # Pre-increment/decrement returns the modified value
            operand_type = 'nt'
            self.advance()  # Move past the variable name

        # Determine the type of the current operand
        elif token_type in ['ntlit', '~ntlit']:
            operand_type = 'nt'
            if current_operator == '/' and token_value == '0':
                raise SemanticError(f"Division by zero detected", line, column)
            self.advance()
        elif token_type in ['dbllit', '~dbllit']:
            operand_type = 'dbl'
            if current_operator == '/' and token_value == '0':
                raise SemanticError(f"Division by zero detected", line, column)
            self.advance()
        elif token_type in ['true', 'false', 'blnlit']:
            operand_type = 'bln'
            self.advance()
        elif token_type == 'chrlit':
            operand_type = 'chr'
            self.advance()
        elif token_type == 'strnglit':
            operand_type = 'strng'
            self.advance()
        elif token_type == 'id':
            # Save position for lookahead
            save_pos = self.current_token_index
            var_name = token_value

            # Look ahead to see what follows the identifier
            self.advance()
            if self.current_token_index < end_pos:
                next_token = self.get_current_token()[0]

                # Check for array access: id[...]
                if next_token == '[':
                    print(f"Found array access in expression: {var_name}[...]")
                    # Go back to array name
                    self.current_token_index = save_pos

                    # Get array element type
                    operand_type = self.analyze_array_element()
                    print(f"Array element type in expression: {operand_type}")

                # Check for post-increment or post-decrement
                elif next_token in ['++', '--']:
                    # These operators can only be applied to 'nt' variables
                    symbol = self.current_scope.lookup(var_name)
                    if not symbol:
                        raise SemanticError(f"Undefined variable '{var_name}'", line, column)

                    if symbol.data_type != 'nt':
                        print(f"ERROR: Increment/decrement only applies to 'nt' variables, not '{symbol.data_type}'")
                        raise SemanticError(f"Increment/decrement operators can only be applied to 'nt' variables, not '{symbol.data_type}'",
                                            line, column)

                    operand_type = 'nt'
                    self.advance()  # Move past the operator

                # Check for function call: id(...)
                elif next_token == '(':
                    # This is a function call within an expression
                    # Go back to function name
                    self.current_token_index = save_pos

                    # Process function call and get its return type
                    operand_type = self.analyze_function_call()

                    # If the function is void, it can't be used in an expression
                    if operand_type == 'vd':
                        print(f"ERROR: Void function '{var_name}' cannot be used in expression")
```

```
                    raise SemanticError(f"Void function '{var_name}' cannot be used in an expression", line, column)

                # Check for struct member access: id.member
                elif next_token == '.':
                    print(f"Processing struct member access in _parse_expression: {var_name}.{self.token_stream[self.current_token_index+1][1]}")
                    # This is a struct member access
                    # Go back to struct instance name
                    self.current_token_index = save_pos

                    # Process struct member access and get its type
                    operand_type = self.analyze_struct_member_access()

                else:
                    # Not a function call or struct member access, just a variable reference
                    self.current_token_index = save_pos

                    # Get type from symbol table
                    symbol = self.current_scope.lookup(var_name)
                    if not symbol:
                        print(f"ERROR: Undefined variable '{var_name}'")
                        raise SemanticError(f"Undefined variable '{var_name}'", line, column)

                    # Check if it's a function being used without parentheses
                    if symbol.type == 'function':
                        print(f"ERROR: Function '{var_name}' used without parentheses")
                        raise SemanticError(f"Function '{var_name}' used without parentheses", line, column)

                    # Check if variable is initialized
                    if not symbol.initialized:
                        print(f"Warning: Variable '{var_name}' may be used before initialization at line {line}, column {column}")

                    operand_type = symbol.data_type
                    self.advance()
            else:
                # We're at the end - treat as variable reference
                self.current_token_index = save_pos

                # Get type from symbol table
                symbol = self.current_scope.lookup(var_name)
                if not symbol:
                    print(f"ERROR: Undefined variable '{var_name}'")
                    raise SemanticError(f"Undefined variable '{var_name}'", line, column)

                operand_type = symbol.data_type
                self.advance()
        elif token_type == '!':
            # Logical NOT operator
            self.advance()  # Skip !

            # Track number of consecutive NOT operators
            not_count = 1
            while self.current_token_index < end_pos and self.token_stream[self.current_token_index][0] == '!':
                not_count += 1
                self.advance()  # Skip additional ! operators

            # Handle subexpression after the series of ! operators
            if self.current_token_index < end_pos:
                next_token_type = self.token_stream[self.current_token_index][0]
                next_token_value = self.token_stream[self.current_token_index][1]
                next_line = self.token_stream[self.current_token_index][2]
                next_column = self.token_stream[self.current_token_index][3]

                if next_token_type == '(':
                    # Process parenthesized expression after !
                    self.advance()  # Skip (

                    # Find closing parenthesis
                    start_pos = self.current_token_index
                    paren_level = 1
                    while paren_level > 0 and self.current_token_index < end_pos:
                        self.advance()
                        if self.current_token_index >= len(self.token_stream):
                            raise SemanticError("Unclosed parenthesis after '!'", line, column)

                        current_token = self.token_stream[self.current_token_index][0]
                        if current_token == '(':
                            paren_level += 1
                        elif current_token == ')':
                            paren_level -= 1

                    # Current token is now at the closing parenthesis
                    subexpr_end = self.current_token_index

                    self.current_token_index = start_pos

                    # Parse the subexpression
                    subexpr_type, _ = self._parse_expression(subexpr_end)

                    # Skip the closing parenthesis
                    self.advance()

                    if subexpr_type != 'bln':
```

```
                    print(f"ERROR: Cannot apply '!' to non-boolean type '{subexpr_type}'")
                    raise SemanticError(f"Type mismatch: Cannot apply '!' to non-boolean type '{subexpr_type}'", line, column)

                    # The result is always boolean, regardless of how many ! operators
                    # If there's an odd number of ! operators, the value is negated
                    # If there's an even number, it's equivalent to the original value
                    current_type = 'bln'
                else:
                    # Not a parenthesized expression - check the identifier or literal
                    if next_token_type == 'id':
                        # Variable reference
                        symbol = self.current_scope.lookup(next_token_value)
                        if not symbol:
                            print(f"ERROR: Undefined variable '{next_token_value}'")
                            raise SemanticError(f"Undefined variable '{next_token_value}'", next_line, next_column)

                        subexpr_type = symbol.data_type
                    elif next_token_type in ['true', 'false', 'blnlit']:
                        subexpr_type = 'bln'
                    else:
                        # For other token types, get their corresponding type
                        subexpr_type = self.get_token_type(next_token_type)

                    if subexpr_type != 'bln':
                        print(f"ERROR: Cannot apply '!' to non-boolean type '{subexpr_type}'")
                        raise SemanticError(f"Type mismatch: Cannot apply '!' to non-boolean type '{subexpr_type}'", next_line, next_column)

                    self.advance()  # Skip the operand
                    current_type = 'bln'

                expecting_operand = False

            else:
                # No operand after ! operator
                raise SemanticError("Unexpected end of expression after '!'", line, column)

            operand_type = 'bln'
            current_type = operand_type
            expecting_operand = False
            continue
        elif token_type == 'npt':
            print(f"Found npt function in expression")
            # This is a built-in function for input
            # Use our function call analyzer which now handles npt
            operand_type = self.analyze_function_call()
        else:
            # Unknown token in expression
            print(f"ERROR: Unexpected token '{token_type}' in expression")
            raise SemanticError(f"Unexpected token '{token_type}' in expression", line, column)

        # Handle the operand in context
        if current_type is None:
            # This is the first operand
            current_type = operand_type
        elif current_operator:
            # This is a right operand - check compatibility with operator and left operand
            if current_operator in self.arithmetic_operators:
                # Arithmetic operators require numeric operands - now allow mixing nt and dbl
                if current_type not in ['nt', 'dbl'] or operand_type not in ['nt', 'dbl']:
                    print(f"ERROR: Cannot apply '{current_operator}' to '{current_type}' and '{operand_type}'")
                    raise SemanticError(
                        f"Type mismatch: Cannot apply '{current_operator}' to '{current_type}' and '{operand_type}'",
                        line, column
                    )

                # If we mix nt and dbl, the result is dbl
                if current_type == 'dbl' or operand_type == 'dbl':
                    current_type = 'dbl'
                else:
                    # Both are nt, result remains nt
                    current_type = 'nt'

            elif current_operator in self.logical_operators:
                # Logical operators require boolean operands
                if current_type != 'bln' or operand_type != 'bln':
                    print(f"ERROR: Cannot apply '{current_operator}' to '{current_type}' and '{operand_type}'")
                    raise SemanticError(
                        f"Type mismatch: Cannot apply '{current_operator}' to '{current_type}' and '{operand_type}'",
                        line, column
                    )
                # Result is boolean
                current_type = 'bln'
            elif current_operator in self.relational_operators:
                # Relational operators behavior depends on specific operator
                if current_operator in self.equality_operators:
                    # == and != work for all types, but operands must match
                    # However, we'll allow comparing nt with dbl
```

```
                    if current_type !=
operand_type:
                        # Allow
comparing nt with dbl
                        if not
((current_type == 'nt' and
operand_type == 'dbl') or
                            (current_type
== 'dbl' and operand_type == 'nt')):

print(f"ERROR: Cannot compare
'{current_type}' with
'{operand_type}'")
                            raise
SemanticError(
                                f"Type
mismatch: Cannot compare
'{current_type}' with
'{operand_type}'",
                                line,
column
                            )
                    else:
                        # <, >, <=, >= only
work for numeric types - now allow
mixing nt and dbl
                        if current_type not
in ['nt', 'dbl'] or operand_type not in
['nt', 'dbl']:
                            print(f"ERROR:
Cannot apply '{current_operator}' to
'{current_type}' and
'{operand_type}'")
                            raise
SemanticError(
                                f"Type
mismatch: Cannot apply
'{current_operator}' to
'{current_type}' and
'{operand_type}'",
                                line, column
                            )

                    # Result is always
boolean
                    current_type = 'bln'

contains_relational_op = True
                elif current_operator in
self.string_concat_operator:
                    # String
concatenation requires string
operands
                    if current_type !=
'strng' or operand_type != 'strng':
                        print(f"ERROR:
Cannot concatenate '{current_type}'
with '{operand_type}'")
                        raise
SemanticError(
                            f"Type
mismatch: Cannot concatenate
'{current_type}' with
'{operand_type}'",
                            line, column
                        )
                    # Result is string
                    current_type = 'strng'

                # Clear the operator
now that it's been applied
                current_operator =
None

            expecting_operand =
False

        # Handle operators
        else:
            if token_type in
self.arithmetic_operators:
                # Verify current type
supports arithmetic
                if current_type not in
['nt', 'dbl']:
                    print(f"ERROR:
Cannot apply arithmetic operator
'{token_type}' to '{current_type}'")
                    raise SemanticError(
                        f"Type mismatch:
Cannot apply '{token_type}' to
'{current_type}'",
                        line, column
                    )
                current_operator =
token_type
                expecting_operand =
True
                self.advance()
            elif token_type in
self.logical_operators:
                # Verify current type is
boolean
                if current_type != 'bln':
                    print(f"ERROR:
Cannot apply logical operator
'{token_type}' to '{current_type}'")
                    raise SemanticError(
                        f"Type mismatch:
Cannot apply '{token_type}' to
'{current_type}'",
                        line, column
                    )
                current_operator =
token_type
                expecting_operand =
True
                self.advance()
            elif token_type in
self.relational_operators:
                # For relational
operators, verify based on specific
operator
                if token_type in
self.equality_operators:
                    # == and != work for
all types
                    # No additional
checks needed here, will check type
match with right operand
                    pass
                else:
                    # <, >, <=, >= only
work with numeric types
                    if current_type not in
['nt', 'dbl']:
                        print(f"ERROR:
Cannot apply relational operator
'{token_type}' to '{current_type}'")
                        raise
SemanticError(
                            f"Type
mismatch: Cannot apply
'{token_type}' to '{current_type}'",
                            line, column
                        )

                # Store the relational
operator
                current_operator =
token_type
                contains_relational_op
= True
                expecting_operand =
True
                self.advance()
            elif token_type == '`':
                # String concatenation
operator
                if current_type !=
'strng':
                    print(f"ERROR:
Cannot apply string concatenation
to non-string type '{current_type}'")
                    raise SemanticError(
                        f"Type mismatch:
Cannot apply '`' to non-string type
'{current_type}'",
                        line, column
                    )
                current_operator =
token_type
                expecting_operand =
True
                self.advance()
            elif token_type == ')':
                # This should be
handled by the parenthesis
processing logic
```

```
            # If we encounter a closing parenthesis here, it means it's not balanced properly
            print(f"ERROR: Unexpected closing parenthesis")
            raise SemanticError(f"Unexpected closing parenthesis", line, column)
        else:
            # Unknown operator
            print(f"ERROR: Unexpected token '{token_type}' in expression")
            raise SemanticError(f"Unexpected token '{token_type}' in expression", line, column)

    if expecting_operand:
        print(f"ERROR: Incomplete expression: expecting operand")
        raise SemanticError("Incomplete expression: expecting operand", line, column)

    # If we have a pending operator, that's an error
    if current_operator:
        print(f"ERROR: Incomplete expression: missing right operand for '{current_operator}'")
        raise SemanticError(f"Incomplete expression: missing right operand for '{current_operator}'", line, column)

    # Print final result for debugging
    print(f"Expression final type: {current_type}, contains relational: {contains_relational_op}")

    # If expression contains a relational operator, result is boolean
    if contains_relational_op:
        return 'bln', True

    return current_type, contains_relational_op

def analyze_array_element(self):
    """
    Analyze an array element access expression and return the element type.
    Used for expressions like arr[1] when they appear on right side of assignments.
    """
    # Get array name
    token_type, var_name, line, column = self.get_current_token()
    if token_type != 'id':
        raise SemanticError(f"Expected array identifier, got {token_type}", line, column)

    print(f"Analyzing array element access for '{var_name}' at line {line}, column {column}")

    # Check if variable exists
    symbol = self.current_scope.lookup(var_name)
    if not symbol:
        raise SemanticError(f"Undefined variable '{var_name}'", line, column)

    # Check that it's an array
    if not symbol.is_array:
        raise SemanticError(f"Variable '{var_name}' is not an array", line, column)

    self.advance()  # Move past array name

    # Process opening bracket
    token_type, token_value, line, column = self.get_current_token()
    if token_type != '[':
        raise SemanticError(f"Expected '[', got {token_type}", line, column)

    self.advance()  # Move past '['

    # Save the current index token for bounds checking
    index1_token_type, index1_value, index1_line, index1_column = self.get_current_token()

    # Find the end of this index expression (should be the closing bracket)
    start_pos = self.current_token_index
    bracket_level = 1

    while bracket_level > 0 and self.current_token_index < len(self.token_stream):
        self.current_token_index += 1
        if self.current_token_index >= len(self.token_stream):
            raise SemanticError("Unclosed bracket", line, column)

        current_token = self.token_stream[self.current_token_index][0]
        if current_token == '[':
            bracket_level += 1
        elif current_token == ']':
            bracket_level -= 1

    end_pos = self.current_token_index
    self.current_token_index = start_pos

    # Validate index expression is of type nt
    index1_type = self.analyze_expression(end_pos)
    if index1_type != 'nt':
        raise SemanticError(f"Array index must be of type 'nt', got '{index1_type}'",
                            index1_line, index1_column)

    # Check bounds if index is a literal
    if index1_token_type == 'ntlit' and isinstance(symbol.array_sizes[0], int):
        index1 = int(index1_value)
        if index1 < 0 or index1 >= symbol.array_sizes[0]:
            raise SemanticError(f"Array index {index1} out of bounds (size {symbol.array_sizes[0]})",
                                index1_line, index1_column)

    if self.get_current_token()[0] != ']':
        raise SemanticError(f"Expected ']', got {self.get_current_token()[0]}",
                            self.get_current_token()[2], self.get_current_token()[3])

    self.advance()  # Move past ']'

    # Check for second dimension if this is a 2D array
    if symbol.array_dimensions == 2:
```

```
        if self.get_current_token()[0]
!= '[':
            raise
SemanticError(f"Expected second
dimension '[' for 2D array, got
{self.get_current_token()[0]}",

self.get_current_token()[2],
self.get_current_token()[3])

        self.advance()  # Move past
'['

        # Save the current index
token for bounds checking
        index2_token_type,
index2_value, index2_line,
index2_column =
self.get_current_token()

        # Find the end of this index
expression (should be the closing
bracket)
        start_pos =
self.current_token_index
        bracket_level = 1

        while bracket_level > 0 and
self.current_token_index <
len(self.token_stream):
            self.current_token_index
+= 1
            if
self.current_token_index >=
len(self.token_stream):
                raise
SemanticError("Unclosed bracket",
line, column)

            current_token =
self.token_stream[self.current_token
_index][0]
            if current_token == '[':
                bracket_level += 1
            elif current_token == ']':
                bracket_level -= 1

        end_pos =
self.current_token_index
        self.current_token_index =
start_pos

        # Validate index expression
is of type nt
        index2_type =
self.analyze_expression(end_pos)
        if index2_type != 'nt':
            raise
SemanticError(f"Array index must
be of type 'nt', got '{index2_type}'",
                          index2_line,
index2_column)

        # Check bounds if index is a
literal
        if index2_token_type ==
'ntlit' and
isinstance(symbol.array_sizes[1],
int):
            index2 =
int(index2_value)
            if index2 < 0 or index2 >=
symbol.array_sizes[1]:
                raise
SemanticError(f"Array index
{index2} out of bounds (size
{symbol.array_sizes[1]})",
                              index2_line,
index2_column)

        if self.get_current_token()[0]
!= ']':
            raise
SemanticError(f"Expected ']', got
{self.get_current_token()[0]}",

self.get_current_token()[2],
self.get_current_token()[3])

        self.advance()  # Move past
']'

      # Return the data type of the
array element
      print(f"Array element type:
{symbol.data_type}")
      return symbol.data_type

   def is_compatible_type(self,
declared_type, value_type):
      """Check if value type is
compatible with declared type"""
      # Map literal types to their
corresponding data types
      if value_type in ('ntlit', '~ntlit'):
         return declared_type == 'nt'
      elif value_type in ('dbllit',
'~dbllit'):
         return declared_type == 'dbl'
      elif value_type in ('true', 'false',
'blnlit'):
         return declared_type == 'bln'
      elif value_type == 'chrlit':
         return declared_type == 'chr'
      elif value_type == 'strnglit':
         return declared_type ==
'strng'

      # Handle exact type matching
(no type conversion)
      return declared_type ==
value_type

   def analyze_assignment(self):
      """Analyze variable
assignment including shortcut
assignment operators"""
      var_token_type, var_name,
line, column =
self.get_current_token()

      # Debug output
      print(f"Analyzing assignment
for variable '{var_name}' at line
{line}, column {column}")

      # Check if variable exists
      symbol =
self.current_scope.lookup(var_name
)
      if not symbol:
         raise
SemanticError(f"Variable
'{var_name}' not declared", line,
column)

      var_type = symbol.data_type
      print(f"Variable '{var_name}'
has type '{var_type}'")

      # Check if this is a constant
      if symbol.is_constant:
         raise
SemanticError(f"Cannot reassign
constant '{var_name}'", line,
column)

      # Move past variable name
      self.advance()

      # Check which assignment
operator is being used
      token_type, token_value,
op_line, op_column =
self.get_current_token()
      print(f"Assignment operator:
{token_type}")

      # Handle increment/decrement
operators (++, --)
      if token_type in ['++', '--']:
         # These operators can only
be applied to 'nt' variables
         if var_type != 'nt':
            raise
SemanticError(f"Increment/decreme
nt operators can only be applied to
'nt' variables, not '{var_type}'",
                          op_line,
op_column)
```

```
            self.advance()  # Move past the operator

            # Check for semicolon
            if self.get_current_token()[0] != ';':
                raise SemanticError(f"Expected ';' after increment/decrement operation", self.get_current_token()[2], self.get_current_token()[3])

            self.advance()  # Move past semicolon
            symbol.initialized = True
            return

        # Handle compound assignment operators (+=, -=, *=, /=, %=)
        if token_type in ['+=', '-=', '*=', '/=', '%=']:
            print(f"Processing shortcut assignment operator '{token_type}'")

            # These operators require numeric operands for variable
            if var_type not in ['nt', 'dbl']:
                raise SemanticError(f"Shortcut assignment operator '{token_type}' can only be applied to numeric types, not '{var_type}'",
                                    op_line, op_column)

            self.advance()  # Move past the operator

            # Save starting position for expression analysis
            start_pos = self.current_token_index

            # Skip ahead to find the end of the expression (semicolon)
            while (self.current_token_index < len(self.token_stream) and
                   self.token_stream[self.current_token_index][0] != ';'):
                self.advance()

            # Reset position to start of expression
            end_pos = self.current_token_index
            self.current_token_index = start_pos

            # Get the tokens being analyzed for debug
            expr_tokens = self.token_stream[start_pos:end_pos]
            expr_str = " ".join([f"{t[0]}('{t[1]}')" for t in expr_tokens])
            print(f"Expression tokens: {expr_str}")

            # Analyze the expression on the right side of the shortcut assignment
            expr_type = self.analyze_expression(end_pos)
            print(f"Expression type: {expr_type}")

            # For shortcut assignments, the right operand must be a compatible numeric type
            if expr_type not in ['nt', 'dbl']:
                raise SemanticError(
                    f"Type mismatch: Cannot use '{token_type}' with non-numeric type '{expr_type}'",
                    op_line, op_column
                )

            # Mark variable as initialized
            symbol.initialized = True

            # Move past the semicolon
            if self.get_current_token()[0] == ';':
                self.advance()  # Move past the semicolon
            return

        # Regular assignment (=)
        elif token_type == '=':
            # ... (rest of your existing regular assignment code)
            self.advance()  # Move past the = operator

            # Check if this is an npt input statement directly
            next_token = self.get_current_token()
            if next_token and next_token[0] == 'npt':
                print(f"Found direct npt function call in assignment for variable '{var_name}'")

                # Use our function call analyzer to process the npt function
                expr_type = self.analyze_function_call()

                # npt function returns strng by default, but we'll allow type coercion
                # Type conversion happens at runtime, so we'll just mark the variable as initialized
                symbol.initialized = True

                # Current position is now at the semicolon
                if self.get_current_token()[0] == ';':
                    self.advance()  # Move past the semicolon

                return

            # Save starting position for expression analysis
            start_pos = self.current_token_index

            # Skip ahead to find the end of the expression (semicolon)
            while (self.current_token_index < len(self.token_stream) and
                   self.token_stream[self.current_token_index][0] != ';'):
                self.advance()

            # Reset position to start of expression
            end_pos = self.current_token_index
            self.current_token_index = start_pos

            # Analyze the expression
            expr_type = self.analyze_expression(end_pos)

            # Validate assignment compatibility - no implicit conversions in Conso
            if var_type != expr_type:
                # Special case: Boolean variable can be assigned result of relational expression
                if var_type == 'bln' and expr_type == 'bln':
                    # This is valid - a boolean variable can hold the result of a relational expression
                    pass
```

```
            else:
                raise SemanticError(
                    f"Type mismatch: Cannot assign {expr_type} to {var_type}",
                    op_line, op_column
                )

            symbol.initialized = True

            # Current position is now at the semicolon
            if self.get_current_token()[0] == ';':
                self.advance()  # Move past the semicolon
        else:
            raise SemanticError(f"Expected assignment operator, got '{token_type}'", op_line, op_column)

    def check_variable_usage(self, var_name, line, column):
        """Check if a variable used in an expression or function call is declared"""
        # Skip check for built-in symbols if applicable
        if var_name in self.built_in_functions:
            return True

        # Look up the variable in the current scope and parent scopes
        symbol = self.current_scope.lookup(var_name)
        if not symbol:
            raise SemanticError(f"Undefined variable '{var_name}'", line, column)

        # Optionally: Check if variable is initialized before use
        if not symbol.initialized:
            # This could be a warning instead of an error
            print(f"Warning: Variable '{var_name}' may be used before initialization at line {line}, column {column}")

        return symbol.data_type

    def analyze_if_statement(self):
        """Analyze an if statement (f statement in Conso)"""
        token_type, token_value, line, column = self.get_current_token()
        if token_type != 'f':
            raise SemanticError(f"Expected 'f' keyword, got '{token_type}'", line, column)

        self.advance()  # Move past 'f'

        # Process opening parenthesis for condition
        token_type, token_value, line, column = self.get_current_token()
        if token_type != '(':
            raise SemanticError(f"Expected '(' after 'f', got '{token_type}'", line, column)

        # Keep track of the outermost parentheses level
        outer_paren_start = self.current_token_index
        self.advance()  # Move past opening parenthesis

        # Need to find the matching closing parenthesis for the outermost level
        paren_level = 1

        while paren_level > 0 and self.current_token_index < len(self.token_stream):
            current_token = self.get_current_token()[0]
            if current_token == '(':
                paren_level += 1
            elif current_token == ')':
                paren_level -= 1

            if paren_level > 0:
                self.advance()

        # Now we should be at the closing parenthesis of the condition
        if paren_level > 0:
            raise SemanticError("Unclosed parenthesis in 'f' statement condition", line, column)

        # Now at the closing parenthesis
        outer_paren_end = self.current_token_index

        # Reset to just after the opening parenthesis
        self.current_token_index = outer_paren_start + 1

        # Analyze the condition using your expression analyzer
        # The condition is everything between the parentheses
        expr_type = self.analyze_expression(outer_paren_end)

        # --- MODIFIED CHECK: Allow 'nt' or 'bln' ---
        if expr_type not in ['bln', 'nt']:
            raise SemanticError(f"Condition in 'f' statement must be of type 'bln' or 'nt', got '{expr_type}'", line, column)
        # --- END MODIFIED CHECK ---

        # Move to the closing parenthesis and advance past it
        self.current_token_index = outer_paren_end
        self.advance()  # Move past the closing parenthesis

        # Process opening brace for if-body
        token_type, token_value, line, column = self.get_current_token()
        if token_type != '{':
            raise SemanticError(f"Expected '{' to start 'f' body, got '{token_type}'", line, column)

        self.advance()  # Move past '{'

        # Create a new scope for the if-body
        if_scope = SymbolTable(parent=self.current_scope, scope_name="if block")
        original_scope = self.current_scope  # Store the original scope
        self.current_scope = if_scope

        # Process if-body statements
        self.analyze_block_statements()

        # Restore original scope
        self.current_scope = original_scope

        # Check for elseif (lsf) or else (ls) statements
        if self.current_token_index < len(self.token_stream):
```

```
        token_type, token_value,
line, column =
self.get_current_token()
        if token_type == 'lsf':

self.analyze_elseif_statement(origin
al_scope)  # Pass the original scope
        elif token_type == 'ls':

self.analyze_else_statement(original
_scope)  # Pass the original scope

    def analyze_elseif_statement(self,
parent_scope):
        """Analyze an else-if statement
(lsf statement in Conso)"""
        token_type, token_value, line,
column = self.get_current_token()
        if token_type != 'lsf':
            raise
SemanticError(f"Expected 'lsf'
keyword, got '{token_type}'", line,
column)

        self.advance()  # Move past
'lsf'

        # Process opening parenthesis
for condition
        token_type, token_value, line,
column = self.get_current_token()
        if token_type != '(':
            raise
SemanticError(f"Expected '(' after
'lsf', got '{token_type}'", line,
column)

        # Keep track of the outermost
parentheses level
        outer_paren_start =
self.current_token_index
        self.advance()  # Move past
opening parenthesis

        # Need to find the matching
closing parenthesis for the
outermost level
        paren_level = 1

        while paren_level > 0 and
self.current_token_index <
len(self.token_stream):
            current_token =
self.get_current_token()[0]
            if current_token == '(':
                paren_level += 1
            elif current_token == ')':
                paren_level -= 1

            if paren_level > 0:
                self.advance()

        # Now we should be at the
closing parenthesis of the condition
        if paren_level > 0:
            raise
SemanticError("Unclosed
parenthesis in 'lsf' statement
condition", line, column)

        # Now at the closing
parenthesis
        outer_paren_end =
self.current_token_index

        # Reset to just after the
opening parenthesis
        self.current_token_index =
outer_paren_start + 1

        # Analyze the condition using
your expression analyzer
        # The condition is everything
between the parentheses
        expr_type =
self.analyze_expression(outer_paren
_end)

        # --- MODIFIED CHECK:
Allow 'nt' or 'bln' ---
        if expr_type not in ['bln', 'nt']:
            raise
SemanticError(f"Condition in 'lsf'
statement must be of type 'bln' or
'nt', got '{expr_type}'", line,
column)
        # --- END MODIFIED
CHECK ---

        # Move to the closing
parenthesis and advance past it
        self.current_token_index =
outer_paren_end
        self.advance()  # Move past the
closing parenthesis

        # Process opening brace for
elseif-body
        token_type, token_value, line,
column = self.get_current_token()
        if token_type != '{':
            raise
SemanticError(f"Expected '{{' to
start 'lsf' body, got '{token_type}'",
line, column)

        self.advance()  # Move past '{'

        # Create a new scope for the
elseif-body, using the passed parent
scope
        elseif_scope =
SymbolTable(parent=parent_scope,
scope_name="elseif block")
        self.current_scope =
elseif_scope

        # Process elseif-body
statements

self.analyze_block_statements()

        # Restore original scope
        self.current_scope =
parent_scope

        # Check for another elseif (lsf)
or else (ls) statement
        if self.current_token_index <
len(self.token_stream):
            token_type, token_value,
line, column =
self.get_current_token()
            if token_type == 'lsf':

self.analyze_elseif_statement(parent
_scope)  # Pass the original scope
            elif token_type == 'ls':

self.analyze_else_statement(parent_
scope)  # Pass the original scope

    def analyze_else_statement(self,
parent_scope):
        """Analyze an else statement
(ls statement in Conso)"""
        token_type, token_value, line,
column = self.get_current_token()
        if token_type != 'ls':
            raise
SemanticError(f"Expected 'ls'
keyword, got '{token_type}'", line,
column)

        self.advance()  # Move past 'ls'

        # Process opening brace for
else-body
        token_type, token_value, line,
column = self.get_current_token()
        if token_type != '{':
            raise
SemanticError(f"Expected '{{' to
start 'ls' body, got '{token_type}'",
line, column)

        self.advance()  # Move past '{'

        # Create a new scope for the
else-body, using the passed parent
scope
```

```
        else_scope = SymbolTable(parent=parent_scope, scope_name="else block")
        self.current_scope = else_scope

        # Process else-body statements
        self.analyze_block_statements()

        # Restore original scope
        self.current_scope = parent_scope

    def analyze_switch_statement(self):
        """Analyze a switch statement (swtch statement in Conso)"""
        token_type, token_value, line, column = self.get_current_token()
        if token_type != 'swtch':
            raise SemanticError(f"Expected 'swtch' keyword, got '{token_type}'", line, column)

        self.advance()  # Move past 'swtch'

        # Process opening parenthesis for switch expression
        token_type, token_value, line, column = self.get_current_token()
        if token_type != '(':
            raise SemanticError(f"Expected '(' after 'swtch', got '{token_type}'", line, column)

        self.advance()  # Move past '('

        # Now get the switch expression token
        token_type, token_value, line, column = self.get_current_token()
        switch_expr_line, switch_expr_column = line, column

        # If it's a struct member access (e.g., student.id)
        if token_type == 'id' and self.peek_next_token()[0] == '.':
            switch_expr_type = self.analyze_struct_member_access()
        elif token_type == 'id':
            # Get the variable type from symbol table
            var_name = token_value
            symbol = self.current_scope.lookup(var_name)
            if not symbol:
                raise SemanticError(f"Undefined variable '{var_name}'", line, column)

            switch_expr_type = symbol.data_type
            self.advance()  # Move past identifier
        else:
            # Not an identifier or struct member access
            raise SemanticError(f"Switch expression must be an identifier", line, column)

        # Verify switch expression type is nt or chr
        if switch_expr_type not in ['nt', 'chr']:
            raise SemanticError(f"Switch expression must be of type 'nt' or 'chr', got '{switch_expr_type}'",
                                switch_expr_line, switch_expr_column)

        # Process closing parenthesis
        token_type, token_value, line, column = self.get_current_token()
        if token_type != ')':
            raise SemanticError(f"Expected ')' after switch expression, got '{token_type}'", line, column)

        self.advance()  # Move past ')'

        # Process opening brace for switch body
        token_type, token_value, line, column = self.get_current_token()
        if token_type != '{':
            raise SemanticError(f"Expected '{' to start switch body, got '{token_type}'", line, column)

        self.advance()  # Move past '{'

        # Save old switch and case block state
        old_in_switch = self.in_switch
        old_in_case_block = self.in_case_block

        # Set new state - we're now in a switch but not yet in a case block
        self.in_switch = True
        self.in_case_block = False

        # Create a new scope for the switch body
        switch_scope = SymbolTable(parent=self.current_scope, scope_name="switch block")
        original_scope = self.current_scope
        self.current_scope = switch_scope

        # Track case labels to ensure no duplicates
        case_labels = set()

        # Process case statements
        has_default = False
        found_break = True  # Initially true to allow the first case

        while self.current_token_index < len(self.token_stream):
            token_type, token_value, line, column = self.get_current_token()

            # Check for end of switch
            if token_type == '}':
                self.advance()  # Move past '}'
                break

            # Handle case statement
            if token_type == 'cs':
                self.in_case_block = True
                if not found_break:
                    raise SemanticError(f"Missing 'brk' statement before new case", line, column)

                self.advance()  # Move past 'cs'

                # Get case label
                token_type, token_value, line, column = self.get_current_token()

                # Ensure case label type matches switch expression type
                if switch_expr_type == 'nt':
                    if token_type != 'ntlit':
                        raise SemanticError(f"Case label must be of type 'nt', got '{token_type}'", line, column)
```

```
            # Check for duplicate case label
            if token_value in case_labels:
                raise SemanticError(f"Duplicate case label '{token_value}'", line, column)

            case_labels.add(token_value)
        elif switch_expr_type == 'chr':
            if token_type != 'chrlit':
                raise SemanticError(f"Case label must be of type 'chr', got '{token_type}'", line, column)

            # Check for duplicate case label
            if token_value in case_labels:
                raise SemanticError(f"Duplicate case label '{token_value}'", line, column)

            case_labels.add(token_value)

        self.advance()  # Move past case label

        # Check for colon
        token_type, token_value, line, column = self.get_current_token()
        if token_type != ':':
            raise SemanticError(f"Expected ':' after case label, got '{token_type}'", line, column)

        self.advance()  # Move past ':'

        # Reset break flag
        found_break = False

        # Process statements until break or next case/default
        while self.current_token_index < len(self.token_stream):
            current_token = self.get_current_token()[0]

            # Check for continue statement
            if current_token == 'cntn':
                raise SemanticError("Continue statement cannot be used directly in a switch case",
                                    self.get_current_token()[2], self.get_current_token()[3])

            # Found break - set flag and process it
            if current_token == 'brk':
                found_break = True
                self.analyze_break_statement()
                break

            # Reached another case or end of switch without finding break
            if current_token in ['cs', 'dflt', '}']:
                raise SemanticError(f"Missing 'brk' statement at end of case", line, column)

            # Now we'll properly handle statements in the case block
            # Handle various statement types (including loops and nested switches)
            if current_token == 'f':
                self.analyze_if_statement()
            elif current_token == 'whl':
                # Allow loops inside switch cases
                self.analyze_while_loop()
            elif current_token == 'fr':
                # Allow for loops inside switch cases
                self.analyze_for_loop()
            elif current_token == 'd':
                # Allow do-while loops inside switch cases
                self.analyze_do_while_loop()
            elif current_token == 'swtch':
                # Allow nested switch statements - REMOVED the restriction
                self.analyze_switch_statement()
            elif current_token == 'cnst':
                self.analyze_constant_declaration()
            elif current_token == 'dfstrct':
                self.analyze_struct_instantiation()
            elif current_token in self.data_types:
                start_pos = self.current_token_index
                self.advance()
                if self.current_token_index >= len(self.token_stream) or self.token_stream[self.current_token_index][0] != 'id':
                    raise SemanticError(f"Expected an identifier after '{current_token}'", self.get_current_token()[2], self.get_current_token()[3])

                next_token = self.peek_next_token()
                if next_token and next_token[0] == '[':
                    self.current_token_index = start_pos
                    self.analyze_array_declaration()
                else:
                    self.current_token_index = start_pos
                    self.analyze_variable_declaration()
            elif current_token == 'id':
                id_token_value = self.get_current_token()[1]
                next_token = self.peek_next_token()
                if next_token and next_token[0] == '.':
                    self.analyze_struct_member_access()
                elif next_token and next_token[0] == '(':
                    self.analyze_function_call()
                elif next_token and next_token[0] == '[':
                    self.analyze_array_access()

                # Skip to end of statement
```

```
                    while
self.current_token_index <
len(self.token_stream) and
self.get_current_token()[0] != ';':
                        self.advance()
                    self.advance()  #
Move past semicolon
                elif next_token and
next_token[0] == '=':

self.analyze_assignment()
                else:

self.check_variable_usage(id_token
_value,

self.get_current_token()[2],
self.get_current_token()[3])
                    self.advance()
            elif current_token ==
'prnt':

self.analyze_print_statement()
            elif current_token ==
'rtrn':

self.analyze_return_in_conditional()
            else:
                self.advance()

        # Handle default case
        elif token_type == 'dflt':
            self.in_case_block = True
            if not found_break:
                raise
SemanticError(f"Missing 'brk'
statement before default case", line,
column)

            if has_default:
                raise
SemanticError(f"Multiple default
cases are not allowed", line,
column)

            has_default = True
            self.advance()  # Move
past 'dflt'

            # Check for colon
            token_type, token_value,
line, column =
self.get_current_token()
            if token_type != ':':
                raise
SemanticError(f"Expected ':' after
'dflt', got '{token_type}'", line,
column)

            self.advance()  # Move
past ':'

            # Reset break flag
            found_break = False

            # Process statements until
break or next case/default (similar
logic to case)
            while
self.current_token_index <
len(self.token_stream):
                current_token =
self.get_current_token()[0]

                # Check for continue
statement
                if current_token ==
'cntn':
                    raise
SemanticError("Continue statement
cannot be used directly in a switch
default block",

self.get_current_token()[2],
self.get_current_token()[3])

                # Found break
                if current_token ==
'brk':
                    found_break = True

self.analyze_break_statement()
                    break

                # Reached another case
or end of switch without finding
break
                if current_token in ['cs',
'dflt', '}']:
                    raise
SemanticError(f"Missing 'brk'
statement at end of default case",
line, column)

                # Now handle all
possible statements in the default
block
                # Process statements
(same as case block)
                if current_token == 'f':

self.analyze_if_statement()
                elif current_token ==
'whl':

self.analyze_while_loop()
                elif current_token ==
'fr':

self.analyze_for_loop()
                elif current_token ==
'd':

self.analyze_do_while_loop()
                elif current_token ==
'swtch':
                    # Allow nested
switch

self.analyze_switch_statement()
                elif current_token ==
'cnst':

self.analyze_constant_declaration()
                elif current_token ==
'dfstrct':

self.analyze_struct_instantiation()
                elif current_token in
self.data_types:
                    start_pos =
self.current_token_index
                    self.advance()
                    if
self.current_token_index >=
len(self.token_stream) or
self.token_stream[self.current_token
_index][0] != 'id':
                        raise
SemanticError(f"Expected an
identifier after '{current_token}'",

self.get_current_token()[2],
self.get_current_token()[3])

                    next_token =
self.peek_next_token()
                    if next_token and
next_token[0] == '[':

self.current_token_index = start_pos

self.analyze_array_declaration()
                    else:

self.current_token_index = start_pos

self.analyze_variable_declaration()
                elif current_token ==
'id':
                    id_token_value =
self.get_current_token()[1]
                    next_token =
self.peek_next_token()
                    if next_token and
next_token[0] == '.':

self.analyze_struct_member_access(
)
                    elif next_token and
next_token[0] == '(':

self.analyze_function_call()
```

```
                elif next_token and
next_token[0] == '[':

self.analyze_array_access()

                    # Skip to end of
statement
                    while
self.current_token_index <
len(self.token_stream) and
self.get_current_token()[0] != ';':
                        self.advance()
                    self.advance()  #
Move past semicolon
                elif next_token and
next_token[0] == '=':

self.analyze_assignment()
                else:

self.check_variable_usage(id_token
_value,

self.get_current_token()[2],
self.get_current_token()[3])
                    self.advance()
            elif current_token ==
'prnt':

self.analyze_print_statement()
            elif current_token ==
'rtrn':

self.analyze_return_in_conditional()
            else:
                self.advance()

        elif token_type == 'brk' and
not self.in_case_block:
            raise
SemanticError("Break statement in
switch must appear within a case or
default block",
                          line, column)
        else:
            # Unexpected token in
switch statement
            self.advance()  # Skip
unexpected tokens to avoid infinite
loops

    # Restore original flags and
scope
    self.in_switch = old_in_switch
    self.in_case_block =
old_in_case_block
    self.current_scope =
original_scope

def analyze_case_body(self):
    """Analyze statements inside a
case body until break statement"""
    # Keep track of whether we
found a nested switch or break
    found_break = False

    while self.current_token_index
< len(self.token_stream):
        token_type, token_value,
line, column =
self.get_current_token()

        # Check for break statement
        if token_type == 'brk':
            return True  # Return True
to indicate a break was found

        # Check for nested switch
statement (not allowed)
        if token_type == 'swtch':
            raise
SemanticError(f"Nested switch
statements are not allowed", line,
column)

        # Check for end of switch
(missing break)
        if token_type == '}':
            return False  # Return
False to indicate no break was found

        # Handle other case
statements (missing break)
        if token_type in ['cs', 'dflt']:
            return False  # Return
False to indicate no break was found

        # Handle if statements
        if token_type == 'f':

self.analyze_if_statement_in_switch
()
            continue

        # Handle other statements
        if token_type == 'cnst':

self.analyze_constant_declaration()
        elif token_type == 'dfstrct':

self.analyze_struct_instantiation()
        elif token_type in
self.data_types:
            start_pos =
self.current_token_index
            self.advance()
            if
self.current_token_index >=
len(self.token_stream) or
self.token_stream[self.current_token
_index][0] != 'id':
                raise
SemanticError(f"Expected an
identifier after '{token_type}'", line,
column)

            next_token =
self.peek_next_token()
            if next_token and
next_token[0] == '[':

self.current_token_index = start_pos

self.analyze_array_declaration()
            else:

self.current_token_index = start_pos

self.analyze_variable_declaration()
        elif token_type == 'id':
            next_token =
self.peek_next_token()
            if next_token and
next_token[0] == '.':

self.analyze_struct_member_access(
)
            elif next_token and
next_token[0] == '(':

self.analyze_function_call()
            elif next_token and
next_token[0] == '[':

self.analyze_array_access()

                # Skip to end of
statement
                while
self.current_token_index <
len(self.token_stream) and
self.get_current_token()[0] != ';':
                    self.advance()
                self.advance()  # Move
past semicolon
            elif next_token and
next_token[0] == '=':

self.analyze_assignment()
            else:

self.check_variable_usage(token_va
lue, line, column)
                self.advance()
        else:
            self.advance()

    return False  # No break found
if we reach end of tokens
```

```
    def
analyze_if_statement_in_switch(self
):
        """Analyze an if statement
inside a switch case (special
handling to prevent nested
switch)"""
        # Create a flag to track that
we're inside a switch case
        old_in_switch = getattr(self,
'in_switch_case', False)
        self.in_switch_case = True

        try:
            # Use regular if statement
analysis
            self.analyze_if_statement()
        finally:
            # Restore the flag
            self.in_switch_case =
old_in_switch

    def
analyze_block_statements(self):
        """Analyze statements inside a
block until closing brace"""
        # Process block statements
until we find closing brace
        while self.current_token_index
< len(self.token_stream):
            token_type, token_value,
line, column =
self.get_current_token()

            # Add debug output
            print(f"Processing token in
block: {token_type} '{token_value}'
at line {line}, column {column}")

            # Check for end of block
            if token_type == '}':
                self.advance()  # Move
past '}'
                break

            # Add special handling for
break and continue
            if token_type == 'brk':
                if self.in_loop:
                    # Break in a loop is
always fine

self.analyze_break_statement()
                    continue
                elif self.in_switch and
self.in_case_block:
                    # Break in a switch is
only fine if inside a case/default
block

self.analyze_break_statement()
                    continue
                elif self.in_switch:
                    # Break in a switch but
not in a case block
                    raise
SemanticError("Break statement in
switch must appear within a case or
default block",
                                  line, column)
                else:
                    # Not in a loop or
switch
                    raise
SemanticError("Break statement
can only be used inside a loop or
switch statement",
                                  line, column)

            if token_type == 'cntn':
                if self.in_switch and not
self.in_loop:
                    # Continue inside
switch but not inside a loop
                    raise
SemanticError("Continue statement
cannot be used in a switch
statement",
                                  line, column)
                elif not self.in_loop:
                    # Not in a loop
                    raise
SemanticError("Continue statement
can only be used inside a loop",
                                  line, column)
                else:
                    # Inside a loop,
continue is fine

self.analyze_continue_statement()
                    continue

            # Handle all statement types
            if token_type == 'f':

self.analyze_if_statement()
            elif token_type == 'whl':
                self.analyze_while_loop()
            elif token_type == 'd':

self.analyze_do_while_loop()
            elif token_type == 'fr':
                self.analyze_for_loop()
            elif token_type == 'swtch':

self.analyze_switch_statement()
            elif token_type == 'prnt':
                # Explicitly call
analyze_print_statement when 'prnt'
token is found
                print("Found print
statement - calling
analyze_print_statement()")  #
Debug

self.analyze_print_statement()
            elif token_type == 'rtrn':
                # Handle return inside
blocks

self.analyze_return_in_conditional()
            elif token_type == 'cnst':

self.analyze_constant_declaration()
            elif token_type == 'dfstrct':

self.analyze_struct_instantiation()
            elif token_type in
self.data_types:
                start_pos =
self.current_token_index
                self.advance()
                if
self.current_token_index >=
len(self.token_stream) or
self.token_stream[self.current_token
_index][0] != 'id':
                    raise
SemanticError(f"Expected an
identifier after '{token_type}'", line,
column)

                next_token =
self.peek_next_token()
                if next_token and
next_token[0] == '[':

self.current_token_index = start_pos

self.analyze_array_declaration()
                else:

self.current_token_index = start_pos

self.analyze_variable_declaration()
            elif token_type == 'id':
                next_token =
self.peek_next_token()

                # Check for
increment/decrement operations
                if next_token and
next_token[0] in ['++', '--']:

self.analyze_increment_operation()
                # Check for shortcut
assignments
                elif next_token and
next_token[0] in ['+=', '-=', '*=', '/=',
'%=']:
                    print(f"Found shortcut
assignment {token_value}
{next_token[0]}")
```

```
self.analyze_assignment()
            elif next_token and
next_token[0] == '.':

self.analyze_struct_member_access(
)
            elif next_token and
next_token[0] == '(':

self.analyze_function_call()
            elif next_token and
next_token[0] == '[':

self.analyze_array_access()

                # Skip to end of
statement
                while
self.current_token_index <
len(self.token_stream) and
self.get_current_token()[0] != ';':
                    self.advance()
                self.advance()  # Move
past semicolon
            elif next_token and
next_token[0] == '=':

self.analyze_assignment()
            else:

self.check_variable_usage(token_va
lue, line, column)
                self.advance()
        else:
            self.advance()  # Skip
other tokens

    def
analyze_return_in_conditional(self):
        """Analyze a return statement
inside a conditional block"""
        token_type, token_value, line,
column = self.get_current_token()
        if token_type != 'rtrn':
            raise
SemanticError(f"Expected 'rtrn'
keyword, got '{token_type}'", line,
column)

        self.advance()  # Move past
'rtrn'

        # Check if we're in the main
function
        is_main_function = False
        current_function_name = None
        temp_scope =
self.current_scope
        while temp_scope:
            if
temp_scope.scope_name.startswith(
"function "):
                current_function_name =
temp_scope.scope_name[len("functi
on "):]
                if current_function_name
== "mn":
                    is_main_function =
True
                break
            temp_scope =
temp_scope.parent

        # --- REMOVED CHECK
THAT DISALLOWED RETURN
IN MAIN ---
        # # No return statements
allowed in main function
        # if current_function == "mn":
        #     raise
SemanticError("Return statements
are not allowed in the main
function", line, column)
        # --- END REMOVED
CHECK ---

        # For other functions (or main),
we need to check against the
function's return type
        if current_function_name: #
Should always be true if we are in a
function scope
            func_symbol =
self.global_scope.lookup(current_fu
nction_name)
            if func_symbol and
func_symbol.type == 'function':
                return_type =
func_symbol.data_type

            # For void functions
(excluding main), there should be
no return value
            if return_type == 'vd' and
not is_main_function:
                token_type,
token_value, line, column =
self.get_current_token()
                if token_type != ';':
                    raise
SemanticError(f"Void function
cannot return a value", line, column)
                self.advance()  # Move
past ';'
            # For main or non-void
functions
            else:
                # --- ADDED: Handle
return in main ---
                if is_main_function:
                    # Allow 'rtrn 0;' or
'rtrn;'
                    next_token_type,
next_token_value, next_line,
next_col = self.get_current_token()
                    if next_token_type
== 'ntlit' and next_token_value ==
'0':
                        self.advance() #
Past 0
                        if
self.get_current_token()[0] == ';':
                            self.advance() #
Past ;
                            return # Valid
rtrn 0;
                        else:
                            raise
SemanticError("Expected ';' after
'rtrn 0' in main function",
self.get_current_token()[2],
self.get_current_token()[3])
                    elif next_token_type
== ';':
                        # Allow 'rtrn;'
                        self.advance() #
Past ;
                        return # Valid
rtrn;
                    else:
                        # Invalid return
value for main
                        raise
SemanticError(f"Main function
('mn') can only return '0' or have no
return value ('rtrn;'), got
'{next_token_value}'", next_line,
next_col)
                # --- END ADDED
check for main ---
                # For non-void,
non-main functions, analyze the
return expression
                else:
                    start_pos =
self.current_token_index

                    # Find the end of the
return expression (semicolon)
                    while
self.current_token_index <
len(self.token_stream) and
self.token_stream[self.current_token
_index][0] != ';':
                        self.advance()

                    if
self.current_token_index >=
len(self.token_stream) or
self.get_current_token()[0] != ';':
```

```
                raise SemanticError("Expected ';' after return value", line, column)

                end_pos = self.current_token_index
                self.current_token_index = start_pos

                # Analyze the expression
                expr_type = self.analyze_expression(end_pos)

                # Check if return type matches function return type
                if expr_type != return_type:
                    raise SemanticError(f"Return type mismatch: expected '{return_type}', got '{expr_type}'", line, column)

                # Move past the semicolon
                # self.advance() # Move to semicolon (analyze_expression leaves us here)
                if self.get_current_token()[0] == ';':
                    self.advance()  # Move past semicolon
                else: # Should not happen if semicolon check above worked
                    raise SemanticError("Internal Error: Expected semicolon after analyzed return expression", line, column)

    def analyze_for_loop(self):
        """Analyze a for loop (fr statement in Conso)"""
        token_type, token_value, line, column = self.get_current_token()
        if token_type != 'fr':
            raise SemanticError(f"Expected 'fr' keyword, got '{token_type}'", line, column)

        self.advance()  # Move past 'fr'

        # Process opening parenthesis
        token_type, token_value, line, column = self.get_current_token()
        if token_type != '(':
            raise SemanticError(f"Expected '(' after 'fr', got '{token_type}'", line, column)

        self.advance()  # Move past '('

        # Process initialization part
        # Note: You mentioned initialization is already validated in syntax
        # We'll just skip past it to the first semicolon
        init_start = self.current_token_index
        while self.current_token_index < len(self.token_stream) and self.get_current_token()[0] != ';':
            self.advance()

        if self.current_token_index >= len(self.token_stream) or self.get_current_token()[0] != ';':
            raise SemanticError(f"Expected ';' after for loop initialization", line, column)

        # Go back to analyze the initialization
        self.current_token_index = init_start

        # Process the initialization (typically an assignment)
        # In your language, this should only be an 'nt' variable initialized with an 'nt' value
        if self.get_current_token()[0] == 'id':
            var_name = self.get_current_token()[1]
            init_line, init_column = self.get_current_token()[2], self.get_current_token()[3]

            # Check if variable exists
            var_symbol = self.current_scope.lookup(var_name)
            if not var_symbol:
                raise SemanticError(f"Undefined variable '{var_name}' in for loop initialization", init_line, init_column)

            # Check if variable is of type 'nt'
            if var_symbol.data_type != 'nt':
                raise SemanticError(f"Variable in for loop initialization must be of type 'nt', got '{var_symbol.data_type}'", init_line, init_column)

            # Skip past variable name and = sign
            self.advance()  # Move past variable name

            # Expect = sign
            if self.get_current_token()[0] != '=':
                raise SemanticError(f"Expected '=' in for loop initialization, got '{self.get_current_token()[0]}'", init_line, init_column)

            self.advance()  # Move past =

            # Check the value being assigned
            init_value_type = self.get_current_token()[0]
            if init_value_type == 'ntlit':
                # Integer literal is OK
                self.advance()  # Move past literal
            elif init_value_type == 'id':
                # Check if the identifier is of type 'nt'
                init_id = self.get_current_token()[1]
                init_id_symbol = self.current_scope.lookup(init_id)
                if not init_id_symbol:
                    raise SemanticError(f"Undefined variable '{init_id}' in for loop initialization", init_line, init_column)

                if init_id_symbol.data_type != 'nt':
                    raise SemanticError(f"Variable in for loop initialization value must be of type 'nt', got '{init_id_symbol.data_type}'", init_line, init_column)

                self.advance()  # Move past identifier
            else:
                raise SemanticError(f"For loop initialization value must be an 'nt' literal or identifier, got '{init_value_type}'", init_line, init_column)
```

```
        # Now we should be at the first semicolon
        if self.get_current_token()[0] != ';':
            raise SemanticError(f"Expected ';' after for loop initialization, got '{self.get_current_token()[0]}'", self.get_current_token()[2], self.get_current_token()[3])

        self.advance()  # Move past the first semicolon

        # Process the condition
        condition_start = self.current_token_index
        condition_line, condition_column = self.get_current_token()[2], self.get_current_token()[3]

        # Find the end of the condition (should be the second semicolon)
        while self.current_token_index < len(self.token_stream) and self.get_current_token()[0] != ';':
            self.advance()

        if self.current_token_index >= len(self.token_stream) or self.get_current_token()[0] != ';':
            raise SemanticError(f"Expected ';' after for loop condition", condition_line, condition_column)

        condition_end = self.current_token_index
        self.current_token_index = condition_start

        # Analyze the condition expression - it must evaluate to a boolean
        # and must only involve 'nt' values in the comparison
        condition_expr_type = self.analyze_expression(condition_end)
        if condition_expr_type != 'bln':
            raise SemanticError(f"For loop condition must evaluate to a boolean, got '{condition_expr_type}'", condition_line, condition_column)

        # Now we should be at the second semicolon
        if self.get_current_token()[0] != ';':
            raise SemanticError(f"Expected ';' after for loop condition, got '{self.get_current_token()[0]}'", self.get_current_token()[2], self.get_current_token()[3])

        self.advance()  # Move past the second semicolon

        # Process the increment/decrement
        increment_start = self.current_token_index
        increment_line, increment_column = self.get_current_token()[2], self.get_current_token()[3]

        # Find the end of the increment (should be the closing parenthesis)
        paren_level = 1  # Start at 1 because we're already inside a parenthesis
        while self.current_token_index < len(self.token_stream):
            if self.get_current_token()[0] == '(':
                paren_level += 1
            elif self.get_current_token()[0] == ')':
                paren_level -= 1
                if paren_level == 0:
                    break

            self.advance()

        if self.current_token_index >= len(self.token_stream) or self.get_current_token()[0] != ')':
            raise SemanticError(f"Expected ')' after for loop increment", increment_line, increment_column)

        increment_end = self.current_token_index
        self.current_token_index = increment_start

        # Analyze the increment expression - it must involve an 'nt' identifier
        # Check for pre-increment/decrement: ++id or --id
        if self.get_current_token()[0] in ['++', '--']:
            incr_op = self.get_current_token()[0]
            self.advance()  # Move past operator

            if self.get_current_token()[0] != 'id':
                raise SemanticError(f"Expected an identifier after '{incr_op}' in for loop increment", self.get_current_token()[2], self.get_current_token()[3])

            incr_var_name = self.get_current_token()[1]
            incr_var_symbol = self.current_scope.lookup(incr_var_name)

            if not incr_var_symbol:
                raise SemanticError(f"Undefined variable '{incr_var_name}' in for loop increment", self.get_current_token()[2], self.get_current_token()[3])

            # Check if variable is of type 'nt'
            if incr_var_symbol.data_type != 'nt':
                raise SemanticError(f"Variable in for loop increment must be of type 'nt', got '{incr_var_symbol.data_type}'", self.get_current_token()[2], self.get_current_token()[3])

            self.advance()  # Move past identifier

        # Check for post-increment/decrement or other forms: id++ or id-- or other assignments
        elif self.get_current_token()[0] == 'id':
            incr_var_name = self.get_current_token()[1]
            incr_var_symbol = self.current_scope.lookup(incr_var_name)

            if not incr_var_symbol:
                raise SemanticError(f"Undefined variable
```

```
'{incr_var_name}' in for loop increment",
                    self.get_current_token()[2], self.get_current_token()[3])

                # Check if variable is of type 'nt'
                if incr_var_symbol.data_type != 'nt':
                    raise SemanticError(f"Variable in for loop increment must be of type 'nt', got '{incr_var_symbol.data_type}'",
                                        self.get_current_token()[2], self.get_current_token()[3])

                self.advance()  # Move past identifier

                # Check for operator: ++, --, +=, -=, *=, /=
                if self.get_current_token()[0] in ['++', '--']:
                    # Simple increment/decrement is fine
                    self.advance()  # Move past operator
                elif self.get_current_token()[0] in ['+=', '-=', '*=', '/=']:
                    operator = self.get_current_token()[0]
                    self.advance()  # Move past operator

                    # Check the value - should be 'nt' literal or identifier
                    if self.get_current_token()[0] == 'ntlit':
                        # Integer literal is OK
                        self.advance()  # Move past literal
                    elif self.get_current_token()[0] == 'id':
                        # Check if the identifier is of type 'nt'
                        right_id = self.get_current_token()[1]
                        right_id_symbol = self.current_scope.lookup(right_id)

                        if not right_id_symbol:
                            raise SemanticError(f"Undefined variable '{right_id}' in for loop increment",
                                                increment_line, increment_column)
                        if right_id_symbol.data_type != 'nt':
                            raise SemanticError(f"Variable in for loop increment must be of type 'nt', got '{right_id_symbol.data_type}'",
                                                increment_line, increment_column)

                        self.advance()  # Move past identifier
                    else:
                        raise SemanticError(f"For loop increment value must be an 'nt' literal or identifier, got '{self.get_current_token()[0]}'",
                                            increment_line, increment_column)
                else:
                    raise SemanticError(f"Expected increment/decrement operator in for loop increment, got '{self.get_current_token()[0]}'",
                                        increment_line, increment_column)

                # Skip to the end of the increment (should be the closing parenthesis)
                while self.current_token_index < increment_end:
                    self.advance()

                # Now we should be at the closing parenthesis
                if self.get_current_token()[0] != ')':
                    raise SemanticError(f"Expected ')' after for loop increment, got '{self.get_current_token()[0]}'",
                                        self.get_current_token()[2], self.get_current_token()[3])

                self.advance()  # Move past the closing parenthesis

                # Process opening brace for loop body
                token_type, token_value, line, column = self.get_current_token()
                if token_type != '{':
                    raise SemanticError(f"Expected '{{' to start for loop body, got '{token_type}'", line, column)

                self.advance()  # Move past '{'

                # ADD THE LOOP FLAG CODE HERE, just before creating the new scope:
                old_in_loop = self.in_loop
                self.in_loop = True

                # Create a new scope for the loop body
                loop_scope = SymbolTable(parent=self.current_scope, scope_name="for loop block")
                original_scope = self.current_scope
                self.current_scope = loop_scope

                # Process loop body statements
                self.analyze_block_statements()

                # Restore original flags and scope
                self.in_loop = old_in_loop
                self.current_scope = original_scope

                # Create a new scope for the loop body
                loop_scope = SymbolTable(parent=self.current_scope, scope_name="for loop block")
                original_scope = self.current_scope
                self.current_scope = loop_scope

                # Process loop body statements
                self.analyze_block_statements()

                # Restore original scope
                self.current_scope = original_scope

            def analyze_while_loop(self):
                """Analyze a while loop (whl statement in Conso)"""
                token_type, token_value, line, column = self.get_current_token()
                if token_type != 'whl':
                    raise SemanticError(f"Expected 'whl' keyword, got '{token_type}'", line, column)

                self.advance()  # Move past 'whl'

                # Process opening parenthesis for condition
```

```
        token_type, token_value, line,
column = self.get_current_token()
        if token_type != '(':
            raise
SemanticError(f"Expected '(' after
'whl', got '{token_type}'", line,
column)

        # Keep track of the outermost
parentheses level
        outer_paren_start =
self.current_token_index
        self.advance()  # Move past
opening parenthesis

        # Find the matching closing
parenthesis for the outermost level
        paren_level = 1

        while paren_level > 0 and
self.current_token_index <
len(self.token_stream):
            current_token =
self.get_current_token()[0]
            if current_token == '(':
                paren_level += 1
            elif current_token == ')':
                paren_level -= 1

            if paren_level > 0:
                self.advance()

        # Now we should be at the
closing parenthesis of the condition
        if paren_level > 0:
            raise
SemanticError("Unclosed
parenthesis in 'whl' condition", line,
column)

        # Now at the closing
parenthesis
        outer_paren_end =
self.current_token_index

        # Reset to just after the
opening parenthesis
        self.current_token_index =
outer_paren_start + 1

        # Analyze the condition using
your expression analyzer
        # The condition is everything
between the parentheses
        expr_type =
self.analyze_expression(outer_paren
_end)

        # --- MODIFIED CHECK:
Allow 'nt' or 'bln' ---
        if expr_type not in ['bln', 'nt']:
            raise
SemanticError(f"Condition in 'whl'
statement must be of type 'bln' or
'nt', got '{expr_type}'", line,
column)
        # --- END MODIFIED
CHECK ---

        # Move to the closing
parenthesis and advance past it
        self.current_token_index =
outer_paren_end
        self.advance()  # Move past the
closing parenthesis

        # Process opening brace for
while-body
        token_type, token_value, line,
column = self.get_current_token()
        if token_type != '{':
            raise
SemanticError(f"Expected '{{' to
start 'whl' body, got '{token_type}'",
line, column)

        self.advance()  # Move past '{'

        old_in_loop = self.in_loop
        self.in_loop = True

        # Existing code for creating a
new scope
        while_scope =
SymbolTable(parent=self.current_sc
ope, scope_name="while block")
        original_scope =
self.current_scope
        self.current_scope =
while_scope

        # Process while-body
statements

self.analyze_block_statements()

        # Just before you restore the
original scope, add this:
        self.in_loop = old_in_loop

        # Existing code to restore
scope
        self.current_scope =
original_scope

    def analyze_do_while_loop(self):
        """Analyze a do-while loop
(d-whl statement in Conso)"""
        token_type, token_value, line,
column = self.get_current_token()
        if token_type != 'd':
            raise
SemanticError(f"Expected 'd'
keyword, got '{token_type}'", line,
column)

        self.advance()  # Move past 'd'

        # Process opening brace for
do-body
        token_type, token_value, line,
column = self.get_current_token()
        if token_type != '{':
            raise
SemanticError(f"Expected '{{' to
start 'd' body, got '{token_type}'",
line, column)

        self.advance()  # Move past '{'

        old_in_loop = self.in_loop
        self.in_loop = True

        # Create a new scope for the
do-body
        do_scope =
SymbolTable(parent=self.current_sc
ope, scope_name="do-while block")
        original_scope =
self.current_scope
        self.current_scope = do_scope

        # Process do-body statements

self.analyze_block_statements()

        # Restore original scope
(parent scope, not the do-while
scope itself)
        self.current_scope =
original_scope

        # Process 'whl' keyword
        token_type, token_value, line,
column = self.get_current_token()
        if token_type != 'whl':
            # If
analyze_block_statements finished
correctly, current token should be
'whl'
            # This error might indicate
an unclosed block or other issue
inside the loop body
            # Find the line/col of the
closing brace '}' if possible
            loop_end_line,
loop_end_col = line, column #
Fallback to current token's pos
            temp_idx =
self.current_token_index -1
```

```
            if temp_idx >= 0 and
self.token_stream[temp_idx][0] ==
'}':
                loop_end_line,
loop_end_col =
self.token_stream[temp_idx][2],
self.token_stream[temp_idx][3]

            raise
SemanticError(f"Expected 'whl'
after do block body",
loop_end_line, loop_end_col)


        self.advance()  # Move past
'whl'

        # Process opening parenthesis
for condition
        token_type, token_value, line,
column = self.get_current_token()
        if token_type != '(':
            raise
SemanticError(f"Expected '(' after
'whl' in do-while, got
'{token_type}'", line, column)

        # Keep track of the outermost
parentheses level
        outer_paren_start =
self.current_token_index
        self.advance()  # Move past
opening parenthesis

        # Find the matching closing
parenthesis for the outermost level
        paren_level = 1

        while paren_level > 0 and
self.current_token_index <
len(self.token_stream):
            current_token =
self.get_current_token()[0]
            if current_token == '(':
                paren_level += 1
            elif current_token == ')':
                paren_level -= 1

            if paren_level > 0:
                self.advance()

        # Now we should be at the
closing parenthesis of the condition
        if paren_level > 0:
            raise
SemanticError("Unclosed
parenthesis in 'd-whl' condition",
line, column)

        # Now at the closing
parenthesis
        outer_paren_end =
self.current_token_index

        # Reset to just after the
opening parenthesis
        self.current_token_index =
outer_paren_start + 1

        # Analyze the condition using
your expression analyzer
        # The condition is everything
between the parentheses
        expr_type =
self.analyze_expression(outer_paren
_end)

        # --- MODIFIED CHECK:
Allow 'nt' or 'bln' ---
        if expr_type not in ['bln', 'nt']:
            raise
SemanticError(f"Condition in
'd-whl' statement must be of type
'bln' or 'nt', got '{expr_type}'", line,
column)
        # --- END MODIFIED
CHECK ---

        # Move to the closing
parenthesis and advance past it
        self.current_token_index =
outer_paren_end
        self.advance()  # Move past the
closing parenthesis

        # FOR DO-WHILE: Look for
semicolon, not opening brace
        token_type, token_value, line,
column = self.get_current_token()
        if token_type != ';':
            raise
SemanticError(f"Expected ';' after
'd-whl' condition, got
'{token_type}'", line, column)

        self.advance()  # Move past ';'

        # Restore the loop flag
        self.in_loop = old_in_loop

    def
analyze_increment_operation(self):
        """Analyze increment or
decrement operation (++, --)"""
        # Get the variable name
        token_type, var_name, line,
column = self.get_current_token()

        # Look up the variable
        symbol =
self.current_scope.lookup(var_name
)
        if not symbol:
            raise
SemanticError(f"Undefined variable
'{var_name}'", line, column)

        # Check if variable is of type
'nt'
        if symbol.data_type != 'nt':
            raise
SemanticError(f"Increment/decreme
nt operators can only be applied to
'nt' variables, not
'{symbol.data_type}'",
                          line, column)

        self.advance()  # Move past
identifier

        # Check if it's an increment or
decrement operator
        token_type, token_value, line,
column = self.get_current_token()
        if token_type not in ['++', '--']:
            raise
SemanticError(f"Expected
increment/decrement operator, got
'{token_type}'", line, column)

        self.advance()  # Move past ++
or --

        # Check for semicolon
        token_type, token_value, line,
column = self.get_current_token()
        if token_type != ';':
            raise
SemanticError(f"Expected ';' after
increment/decrement operation",
line, column)

        self.advance()  # Move past
semicolon

    def
analyze_print_statement(self):
        """
        Analyze a print statement with
proper type checking.
        This function enforces the
same strict type constraints
        as regular expressions in the
Conso language.
        """
        # Current token should be 'prnt'
        token_type, token_value, line,
column = self.get_current_token()
        if token_type != 'prnt':
            raise
SemanticError(f"Expected 'prnt'
keyword, got '{token_type}'", line,
column)
```

```
        print(f"Analyzing print statement at line {line}, column {column}")  # Debug output

        self.advance()  # Move past 'prnt'

        # Check for opening parenthesis
        token_type, token_value, line, column = self.get_current_token()
        if token_type != '(':
            raise SemanticError(f"Expected '(' after 'prnt', got '{token_type}'", line, column)

        self.advance()  # Move past '('

        # Handle empty print statement: prnt();
        if self.get_current_token()[0] == ')':
            self.advance()  # Skip ')'

            # Check for semicolon
            token_type, token_value, line, column = self.get_current_token()
            if token_type != ';':
                raise SemanticError(f"Expected ';' after print statement", line, column)

            self.advance()  # Skip ';'
            return

        # Process expressions until closing parenthesis
        while True:
            # Mark the start of the current print argument
            expr_start = self.current_token_index

            # Find the end of this expression (comma or closing parenthesis)
            paren_level = 0
            while self.current_token_index < len(self.token_stream):
                token_type = self.get_current_token()[0]

                if token_type == '(':
                    paren_level += 1
                elif token_type == ')':
                    if paren_level == 0:
                        # This is the closing parenthesis of the print statement
                        break
                    paren_level -= 1
                elif token_type == ',' and paren_level == 0:
                    # This is a comma separating expressions
                    break

                self.advance()

            # Save the end position
            expr_end = self.current_token_index

            # Reset to start of expression for analysis
            self.current_token_index = expr_start

            # Get the tokens being analyzed for debug
            expr_tokens = self.token_stream[expr_start:expr_end]
            expr_str = " ".join([f"{t[0]}('{t[1]}')" for t in expr_tokens])
            print(f"Print expression tokens: {expr_str}")  # Debug output

            # Analyze the expression using our standard expression analyzer
            try:
                # The standard analyze_expression will properly enforce all type constraints
                # including arithmetic, relational, logical, and string operations
                expr_type = self.analyze_expression(expr_end)
                print(f"Print expression type result: {expr_type}")  # Debug output
            except SemanticError as e:
                print(f"Semantic error in print expression: {e}")  # Debug output
                # Re-raise the error
                raise e

            # Move to the end of this expression
            self.current_token_index = expr_end

            # Check if we're at a comma or closing parenthesis
            if self.get_current_token()[0] == ',':
                self.advance()  # Move past comma to the next expression
            else:
                # We should be at the closing parenthesis
                break

        # We should now be at the closing parenthesis
        token_type, token_value, line, column = self.get_current_token()
        if token_type != ')':
            raise SemanticError(f"Expected ')' at end of print statement", line, column)

        self.advance()  # Skip ')'

        # Check for semicolon
        token_type, token_value, line, column = self.get_current_token()
        if token_type != ';':
            raise SemanticError(f"Expected ';' after print statement", line, column)

        self.advance()  # Skip ';'
        print("Print statement analysis completed successfully")  # Debug output

    def analyze_break_statement(self):
        """Analyze a break statement"""
        token_type, token_value, line, column = self.get_current_token()
        if token_type != 'brk':
            raise SemanticError(f"Expected 'brk' keyword, got '{token_type}'", line, column)

        self.advance()  # Move past 'brk'

        # Check for semicolon
        token_type, token_value, line, column = self.get_current_token()
        if token_type != ';':
            raise SemanticError(f"Expected ';' after 'brk', got '{token_type}'", line, column)

        self.advance()  # Move past ';'
```

```
    def
analyze_continue_statement(self):
        """Analyze a continue
statement"""
        token_type, token_value, line,
column = self.get_current_token()
        if token_type != 'cntn':
            raise
SemanticError(f"Expected 'cntn'
keyword, got '{token_type}'", line,
column)

        self.advance()  # Move past
'cntn'

        # Check for semicolon
        token_type, token_value, line,
column = self.get_current_token()
        if token_type != ';':
            raise
SemanticError(f"Expected ';' after
'cntn', got '{token_type}'", line,
column)
        self.advance()  # Move past ';'

    def advance(self):
        """Move to the next token"""
        self.current_token_index += 1

    def peek_next_token(self):
        """Peek at the next token
without advancing"""
        if self.current_token_index + 1
< len(self.token_stream):
            return
self.token_stream[self.current_token
_index + 1]
        return None, None, None,
None

    def get_current_token(self):
        """Get the current token tuple
(type, value, line, column)"""
        if self.current_token_index <
len(self.token_stream):
            return
self.token_stream[self.current_token
_index]
        return None, None, None,
None

    def
debug_print_scope_hierarchy(self,
scope=None, indent=0):
        if scope is None:
            scope = self.current_scope
        print(" " * indent + f"Scope:
{scope.scope_name}")
        print(" " * indent + "Symbols:
" + ", ".join(scope.symbols.keys()))
        if scope.parent:

self.debug_print_scope_hierarchy(s
cope.parent, indent + 2)
```

Code Generation

```
"""
Conso to C Transpiler (V7 -
Token-Based Sequential - Input
Brace Fix)
This module converts Conso code to
C code using a token stream
provided
by earlier compiler phases (Lexer,
Parser, Semantic Analyzer).
Processes top-level blocks
sequentially based on tokens.
Includes fixes for global
declarations, array initializers,
default values,
and removes extra braces around
input statement code.
Does NOT expect an EOF token in
the input list.
"""
import sys
import re

# --- Custom Exception Class ---
class TranspilerError(Exception):
    """Custom exception for errors
during the transpilation process."""
    def __init__(self, message,
line_num=None):
        self.message = message
        self.line_num = line_num
        super().__init__(self.message)

    def __str__(self):
        if self.line_num is not None:
            return f"Transpiler Error at
line {self.line_num}:
{self.message}"
        else:
            return f"Transpiler Error:
{self.message}"

# --- Transpiler Class ---
class ConsoTranspilerTokenBased:
    # --- MODIFIED __init__ ---
    # Add user_inputs to __init__
    def __init__(self, token_list,
symbol_table=None,
function_scopes=None): # Added
function_scopes
        """
        Initializes the transpiler.

        Args:
            token_list: List of tokens
from the lexer (EOF token should
be removed).
            symbol_table: The global
SymbolTable instance (used as
default).
            function_scopes: A
dictionary mapping function names
to their SymbolTable instances.
        """
        self.tokens = token_list
        self.symbol_table =
symbol_table # This will be the
default (global) table
        self.function_scopes =
function_scopes if function_scopes
is not None else {} # Store function
scopes
        self.current_pos = 0
        self.output_parts = []
        self.current_indent_level = 0

self.input_buffer_declared_in_scope
= set()

        # Mappings (Keep existing
mappings)
        self.type_mapping = {
            "nt": "int", "dbl": "double",
"strng": "char*",
            "bln": "int", "chr": "char",
"vd": "void",
```

```
        "dfstrct": "struct"
    }
    self.keyword_mapping = {
        "f": "if", "ls": "else", "lsf":
"else if", "whl": "while",
        "fr": "for", "d": "do",
"swtch": "switch", "cs": "case",
        "dflt": "default", "brk":
"break", "cntn": "continue"
    }
    self.bool_mapping = {"tr": "1",
"fls": "0"}
    self.default_values = {
        "nt": "0",
        "dbl": "0.00",
        "strng": "NULL", # Be
cautious with NULL for char*, ""
might be safer if not checking
        "bln": "0",
        "chr": "'\\0'"
    }

  # --- Token Helpers ---
  def _get_token_info(self, token):
    """Safely extracts type, value,
line from token tuple or object."""
    token_type, token_value, line =
None, None, '?'
    if isinstance(token, tuple):
        token_type = token[0] if
len(token) > 0 else None
        token_value = token[1] if
len(token) > 1 else None
        line = token[2] if len(token)
> 2 else '?'
    elif hasattr(token, 'type'):
        token_type = token.type
        token_value = getattr(token,
'value', None)
        line = getattr(token, 'line',
'?')
    return token_type,
token_value, line

  def _peek(self, offset=0):
    """Look at the token type at
current position + offset."""
    peek_pos = self.current_pos +
offset
    if 0 <= peek_pos <
len(self.tokens):
        token =
self.tokens[peek_pos]
        token_type, _, _ =
self._get_token_info(token)
        return token_type
    return None

  def _consume(self,
expected_type=None,
expected_value=None):
    """Consume current token,
check expectations, return (type,
value, full_token)."""
    if self.current_pos >=
len(self.tokens):
        raise
TranspilerError("Unexpected end of
token stream")
    token =
self.tokens[self.current_pos]
    token_type, token_value, line =
self._get_token_info(token)
    if expected_type and
token_type != expected_type:
        raise
TranspilerError(f"Expected token
type '{expected_type}' but got
'{token_type}'", line)
    if expected_value and
token_value != expected_value:
        raise
TranspilerError(f"Expected token
value '{expected_value}' but got
'{token_value}'", line)
    self.current_pos += 1
    return token_type,
token_value, token

  def _skip_token(self, count=1):
    """Advance the current
position."""
    self.current_pos =
min(self.current_pos + count,
len(self.tokens))

  # --- Core Transpilation Method
---
  def transpile(self):
    """Transpile the token stream
sequentially, handling globals."""
    self.output_parts = [
        self._generate_headers(),

self._generate_helper_functions()
    ]
    struct_defs_c = [];
global_vars_c = []; func_defs_c =
[]; main_func_c = None

    while self.current_pos <
len(self.tokens):
        token_type = self._peek()
        if token_type is None: break

        start_pos_before_statement
= self.current_pos
        processed_something =
False

        try:
            if token_type == 'strct':
                struct_def =
self._process_struct_definition_fro
m_tokens()
                if struct_def:
struct_defs_c.append(struct_def)
                processed_something =
True
            elif token_type == 'fnctn':
                func_def =
self._process_function_definition_fr
om_tokens()
                if func_def:
func_defs_c.append(func_def)
                processed_something =
True
            elif token_type == 'mn':
                main_def =
self._process_main_definition_from
_tokens()
                if main_def:
main_func_c = main_def
                processed_something =
True
            elif token_type in
self.type_mapping and token_type
!= 'dfstrct':
                global_decl_c =
self._process_statement_from_toke
ns(is_global=True)
                if global_decl_c:
global_vars_c.append(global_decl_
c)
                processed_something =
True
            elif token_type ==
'dfstrct':
                global_inst_c =
self._process_statement_from_toke
ns(is_global=True)
                if global_inst_c:
global_vars_c.append(global_inst_c
)
                processed_something =
True
            else:
                line = '?';
                try: token =
self.tokens[self.current_pos]; line =
self._get_token_info(token)[2]
                except IndexError: pass
                print(f"Warning:
Ignoring unexpected top-level token
'{token_type}' near line {line}")
                self._skip_token();
processed_something = True

        except TranspilerError as e:
            print(f"Error during
top-level processing: {e}",
file=sys.stderr)
```

```
            if self.current_pos ==
start_pos_before_statement:
self._skip_token()
            processed_something =
True
        except Exception as e:
            print(f"Unexpected error
during top-level processing: {e}",
file=sys.stderr)
            if self.current_pos ==
start_pos_before_statement:
self._skip_token()
            processed_something =
True

        if not processed_something
and self.current_pos ==
start_pos_before_statement:
            print(f"Failsafe:
Advancing past unhandled token
{self._peek()}")
            self._skip_token()

    # --- Assemble the final C code
in Order ---
    if struct_defs_c:
self.output_parts.extend(["// Struct
Definitions"] + struct_defs_c + [""])
    if global_vars_c:
self.output_parts.extend(["// Global
Declarations"] + global_vars_c +
[""])
    if func_defs_c:
self.output_parts.extend(["//
Function Definitions"] +
func_defs_c + [""])
    if main_func_c:
self.output_parts.extend(["// Main
Function", main_func_c, ""])
    else:
self.output_parts.extend(["//
ERROR: 'mn' function block not
found or failed to process.", ""])

    return
"\n".join(self.output_parts)


  # --- Token-Based Definition
Processors ---
  def
_process_struct_definition_from_to
kens(self):
    """Processes struct definition
from tokens."""
    start_pos = self.current_pos;
struct_name = "<unknown>"
    try:
        self._consume('strct'); _,
struct_name, _ =
self._consume('id');
self._consume('{')
        definition_lines = [f"typedef
struct {struct_name} {{"]; indent =
"    "
        while self._peek() != '}':
            if self._peek() is None:
raise TranspilerError(f"Unexpected
end of stream inside struct
{struct_name}")
            member_line =
self._process_statement_from_toke
ns(is_struct_member=True)
            if member_line:
definition_lines.append(indent +
member_line)
        self._consume('}')
        if self._peek() == ';':
self._consume(';')
        definition_lines.append(f"}}
{struct_name};")
        return
"\n".join(definition_lines)
    except TranspilerError as e:
print(f"Error processing struct
'{struct_name}': {e}");
self.current_pos = start_pos;
    try: self._consume('strct')
    except: pass; return None

  def
_process_function_definition_from_
tokens(self):
    """Processes function
definition from tokens, correctly
handling nested braces."""
    start_pos = self.current_pos
    func_name = "<unknown>"
    # --- Store the current (likely
global) table ---
    original_symbol_table =
self.symbol_table
    func_body_tokens = []

    try:
        # Consume fnctn, return
type, func_name, params
        self._consume('fnctn')
        type_token_type,
type_token_value, _ =
self._consume()
        if type_token_type == 'id':
c_return_type = type_token_value
        elif type_token_type in
self.type_mapping: c_return_type =
self.type_mapping[type_token_type
]
        else: raise
TranspilerError(f"Invalid function
return type token:
{type_token_type}")
        _, func_name, _ =
self._consume('id');
self._consume('(')
        params_c =
self._process_parameters_from_tok
ens(); self._consume(')')
        self._consume('{'); #
Consume function's opening brace

        # Find the end of the
function block to get its tokens (for
buffer check)
        # This part is okay, used
only for the 'has_npt' check
        temp_pos = self.current_pos
        find_brace_level = 1
        while find_brace_level > 0:
            token_type =
self._peek(temp_pos -
self.current_pos)
            if token_type is None:
raise
TranspilerError(f"Unterminated
function '{func_name}'")
            if token_type == '{':
find_brace_level += 1
            elif token_type == '}':
find_brace_level -= 1
            if find_brace_level == 0:
                func_body_tokens =
self.tokens[self.current_pos :
temp_pos]
                break
            temp_pos += 1
            if temp_pos >=
len(self.tokens): raise
TranspilerError(f"Cannot find
closing brace for '{func_name}'")

        definition_lines =
[f"{c_return_type}
{func_name}({params_c}) {{"]
        self.current_indent_level = 1

        # Declare input buffer if
needed (Keep this logic)
        has_npt =
any(self._get_token_info(t)[0] ==
'npt' for t in func_body_tokens)
        if has_npt:
            buffer_size = 1024

definition_lines.append(self._indent
(1) + f"char
conso_input_buffer[{buffer_size}];
// Input buffer for this scope")

self.input_buffer_declared_in_scope
.add(func_name)
```

```
        # Temporarily switch
symbol table context (Keep this
logic)
        if self.function_scopes and
func_name in self.function_scopes:
            print(f"[Transpiler]
Switching symbol table to scope:
{func_name}") # DEBUG
            self.symbol_table =
self.function_scopes[func_name]
        else:
            print(f"[Transpiler
Warning] Function scope
'{func_name}' not found. Using
previous table.")
            self.symbol_table =
original_symbol_table # Fallback

        # --- MODIFIED LOOP:
Track brace level ---
        body_lines = []
        body_brace_level = 1 # Start
at 1 because we consumed the
function's {
        while body_brace_level > 0:
            # Check for premature
end of stream
            if self._peek() is None:
                raise
TranspilerError(f"Unexpected end
of stream inside function
'{func_name}' body")

            # Peek at the next token to
see if it's the closing brace we
expect
            next_token_type =
self._peek()

            # Process the statement
            statement_start_pos =
self.current_pos
            statement_c =
self._process_statement_from_toke
ns(current_scope_name=func_name
)

            if statement_c is not
None:
                # Adjust indent
*before* adding the closing brace
                indent_level =
self.current_indent_level
                if statement_c == "}":
indent_level = max(0, indent_level -
1)


body_lines.append(self._indent(inde
nt_level) + statement_c)
                # Adjust indent *after*
adding the opening brace
                if
statement_c.endswith("{"):
self.current_indent_level += 1
                # Adjust indent *after*
adding the closing brace
                if statement_c == "}":
self.current_indent_level = max(0,
self.current_indent_level - 1)

                # --- Update brace level
based on the processed statement ---
                if
statement_c.endswith("{"):
                    body_brace_level +=
1
                elif statement_c == "}":
                    body_brace_level -=
1
                # --- End brace level
update ---

            elif self.current_pos ==
statement_start_pos:
                # Failsafe: If no
statement was processed and
position didn't change, skip token
                skipped_token =
self._consume()[0]
                print(f"Warning:
Skipping unhandled token
'{skipped_token}' inside function
'{func_name}'")

            # Check brace level after
processing
            if body_brace_level == 0:
                # We have just
processed the function's closing
brace
                break
            elif body_brace_level < 0:
                # Should not happen
with balanced braces
                raise
TranspilerError(f"Mismatched
braces detected inside function
'{func_name}'")

        # --- REMOVED
self._consume('}') ---
        # The loop now correctly
consumes the function's closing
brace.

        self.current_indent_level = 0
# Reset indent level


definition_lines.extend(body_lines)
        # --- REMOVED
definition_lines.append("}") ---
        # The closing brace is now
part of the body_lines when
processed by the loop.

        return
"\n".join(definition_lines)

    except TranspilerError as e:
        print(f"Error processing
function '{func_name}': {e}",
file=sys.stderr)
        self.current_pos = start_pos
# Attempt reset
        return f"// ERROR
processing function '{func_name}':
{e}"
    finally:
        # --- Restore original symbol
table ---
        print(f"[Transpiler]
Restoring symbol table after
processing function: {func_name}")
# DEBUG
        self.symbol_table =
original_symbol_table
        # --- End restore ---
        # Cleanup buffer tracking
        if func_name !=
"<unknown>" and func_name in
self.input_buffer_declared_in_scope
:

self.input_buffer_declared_in_scope
.remove(func_name)

  def
_process_main_definition_from_tok
ens(self):
    """Processes main function
definition from tokens, switching
symbol table context."""
    start_pos = self.current_pos
    original_symbol_table =
self.symbol_table # Store the
current (global) table
    main_body_tokens = []
    try:
        self._consume('mn');
self._consume('(');
self._consume(')');
self._consume('{')

        # Find the end of the main
block to get its tokens (for buffer
check)
        temp_pos = self.current_pos
        brace_level = 1
        while True:
```

```
            token_type =
self._peek(temp_pos -
self.current_pos)
            if token_type is None:
raise TranspilerError("Unterminated
'mn' block")
            if token_type == '{':
brace_level += 1
            elif token_type == '}':
brace_level -= 1
            # Check for 'end' at the
correct brace level
            elif token_type == 'end'
and brace_level <= 1:
                main_body_tokens =
self.tokens[self.current_pos :
temp_pos]
                break
            temp_pos += 1
            if temp_pos >=
len(self.tokens): raise
TranspilerError("Cannot find 'end;'
for 'mn' block")

        definition_lines = ["int
main(int argc, char *argv[]) {"]
        self.current_indent_level = 1

        # Declare input buffer if
needed
        has_npt =
any(self._get_token_info(t)[0] ==
'npt' for t in main_body_tokens)
        if has_npt:
            buffer_size = 1024

definition_lines.append(self._indent
(1) + f"char
conso_input_buffer[{buffer_size}];
// Input buffer for this scope")

self.input_buffer_declared_in_scope
.add("main")

        # --- NEW: Switch to the
'main' specific symbol table (if
exists) ---
        # Assuming semantic
analyzer stores 'mn' scope in
function_scopes['mn']
        main_scope_name = "mn" #
Or whatever key your semantic
analyzer uses for main
        if self.function_scopes and
main_scope_name in
self.function_scopes:
            print(f"[Transpiler]
Switching symbol table to scope:
{main_scope_name}") # DEBUG
            self.symbol_table =
self.function_scopes[main_scope_n
ame]
        else:
            # If no specific 'mn' scope
found, maybe vars are global? Or
error?
            # Let's assume for now it
should exist if declared inside mn.
            # If it might fall back to
global, keep original_symbol_table.
            print(f"[Transpiler
Warning] Scope
'{main_scope_name}' not found in
function_scopes. Using previous
table (likely global).")
            self.symbol_table =
original_symbol_table # Fallback
        # --- END NEW ---

        # Process main body
statements (will use the switched
self.symbol_table)
        body_lines = []
        while self._peek() != 'end':
            if self._peek() is None:
raise TranspilerError("Unexpected
end of stream inside main
definition, missing 'end;'?")
            statement_start_pos =
self.current_pos
            # Pass scope name "main"
for consistency if needed by other
funcs
            statement_c =
self._process_statement_from_toke
ns(current_scope_name="main")
            if statement_c is not
None:
                indent_level =
self.current_indent_level
                if statement_c == "}":
indent_level = max(0, indent_level -
1)

body_lines.append(self._indent(inde
nt_level) + statement_c)
                if
statement_c.endswith("{"):
self.current_indent_level += 1
                if statement_c == "}":
self.current_indent_level = max(0,
self.current_indent_level - 1)
            elif self.current_pos ==
statement_start_pos:
                skipped_token =
self._consume()[0]
                print(f"Warning:
Skipping unhandled token
'{skipped_token}' inside 'mn'
block")
        self._consume('end');
self._consume(';')
        self.current_indent_level = 0


definition_lines.extend(body_lines)

definition_lines.append(self._indent
(1) + "return 0; // Corresponds to
Conso 'end;'")
        definition_lines.append("}")
        return
"\n".join(definition_lines)

    except TranspilerError as e:
        print(f"Error processing
main function: {e}", file=sys.stderr)
        self.current_pos = start_pos
        return f"// ERROR
processing 'mn' function: {e}"
    finally:
        # --- Restore original symbol
table ---
        print(f"[Transpiler]
Restoring symbol table after
processing 'mn'.") # DEBUG
        self.symbol_table =
original_symbol_table
        # --- End restore ---
        # Cleanup buffer tracking
        if "main" in
self.input_buffer_declared_in_scope
:

self.input_buffer_declared_in_scope
.remove("main")

  def
_process_parameters_from_tokens(s
elf):
    """Processes parameters from
token stream until ')' is found."""
    params = []
    if self._peek() == ')': return
"void"
    while self._peek() != ')':
        if self._peek() is None: raise
TranspilerError("Unexpected end of
stream in parameter list")
        type_token =
self._consume()
        param_type_conso =
type_token[0]
        if param_type_conso not in
self.type_mapping:
            if param_type_conso ==
'id': c_type = type_token[1]
            else: raise
TranspilerError(f"Expected type
```

```
token in parameter list, got
'{param_type_conso}'")
            else: c_type =
self.type_mapping.get(param_type_
conso, param_type_conso)
            _, param_name, _ =
self._consume('id')
            params.append(f"{c_type}
{param_name}")
            if self._peek() == ',':
self._consume(',')
            elif self._peek() == ')': break
            else: raise
TranspilerError(f"Unexpected token
'{self._peek()}' in parameter list")
        return ", ".join(params) if
params else "void"

    # --- Statement Processing
(Token-Based Dispatcher) ---
    def
_process_statement_from_tokens(se
lf, is_struct_member=False,
is_global=False,
current_scope_name=None):
        """
        Processes a single statement
from the current token position.
        This function dispatches to
specific processing methods based
on the
        initial token(s) of the
statement.
        Modified to correctly identify
input statements with complex
targets
        and pass the
current_scope_name to the input
processor.
        """
        token_type = self._peek()
        if token_type is None:
            # Return empty string if
there are no more tokens
            return ""

        start_pos = self.current_pos #
Store the starting position for error
recovery
        line_num = '?'
        try:
            # Attempt to get the line
number from the first token
            line_num =
self._get_token_info(self.tokens[star
t_pos])[2]
        except IndexError:
            # Handle case where there's
no token at start_pos (should be
caught by token_type is None
check, but defensive)
            pass
        except Exception:
            # Ignore other errors getting
line number for error reporting itself
            pass

        try:
            # --- Handle struct members
first ---
            if is_struct_member:
                # Struct members can
only be declarations (type followed
by id, possibly array)
                if token_type in
self.type_mapping and token_type
!= 'dfstrct':
                    # Process as a
declaration. Struct members are not
const and don't get default
assignment here.
                    return
self._process_declaration_from_tok
ens(assign_default=False,
is_const=False)
                else:
                    # If an unexpected
token is found within a struct
definition, print a warning and skip
it.
                    print(f"Warning:
Skipping non-declaration token
'{token_type}' inside struct near line
{line_num}");
                    self._consume(); #
Consume the unexpected token
                    return "" # Return
empty string, effectively skipping
this line in the output

            # --- Processing for global,
function, or main scope ---

            # Handle block delimiters ({
and }) immediately
            if token_type == '{':
                self._consume(); #
Consume the opening brace
                return "{" # Return the C
equivalent
            if token_type == '}':
                self._consume(); #
Consume the closing brace
                return "}" # Return the C
equivalent

            # --- Check for Constant
Declaration (cnst type id ...) ---
            is_const_decl = False
            if token_type == 'cnst':
                # Peek ahead to see if a
valid type follows 'cnst'
                next_type_peek =
self._peek(1)
                # Check if the next token
is a recognized data type (excluding
struct definition itself)
                if next_type_peek in
self.type_mapping and
next_type_peek != 'dfstrct':
                    # It's a constant
declaration, consume the 'cnst' token
                    self._consume('cnst')
                    is_const_decl = True
                    # Update token_type to
the actual data type token that
follows 'cnst'
                    token_type =
self._peek()
                else:
                    # 'cnst' not followed by
a valid type - this might be an error
or part of another expression.
                    # For now, let it fall
through to be handled as an 'other'
statement or error later.
                    pass # Semantic
analysis should ideally catch this
error earlier.

            # --- Check for Regular or
Constant Declaration (type id ...) ---
            # Now token_type is either
the original peeked type or the type
after 'cnst'
            if token_type in
self.type_mapping and token_type
!= 'dfstrct':
                # Process as a variable
declaration (regular or constant)
                # Pass the is_const flag
determined above.
                # assign_default=True
means non-array, non-const
variables without initializer get
default values.
                return
self._process_declaration_from_tok
ens(assign_default=True,
is_const=is_const_decl)

            # --- Check for Struct
Instance Declaration (dfstrct
struct_name id ...) ---
            elif token_type == 'dfstrct':
                # Process as a struct
instance declaration.
                # Struct instances cannot
be declared 'const' using the 'cnst'
keyword in this grammar.
                return
self._process_dfstrct_from_tokens()
```

```
        # --- Statements only valid inside functions/main ---
        if is_global:
            # If we reach here in global scope with an unhandled token type, it's an error
            print(f"Error: Statement type '{token_type}' not allowed at global scope near line {line_num}", file=sys.stderr)
            # Attempt to recover by skipping tokens until a potential statement boundary (semicolon, brace)
            while self._peek() not in [';', '}', None] and self.current_pos < len(self.tokens):
                self._skip_token()
            # Consume the boundary token if it's a semicolon
            if self._peek() == ';': self._skip_token()
            # Return a C comment indicating the error
            return f"// ERROR: Statement type '{token_type}' not allowed at global scope"

        # --- Function/Main Scope Statements ---
        # These should only be processed if we are NOT in the global scope

        # Check for specific keywords first
        if token_type == 'prnt': return self._process_print_from_tokens()
        elif token_type == 'rtrn': return self._process_return_from_tokens()
        elif token_type == 'f': return self._process_if_from_tokens()
        elif token_type == 'lsf': return self._process_else_if_from_tokens()
        elif token_type == 'ls': return self._process_else_from_tokens()
        elif token_type == 'whl': return self._process_while_from_tokens()
        elif token_type == 'fr': return self._process_for_from_tokens()
        elif token_type == 'd': return self._process_do_from_tokens()
        elif token_type == 'swtch': return self._process_switch_from_tokens()
        elif token_type == 'cs': return self._process_case_from_tokens()
        elif token_type == 'dflt': return self._process_default_from_tokens()
        elif token_type == 'brk':
            self._consume('brk'); self._consume(';'); # Consume 'brk' and ';'
            return "break;" # Return the C equivalent
        elif token_type == 'cntn':
            self._consume('cntn'); self._consume(';'); # Consume 'cntn' and ';'
            return "continue;" # Return the C equivalent
        # Add other specific statements here

        # --- NEW/MODIFIED: Handle Input Statement (target = npt(...)) ---
        # This requires looking ahead to find the '=' and 'npt' tokens.
        # We need to consume tokens until we find '=' at the top level (outside any parens/brackets).
        temp_pos = self.current_pos # Use a temporary position to peek ahead without consuming
        temp_paren_level = 0
        temp_bracket_level = 0
        found_assignment = False

        # Scan ahead to find the '=' token at the top level
        while temp_pos < len(self.tokens):
            peek_token_type = self._peek(temp_pos - self.current_pos)
            if peek_token_type is None: break # Reached end of stream

            if peek_token_type == '(': temp_paren_level += 1
            elif peek_token_type == ')': temp_paren_level -= 1
            elif peek_token_type == '[': temp_bracket_level += 1
            elif peek_token_type == ']': temp_bracket_level -= 1
            elif peek_token_type == '=' and temp_paren_level == 0 and temp_bracket_level == 0:
                found_assignment = True
                break # Found the top-level assignment operator

            temp_pos += 1 # Move to the next token

        # If an assignment was found, check if the RHS starts with 'npt'
        if found_assignment:
            # Peek at the token immediately after the '='
            token_after_assignment = self._peek(temp_pos - self.current_pos + 1)
            if token_after_assignment == 'npt':
                # This is an input statement! Dispatch to the input processor.
                # Pass the current_scope_name to the input processor.
                return self._process_input_from_tokens(current_scope_name=current_scope_name)
            # If the token after '=' is not 'npt', it's a regular assignment or other expression.
            # Let it fall through to _process_other_statement_from_tokens.


        # --- Handle Other Statements (Assignments, Function Calls, Expressions) ---
        # If none of the specific statement types matched, process it as a general statement.
        # This includes assignments that are NOT input statements, function calls,
        # standalone expressions (though less common in C), etc.
        return self._process_other_statement_from_tokens()

    except TranspilerError as e:
        # Catch custom transpiler errors
        print(f"Error processing statement near line {e.line_num if e.line_num else line_num}: {e.message}", file=sys.stderr)
        # Attempt to reset the token position to the start of the statement for recovery
        self.current_pos = start_pos
```

```
            # Skip tokens until a
potential statement boundary
(semicolon, brace) to avoid infinite
loops
            while self._peek() not in [';',
'}', '{', None] and self.current_pos <
len(self.tokens):
                self._skip_token()
            # Consume the boundary
token if it's a semicolon
            if self._peek() == ';':
self._skip_token()
            # Return a C comment
indicating the error
            return f"// TRANSPILER
ERROR: {e.message}"
        except Exception as e:
            # Catch any unexpected
Python exceptions
            # Safely get the line number
if possible for the error report
            err_line = line_num
            print(f"Unexpected error
processing statement near line
{err_line}: {e}", file=sys.stderr)
            # Print a traceback for
debugging unexpected errors
            import traceback

traceback.print_exc(file=sys.stderr)
            # Advance at least one
token if the position didn't change to
prevent infinite loops
            if self.current_pos ==
start_pos: self._skip_token()
            # Return a C comment
indicating the unexpected error
            return f"// UNEXPECTED
TRANSPILER ERROR:
{type(e).__name__}: {e}"


    # --- Token-Based Specific
Statement Processors ---
    def
_process_declaration_from_tokens(
self, assign_default=True,
is_const=False): # Added is_const
parameter
        """
        Processes declaration statement
from tokens. Handles const.
        Handles array initializers.
Assigns default values ONLY to
non-array variables
        if assign_default is True and no
explicit initializer is provided.
        Arrays are NOT initialized by
default. Const requires initializer
(semantic check).
        """
        # Consume the actual type
token (e.g., nt, dbl) which follows
'cnst' if present
        type_token = self._consume()
        conso_type = type_token[0]

        # Map to base C type
        base_c_type =
self.type_mapping.get(conso_type,
conso_type)

        # Prepend "const " if this is a
constant declaration
        c_type = f"const
{base_c_type}" if is_const else
base_c_type

        processed_decls = []

        while True: # Loop for
comma-separated parts (e.g., nt x,
y=1;)
            _, var_name, token_full =
self._consume('id')
            array_suffix = ""
            initializer_tokens = None
            is_array = False
            line_num =
self._get_token_info(token_full)[2]

            # Parse array dimensions
            dimensions = []
            if self._peek() == '[':
                is_array = True
                while self._peek() == '[':
                    self._consume('[')
                    size_tokens = []
                    while self._peek() != ']':
                        if self._peek() is
None: raise
TranspilerError("Unterminated
array size", line_num)
                        size_tok_type,
size_tok_val, _ = self._consume()
                        if size_tok_type ==
'blnlit':
size_tokens.append(self.bool_mappi
ng.get(size_tok_val, '0'))
                        else:
size_tokens.append(str(size_tok_val
))
                    self._consume(']')
                    dimension_expr =
"".join(size_tokens)

dimensions.append(dimension_expr
)
                    array_suffix +=
f"[{dimension_expr}]"
            # Get default value based on
type (only used for non-arrays,
non-const without initializer)
            default_val =
self.default_values.get(conso_type,
"0")

            # Handle explicit initializer
            if self._peek() == '=':
                self._consume('=')
                initializer_tokens = []
                brace_level = 0;
paren_level = 0
                while True:
                    next_token_type =
self._peek()
                    if next_token_type is
None: raise
TranspilerError("Unterminated
initializer", line_num)
                    if (next_token_type ==
',' or next_token_type == ';') and
brace_level == 0 and paren_level
== 0: break
                    tok_type, tok_val,
token_full = self._consume()

initializer_tokens.append((tok_type,
tok_val))
                    if tok_type == '{':
brace_level += 1
                    elif tok_type == '}':
brace_level -= 1
                    elif tok_type == '(':
paren_level += 1
                    elif tok_type == ')':
paren_level -= 1
                    if brace_level < 0 or
paren_level < 0: raise
TranspilerError("Mismatched
braces/parentheses in initializer",
line_num)

                init_str =
self._tokens_to_c_expression(initial
izer_tokens)
                # Use the potentially
'const' modified c_type here
                c_decl_part = f"{c_type}
{var_name}{array_suffix} =
{init_str}"

            else:
                # No explicit initializer
provided
                # Semantic analysis
should ensure 'const' variables ARE
initialized.
                # If we reach here for a
const variable, it's technically an
error,
```

```
            # but we proceed
assuming semantics handled it. We
just won't assign default.
            if is_const:
                # Const variables must
be initialized. If no '=' found,
generate declaration without value.
                # Rely on C compiler
to catch the missing initializer error.
                c_decl_part =
f"{c_type}
{var_name}{array_suffix}"
            elif is_array:
                # Regular arrays are
NOT initialized by default.
                c_decl_part =
f"{c_type}
{var_name}{array_suffix}"
            else:
                # Regular (non-array,
non-const) variable without
initializer
                if assign_default:
                    # Assign default
value ONLY if assign_default is
True
                    c_decl_part =
f"{c_type} {var_name} =
{default_val}"
                else:
                    # Otherwise, just
declare the variable
                    c_decl_part =
f"{c_type} {var_name}"


processed_decls.append(c_decl_part
)

        # Check for end of
declaration list (semicolon) or next
declaration (comma)
        if self._peek() == ';':
            self._consume(';')
            break
        elif self._peek() == ',':
            self._consume(',')
            # Continue loop for the
next variable in the list
        else:
            # Assume end of
statement if unexpected token
            break

    # Join declarations if multiple
were on one line (e.g., cnst nt x=1,
y=2;)
    # Need careful joining if they
were split by C compiler needs (like
separate statements)
    # For now, join with "; " which
might require C compiler fixups if
types differ, but okay for same type.
    # A better approach might
return a list of declarations.
    # Let's assume declarations on
one line are typically the same base
type.
    return "
".join(processed_decls) + ";"

  def _count_array_elements(self,
initializer_tokens):
    """Count the number of
elements in a 1D array initializer."""
    if not initializer_tokens or
initializer_tokens[0][0] != '{':
        return 0

    elements = 0
    brace_level = 0
    in_element = False

    for token_type, _ in
initializer_tokens:
        if token_type == '{':
            brace_level += 1
            if brace_level == 1:
                # Start of outermost
array
                continue
        elif token_type == '}':
            brace_level -= 1
            if brace_level == 0 and
in_element:
                # End of element
                elements += 1
                in_element = False
        elif brace_level == 1:
            if token_type == ',':
                if in_element:
                    elements += 1
                in_element = False
            elif token_type in ['ntlit',
'dbllit', 'blnlit', 'chrlit', 'strnglit', 'id',
'tr', 'fls']:
                if not in_element:
                    in_element = True

    return elements


  # (Other _process_*_from_tokens
methods remain largely the same as
V6)
  def
_process_dfstrct_from_tokens(self):
    self._consume('dfstrct'); _,
struct_type, _ = self._consume('id');
var_names = []
    while self._peek() != ';':
        if self._peek() == 'id':
var_names.append(self._consume('i
d')[1])
        elif self._peek() == ',':
self._consume(',')
        else: raise
TranspilerError(f"Unexpected token
'{self._peek()}' in dfstrct")
    self._consume(';');
c_declarations = [f"{struct_type}
{var_name}" for var_name in
var_names]
    return "; ".join(c_declarations)
+ ";"

  def
_process_print_from_tokens(self):
    """
    Processes a print statement
(prnt(...)) from tokens, generating
    a C printf call with appropriate
format specifiers based on argument
types,
    including handling for array
elements, literals, variables, struct
members,
    function calls, and expressions
(including arithmetic results).
    Requires access to the
symbol_table to determine
variable/member/return types.
    """
    self._consume('prnt')
    self._consume('(')
    arg_groups_tokens = [] # List
to hold lists of tokens for each
argument
    current_arg_tokens = [] #
Temp list for tokens of the current
argument
    paren_level = 0 # To handle
nested parentheses within
arguments

    # Handle empty print() case
first
    if self._peek() == ')':
        self._consume(')')
        self._consume(';')
        # Return printf with an
empty string and flush stdout
        return 'printf("");
fflush(stdout);'

    # Collect tokens for each
argument, splitting by top-level
commas
    while not (self._peek() == ')'
and paren_level == 0):
        if self._peek() is None:
```

```
            # Get line number for
error reporting if possible
            line = '?'
            if current_arg_tokens:
                try: line =
self._get_token_info(current_arg_to
kens[-1])[2]
                except: pass
            elif self.current_pos > 0:
                try: line =
self._get_token_info(self.tokens[self
.current_pos-1])[2]
                except: pass
            raise
TranspilerError("Unexpected end of
stream in print statement", line)

        # Consume the next token
        tok_type, tok_val, token_full
= self._consume()

        # Track parenthesis level to
correctly identify argument
boundaries
        if tok_type == '(':
paren_level += 1
        elif tok_type == ')':
paren_level -= 1

        # If we encounter a comma
at the top level (paren_level == 0),
        # it signifies the end of the
current argument.
        if tok_type == ',' and
paren_level == 0:
            if current_arg_tokens: #
Add the completed argument tokens
to the main list

arg_groups_tokens.append(current_
arg_tokens)
            current_arg_tokens = [] #
Reset for the next argument
        else:
            # Otherwise, add the
token to the current argument being
built

current_arg_tokens.append(token_f
ull)

    # Add the last argument's
tokens after the loop finishes
    if current_arg_tokens:

arg_groups_tokens.append(current_
arg_tokens)

    # Consume the closing
parenthesis and semicolon of the
prnt statement
    self._consume(')')
    self._consume(';')

    format_parts = [] # List to store
the C format specifiers (e.g., "%d",
"%.2f")
    c_args = []      # List to store
the C expressions for each argument

    # Process each argument group
to determine format specifier and C
code
    for arg_tokens in
arg_groups_tokens:
        if not arg_tokens: continue #
Skip if an argument group is empty
for some reason

        # Convert the argument's
tokens into a C expression string
        # We need token pairs (type,
value) for _tokens_to_c_expression
        token_pairs =
[(self._get_token_info(t)[0],
self._get_token_info(t)[1]) for t in
arg_tokens]
        arg_c_expr =
self._tokens_to_c_expression(token
_pairs)

        fmt = None # Initialize
format specifier for this argument

        # --- Logic to determine the
format specifier ('fmt') ---

        # 1. Check for Array
Element Access (e.g.,
myArr[index], myArr[i][j])
        #    Pattern: Starts with 'id',
followed by '['
        if len(arg_tokens) >= 3 and \

self._get_token_info(arg_tokens[0])
[0] == 'id' and \

self._get_token_info(arg_tokens[1])
[0] == '[':
            base_var_name =
self._get_token_info(arg_tokens[0])
[1]
            # Look up the base array
variable in the symbol table
            if self.symbol_table and
hasattr(self.symbol_table, 'lookup'):
                symbol_entry =
self.symbol_table.lookup(base_var_
name)
                # Check if symbol
exists and has a data type
                if symbol_entry and
hasattr(symbol_entry, 'data_type'):
                    var_type =
symbol_entry.data_type # Get the
type (nt, dbl, etc.)
                    # Assign format
specifier based on the array's base
type
                    if var_type == 'strng':
fmt = "%s"
                    elif var_type == 'dbl':
fmt = "%.2f" # Use .2f for doubles
                    elif var_type == 'chr':
fmt = "%c"
                    elif var_type == 'nt':
fmt = "%d"
                    elif var_type == 'bln':
fmt = "%d" # Booleans printed as
integers (0 or 1)
                    # Add other Conso
types here if needed

        # 2. If not an array access,
check for simple literals or single
variables
        elif len(arg_tokens) == 1:
            arg_type, arg_val =
self._get_token_info(arg_tokens[0])
[:2]

            if arg_type == 'strnglit':
fmt = "%s"
            elif arg_type == 'chrlit':
fmt = "%c"
            elif arg_type == 'dbllit' or
arg_type == 'NEGDOUBLELIT':
fmt = "%.2f"
            elif arg_type == 'ntlit' or
arg_type == 'NEGINTLIT': fmt =
"%d"
            # Handle boolean literals
tr/fls mapped to 1/0 in C expression,
print as int
            elif arg_type == 'blnlit':
fmt = "%d"
            elif arg_type == 'id':
                # It's a single variable
identifier (not array access)
                if self.symbol_table and
hasattr(self.symbol_table, 'lookup'):
                    symbol_entry =
self.symbol_table.lookup(arg_val)
                    if symbol_entry:
                        # Check if it's
NOT an array before assigning
format here
                        # (Array elements
handled above)
                        if not
getattr(symbol_entry, 'is_array',
False):
```

```
                    var_type =
getattr(symbol_entry, 'data_type',
None)
                    if var_type ==
'strng': fmt = "%s"
                    elif var_type ==
'dbl': fmt = "%.2f"
                    elif var_type ==
'chr': fmt = "%c"
                    elif var_type ==
'nt': fmt = "%d"
                    elif var_type ==
'bln': fmt = "%d"

        # 3. If not handled above,
check for complex expressions like
        #    struct access, function
calls, or arithmetic/logical
operations.
        else:
            # Initialize flags for
expression type detection
            is_struct_access = False
            is_function_call = False
            fmt = None # Reset fmt
for this block

            # --- Check if the
expression is primarily a
comparison ---
            # Comparisons should
always result in an integer (0 or 1)
            is_comparison =
any(self._get_token_info(t)[0] in
['==', '!=', '<', '>', '<=', '>='] for t in
arg_tokens)
            if is_comparison:
                fmt = "%d" # Force
format to integer for comparison
results
            else:
                # --- If not a
comparison, check for Struct
Member Access ---
                # Iterate through tokens
looking for the id . id pattern
                for i in
range(len(arg_tokens) - 2):
                    if
self._get_token_info(arg_tokens[i])[
0] == 'id' and \

self._get_token_info(arg_tokens[i+1
])[0] == '.' and \

self._get_token_info(arg_tokens[i+2
])[0] == 'id':
                        # ... (struct lookup
logic as before) ...
                        struct_var_name =
self._get_token_info(arg_tokens[i])[
1]
                        member_name =
self._get_token_info(arg_tokens[i+2
])[1]
                        if
self.symbol_table and
hasattr(self.symbol_table, 'lookup'):

struct_var_symbol =
self.symbol_table.lookup(struct_var
_name)
                            if
struct_var_symbol and
hasattr(struct_var_symbol,
'data_type'):

struct_type_name =
struct_var_symbol.data_type

struct_def_symbol =
self.symbol_table.lookup(struct_typ
e_name)
                                if
struct_def_symbol and
hasattr(struct_def_symbol,
'members') and
hasattr(struct_def_symbol.members,
'get'):

member_symbol =
struct_def_symbol.members.get(me
mber_name)
                                    if
member_symbol and
hasattr(member_symbol,
'data_type'):

member_type =
member_symbol.data_type
                                        if
member_type == 'strng': fmt = "%s"
                                        elif
member_type == 'dbl': fmt = "%.2f"
                                        elif
member_type == 'chr': fmt = "%c"
                                        elif
member_type == 'nt': fmt = "%d"
                                        elif
member_type == 'bln': fmt = "%d"

is_struct_access = True
                                        break #
Found struct member type

                # --- Check for
Function Call (if not struct access)
---
                if not is_struct_access
and len(arg_tokens) >= 2 and \
self._get_token_info(arg_tokens[0])
[0] == 'id' and \

self._get_token_info(arg_tokens[1])
[0] == '(':
                    # ... (function lookup
logic as before) ...
                    func_name =
self._get_token_info(arg_tokens[0])
[1]
                    if self.symbol_table
and hasattr(self.symbol_table,
'lookup'):
                        func_symbol =
self.symbol_table.lookup(func_nam
e)
                        if func_symbol
and hasattr(func_symbol,
'data_type'):
                            return_type =
func_symbol.data_type
                            if return_type
== 'strng': fmt = "%s"
                            elif return_type
== 'dbl': fmt = "%.2f"
                            elif return_type
== 'chr': fmt = "%c"
                            elif return_type
== 'nt': fmt = "%d"
                            elif return_type
== 'bln': fmt = "%d"
                            elif return_type
== 'vd':

print(f"Warning: Attempting to print
result of void function
'{func_name}' near line
{self._get_token_info(arg_tokens[0]
)[2]}. Output may be
unpredictable.")
                                fmt = None #
Let it default
                            is_function_call
= True

                # --- Fallback checks if
not comparison/struct/function ---
                if fmt is None:
                    # Check for logical
operators (results in int)
                    has_logical_op =
any(self._get_token_info(t)[0] in
['&&', '||'] for t in arg_tokens)
                    if has_logical_op:
                        fmt = "%d"
                    else:
                        # --- MODIFIED
PART ---
```

```
                # Check the
overall expression type using the
helper.
                # This helps catch
arithmetic operations resulting in
double.
                # Need token pairs
(type, value) for
get_expression_type
                arg_token_pairs =
[(self._get_token_info(t)[0],
self._get_token_info(t)[1]) for t in
arg_tokens]

expression_result_type =
self.get_expression_type(arg_token
_pairs)

                if
expression_result_type == 'dbl':
                    fmt = "%.2f"
                # Add elif for
other potential result types if needed
                # elif
expression_result_type == 'strng':
                #    fmt = "%s" #
e.g., for string concatenation if
supported
                # --- END
MODIFIED PART ---


        # 4. Default format specifier
if none of the above matched
        if fmt is None:
            # If the type is still
unknown after all checks, default to
integer.
            # This might happen for
complex arithmetic resulting in int,
or errors.
            fmt = "%d"

        # Add the determined format
specifier and the C expression to our
lists
        format_parts.append(fmt)
        c_args.append(arg_c_expr)

    # Construct the final printf
statement
    format_str = "
".join(format_parts) # Join format
specifiers with spaces
    # Join C arguments with
commas
    args_str = ", ".join(c_args)

    # Ensure there's always a
format string, even if empty
    if not format_str and not
args_str:
        # Handle case like prnt(); ->
printf("");
        return 'printf("");
fflush(stdout);'
    elif not args_str:
        # Handle case like
prnt("Literal"); -> printf("Literal");
        # Assumes single literal was
handled correctly by section #2.
        return
f'printf("{format_str}");
fflush(stdout);' # format_str should
be %s etc.
    else:
        # Normal case with format
specifiers and arguments
        return
f'printf("{format_str}", {args_str});
fflush(stdout);'

  # Helper method to check if a
variable is a string
  def _is_string_var(self, token):
    """Check if a token represents
a string variable"""
    token_type, token_value =
self._get_token_info(token)[:2]
    if token_type != 'id':
        return False

    if self.symbol_table and
hasattr(self.symbol_table, 'lookup'):
        symbol =
self.symbol_table.lookup(token_val
ue)
        return symbol and
getattr(symbol, 'data_type', None)
== 'strng'

    return False

  def
_process_input_from_tokens(self,
current_scope_name=None):
    """
    Processes an input statement
(target = npt("prompt");) by
generating
    C code. Handles single
variables, array elements, struct
members,
    and array shortcut input.

    This function parses the
left-hand side (LHS) of the
    assignment to correctly
identify the target (simple variable,
    array index, struct member)
and generate appropriate C input
code.
    It distinguishes the array
shortcut input (e.g., `arr = npt(...)`)
    which implies multiple values,
from single-value assignments.
    Includes fix for string target C
code generation structure.
    Uses exit(1) on input failure
for all scopes.
    """
    start_pos = self.current_pos
    line_num = '?'
    target_tokens = [] # Tokens for
the LHS (the target
variable/member/element)

    try:
        # 1. Consume the target
tokens (LHS) until '='
        # This loop collects all
tokens on the LHS, handling nested
parentheses/brackets
        paren_level = 0
        bracket_level = 0
        # Consume tokens until we
see '=' at the top level (outside any
parens/brackets)
        while self._peek() != '=' or
paren_level > 0 or bracket_level >
0:
            if self._peek() is None:
                raise
TranspilerError("Unexpected end of
token stream in input statement
target", line_num)

            # Consume the next
token
            tok_type, tok_val,
token_full = self._consume()
            # Update line number
from the consumed token
            line_num =
self._get_token_info(token_full)[2]

            # Track parenthesis and
bracket levels
            if tok_type == '(':
paren_level += 1
            elif tok_type == ')':
paren_level -= 1
            elif tok_type == '[':
bracket_level += 1
            elif tok_type == ']':
bracket_level -= 1

            # Add the consumed
token (type and value) to the target
tokens list
```

```
target_tokens.append((tok_type, tok_val))

        # Ensure we actually consumed some tokens for the target
        if not target_tokens:
            raise TranspilerError("Missing target for input statement", line_num)

        # Convert target tokens into a C expression string.
        # This string will be used directly in the scanf/assignment C code.
        target_c_expression = self._tokens_to_c_expression(target_tokens)

        # 2. Consume '=' and the npt(...) call
        self._consume('=') # Consume the '=' token
        self._consume('npt') # Consume the 'npt' keyword
        self._consume('(') # Consume the opening parenthesis of the npt call

        # Consume the prompt string literal inside npt()
        prompt_text = ""
        if self._peek() == 'strnglit':
            _, prompt_text, _ = self._consume('strnglit')
        # Escape the prompt text for use in a C string literal (e.g., for printf)
        c_prompt = prompt_text.replace('\\', '\\\\').replace('"', '\\"').replace('\n', '\\n')

        self._consume(')') # Consume the closing parenthesis of the npt call
        self._consume(';') # Consume the semicolon ending the statement

        # 3. Analyze Target Tokens to Determine Handling Method (Array Shortcut vs Single Value)
        is_array_shortcut = False
        array_type = None
        array_size_value = 0
        # Check if the target is a single identifier and that identifier is an array
        if len(target_tokens) == 1 and target_tokens[0][0] == 'id':
            var_name = target_tokens[0][1]
            # Look up the variable in the symbol table (current scope)
            symbol = self.symbol_table.lookup(var_name)

            # Check if the symbol exists and is marked as an array
            if symbol and getattr(symbol, 'is_array', False):
                is_array_shortcut = True
                # Retrieve array details needed for processing multiple inputs
                array_type = getattr(symbol, 'data_type', None)
                array_sizes_attr = getattr(symbol, 'array_sizes', None)

                # Get the size of the first dimension for the loop
                if isinstance(array_sizes_attr, list) and len(array_sizes_attr) > 0:
                    try:
                        # Attempt to convert the first dimension size to an integer
                        array_size_value = int(array_sizes_attr[0])
                    except (ValueError, TypeError):
                        # Handle cases where the size is not a simple integer literal
                        # (e.g., defined by a variable, which is not supported for input loops here)
                        raise TranspilerError(f"Array size for '{var_name}' must be a positive integer literal for shortcut input (found: {array_sizes_attr[0]})", line_num)
                else:
                    # Handle cases where array_sizes attribute is missing or invalid
                    raise TranspilerError(f"Could not determine valid size for array '{var_name}' from 'array_sizes': {repr(array_sizes_attr)}", line_num)

                # Ensure the array size is positive
                if array_size_value <= 0:
                    raise TranspilerError(f"Array size must be positive for shortcut input '{var_name}' (found: {array_size_value})", line_num)

                # Ensure the array type is supported for shortcut input
                if array_type not in ['chr', 'strng', 'nt', 'dbl', 'bln']:
                    raise TranspilerError(f"Array shortcut input ('npt') not supported for array type '{array_type}' of variable '{var_name}'", line_num)


        # 4. Generate C Code based on Handling Method (Array Shortcut vs Single Value)
        c_input_code = []
        # Print the prompt before reading input
        c_input_code.append(f'printf("{c_prompt}"); fflush(stdout);')

        # Define the fixed size buffer name and size (assuming it's declared in the scope)
        fixed_buffer_name = "conso_input_buffer"
        buffer_size = 1024 # This size should match the buffer declaration

        if is_array_shortcut:
            # --- ARRAY SHORTCUT INPUT (Multiple values expected, e.g., comma/space separated) ---
            # This logic is adapted from your original code for handling array shortcut input.
            # It reads the entire line and then parses it.

            if array_type == 'chr':
                # For char arrays, use scanf with a width specifier to read a fixed number of characters
                # Note: scanf leaves the newline, so we need to consume it.
                if array_size_value > 0:
                    c_input_code.append(f'scanf("%{arr
```

```
ay_size_value - 1}s",
{var_name});')
                # Consume the rest
of the line including the newline

c_input_code.append('int c; while
((c = getchar()) != \'\\n\' && c !=
EOF);')
            else:
                # Handle zero-size
char array (cannot use scanf)
                raise
TranspilerError(f"Cannot use scanf
for zero-size char array
'{var_name}'", line_num)

        elif array_type == 'strng':
            # For string arrays
(strng[]), read the line, then tokenize
using strtok_r
            # and duplicate each
token using strdup.

c_input_code.append(f'int
items_read_{var_name} = 0;')
            # Before assigning new
strings, free any existing strings in
the array

c_input_code.append(f'for (int
k_{var_name} = 0; k_{var_name}
< {array_size_value};
++k_{var_name}) {{')
            c_input_code.append(f'
free({var_name}[k_{var_name}]);
// Free existing string if any')
            c_input_code.append(f'
{var_name}[k_{var_name}] =
NULL; // Set pointer to NULL after
freeing')

c_input_code.append(f'}}')

            # Read the entire line
into the buffer

c_input_code.append(f'if
(fgets({fixed_buffer_name},
{buffer_size}, stdin) != NULL) {{')
            # Remove the trailing
newline from the buffer
            c_input_code.append(f'
{fixed_buffer_name}[strcspn({fixed
_buffer_name}, "\\n")] = 0;')

            # Tokenize the buffer
using space and tab as delimiters
            c_input_code.append(f'
char *token_{var_name};')
            c_input_code.append(f'
char *saveptr_{var_name}; //
Pointer for strtok_r to maintain
state')
            c_input_code.append(f'
char *str_ptr_{var_name} =
{fixed_buffer_name}; // Pointer to
the current position in the buffer')

            # Loop through the
array size, tokenizing and assigning
            c_input_code.append(f'
for (items_read_{var_name} = 0;
items_read_{var_name} <
{array_size_value};
++items_read_{var_name},
str_ptr_{var_name} = NULL) {{')
            # Get the next token
(using strtok_r for thread
safety/re-entrancy)
            c_input_code.append(f'
token_{var_name} =
strtok_r(str_ptr_{var_name}, " \\t",
&saveptr_{var_name});')
            # If no more tokens are
found, break the loop
            c_input_code.append(f'
if (token_{var_name} == NULL)
{{')
            c_input_code.append(f'
break; // Stop if fewer items were
entered than array size')
            c_input_code.append(f'
}}')
            # Duplicate the token
string and assign the new pointer to
the array element
            c_input_code.append(f'
//
{var_name}[items_read_{var_name
}] was freed or is NULL')
            c_input_code.append(f'
{var_name}[items_read_{var_name
}] = strdup(token_{var_name});')
            # Check if strdup failed
(memory allocation error)
            c_input_code.append(f'
if
({var_name}[items_read_{var_nam
e}] == NULL) {{')
            c_input_code.append(f'
fprintf(stderr, "Memory allocation
failed for string array input.\\n");')
            # --- Unconditional
exit(1) on failure ---
            c_input_code.append(f'
exit(1); // Indicate failure')
            # --- End Unconditional
exit(1) ---
            c_input_code.append(f'
}}')
            c_input_code.append(f'
}}') # End for loop

            # Handle the case
where fgets failed

c_input_code.append(f'}} else {{ /*
Handle fgets error */
items_read_{var_name} = 0; }}')


        elif array_type in ['nt',
'dbl', 'bln']:
            # For numeric and
boolean arrays, read the line, then
parse values using sscanf in a loop

c_input_code.append(f'int
items_read_{var_name} = 0;')

c_input_code.append(f'char*
parse_ptr_{var_name}; // Pointer to
the current parsing position in the
buffer')

c_input_code.append(f'int
offset_{var_name}; // To store the
number of characters consumed by
sscanf')

            # Read the entire line
into the buffer

c_input_code.append(f'if
(fgets({fixed_buffer_name},
{buffer_size}, stdin) != NULL) {{')
            # Remove the trailing
newline
            c_input_code.append(f'
{fixed_buffer_name}[strcspn({fixed
_buffer_name}, "\\n")] = 0;')
            c_input_code.append(f'
parse_ptr_{var_name} =
{fixed_buffer_name}; // Start
parsing from the beginning of the
buffer')

            # Determine the sscanf
format specifier and assignment
logic based on array element type
            sscanf_fmt = "";
temp_var_decl = ""; assign_logic =
f"{var_name}[items_read_{var_na
me}]"
            if array_type == 'nt':
sscanf_fmt = "%d%n" # %n
captures characters consumed
            elif array_type == 'dbl':
sscanf_fmt = "%lf%n" # %lf for
double
            elif array_type == 'bln':
                # Read boolean as int
first, then convert
```

```
                sscanf_fmt =
"%d%n"; temp_var_decl = f"int
temp_bln_{var_name};";
assign_logic =
f"temp_bln_{var_name}"

            # Declare a temporary
variable if needed for boolean
conversion
            if temp_var_decl:
c_input_code.append(f'
{temp_var_decl}')

            # Loop to read multiple
values using sscanf
            c_input_code.append(f'
for (items_read_{var_name} = 0;
items_read_{var_name} <
{array_size_value};
++items_read_{var_name}) {{')
            # Attempt to scan the
next value from the current
parse_ptr position
            c_input_code.append(f'
if (sscanf(parse_ptr_{var_name},
"{sscanf_fmt}", &{assign_logic},
&offset_{var_name}) == 1) {{')
            # If scanning was
successful:
            if array_type == 'bln':
                # Convert the
temporary int to boolean (0 or
non-zero)

c_input_code.append(f'
{var_name}[items_read_{var_name
}] = (temp_bln_{var_name} != 0);')
            # Advance the parse
pointer by the number of characters
consumed
            c_input_code.append(f'
parse_ptr_{var_name} +=
offset_{var_name};')
            # Skip any whitespace
after the number
            c_input_code.append(f'
while (*parse_ptr_{var_name} == \'
\' || *parse_ptr_{var_name} ==
\'\\t\') parse_ptr_{var_name}++;')
            # Check if the next
character is a comma (if not the last
item)
            c_input_code.append(f'
if (items_read_{var_name} <
{array_size_value} - 1 &&
*parse_ptr_{var_name} == \',\') {{
parse_ptr_{var_name}++; }}')
            # Skip any whitespace
after the comma
            c_input_code.append(f'
while (*parse_ptr_{var_name} == \'
\' || *parse_ptr_{var_name} ==
\'\\t\') parse_ptr_{var_name}++;')
            c_input_code.append(f'
}} else {{')
            # If sscanf failed, stop
reading values
            c_input_code.append(f'
break; // Stop if input doesn\'t match
format or fewer items were entered')
            c_input_code.append(f'
}}')
            c_input_code.append(f'
}}') # End for loop

            # Handle the case
where fgets failed

c_input_code.append(f'}} else {{ /*
Handle fgets error */
items_read_{var_name} = 0; }}')

            # No explicit 'return 1' on
invalid input for array shortcut,
            # as it reads as many as
possible. Error messages printed by
sscanf/fprintf.

        else:
            # --- SINGLE VALUE
INPUT (Simple var, array element
like arr[0], struct member like
person.age) ---
            # Determine the final type
of the target expression (e.g., nt for
arr[0], dbl for person.age)
            # This uses the helper
function that analyzes the target
tokens.
            target_type =
self.get_expression_type(target_tok
ens)

            if target_type is None:
                # This indicates an
issue in get_expression_type or an
unsupported target expression
                raise
TranspilerError(f"Could not
determine type of input target
'{target_c_expression}'", line_num)

            # Read the input line into
the buffer
            c_input_code.append(f'if
(fgets({fixed_buffer_name},
{buffer_size}, stdin) != NULL) {{')
            # Remove the trailing
newline character from the buffer
            c_input_code.append(f'
{fixed_buffer_name}[strcspn({fixed
_buffer_name}, "\\n")] = 0;')

            # Generate C code to
parse the input based on the target
type
            if target_type == 'strng':
                # Handle char* targets
(simple var, array element strng[],
struct member char*)
                # Use strdup to
allocate memory and copy the string
from the buffer
                # Need to free any
previously allocated string memory
first, but NOT for string literals
                c_input_code.append(f'
// Free existing string if it was
dynamically allocated (not a
literal)')
                c_input_code.append(f'
if ({target_c_expression} != NULL
&& {target_c_expression} != ""
&& {target_c_expression} != " ")
{{ // Basic check to avoid freeing
literals')
                c_input_code.append(f'
free({target_c_expression});')
                c_input_code.append(f'
}}')
                c_input_code.append(f'
{target_c_expression} =
strdup({fixed_buffer_name});')
                # Check if strdup
failed (memory allocation error)
                c_input_code.append(f'
if ({target_c_expression} ==
NULL) {{')
                c_input_code.append(f'
fprintf(stderr, "Memory allocation
failed for string input.\\n");')
                # --- Unconditional
exit(1) on failure ---
                c_input_code.append(f'
exit(1); // Indicate failure and exit
the current function')
                # --- End
Unconditional exit(1) ---
                c_input_code.append(f'
}}')
                # Add the closing
brace for the fgets if block
                # Corrected else block:
Removed free, only set to NULL on
fgets error

c_input_code.append(f'}} else {{ /*
Handle fgets error or EOF */
{target_c_expression} = NULL;
}}')

            elif target_type == 'chr'
and '[' in target_c_expression:
```

```
            # Handle char array targets (e.g., char_array[index] or struct member char array)
            # Use strncpy to copy the string from the buffer to the char array element/member
            # This requires knowing the size of the destination char array.
            # A robust solution would look up the symbol for the target and get its size.
            # For simplicity here, we'll use sizeof() on the target expression, which works
            # if the target is a char array element or a struct member char array.
            c_input_code.append(f'// Assuming {target_c_expression} is a char array element or struct member char array')
            c_input_code.append(f'strncpy({target_c_expression}, {fixed_buffer_name}, sizeof({target_c_expression}) - 1);')
            # Ensure null termination in case the input string is longer than the array capacity
            c_input_code.append(f'{target_c_expression}[sizeof({target_c_expression}) - 1] = \'\\0\'; // Ensure null termination')
            # Add the closing brace for the fgets if block

c_input_code.append(f'}} else {{ /* Handle fgets error or EOF */ {target_c_expression}[0] = \'\\0\'; }}') # Added else block and closing brace

        elif target_type == 'nt':
            # Use sscanf to parse an integer from the buffer and assign to the target
            c_input_code.append(f'if (sscanf({fixed_buffer_name}, "%d", &({target_c_expression})) != 1) {{')
            # If sscanf fails (input is not a valid integer)
            c_input_code.append(f'fprintf(stderr, "\\nError: Invalid integer input for {target_c_expression}.\\n");')
            # --- Unconditional exit(1) on failure ---
            c_input_code.append(f'exit(1); // Indicate failure and exit the current function')
            # --- End Unconditional exit(1) ---
            c_input_code.append(f'}}')
            # Add the closing brace for the fgets if block

c_input_code.append(f'}} else {{ /* Handle fgets error or EOF */ }}') # Added else block and closing brace

        elif target_type == 'dbl':
            # Use sscanf to parse a double from the buffer and assign to the target
            c_input_code.append(f'if (sscanf({fixed_buffer_name}, "%lf", &({target_c_expression})) != 1) {{')
            # If sscanf fails (input is not a valid double)
            c_input_code.append(f'fprintf(stderr, "\\nError: Invalid double input for {target_c_expression}.\\n");')
            # --- Unconditional exit(1) on failure ---
            c_input_code.append(f'exit(1); // Indicate failure and exit the current function')
            # --- End Unconditional exit(1) ---
            c_input_code.append(f'}}')
            # Add the closing brace for the fgets if block

c_input_code.append(f'}} else {{ /* Handle fgets error or EOF */ }}') # Added else block and closing brace

        elif target_type == 'chr':
            # Handle single character input: take the first character of the buffer
            c_input_code.append(f'if ({fixed_buffer_name}[0] != \'\\n\' && {fixed_buffer_name}[0] != \'\\0\') {{')
            c_input_code.append(f'{target_c_expression} = {fixed_buffer_name}[0];')
            c_input_code.append(f'}} else {{')
            # If the buffer is empty or only contains a newline
            c_input_code.append(f'fprintf(stderr, "\\nError: Invalid character input for {target_c_expression}.\\n");')
            # --- Unconditional exit(1) on failure ---
            c_input_code.append(f'exit(1); // Indicate failure and exit the current function')
            # --- End Unconditional exit(1) ---
            c_input_code.append(f'}}')
            # Add the closing brace for the fgets if block

c_input_code.append(f'}} else {{ /* Handle fgets error or EOF */ }}') # Added else block and closing brace

        elif target_type == 'bln':
            # Handle boolean input: accept 0, 1, "tr", or "fls"
            c_input_code.append(f'int temp_bln_input;') # Temporary variable to read integer boolean
            c_input_code.append(f'if (sscanf({fixed_buffer_name}, "%d", &temp_bln_input) == 1) {{')
            # If an integer (0 or 1) was successfully scanned, assign the boolean value
            c_input_code.append(f'{target_c_expression} = (temp_bln_input != 0);')
            c_input_code.append(f'}} else {{')
            # If integer scan failed, check if the input string is "tr" or "fls"
            c_input_code.append(f'if (strcmp({fixed_buffer_name}, "tr") == 0) {{ {target_c_expression} = 1; }}')
            c_input_code.append(f'else if (strcmp({fixed_buffer_name}, "fls") == 0) {{ {target_c_expression} = 0; }}')
            c_input_code.append(f'else {{')
            # If none of the expected formats match
            c_input_code.append(f'fprintf(stderr, "\\nError: Invalid boolean input for {target_c_expression} (expected 0, 1, tr, or fls).\\n");')
            # --- Unconditional exit(1) on failure ---
            c_input_code.append(f'exit(1); // Indicate failure and exit the current function')
            # --- End Unconditional exit(1) ---
```

```
                c_input_code.append(f'
}}')
                c_input_code.append(f'
}}')
                # Add the closing
brace for the fgets if block

c_input_code.append(f'}} else {{ /*
Handle fgets error or EOF */ }}') #
Added else block and closing brace

            else:
                # This case should not
be reached if get_expression_type is
comprehensive
                raise
TranspilerError(f"Input ('npt') not
supported for target type
'{target_type}'", line_num)



        # Join all the generated C
lines
        return
"\n".join(c_input_code)

    # --- Error Handling (Keep
existing blocks) ---
    except TranspilerError as e:
        # Print the custom transpiler
error message
        print(f"Error processing
input statement near line
{e.line_num if e.line_num else
line_num}: {e.message}",
file=sys.stderr)
        # Attempt to reset the token
position to the start of the statement
for recovery
        self.current_pos = start_pos
        # Skip tokens until a
potential statement boundary
(semicolon, brace) to avoid infinite
loops
        while self._peek() not in [';',
'}', '{', None] and self.current_pos <
len(self.tokens):
            self._skip_token()
        # Consume the boundary
token if it's a semicolon
        if self._peek() == ';':
self._skip_token()
        # Return a C comment
indicating the error
        return f"// TRANSPILER
ERROR (Input): {e.message}"
    except Exception as e:
        # Handle any unexpected
Python exceptions during
processing
        # Safely get the line number
if possible for the error report
        err_line = line_num
        print(f"Unexpected error
processing input for
'{target_c_expression if
'target_c_expression' in locals() else
'unknown target'}' near line
{err_line}: {e}", file=sys.stderr)
        # Print a traceback for
debugging unexpected errors
        import traceback

traceback.print_exc(file=sys.stderr)
        # Advance at least one token
if the position didn't change to
prevent infinite loops
        if self.current_pos ==
start_pos: self._skip_token()
        # Return a C comment
indicating the unexpected error
        return f"// UNEXPECTED
TRANSPILER ERROR (Input):
{type(e).__name__}: {e}"


    def
_process_return_from_tokens(self):
        self._consume('rtrn')
        if self._peek() == ';':
self._consume(';'); return "return;"
        expr_tokens = []
        while self._peek() != ';':
expr_tokens.append(self._consume(
)[:2])
        self._consume(';'); value_c =
self._tokens_to_c_expression(expr_
tokens)
        return f"return {value_c};"

    def
_process_if_from_tokens(self):
        self._consume('f');
self._consume('('); condition_tokens
= []; paren_level = 1
        while paren_level > 0:
            tt, tv, _ = self._consume();
            if tt == '(': paren_level += 1;
            elif tt == ')': paren_level -=
1
            if paren_level > 0:
condition_tokens.append((tt, tv))
        condition_c =
self._tokens_to_c_expression(condit
ion_tokens)
        return f"if ({condition_c})"

    def
_process_else_if_from_tokens(self):
        self._consume('lsf');
self._consume('('); condition_tokens
= []; paren_level = 1
        while paren_level > 0:
            tt, tv, _ = self._consume();
            if tt == '(': paren_level += 1;
            elif tt == ')': paren_level -=
1
            if paren_level > 0:
condition_tokens.append((tt, tv))
        condition_c =
self._tokens_to_c_expression(condit
ion_tokens)
        return f"else if
({condition_c})"

    def
_process_else_from_tokens(self):
self._consume('ls'); return "else"
    def
_process_while_from_tokens(self):
        self._consume('whl');
self._consume('('); condition_tokens
= []; paren_level = 1
        while paren_level > 0:
            tt, tv, _ = self._consume();
            if tt == '(': paren_level += 1;
            elif tt == ')': paren_level -=
1
            if paren_level > 0:
condition_tokens.append((tt, tv))
        condition_c =
self._tokens_to_c_expression(condit
ion_tokens)
        return f"while ({condition_c})"

    def
_process_for_from_tokens(self):
        self._consume('fr');
self._consume('('); it = []; ct = []; ut
= []; p = 1; pl = 1
        while pl > 0:
            tt, tv, _ = self._consume();
            if tt == '(': pl += 1;
            elif tt == ')': pl -= 1
            if pl == 0: break
            if tt == ';' and pl == 1: p +=
1
            else: t = (tt, tv); (it if p == 1
else ct if p == 2 else ut).append(t)
        ic =
self._tokens_to_c_expression(it) if
it else ""; cc =
self._tokens_to_c_expression(ct) if
ct else ""; uc =
self._tokens_to_c_expression(ut) if
ut else ""
        return f"for ({ic}; {cc}; {uc})"
```

```
    def
_process_do_from_tokens(self):
self._consume('d'); return "do"
    def
_process_switch_from_tokens(self):
        self._consume('swtch');
self._consume('('); expr_tokens = [];
paren_level = 1
        while paren_level > 0:
            tt, tv, _ = self._consume();
            if tt == '(': paren_level += 1;
            elif tt == ')': paren_level -=
1
            if paren_level > 0:
expr_tokens.append((tt, tv))
        expr_c =
self._tokens_to_c_expression(expr_
tokens)
        return f"switch ({expr_c})"

    def
_process_case_from_tokens(self):
        self._consume('cs');
value_tokens = []
        while self._peek() != ':':
value_tokens.append(self._consume
()[:2])
        self._consume(':'); value_c =
self._tokens_to_c_expression(value
_tokens)
        return f"case {value_c}:"

    def
_process_default_from_tokens(self)
: self._consume('dflt');
self._consume(':'); return "default:"

    def
_process_other_statement_from_tok
ens(self):
        """Processes assignments,
function calls etc. from tokens until
semicolon."""
        stmt_tokens = []; paren_level =
0; bracket_level = 0
        while not (self._peek() == ';'
and paren_level == 0 and
bracket_level == 0):
            if self._peek() is None: raise
TranspilerError("Unexpected end of
stream in statement")
            tok_type, tok_val, _ =
self._consume()
            if tok_type == '(':
paren_level += 1
            elif tok_type == ')':
paren_level -= 1
            elif tok_type == '[':
bracket_level += 1
            elif tok_type == ']':
bracket_level -= 1
            stmt_tokens.append((tok_type,
tok_val))
        self._consume(';')
        return
self._tokens_to_c_expression(stmt_t
okens) + ";"

    # --- Helper Methods for
Expression Processing (with
Debugging) ---

    def format_token(self, tok_type,
tok_val):
        """
        Formats a single token into its
C string representation.
        Handles special cases for
literals like strings, chars, and
booleans.
        """
        if tok_type == 'blnlit':
            # Map boolean literals 'tr'
and 'fls' to '1' and '0'
            return
self.bool_mapping.get(tok_val, '0')
        elif tok_type == 'strnglit':
            # Enclose string literals in
double quotes
            return f'"{tok_val}"'
        elif tok_type == 'chrlit':
            # Enclose character literals
in single quotes
            return f"'{tok_val}'"
        else:
            # Default case: convert the
token value to a string
            return str(tok_val)

    def get_expression_type(self,
expr_tokens):
        """
        Determines the resulting C
type ('nt', 'dbl', 'strng', 'bln', 'chr', or
None)
        of an expression represented by
a list of (type, value) tokens.
        Prioritizes operator type over
initial operand type (e.g., division of
ints
        might result in double in some
contexts, though C integer division
truncates).
        Relies on symbol table lookup
to determine variable and member
types.
        Handles simple literals,
variables, array elements, struct
members,
        function calls, and basic
arithmetic/logical/comparison
operations.
        """
        # print(f"[DEBUG
get_expression_type] Input tokens:
{expr_tokens}") # Uncomment for
deep debug
        if not expr_tokens:
            # print("[DEBUG
get_expression_type] Returning:
None (empty tokens)") #
Uncomment for deep debug
            return None # Return None
for empty expressions

        result_type = None # Initialize
the determined result type

        # --- Operator-based type
determination FIRST ---
        # Check for comparison or
logical operators, as they always
result in a boolean (int in C)
        is_comparison_or_logical =
any(
            t[0] in ['==', '!=', '<', '>', '<=',
'>=', '&&', '||']
            for t in expr_tokens
        )
        if is_comparison_or_logical:
            result_type = 'bln' # The
result of a comparison or logical
operation is boolean (true/false)

        # If not a comparison/logical
operation, check for arithmetic
operations
        has_arithmetic_op = any(t[0]
in ['+', '-', '*', '/'] for t in
expr_tokens)
        if has_arithmetic_op and
result_type is None: # Only check if
type hasn't been determined yet
            promotes_to_double = False
            # Iterate through tokens to
check operand types for potential
double promotion
            i = 0
            while i < len(expr_tokens):
                tok_type, tok_val =
expr_tokens[i]

                # If we find a double
literal or a division operator '/', the
result might be double
                if tok_type in ['dbllit',
'NEGDOUBLELIT', '/']:
                    promotes_to_double =
True
```

```
                break # Found a source that promotes to double, no need to check further

            # If an identifier is involved, check its type from the symbol table
            if tok_type == 'id':
                # Check for struct member access pattern (id . id)
                if i + 2 < len(expr_tokens) and expr_tokens[i+1][0] == '.' and expr_tokens[i+2][0] == 'id':
                    struct_var_name = tok_val
                    member_name = expr_tokens[i+2][1]
                    # print(f"[DEBUG get_expression_type] Checking struct member: {struct_var_name}.{member_name}") # DEBUG
                    if self.symbol_table and hasattr(self.symbol_table, 'lookup'):
                        # Look up the symbol for the struct variable
                        struct_var_symbol = self.symbol_table.lookup(struct_var_name)
                        if struct_var_symbol and hasattr(struct_var_symbol, 'data_type'):

                            struct_type_name = struct_var_symbol.data_type # Get the name of the struct type
                            # Look up the symbol for the struct definition itself

                            struct_def_symbol = self.symbol_table.lookup(struct_type_name)
                            # Check if the struct definition symbol exists and has members
                            if struct_def_symbol and hasattr(struct_def_symbol, 'members') and hasattr(struct_def_symbol.members, 'get'):
                                # Get the symbol for the specific member

                                member_symbol = struct_def_symbol.members.get(member_name)
                                # Check if the MEMBER's data type is 'dbl'
                                if member_symbol and getattr(member_symbol, 'data_type', None) == 'dbl':
                                    # print(f"[DEBUG get_expression_type] Struct member {member_name} is dbl.") # DEBUG

                                    promotes_to_double = True
                                    break # Found a double source, stop checking

                    # Skip the '.' and member 'id' tokens in the next iteration since they are part of this access
                    i += 2
                else:
                    # Regular ID lookup (not part of struct access pattern)
                    # print(f"[DEBUG get_expression_type] Checking identifier: {tok_val}") # DEBUG
                    if self.symbol_table and hasattr(self.symbol_table, 'lookup'):
                        # Look up the symbol for the identifier
                        symbol = self.symbol_table.lookup(tok_val)
                        # Check if the IDENTIFIER's data type is 'dbl'
                        if symbol and getattr(symbol, 'data_type', None) == 'dbl':
                            # print(f"[DEBUG get_expression_type] Identifier {tok_val} is dbl.") # DEBUG

                            promotes_to_double = True
                            break # Found a double source, stop checking

                i += 1 # Move to the next token

        # Assign the result type based on whether any double source was found
        result_type = 'dbl' if promotes_to_double else 'nt'

    # --- If no operators determined the type, check the structure of the expression ---
    # This handles cases like single literals, variables, array elements, struct members, function calls.
    if result_type is None:
        # Check simple cases first: a single literal or a single variable identifier
        if len(expr_tokens) == 1:
            tok_type, tok_val = expr_tokens[0]
            if tok_type == 'strnglit': result_type = 'strng'
            elif tok_type == 'chrlit': result_type = 'chr'
            elif tok_type in ['dbllit', 'NEGDOUBLELIT']: result_type = 'dbl'
            elif tok_type in ['ntlit', 'NEGINTLIT']: result_type = 'nt'
            elif tok_type == 'blnlit': result_type = 'bln'
            elif tok_type == 'id':
                # Look up the identifier in the symbol table
                if self.symbol_table and hasattr(self.symbol_table, 'lookup'):
                    symbol = self.symbol_table.lookup(tok_val)
                    # The type of a single variable is its data type from the symbol table
                    result_type = getattr(symbol, 'data_type', None) if symbol else None
        # Check complex structures - Array Element Access (e.g., arr[index])
        # Pattern: Starts with 'id', contains '[', ends with ']'
        elif len(expr_tokens) >= 3 and expr_tokens[0][0] == 'id' and expr_tokens[1][0] == '[' and expr_tokens[-1][0] == ']':
            base_var_name = expr_tokens[0][1]
            # Look up the base array variable in the symbol table
            if self.symbol_table and hasattr(self.symbol_table, 'lookup'):
                symbol = self.symbol_table.lookup(base_var_name)
                # The type of an array element is the base data type of the array
```

```
            result_type =
getattr(symbol, 'data_type', None) if
symbol else None
        # Check complex structures
- Struct Member Access (e.g.,
struct.member)
        # Pattern: Ends with 'id',
preceded by '.', preceded by
something (often 'id')
        # This heuristic assumes
simple id.id access. More complex
chains might need recursive
analysis.
        elif len(expr_tokens) >= 3
and expr_tokens[-1][0] == 'id' and
expr_tokens[-2][0] == '.':
            # Assuming a simple
`id.id` structure for now
            if len(expr_tokens) == 3
and expr_tokens[0][0] == 'id':
                struct_var_name =
expr_tokens[0][1]
                member_name =
expr_tokens[-1][1]
                if self.symbol_table
and hasattr(self.symbol_table,
'lookup'):
                    # Look up the
symbol for the struct variable
                    struct_var_symbol =
self.symbol_table.lookup(struct_var
_name)
                    if struct_var_symbol
and hasattr(struct_var_symbol,
'data_type'):
                        struct_type_name
= struct_var_symbol.data_type #
Get the name of the struct type
                        # Look up the
symbol for the struct definition
itself

struct_def_symbol =
self.symbol_table.lookup(struct_typ
e_name)
                        # Check if the
struct definition symbol exists and
has members
                        if
struct_def_symbol and
hasattr(struct_def_symbol,
'members') and
hasattr(struct_def_symbol.members,
'get'):
                            # Get the
symbol for the specific member

member_symbol =
struct_def_symbol.members.get(me
mber_name)
                            # The type of a
struct member is its data type from
the struct definition
                            if
member_symbol: result_type =
getattr(member_symbol, 'data_type',
None)
        # Check complex structures
- Function Call (e.g., func(args))
        # Pattern: Starts with 'id',
followed by '(', ends with ')'
        elif len(expr_tokens) >= 3
and expr_tokens[0][0] == 'id' and
expr_tokens[1][0] == '(' and
expr_tokens[-1][0] == ')':
            func_name =
expr_tokens[0][1]
            # Look up the function
symbol in the symbol table
            if self.symbol_table and
hasattr(self.symbol_table, 'lookup'):
                symbol =
self.symbol_table.lookup(func_nam
e)
                # The type of a
function call expression is the
function's return type
                result_type =
getattr(symbol, 'data_type', None) if
symbol else None

    # print(f"[DEBUG
get_expression_type] Returning
type: {result_type}") # Uncomment
for deep debug
    return result_type

def _format_token_sequence(self,
tokens_to_format):
    """
    Helper to format a sequence of
tokens into C code with basic
spacing rules.
    Attempts to add spaces
between tokens where appropriate
(e.g., between
    identifiers and operators) but
not around punctuation like '.', '(', '[',
etc.
    """
    parts = []
    if not tokens_to_format: return
""


    for i, (tok_type, tok_val) in
enumerate(tokens_to_format):
        needs_space = False
        # Check if a space is needed
before the current token
        if parts:
            last_part = parts[-1] # Get
the last part added to the result

            # Add a space if the
current token is not punctuation that
attaches to the previous token
            # and the last part is not
punctuation that attaches to the
current token.
            # Avoid space before ')',
']', '.', ';', ','
            if tok_type not in [')', ']',
'.', ';', ',']:
                # Avoid space after '(',
'[', '.'
                if not
last_part.endswith(('(', '[', '.')):
                    # Avoid space
between
alphanumeric/closing-paren/bracket
and opening-paren/bracket (function
calls, array access)
                    if not
((last_part.isalnum() or
last_part.endswith((')', ']'))) and
tok_type in ['(', '[']):
                        # Avoid space
before '++' or '--' if preceded by
alphanumeric
                        if not (tok_type
in ['++', '--'] and
last_part.isalnum()):
                            # In most
other cases, add a space
                            needs_space
= True

        # Add a space if needed
        if needs_space:
parts.append(" ")

        # Add the formatted token
value

parts.append(self.format_token(tok_
type, tok_val))

    # Join all parts into a single
string
    return "".join(parts)

def
_process_comparison_segment(self,
segment_tokens):
    """
    Processes a segment of tokens
(typically between logical
operators)
    to handle comparisons,
especially string comparisons using
strcmp.
```

```
        Finds the first top-level
comparison operator and splits the
segment.
        """
        if not segment_tokens: return
""


        comparison_index = -1
        paren_level = 0
        bracket_level = 0
        comparison_ops = ['==', '!=',
'<', '>', '<=', '>='] # List of
comparison operators

        # Iterate through the tokens to
find a top-level comparison operator
        for idx, (tok_type, _) in
enumerate(segment_tokens):
            if tok_type == '(':
paren_level += 1
            elif tok_type == ')':
paren_level -= 1
            elif tok_type == '[':
bracket_level += 1
            elif tok_type == ']':
bracket_level -= 1
            # If the token is a
comparison operator and we are at
the top level (outside
parens/brackets)
            elif paren_level == 0 and
bracket_level == 0 and tok_type in
comparison_ops:
                comparison_index = idx #
Found the operator
                break # Stop at the first
top-level comparison operator

        # If a comparison operator was
found
        if comparison_index != -1:
            op_tok_type =
segment_tokens[comparison_index]
[0] # Get the type of the operator
            left_operand_tokens =
segment_tokens[:comparison_index
] # Tokens before the operator
            right_operand_tokens =
segment_tokens[comparison_index
+ 1:] # Tokens after the operator

            # Determine the types of the
left and right operands
            left_type =
self.get_expression_type(left_opera
nd_tokens)
            right_type =
self.get_expression_type(right_oper
and_tokens)

            # If both operands are
strings and the operator is == or !=,
use strcmp
            if left_type == 'strng' and
right_type == 'strng' and
op_tok_type in ['==', '!=']:
                # Convert operands to C
expressions
                left_c =
self._tokens_to_c_expression(left_o
perand_tokens)
                right_c =
self._tokens_to_c_expression(right_
operand_tokens)
                # Determine the C
comparison operator based on the
Conso operator
                op_str = "==" if
op_tok_type == '==' else "!="
                # Generate the C code
using strcmp
                return f"(strcmp({left_c},
{right_c}) {op_str} 0)"
            else:
                # For non-string or other
comparison operators, generate
standard C comparison
                left_c =
self._tokens_to_c_expression(left_o
perand_tokens)
                right_c =
self._tokens_to_c_expression(right_
operand_tokens)
                # Combine the C
expressions with the operator
                if left_c and right_c:
return f"{left_c} {op_tok_type}
{right_c}"
                elif left_c: return
f"{left_c} {op_tok_type}" # Handle
unary operators if needed (though
not typical for comparison)
                elif right_c: return
f"{op_tok_type} {right_c}" #
Handle unary operators
                else: return
f"{op_tok_type}" # Should not
happen for binary operators


        else:
            # If no top-level comparison
operator was found, format the
segment as a simple sequence of
tokens
            return
self._format_token_sequence(segme
nt_tokens)

    def _tokens_to_c_expression(self,
tokens):
        """
        Converts a list of (type, value)
tokens representing an expression
        into a C expression string.
        Handles operator precedence
by splitting first by logical operators
(&&, ||),
        then processing each segment
for comparisons (using
_process_comparison_segment),
        and finally formatting the
remaining tokens within segments.
        Also replaces boolean literals
'tr' and 'fls' with '1' and '0'.
        """
        if not tokens: return ""

        segments = [] # List to hold
token lists for segments separated
by logical operators
        operators = [] # List to hold the
logical operators (&&, ||)
        current_start = 0 # Index to
track the start of the current segment

        paren_level = 0
        bracket_level = 0

        # Iterate through tokens to split
by top-level logical operators (&&,
||)
        for i, (tok_type, _) in
enumerate(tokens):
            if tok_type == '(':
paren_level += 1
            elif tok_type == ')':
paren_level -= 1
            elif tok_type == '[':
bracket_level += 1
            elif tok_type == ']':
bracket_level -= 1
            # If a logical operator is
found at the top level
            elif paren_level == 0 and
bracket_level == 0 and tok_type in
['&&', '||']:
                # Add the tokens from the
current segment to the segments list

segments.append(tokens[current_sta
rt:i])
                # Add the operator to the
operators list

operators.append(tok_type)
                # Update the start index
for the next segment
                current_start = i + 1

        # Add the last segment after
the loop finishes
```

```
segments.append(tokens[current_start:])

        # Process each segment to handle comparisons within it
        processed_segments = [self._process_comparison_segment(segment) for segment in segments]

        # Reconstruct the full C expression by joining the processed segments with the logical operators
        result_parts = []
        if processed_segments:
            result_parts.append(processed_segments[0]) # Add the first processed segment
            # Add the operators and the subsequent processed segments
            for i, op in enumerate(operators):
                result_parts.append(f" {op} ") # Add the operator with spacing
                result_parts.append(processed_segments[i+1]) # Add the next processed segment

        # Join all parts into a single string
        result = "".join(result_parts)

        # Replace Conso boolean literals ('tr', 'fls') with C equivalents ('1', '0')
        result = self._replace_bool_literals(result)

        return result

    def _replace_bool_literals(self, text):
        """
        Replaces whole-word occurrences of 'tr' and 'fls' with '1' and '0'
        using regular expressions to avoid replacing substrings.
        """
        # Replace 'tr' only when it's a whole word
        text = re.sub(r'\btr\b', '1', text)
        # Replace 'fls' only when it's a whole word
        text = re.sub(r'\bfls\b', '0', text)
        return text

    # --- Print Processing (with Debugging) ---

    def _process_print_from_tokens(self):
        """
        Processes a print statement (prnt(...)) from tokens, generating
        a C printf call with appropriate format specifiers based on argument types.
        Relies on get_expression_type to determine the resulting type for format selection.
        Includes DEBUG prints.
        """
        self._consume('prnt')
        self._consume('(')
        arg_groups_tokens = []
        current_arg_tokens = []
        paren_level = 0

        if self._peek() == ')':
            self._consume(')')
            self._consume(';')
            return 'printf(""); fflush(stdout);'

        while not (self._peek() == ')' and paren_level == 0):
            if self._peek() is None:
                line = '?'
                if current_arg_tokens:
                    try: line = self._get_token_info(current_arg_tokens[-1])[2]
                    except: pass
                elif self.current_pos > 0:
                    try: line = self._get_token_info(self.tokens[self.current_pos-1])[2]
                    except: pass
                raise TranspilerError("Unexpected end of stream in print statement", line)

            tok_type, tok_val, token_full = self._consume()
            if tok_type == '(': paren_level += 1
            elif tok_type == ')': paren_level -= 1

            if tok_type == ',' and paren_level == 0:
                if current_arg_tokens: arg_groups_tokens.append(current_arg_tokens)
                current_arg_tokens = []
            else:
                current_arg_tokens.append(token_full)

        if current_arg_tokens: arg_groups_tokens.append(current_arg_tokens)
        self._consume(')')
        self._consume(';')

        format_parts = []
        c_args = []

        for arg_tokens in arg_groups_tokens:
            if not arg_tokens: continue

            # print(f"\n[DEBUG _process_print] Processing arg tokens: {arg_tokens}") # DEBUG

            token_pairs = [(self._get_token_info(t)[0], self._get_token_info(t)[1]) for t in arg_tokens]
            arg_c_expr = self._tokens_to_c_expression(token_pairs)

            fmt = None

            # --- Universal Type Checking using get_expression_type ---
            expression_result_type = self.get_expression_type(token_pairs)
            # print(f"[DEBUG _process_print] Determined type: {expression_result_type}") # DEBUG

            if expression_result_type == 'dbl':
                fmt = "%.2f"
            elif expression_result_type == 'strng':
                fmt = "%s"
            elif expression_result_type == 'chr':
                fmt = "%c"
            elif expression_result_type == 'nt':
                fmt = "%d"
            elif expression_result_type == 'bln':
                fmt = "%d" # Boolean results printed as integers

            # --- Default format specifier ---
```

```
        if fmt is None:
            line_num =
self._get_token_info(arg_tokens[0])
[2] if arg_tokens else '?'
            print(f"Warning: Could
not determine print format for
expression near line {line_num}.
Defaulting to %d.")
            fmt = "%d"

        # print(f"[DEBUG
_process_print] Final format
specifier: {fmt}") # DEBUG
        format_parts.append(fmt)
        c_args.append(arg_c_expr)

    # Construct the final printf
statement
    format_str = "
".join(format_parts)
    args_str = ", ".join(c_args)

    if not format_str and not
args_str: return 'printf("");
fflush(stdout);'
    elif not args_str: return
f'printf("{format_str}");
fflush(stdout);'
    else: return
f'printf("{format_str}", {args_str});
fflush(stdout);'

  # --- Helper Methods ---
  def _indent(self, level): return "
" * max(0, level)
  def _generate_headers(self):
    return ("#include
<stdio.h>\n#include
<stdlib.h>\n#include
<string.h>\n#include
<stdbool.h>\n#include
<stddef.h>\n\n")
  def
_generate_helper_functions(self):
return """// Helper function for
string input
char* conso_input(const char*
prompt) { printf("%s", prompt);
fflush(stdout); char buffer[1024];
char* line = NULL; size_t cl = 0;
size_t bl = sizeof(buffer); if
(fgets(buffer, bl, stdin) == NULL) {
if (feof(stdin)) return NULL;
fprintf(stderr, "Input Error\\n");
exit(1); } cl = strlen(buffer); if (cl >
0 && buffer[cl - 1] == '\\n') {
buffer[cl - 1] = '\\0'; cl--; } line =
malloc(cl + 1); if (line == NULL) {
fprintf(stderr, "Malloc Error\\n");
exit(1); } strcpy(line, buffer); return
line; }
// Helper function for string
concatenation
char* conso_concat(const char* s1,
const char* s2) { if (s1 == NULL)
s1 = ""; if (s2 == NULL) s2 = "";
size_t l1 = strlen(s1); size_t l2 =
strlen(s2); char* r = malloc(l1 + l2 +
1); if (r == NULL) { fprintf(stderr,
"Malloc Error\\n"); exit(1); }
strcpy(r, s1); strcat(r, s2); return r;
}"""
  def _split_args(self, content): #
Keep for legacy string processing if
needed elsewhere
    if not content: return []
    args = []; current_arg = ""; pl =
0; bl = 0; brl = 0; isq = False; idq =
False; en = False
    for char in content:
      if en: current_arg += char;
en = False; continue
      if char == '\\': current_arg +=
char; en = True; continue
      if char == '"' and not idq: isq
= not isq
      elif char == "'" and not isq:
idq = not idq
      if isq or idq: current_arg +=
char; continue
      if char == '(': pl += 1;
      elif char == ')': pl -= 1
      elif char == '[': bl += 1;
      elif char == ']': bl -= 1
      elif char == '{': brl += 1;
      elif char == '}': brl -= 1
      if char == ',' and pl == 0 and
bl == 0 and brl == 0:
args.append(current_arg);
current_arg = ""
      else: current_arg += char
    args.append(current_arg); sa =
[a.strip() for a in args]; return [a for
a in sa if a]
  def _split_declaration_args(self,
declaration_part): return
self._split_args(declaration_part) #
Keep using robust splitter
  def _replace_bool_literals(self,
text): text = re.sub(r'\btr\b', '1', text);
text = re.sub(r'\bfls\b', '0', text);
return text


# --- Standalone Functions ---
def transpile(conso_code):
  """Standalone function to
transpile Conso code string
(legacy)."""
  print("Warning: Calling
string-based transpile. Token-based
is preferred.")
  return "// String-based transpile
function needs lexer/parser
integration."

def
transpile_from_tokens(token_list,
symbol_table=None,
function_scopes=None):
  # ... (implementation from
previous step) ...
  if token_list:
    last_token = token_list[-1]
    temp_transpiler =
ConsoTranspilerTokenBased([])
    token_type, _, _ =
temp_transpiler._get_token_info(las
t_token)
    if token_type == 'EOF':
      token_list = token_list[:-1]
  transpiler =
ConsoTranspilerTokenBased(token_
list, symbol_table, function_scopes)
  try:
    return transpiler.transpile()
  except TranspilerError as e:
    print(f"Transpilation Error:
{e}", file=sys.stderr)
    return f"// TRANSPILER
ERROR: {e}"
  except Exception as e:
    print(f"Unexpected Transpiler
Error: {type(e).__name__}: {e}",
file=sys.stderr)
    import traceback
    print(traceback.format_exc(),
file=sys.stderr)
    return f"// UNEXPECTED
TRANSPILER ERROR:
{type(e).__name__}: {e}"

# --- Example Usage ---
if __name__ == "__main__":
  # Example token list simulating
output from previous stages
  # --- Make sure these token types
match your lexer ---
  # --- REMOVED EOF from this
example list ---
  test_token_list = [
    # strng name = "hello";
    ('strng', 'strng', 1, 1), ('id',
'name', 1, 7), ('=', '=', 1, 12),
('strnglit', 'hello', 1, 14), (';', ';', 1,
21),
    # nt num[3];
    ('nt', 'nt', 2, 1), ('id', 'num', 2, 4),
('[', '[', 2, 7), ('ntlit', 3, 2, 8), (']', ']', 2,
9), (';', ';', 2, 10),
    # dbl frac[3];
```

```
        ('dbl', 'dbl', 3, 1), ('id', 'frac', 3,
5), ('[', '[', 3, 9), ('ntlit', 3, 3, 10), (']',
']', 3, 11), (';', ';', 3, 12),
        # bln flag[2];
        ('bln', 'bln', 4, 1), ('id', 'flag', 4,
5), ('[', '[', 4, 9), ('ntlit', 2, 4, 10), (']',
']', 4, 11), (';', ';', 4, 12),
        # chr letter[2];
        ('chr', 'chr', 5, 1), ('id', 'letter', 5,
5), ('[', '[', 5, 11), ('ntlit', 2, 5, 12),
(']', ']', 5, 13), (';', ';', 5, 14),
        # strng nameArr;
        ('strng', 'strng', 6, 1), ('id',
'nameArr', 6, 7), (';', ';', 6, 14),
        # chr newLetter;
        ('chr', 'chr', 7, 1), ('id',
'newLetter', 7, 5), (';', ';', 7, 14),
        # dfstrct myStruct struct1,
struct2; (Assuming myStruct is
defined)
        # ('dfstrct', 'dfstrct', 8, 1), ('id',
'myStruct', 8, 9), ('id', 'struct1', 8,
18), (',', ',', 8, 25), ('id', 'struct2', 8,
27), (';', ';', 8, 34),

        # dbl fra3[2] = { 3.2 };
        ('dbl', 'dbl', 10, 1), ('id', 'fra3',
10, 5), ('[', '[', 10, 9), ('ntlit', 2, 10,
10), (']', ']', 10, 11), ('=', '=', 10, 13),
('{', '{', 10, 15), ('dbllit', 3.2, 10, 17),
('}', '}', 10, 21), (';', ';', 10, 22),
        # frac4[2][1] = {{3.3},{2.5}};
        ('id', 'frac4', 10, 24), ('[', '[', 10,
29), ('ntlit', 2, 10, 30), (']', ']', 10,
31), ('[', '[', 10, 32), ('ntlit', 1, 10,
33), (']', ']', 10, 34), ('=', '=', 10, 36),
('{', '{', 10, 38), ('{', '{', 10, 39),
('dbllit', 3.3, 10, 40), ('}', '}', 10, 43),
(',', ',', 10, 44), ('{', '{', 10, 45),
('dbllit', 2.5, 10, 46), ('}', '}', 10, 49),
('}', '}', 10, 50), (';', ';', 10, 51),
        # bln flag2[2] = { tr, fls };
        ('bln', 'bln', 11, 1), ('id', 'flag2',
11, 5), ('[', '[', 11, 10), ('ntlit', 2, 11,
11), (']', ']', 11, 12), ('=', '=', 11, 14),
('{', '{', 11, 16), ('blnlit', 'tr', 11, 18),
(',', ',', 11, 20), ('blnlit', 'fls', 11, 22),
('}', '}', 11, 26), (';', ';', 11, 27),
        # flag3[2][1] = {{tr},{tr}};
        ('id', 'flag3', 11, 29), ('[', '[', 11,
34), ('ntlit', 2, 11, 35), (']', ']', 11,
36), ('[', '[', 11, 37), ('ntlit', 1, 11,
38), (']', ']', 11, 39), ('=', '=', 11, 41),
('{', '{', 11, 43), ('{', '{', 11, 44),
('blnlit', 'tr', 11, 45), ('}', '}', 11, 47),
(',', ',', 11, 48), ('{', '{', 11, 49),
('blnlit', 'tr', 11, 50), ('}', '}', 11, 52),
('}', '}', 11, 53), (';', ';', 11, 54),
        # chr letter2[2] = { 'c' };
        ('chr', 'chr', 12, 1), ('id', 'letter2',
12, 5), ('[', '[', 12, 12), ('ntlit', 2, 12,
13), (']', ']', 12, 14), ('=', '=', 12, 16),
('{', '{', 12, 18), ('chrlit', 'c', 12, 20),
('}', '}', 12, 24), (';', ';', 12, 25),
        # letter3[2][1] = {{'d'},{'c'}};
        ('id', 'letter3', 12, 27), ('[', '[',
12, 34), ('ntlit', 2, 12, 35), (']', ']', 12,
36), ('[', '[', 12, 37), ('ntlit', 1, 12,
38), (']', ']', 12, 39), ('=', '=', 12, 41),
('{', '{', 12, 43), ('{', '{', 12, 44),
('chrlit', 'd', 12, 45), ('}', '}', 12, 48),
(',', ',', 12, 49), ('{', '{', 12, 50),
('chrlit', 'c', 12, 51), ('}', '}', 12, 54),
('}', '}', 12, 55), (';', ';', 12, 56),

        # mn(){ ... }
        ('mn', 'mn', 14, 1), ('(', '(', 14,
3), (')', ')', 14, 4), ('{', '{', 14, 5),
        # Example input statement
inside main
        ('nt', 'nt', 15, 5), ('id',
'userInput', 15, 8), (';', ';', 15, 17), #
Declare var
        ('id', 'userInput', 16, 5), ('=', '=',
16, 15), ('npt', 'npt', 16, 17), ('(', '(',
16, 20), ('strnglit', 'Enter value: ', 16,
21), (')', ')', 16, 36), (';', ';', 16, 37), #
Input statement
        ('end', 'end', 17, 7), (';', ';', 17,
10),
        # ('}', '}', 18, 1) # No closing
brace for mn if using end;
    ]


    print("--- Transpiling User's
Conso Code from Tokens ---")
    # Pass None for symbol_table for
now
    generated_c_code =
transpile_from_tokens(test_token_li
st, None) # Use token-based
function
    print("\n--- Generated C Code
---")
    print(generated_c_code)
```